

武汉大学《数据库系统/原理》

期末试卷解析册

Maple

2026 年 6 月 17 日

内容说明

本解析册整理了武汉大学计算机学院多年来的《数据库原理》与《数据库系统》期末考试及期中考试试卷，包含完整题目、参考答案、逐题解析与知识点评注。

部分试卷原无官方答案，解析者为补全答案并标注说明。

ER 图均使用 TikZ 重新绘制以保证矢量清晰。

适用课程：数据库原理、数据库原理与设计、数据库系统

试卷年份范围：2002–2024

目录

第一部分 期末试卷解析	5
1 2010–2011 学年《数据库原理与设计》期末试卷 (A)	5
1.1 一、判断题 (每小题 1 分, 共 10 分)	5
1.2 二、单项选择题 (每小题 1 分, 共 10 分)	12
1.3 三、查询设计题 (共 6 小题, 每小题 5 分, 共 30 分)	19
1.4 四、数据库分析、设计 (共 38 分)	25
1.5 五、简答题 (共 2 小题, 每小题 6 分, 共 12 分)	33
2 2008–2009 学年《数据库原理》期末试卷 (A)	36
2.1 一、单项选择题 (每小题 1 分, 共 15 分)	36
2.2 二、判断题 (正确的打“√”, 错误的打“×”。每小题 1 分, 共 10 分)	46
2.3 三、查询设计题 (每小题 5 分, 共 25 分)	54
2.4 1. 用关系代数表示下列查询	54
2.5 2. 用 SQL 语句表示下列查询	55
2.6 四、分析与设计题 (共 50 分)	58
2.7 1. 指出下列各关系模式的候选码、判断其属于第几范式 (1NF~BCNF)。(9 分)	58
2.8 2. 关系模式分解及判断 (12 分)	61
2.9 3. 企业数据库综合设计 (29 分)	63
3 2011–2012 学年《数据库原理与设计》期末试卷 (A)	72
3.1 一、判断题 (每小题 1 分, 共 10 分)	72
3.2 二、单项选择题 (每小题 1 分, 共 10 分)	75
3.3 三、查询设计题 (共 6 小题, 每小题 5 分, 共 30 分)	78
3.4 四、数据库分析、设计 (共 38 分)	83
3.5 五、简答题 (共 2 小题, 每小题 6 分, 共 12 分)	89
4 2022–2023 学年《数据库系统》期末试卷 (A 卷)	90
4.1 一、单项选择题 (共 10 小题, 每小题 2 分, 共 20 分)	91
4.2 二、判断题 (正确的打“√”, 错误的打“×”, 共 10 小题, 每小题 2 分, 共 20 分)	98
4.3 三、实体联系 (E-R) 模型设计及关系模式转化 (共 10 分)	106
4.4 四、数据操作 (共 30 分)	108
4.5 四.1 学生-学院-俱乐部管理系统 (共 9 分)	108
4.6 四.2 学生选课系统——关系代数与 SQL 综合 (共 21 分)	110

4.7	五、关系模式规范化（共 10 分）	113
4.8	六、事务、并发控制与恢复（共 10 分）	115
4.9	六.1 银行账务系统——基于封锁的并发控制	115
4.10	六.2 基于日志和检查点的数据库恢复	117
5	2002–2003 学年《数据库原理》期末试卷 (A 卷)	119
5.1	一、填空题（每小题 2 分，共 10 分）	120
5.2	二、单项选择题（每小题 1 分，共 10 分）	124
5.3	三、判断改错题（每小题 2 分，共 10 分）	130
5.4	四、简答题（每小题 5 分，共 20 分）	134
5.5	五、查询设计题（每小题 4 分，共 24 分）	138
5.6	六、数据库设计（每小题 10 分，共 20 分）	144
5.7	七、证明题（共 6 分）	150
5.8	选做题（参考分：5 分）	152
5.9	附：《数据库原理》测试试题	153
5.10	一、单项选择题（每小题 1 分，共 10 分）	153
6	2009–2010 学年《数据库原理与设计》期末试卷 (A)	159
6.1	一、判断题（每小题 1 分，共 10 分）	159
6.2	二、单项选择题（每小题 1 分，共 10 分）	165
6.3	三、查询设计题（共 6 小题，每小题 5 分，共 30 分）	170
6.4	四、数据库分析、设计（共 34 分）	175
6.5	五、简答题（共 2 小题，每小题 8 分，共 16 分）	181
7	2019–2020 学年《数据库系统》期末试卷 (A 卷，开卷)	186
8	2023–2024 学年《数据库系统》期末试卷	211
8.1	A 卷	211
8.2	一、单项选择题（每题 2 分，共 20 分）	212
8.3	二、多项选择题（每题均有两个答案，每题 2 分，共 10 分）	217
8.4	三、规范化设计（15 分，每题 5 分）	220
8.5	四、关系模式规范化（15 分，每题 5 分）	224
8.6	五、SQL 综合查询（15 分，每题 3 分）	228
8.7	六、ER 图设计与关系模式转化（10 分）	230
8.8	B 卷	232
8.9	一、单项选择题（每题 2 分，共 20 分，题号 3–12）	232
9	2021–2022 学年《数据库系统》期末试卷	237
9.1	一、选择题（本大题共 10 小题，每题 2 分，共 20 分）	237

9.2 二、判断题（本大题共 10 小题，每题 2 分，共 20 分。正确的打 ✓，错误的打 ×）	246
9.3 三、综合题	253
第二部分 期中试卷解析	260
10 2008–2009 学年《数据库原理》期中试卷	260
10.1 一、关系代数与 SQL 查询（40 分）	261
10.2 2. 用 SQL 语句完成下列查询	263
10.3 二、关系模式 BCL 的规范化（18 分）	268
10.4 三、判定关系模式的范式等级（18 分）	273
10.5 四、关系模式的分解（24 分）	277
11 2023–2024 学年《数据库系统》期中测试	283
11.1 一、单项选择题（第 1–9 题）	283
11.2 二、教材习题（第 10–12 题）	289
11.3 三、SQL 应用题（第 13–15 题）	291
第三部分 附录	299
12 知识点索引	299
13 TikZ 样式参考	301

第一部分 期末试卷解析

1 2010–2011 学年《数据库原理与设计》期末试卷 (A)

1.1 一、判断题 (每小题 1 分, 共 10 分)

1. 关系代数中的基本运算是交、差、投影、选择和笛卡儿积。

参考答案

错误 (×)。正确的基本运算应为: 并、差、笛卡儿积、投影和选择。交运算可由差运算导出 ($R \cap S = R - (R - S)$), 不属于基本运算。

解析与知识点

本题考察对关系代数基本运算体系的精确掌握。关系代数由 E.

F. Codd 提出, 其基本运算仅有五种: 并 ($\cup$)、差 ($-$)、笛卡儿积 ($\times$)、投影 (π) 和选择 (σ)。这五种运算是不可再简化的最小运算集, 所有其他关系代数运算 (如交、连接、除) 都可以通过这五种基本运算的组合等价表达。

题目将“交”列为基本运算而遗漏了“并”, 犯了两个错误。交运算可以通过 $R \cap S = R - (R - S)$ 用两次差运算导出, 因此不是基本运算。并运算虽然看似简单, 但无法用其他四种基本运算导出, 所以它必须被列为基础运算。从数据库系统角度看, 这五种基本运算为 SQL 的 SELECT-FROM-WHERE 结构提供了理论基础, 理解运算的底层实现也有助于编写高效的数据库查询语句。

关系代数的完整性和封闭性保证了任何合法查询都可以转化为这五种基本运算的组合, 这是查询优化器进行代数优化的理论基础。在考试中, 常有考生将 θ -连接或自然连接误视为基本运算, 或认为只有四种基本运算 (遗漏并或遗漏笛卡儿积), 这些都是需要警惕的常见误区。知识点: 第二章——关系代数基本运算

2. 可通过视图进行各种数据查询和更新操作。

参考答案

错误 (×)。视图可用于数据查询, 但不是所有视图都可更新。视图的更新操作受到严格限制, 只有行列子集视图等少数类型的视图允许进行更新操作。

解析与知识点

本题考察对视图可更新性的正确认知, 这是区分视图与基本表的关键特性之一。视图是从一个或多个基本表导出的虚拟表, 数据库中仅存储其定义而不存储实际

数据。

SQL 标准对视图更新有严格限制：更新操作必须能够无歧义地映射回基本表上的对应更新。具体而言，若视图定义中包含聚合函数（SUM、AVG、COUNT 等）、GROUP BY 子句、DISTINCT 关键字、多表连接、集合操作（UNION、INTERSECT、EXCEPT）或计算列，则该视图通常是不可更新的。

只有行列子集视图（从单个基本表导出的简单视图，保留原表的主码和部分行列）才支持更新。本题的错误在于“各种”二字泛化了视图的更新能力——实际上绝大多数复杂视图都无法进行更新操作。从数据库系统设计角度看，视图的不可更新性既是性能优化的代价也是数据安全的保障：通过视图隐藏底层复杂查询的同时也阻止了对多表结构的无意破坏。

此外，WITH CHECK OPTION 子句可以在行列子集视图上附加额外的更新约束，防止通过视图插入的数据不再满足视图定义条件。常见误区是将视图等同于基本表，认为视图与表在操作上完全等同。知识点：第三章——视图定义、查询与更新限制

3. 一个关系可以没有主属性。

参考答案

错误（×）。根据关系的实体完整性规则，每个关系必须有一个主码，主码中的属性即为主属性。既然必须有主码，也必然存在主属性，因此一个关系不可能没有主属性。

解析与知识点

本题考察关系模型中实体完整性与主属性概念之间的逻辑关系。主属性定义为包含在任一候选码中的属性。

关系的三大完整性约束之首——实体完整性——要求每个关系必须定义主码且主码值唯一、非空。每个关系至少存在一个候选码（被选为主码），因此至少存在一个属性包含在该候选码中，即主属性必然存在。

极端情况下，如果关系是所有属性构成的“全码”关系（如教师授课关系 TM(T#, M#)，只有所有属性联合才能唯一标识元组），则所有属性都是主属性。因此没有主属性的关系在关系模型中是理论上的不可能情况。这种理解对于后续学习范式理论至关重要，因为 3NF 的判定条件正是基于主属性/非主属性的区分：若每个非主属性既不部分依赖也不传递依赖于码，则满足 3NF。

若连主属性都不存在，整个范式体系便无从谈起。常见误区是将“主属性”与“非空属性”或“主码属性”混淆——主属性涵盖所有候选码的属性，而不只是主码属性。知识点：第二章——关系完整性约束（实体完整性）；第六章——候选码与主

属性的定义

4. 如果关系 R 的候选码是一个全码，则 R 一定是 3NF 也一定是 BCNF。

参考答案

正确 (✓)。全码意味着关系的所有属性构成唯一候选码。此时关系中不存在非主属性，因此 3NF 和 BCNF 的条件自动满足。

解析与知识点

本题考察全码关系与高级范式之间的蕴含关系，是范式理论中一个重要的边界情形。全码意味着关系的唯一候选码由全体属性构成，因此所有属性都是主属性，不存在任何非主属性。

3NF 的条件是“每个非主属性既不部分依赖也不传递依赖于候选码”——既然无非主属性，该条件自然满足。BCNF 的条件更为严格：“每个非平凡函数依赖的决定因素都是超码”。

在全码关系中，若存在非平凡函数依赖 $X \rightarrow Y$ ，则 $X \subset U$ 且 $X \neq U$ 。但此时 $X \rightarrow Y$ 意味着 X 可以决定 $U - X$ 中的某些属性，结合 X 自身， X 实际上能够决定 U 的全部属性，从而 X 也是一个超码（甚至是候选码），这与“全码是唯一候选码”矛盾。因此全码关系中不存在非平凡函数依赖，BCNF 条件也平凡地成立。

这一结论在模式分解和数据库设计中具有实际意义：全码关系已经是最优化的，无需进一步分解。常见误区是认为全码关系只满足 3NF 不满足 BCNF（混淆了“全主属性”和“全码”两个概念——全主属性不一定是全码，且全主属性关系只保底 3NF 不一定 BCNF；但全码关系必定同时满足两者）。知识点：第六章——3NF 与 BCNF 的判定条件；第六章——全码关系的范式性质

5. 一个关系的非聚集索引可以建立多个。

参考答案

正确 (✓)。聚簇索引（聚集索引）最多只能有一个，因为它决定了数据的物理存储顺序；而非聚簇索引（非聚集索引）可以建立多个，这是关系数据库的常规做法。

解析与知识点

本题考察聚簇索引与非聚簇索引在数量限制上的本质区别。

聚簇索引（聚集索引）将表中的数据行按索引键值进行物理排序存储，由于物理存储顺序只能有一种，因此每个表最多只能有一个聚簇索引。

非聚簇索引则独立于数据的物理存储，其叶结点存储的是指向数据行所在位置的指针（行定位符或主键引用），不改变数据行的物理排放方式。

由于没有物理存储顺序的限制，非聚簇索引理论上可以建立任意多个——实际 DBMS 如 SQL Server 最多支持 999 个非聚簇索引。在数据库性能优化中，这种索引类型的选择至关重要：聚簇索引由于数据与索引一体，适合范围查询和排序操作，且节省一次回表查询的开销；非聚簇索引适合单值查询和覆盖索引场景。

常见误区是混淆“可以建立多个非聚集索引”与“应该建立多个非聚集索引”——每增加一个索引都会在插入、删除和更新时带来额外的维护开销，因此需要在查询加速和维护代价之间权衡。

理解两种索引的物理本质差异，是编写高效 SQL 和进行数据库物理设计的基础。

知识点：第三章——索引分类：聚簇索引（聚集索引）与非聚簇索引（非聚集索引）

6. 若事务 T_1 对数据对象 A 上了 IS 锁，则事务 T_2 不能对数据对象 A 上 SIX 锁。

参考答案

错误（×）。根据多粒度封锁的锁相容矩阵，IS 锁与 SIX 锁是相容的。当 T_1 对数据对象 A 持有 IS 锁时， T_2 可以对同一对象加 SIX 锁。

解析与知识点

本题考察多粒度封锁协议中意向锁的相容性判断，是并发控制部分的核心考点。

在多粒度封锁中，IS（意向共享锁）表示事务打算在子孙结点上加共享锁（S 锁），而 SIX（共享意向排他锁）是 S 锁与 IX 锁的组合，即在当前结点加 S 锁同时允许在子孙结点加 X 锁。

根据多粒度锁相容矩阵，IS 与 SIX 是相容的——原因在于 IS 只表示在更低粒度读取数据（加 S 锁），而 SIX 在当前粒度加 S 锁同样允许读取，两者不存在直接冲突。

如果两者在子孙结点上请求冲突的锁（如都请求 X 锁），冲突将发生在子结点层面而非当前结点。多粒度封锁协议设计了 5 种锁类型（IS、IX、S、SIX、X），各锁之间的相容性规律是：任何意向锁（IS、IX）之间总是相容的，S 锁与 IS 相容但不与 IX 相容，X 锁与任何锁都不相容。

常考易混点：IS 与 IX 相容，IS 与 SIX 相容，但 IX 与 SIX 不相容。

深入理解意向锁的设计动机有助于把握数据库系统中多粒度并发控制的核心机制，即将不同粒度的封锁请求统一管理，通过意图锁在高层封装对低层的访问意图，避免对整个数据对象加锁导致并发度下降。知识点：第十章——多粒度封锁协议；第十章——意向锁类型与相容矩阵

7. 若 $X \twoheadrightarrow Y$ ，其中 $XY = U$ ，则称 $X \twoheadrightarrow Y$ 为平凡的多值依赖。

参考答案

正确(√)。当 $XY = U$ (即 X 与 Y 的并集等于全属性集 U)，且 $Z = U - X - Y = \emptyset$ 时， $X \twoheadrightarrow Y$ 是平凡多值依赖。

解析与知识点

本题考察多值依赖的平凡性判定条件。多值依赖 $X \twoheadrightarrow Y$ 称为平凡的当且仅当满足两个条件之一：(1) $Y \subseteq X$ ，或 (2) $X \cup Y = U$ 。

当 $X \cup Y = U$ (即 $XY = U$) 时，剩余属性集 $Z = U - X - Y = \emptyset$ ，多值依赖的定义“对于任意一对 (x, z_1) 和 (x, z_2) ，对应的 y 值集合相同”中 Z 为空集，该条件自动成立，因此 $X \twoheadrightarrow Y$ 是平凡的。

通俗理解：当 X 与 Y 的并已经是全部属性时，多值依赖不施加任何实际约束，因为它要求 Y 的值与不存在的 Z 无关。

非平凡多值依赖才是 4NF 的关注对象——若 $X \twoheadrightarrow Y$ 非平凡且 X 不是超码，则该关系不属于 4NF。多值依赖是理解 4NF 和模式设计中更高级冗余问题的前提，它描述了属性集之间独立的多对多语义关系。

常见误区是将平凡多值依赖与非平凡多值依赖的判定条件记反，或将多值依赖与函数依赖的平凡性条件混淆（函数依赖平凡条件为 $Y \subseteq X$ ，多值依赖多了一个 $X \cup Y = U$ 的条件）。多值依赖产生的根本原因是关系中存在两个或多个相互独立的多值属性，如一门课有多个教师和多本教材，教师和教材之间独立无关。知识点：第六章——多值依赖的定义与平凡/非平凡判定条件

8. 登记日志文件必须先写日志文件，后写数据库。

参考答案

正确(√)。这是 WAL (Write-Ahead Logging, 预写式日志) 协议的核心原则，可确保发生故障时能够通过日志恢复数据库的一致性状态。

解析与知识点

本题考察数据库恢复技术中最为核心的“先写日志”原则（Write-Ahead Logging, WAL）。WAL 规定：在对数据库的数据页做任何修改之前，必须先将对应的日志记录（包含事务标识、修改前后的值等）写入稳定存储上的日志文件。

这样做的基本原理是：一旦系统发生故障（如断电、系统崩溃），内存中的数据全部丢失，但稳定存储上的日志得以保留。

此时恢复子系统可通过日志中记录的信息进行 Redo（重做已提交但未写盘的事务）和 Undo（撤销未提交但已写盘的事务）操作，将数据库恢复到最近的一致性状态。

如果反过来先写数据库后写日志，在写入数据库之后、写日志之前若发生故障，数据库已修改但日志中没有对应记录，恢复系统无法知晓修改的存在，也无法撤销或重做，导致数据库处于不可恢复的不一致状态。WAL 是几乎所有现代 DBMS（PostgreSQL、Oracle、MySQL InnoDB 等）日志恢复机制的基础。

与 WAL 密切相关的还有检查点机制——通过在日志中标记检查点，恢复时只需从最近的检查点开始扫描日志，大幅缩短恢复耗时。这一原则也体现了数据库恢复方案的设计哲学：宁可容忍部分提交事务的数据丢失（通过重做恢复），也绝不能容忍数据库进入不可恢复的状态。知识点：第十一章——先写日志原则（WAL）与数据库恢复机制

9. 数据镜像是在并发控制采用的措施之一。

参考答案

错误（×）。数据镜像是数据库恢复（高可用性）技术，而非并发控制措施。它通过在不同物理位置维护数据副本，应对介质故障，与事务并发控制无关。

解析与知识点

本题考察数据库系统中两大功能模块——并发控制与恢复技术——的正确归属区分。并发控制解决的是多个事务同时访问同一数据时可能产生的不一致性问题，包括丢失修改（Lost Update）、不可重复读（Non-repeatable Read）和读脏数据（Dirty Read）。

并发控制的主要手段包括封锁协议（两段锁协议）、时间戳排序、多版本并发控制（MVCC）等。

而数据镜像（Mirroring）是将整个数据库或部分数据复制到另一个物理存储设备甚至异地机房的技术手段，属于数据库恢复和高可用性范畴。

镜像的主要用途是对抗介质故障（如磁盘损坏、机房火灾）：当主存储介质失效时，

系统可自动切换到镜像副本继续提供服务，不影响业务连续性。RAID1（磁盘镜像）、数据库镜像（如 SQL Server Database Mirroring）、异步复制（如 MySQL Replication）等技术均属于此范畴。

常见误区是将“镜像”与“多版本”混淆——二者的本质区别在于：镜像的目的是灾难恢复和容错，多版本的目的是实现并发隔离（如 MVCC 中用版本快照避免读写冲突）。理解这两个概念的技术定位，有助于全面把握数据库系统的整体架构：事务管理（含并发控制）、恢复管理和存储管理是 DBMS 的三大核心组件。知识点：第十一章——数据镜像与介质故障恢复；第十章——并发控制技术的范畴与目标

10. 函数依赖集中，若属性 X 仅在任一函数依赖的左部出现，则候选码中必包含 X 。

参考答案

正确 (✓)。这是候选码求解的经典法则之一：若某属性仅出现在所有函数依赖的左侧（从未出现在右侧），则该属性必定属于每一个候选码。

解析与知识点

本题考察候选码求解算法中的属性分类法则。在给定关系模式 $R(U, F)$ 中求解所有候选码时，首先将属性分为四类：(1) L 类：仅出现在函数依赖左部的属性；(2) R 类：仅出现在函数依赖右部的属性；(3) N 类：左右部均未出现的属性；(4) LR 类：左右部均出现的属性。

其中 L 类和 N 类属性必定属于所有候选码。

原因在于：R 类属性可以被其他属性通过函数依赖推导出来，但 L 类属性不会被任何函数依赖推导产生（因为没有任何函数依赖的右部包含它），所以 L 类属性只能从候选码本身出发得到，必须包含在候选码中。

同理，N 类属性完全孤立于函数依赖集之外，也只能通过候选码自身来提供。而 R 类属性必定不属于任何候选码（否则候选码将包含冗余属性）。

LR 类属性是否属于候选码需要通过闭包计算逐个验证。

理解这四类属性的划分及其与候选码的关系，对于求解候选码、进行模式分解和判定范式等级都是基础且关键的能力。考试中常见的错误是忽略 N 类属性（以为无关紧要），或错误地将 LR 类属性断定为必属于/必不属于候选码。知识点：第六章——候选码求解算法（属性分类法）；第六章——函数依赖闭包计算

1.2 二、单项选择题（每小题 1 分，共 10 分）

1. 数据模型的三要素是

- A. 外模式、模式和内模式
B. 关系模型、层次模型、网状模型
C. 实体、属性和联系
D. 数据结构、数据操作和完整性约束

参考答案

D。数据模型的三要素包括：数据结构（描述数据库的组成对象及对象间的联系）、数据操作（对数据库中对象允许执行的操作集合）和数据的完整性约束（一组数据完整性规则）。

解析与知识点

本题考察数据库领域最基础的概念——数据模型的定义与其三要素。数据模型是对现实世界数据特征的抽象，用于描述数据的组织方式及其操作规范。

其三个核心要素相互关联：数据结构定义了数据库静态特性（如关系模型中的二维表），是数据模型的基础；数据操作定义了数据库动态特性（如查询、插入、删除、更新），规定了允许执行的操作类型；完整性约束定义了语义约束（如实体完整性、参照完整性、用户定义完整性），确保数据的正确性与相容性。

选项 A “外模式、模式和内模式” 是数据库系统的三级模式结构，属于体系结构层面而非数据模型层面的概念。

选项 B 列举了三种具体的数据模型（层次、网状、关系），是对数据模型的分类而非要素。选项 C “实体、属性和联系” 是 E-R 模型（概念模型）的构成元素，而 E-R 模型本身也是一种特定的数据模型，并非数据模型的通用要素定义。

区分这些易混淆概念的关键在于理解抽象层次：三要素是数据模型的元定义，三级模式是数据库系统的体系结构，E-R 元素是概念建模工具。该知识点在数据库课程中反复出现，是贯穿全书的核心概念，后续学习中涉及的每种数据模型都可以从这三要素角度加以分析和理解。知识点：第一章——数据模型的定义与三要素

2. 一个表的主码

- A. 可以没有
B. 可以有多个
C. 有且只有一个
D. 一个或多个

参考答案

C。根据实体完整性的要求，每个关系必须有一个且仅有一个主码。候选码可以有多个（如一个或多个），但主码只能从中选择一个。

解析与知识点

本题考察对主码 (Primary Key) 与候选码 (Candidate Key) 概念的精确认知, 这是数据库基础中最容易混淆的一组概念。

根据关系模型的实体完整性规则, 每个基本关系必须有且仅有一个主码。

候选码是关系中能够唯一标识每个元组的最小属性集合, 一个关系可能存在多个候选码 (如学生关系中, 学号和身份证号均可唯一标识学生), 但在定义关系时只能从这些候选码中选取一个作为主码。

因此选项 A 错误——不能没有主码; 选项 B 表述模糊——可以有多个候选码但不能有多个主码; 选项 D 正确的描述对象是候选码而非主码。在实际 SQL 建表中, PRIMARY KEY 子句只能出现一次 (虽然可以包含多个属性构成联合主码, 但那仍然是一个主码)。

此外, 主码的选择需要遵循一定原则: 选择长度短、值稳定不变、使用频率高的候选码作为主码。

理解主码的唯一性有助于把握关系数据库中实体标识的基本机制, 也为后续学习外码参照、索引设计和查询优化奠定基础。知识点: 第二章——实体完整性与主码定义; 第二章——候选码与主码的区别

3. 为以最快的速度恢复数据库系统, DBMS 需定时将对 DB 更新等的全部信息存入
- A. 日志文件 B. 用户的数据文件 C. 检查点的有关文件 D. 死锁文件

参考答案

C (原著参考答案), 也可选 **A**。检查点机制可缩短恢复时间, 因为恢复时只需从最近的检查点开始扫描日志, 无需从头开始。

解析与知识点

本题考察恢复技术中检查点机制的核心价值——缩短恢复时间。

虽然日志文件 (选项 A) 登记了所有数据库更新操作, 是实现恢复的基础, 但单纯依赖日志文件时, 系统故障后的恢复过程必须从日志的开头扫描到末尾, 对相关事务逐一重做或撤销, 耗时可能很长。

检查点机制通过在特定时刻执行以下操作来解决问题: (1) 将当前内存中的所有日志缓冲区和数据缓冲区刷写到磁盘; (2) 在日志文件中写入一条检查点记录, 标记此时哪些事务正在活跃执行。

恢复时从最近的检查点开始扫描日志即可——检查点之前的所有已提交事务的数据都已写入磁盘无需重做, 检查点之前的所有未提交事务也无需撤销 (因为它们可能已在检查点时被刷盘处理)。这样大幅缩短了恢复时间, 因此题目问“最快的

速度恢复”时，检查点是最佳答案。

选项 B 用户数据文件只是常规数据存储在，不含日志信息；选项 D 死锁文件用于死锁检测与恢复，属于并发控制范畴。

理解检查点的工作原理，需要将其放在完整的恢复流程中看待：日志提供”做了什么”的记录，检查点提供”从哪开始”的起点，二者协同工作共同构成高效的恢复机制。知识点：第十一章——检查点机制与恢复效率；第十一章——日志文件的角色与局限

4. 使用 SQL 语句进行查询操作时，在 WHERE 子句的条件表达式中，可以与所在位置的零个或多个字符相匹配的通配符是

- A. * B. % C. ? D. _

参考答案

B. 在 SQL 的 LIKE 谓词中，% 代表任意长度（包括零个字符）的字符串，_ 代表任意单个字符。

解析与知识点

本题考察 SQL 中 LIKE 谓词的通配符机制。

SQL 标准定义了两个通配符：%（百分号）匹配零个或多个字符的任意序列，如 WHERE name LIKE 'A%' 匹配以 A 开头的名称，WHERE name LIKE '%A' 匹配以 A 结尾的所有名称，WHERE name LIKE '%A%' 匹配包含 A 的所有名称；_（下划线）匹配任意单个字符，如 WHERE name LIKE 'A_' 匹配以 A 开头且总长度为 2 的名称，WHERE name LIKE '____' 匹配任意长度为 3 的名称。

常见误区有三个：第一是将 SQL 通配符与操作系统通配符混淆——* 和 ? 是 DOS/Unix Shell 以及一些文件搜索工具中的通配符，SQL 不使用它们；第二是忘记 % 可以匹配零个字符（即空字符串也算匹配）；第三是在模糊查询时遗漏了 ESCAPE 子句的使用——当需要精确搜索包含 % 或 _ 字符本身的数据时，必须使用 ESCAPE 定义转义字符。

通配符查询在实际应用中极为常见（如网站的搜索功能），但需要注意 % 和 _ 在 LIKE 中会导致全表扫描而无法利用 B+ 树索引的优势，在大数据量场景下应考虑使用全文索引替代。知识点：第三章——SQL LIKE 谓词与通配符 % 和 _ 的语义

5. 多用户的数据库系统的目标之一是使它的每个用户好像面对着一个单用户的数据库一样使用它，为此数据库系统必须进行

- A. 安全性控制 B. 完整性控制 C. 并发控制 D. 可靠性控制

参考答案

C。并发控制的目标就是使多个并发执行的事务在效果上等价于某种串行执行，从而让每个用户感到好像独占使用数据库一样。

解析与知识点

本题考察并发控制的核心目标——事务隔离性与可串行化调度。在多用户数据库系统中，多个事务可能同时访问相同的数据，若不加控制会导致丢失修改、不可重复读和读脏数据等数据不一致问题。

并发控制子系统通过封锁协议、时间戳排序、多版本并发控制（MVCC）等机制，确保并发事务的调度是可串行化的——即并发执行的结果等价于这些事务按某种顺序串行执行的结果。

一旦实现了可串行化，每个事务就好像独占数据库一样，不会感知到其他并发事务的存在，这就是事务隔离性的本质含义。

SQL 标准定义了四个隔离级别（READ UNCOMMITTED、READ COMMITTED、REPEATABLE READ、SERIALIZABLE），从低到高提供不同程度的隔离保证，其中可串行化（SERIALIZABLE）是完全隔离。隔离级别越高，数据一致性越强，但并发度越低——这是数据库系统设计中一致性与性能的经典权衡。

选项 A 安全性控制关注的是防止非法用户的未授权访问；选项 B 完整性控制维护数据的语义正确性；选项 D 可靠性控制（恢复机制）应对系统故障后的数据恢复。四个选项各自对应 DBMS 不同子系统的功能，理解它们的分工有助于把握数据库系统的整体架构。知识点：第十章——并发控制目标：可串行化调度与事务隔离性

6. 在新增记录时，某个字段未指定值，系统给它赋的值是
A. NULL B. 零 C. 默认值 D. A 或 C

参考答案

D（或 **A**）。若该字段定义了 DEFAULT 默认值，则系统赋默认值；若未定义默认值，则系统自动赋 NULL。注意：当字段有 NOT NULL 约束且未指定值也未定义默认值时，插入会失败。

解析与知识点

本题考察 SQL INSERT 操作中未指定字段的默认赋值规则。

SQL 标准规定的处理逻辑如下：(1) 若该字段定义了 DEFAULT 子句且插入语句未指定该字段的值，则填写 DEFAULT 指定的默认值；(2) 若未定义 DEFAULT 但字段允许 NULL（未加 NOT NULL 约束），则填写 NULL；(3) 若既未定义 DEFAULT 又定义了 NOT NULL 约束，则系统拒绝插入操作并返回错误。

因此实际结果是“若定义了默认值则填写默认值，否则填写 NULL”，这正是选项 D 的含义。

选项 A（仅 NULL）不完整，因为当定义了 DEFAULT 时不会填 NULL；选项 B（零）错误，数据库不会自动填零；选项 C（仅默认值）也不完整，因为很多字段没有定义 DEFAULT。此外还有一个重要细节：即使字段定义了 DEFAULT，也仅在 INSERT 语句未指定该字段时生效——如果显式指定了 NULL 且字段允许 NULL，DEFAULT 不会生效。

理解这些规则对于编写健壮的 SQL 插入语句和设计数据库表结构至关重要：在定义表时应充分考虑业务需求，合理设置默认值或 NOT NULL 约束，避免因未处理 NULL 值而导致查询结果偏差。

这也是数据库完整性约束（用户定义完整性）在实际工程中的典型应用场景。知识点：第三章——SQL INSERT 语句与 DEFAULT/NULL 赋值机制

7. 下面哪种故障会破坏磁盘上的物理数据和日志文件

- A. 中央处理器故障 B. 操作系统故障 C. 突然停电 D. 瞬时的强磁场干扰

参考答案

D。只有介质故障（如强磁场干扰、磁盘损坏）才会导致磁盘上物理存储数据的破坏。

解析与知识点

本题考察数据库系统中三类故障的特征区分，尤其是介质故障（硬故障）与其他故障的本质差异。

数据库故障分为三类：(1) 事务故障——由事务内部的逻辑错误（如除以零、违反完整性约束）或系统强制的死锁回滚引起，只影响单个事务，不破坏数据；(2) 系统故障（软故障）——包括 CPU 故障、操作系统崩溃、突然停电等，会导致内存中数据丢失和所有正在运行的事务非正常终止，但磁盘上的物理数据完整无损，恢复时只需重做已提交事务和撤销未完成事务；(3) 介质故障（硬故障）——包括磁盘损坏、磁头碰撞、强磁场干扰、自然灾害等，直接破坏外存上的物理数据甚

至日志文件，是最严重的一类故障，恢复必须依赖数据备份（数据库转储）和日志文件的异地保存。

选项 A（CPU 故障）、B（操作系统故障）、C（突然停电）都属于系统故障，只影响内存而非磁盘。

选项 D 强磁场干扰会改变磁盘磁性介质的磁场分布导致数据不可逆损坏，属于介质故障。

理解故障分类是掌握数据库恢复技术的先决条件——不同类别故障需要不同的恢复策略和代价，数据库系统针对各类故障设计了多层次的容灾方案（从日志恢复到镜像再到异地备份）。知识点：第十一章——数据库故障分类：事务故障、系统故障与介质故障的区分

8. 数据库三级模式体系结构的划分，有利于保持数据库的

A. 数据独立性 B. 数据安全性 C. 结构规范化 D. 操作可行性

参考答案

A。三级模式结构（外模式/模式/内模式）的设计目的是实现数据独立性，使得应用程序不依赖于数据的物理存储方式和逻辑结构。

解析与知识点

本题考察数据库系统体系结构中三级模式结构的核心价值——数据独立性。

三级模式体系结构由外模式（子模式/用户模式）、模式（逻辑模式/概念模式）和内模式（存储模式/物理模式）三个层次组成，并对外提供两级映像：外模式/模式映像和模式/内模式映像。

当数据库的物理存储结构（内模式）改变时，只需修改模式/内模式映像，上层模式 and 应用程序保持不变，由此实现物理数据独立性。

当数据库的全局逻辑结构（模式）改变时，只需修改外模式/模式映像使外模式保持不变，应用程序同样无需修改，由此实现逻辑数据独立性。正是这种分层隔离的设计，使数据库系统在物理存储方案更换或数据库逻辑结构扩展时无需大量修改应用程序代码，大幅度降低了软件维护成本。

其他选项的分析：选项 B（数据安全性）主要由存取控制（GRANT/REVOKE）、视图机制等实现；选项 C（结构规范化）指的是通过范式理论消除数据冗余；选项 D（操作可行性）是数据操作语言的设计目标。

三级模式结构是 ANSI/SPARC 提出的数据库体系结构标准，几乎所有现代 DBMS 都遵循这一基本框架。知识点：第一章——数据库三级模式体系结构与两级映像；第一章——物理数据独立性与逻辑数据独立性

9. 在集合成员的算术比较运算中, 与 NOT IN 等价的是

A. $\neq$ ANY

B. $=$ ANY

C. $\neq$ ALL

D. $=$ ALL

参考答案

C。 $x \text{ NOT IN } S$ 等价于 $x \neq \text{ALL } S$, 即要求 x 不等于集合 S 中的每一个元素, 即 x 不在 S 中。

解析与知识点

本题考察 SQL 中集合比较运算符的语义等价关系, 是查询设计中的基础知识点。SQL 提供了一组量词运算符用于集合间的比较: SOME (或 ANY) 表示“存在性”($\exists$), ALL 表示“全称性”($\forall$)。

具体等价关系如下: $x \text{ NOT IN } S \iff x \neq \text{ALL } S$ (x 不等于 S 中所有元素, 即 x 不在 S 中); $x \text{ IN } S \iff x = \text{ANY } S$ (x 等于 S 中某个元素, 即 x 在 S 中)。

常见混淆点: $x \neq \text{ANY } S$ 的语义是“存在 S 中某个元素与 x 不等”, 这几乎总是为真 (除非 S 中所有元素都等于 x), 语义比 NOT IN 弱得多, 因此选项 A 错误。 $x = \text{ALL } S$ 表示 x 等于 S 中所有元素, 通常只在 S 为空集或单元素集时才可能成立, 选项 D 错误。 $x = \text{ANY } S$ 等价于 IN, 选项 B 是 IN 的等价表达而非 NOT IN。

另外需要注意 NULL 值的影响: 当集合 S 包含 NULL 时, NOT IN 的行为可能不符合直觉 (所有比较对 NULL 均返回 UNKNOWN)。理解这些等价关系有助于编写正确且高效的子查询语句, 也是数据库查询优化器进行谓词转换的理论基础。
知识点: 第三章——SQL 集合比较运算: $\text{NOT IN} \iff \neq \text{ALL}$, $\text{IN} \iff = \text{ANY}$

10. 可以用来测试数据库系统中是否存在死锁的是

A. 数据流图

B. 用户活动图

C. 函数依赖图

D. 事务等待图

参考答案

D。 事务等待图 (Transaction Wait-for Graph) 是检测死锁的经典方法, 通过检测图中是否存在回路 (环) 来判断系统中是否发生死锁。

解析与知识点

本题考察并发控制中死锁检测的经典方法。事务等待图是一个有向图, 其中结点表示当前活跃的事务, 有向边 $T_i \rightarrow T_j$ 表示事务 T_i 正在等待事务 T_j 释放其持有的锁。

死锁检测的基本原理是: 在事务等待图中执行环检测算法 (如 DFS 深度优先搜索), 若发现存在有向回路, 如 $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$, 则环路中的所有事务均处于

相互等待的死锁状态。

DBMS 通常在后台定时（如每几秒）或在新请求锁被阻塞时触发死锁检测。

一旦检测到死锁，系统会选择代价最小的事务（如已运行时间最短、已锁资源最少的事务）作为牺牲者进行回滚，释放其持有的锁以打破死锁循环。其他选项与死锁检测无关：数据流图（DFD）是结构化分析方法中描述数据流动和处理的工具；用户活动图是 UML 中用于业务流程建模的图；函数依赖图用于关系规范化理论中分析属性间的函数依赖关系。

死锁问题的根源在于并发控制机制（封锁）本身——锁机制在保证数据一致性的同时引入了事务间等待的可能性，而循环等待则构成死锁。除死锁检测外，还有死锁预防策略（如一次封锁法、顺序封锁法）和死锁避免策略，各自在系统开销和并发度之间存在不同权衡。知识点：第十章——死锁检测方法：事务等待图与环检测

1.3 三、查询设计题（共 6 小题，每小题 5 分，共 30 分）

设有三个关系：

- 学生表 $s(\underline{sno}, sname, age, sex, dept)$ ，其属性表示学号、姓名、年龄、性别、所在系。
- 课程表 $c(\underline{cno}, cname, credit, pcno)$ ，其属性表示课程号、课程名、学分、先修课程号。
- 学习表 $sc(\underline{sno}, \underline{cno}, grade)$ ，其属性表示学号、课程号、成绩。

(1) 查询至少选修了两门课程的学生学号和姓名。（关系代数）

参考答案

方法一（自连接法）：

$$\pi_{sno, sname} \left(S \bowtie \pi_{[1]=[4] \wedge [2] \neq [5]} (SC \times SC) \right)$$

方法二（分组计数法，关系代数本身不支持 COUNT，此处用自连接）：

$$\pi_{sno, sname} \left(S \bowtie \pi_{sno} \left(\sigma_{[2] \neq [5]} \left(\pi_{sno, cno} (SC) \times \pi_{sno, cno} (SC) \right) \right) \right)$$

核心思想：通过 SC 自连接并找同一学号但不同课程号的记录对，再与 S 自然连接获取姓名。

解析与知识点

本题考察在关系代数框架下表达“至少 N 个”语义的技术——因为没有直接的聚合函数，必须借助自连接（self-join）枚举不同组合。核心思路：将 SC 表与自身做笛卡儿积（或自连接），寻找同一学号但不同课程号的记录对。

具体而言，先把 SC 投影为 (sno, cno)，再做笛卡儿积，结果为所有选课记录的两两配对。

然后通过选择条件筛选出同一学号 ([1] = [4]，即第一个关系的 sno 等于第二个关系的 sno) 但课程号不同 ([2] ≠ [5]) 的配对。

若某个学号存在这样的配对，说明该学生至少选修了两门不同课程。最后投影出学号并与 s 表自然连接获得姓名。

需要注意三个技术细节：第一，自连接前必须对 SC 进行投影 $\pi_{sno, cno}$ 去掉 grade 列，避免同名属性冲突；第二，条件 [2] ≠ [5] 不可遗漏，否则会包括同一门课程与自身的配对（一个学生选修一门课也算“配对”，导致误判）；第三，最终需通过自然连接 s 表获取姓名，因为 SC 表中没有 sname 信息。

此题也体现了关系代数与 SQL 的对照关系：SQL 中可以用 GROUP BY sno HAVING COUNT(DISTINCT cno) ≥ 2 轻松实现，但关系代数必须借助自连接和选择来模拟分组计数逻辑。这种对照有助于理解 SQL 查询引擎内部如何将聚合查询转化为基本关系代数运算。知识点：第二章——关系代数：投影、选择、笛卡儿积与自连接的综合运用

(2) 查询全部学生都选修的课程号和课程名。(关系代数)

参考答案

使用除法运算：

$$\pi_{cno, cname} \left(C \bowtie \left(\pi_{cno, sno}(SC) \div \pi_{sno}(S) \right) \right)$$

第一步： $\pi_{cno, sno}(SC)$ 得到所有选课记录（课程号-学号对）。第二步： $\pi_{sno}(S)$ 得到全部学生的学号集合。第三步： $\pi_{cno, sno}(SC) \div \pi_{sno}(S)$ 用除法找出那些“被全部学生选修”的课程号。第四步：与课程表连接获取课程名。

解析与知识点

本题考察关系代数中除法运算 (÷) 在“全部/所有”语义下的典型应用。

除法的数学定义为：设 $R(A, B)$ 和 $S(B)$ 是两个关系， $R \div S$ 的结果为满足“A 中的每个元组与 S 中所有元组的组合都在 R 中”的那些 A 值。

对应到本题场景： $\pi_{cno, sno}(SC)$ 表示每个学生选修的课程（二元关系）， $\pi_{sno}(S)$ 表示全体学生集合（一元关系）。

做除法后得到的是那些”在 sc 表中与所有学生都配对出现”的课程号——即被每位学生都选修了的课程。最后与课程表 c 做自然连接获取课程名。

需要深入理解的关键点：(1) 除法运算的前提是被除数关系至少是二元的，除数关系是一元的且其属性必须是被除数的真子集；(2) 除法可以通过基本运算等价表达为 $R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$ ，这意味着先找到那些”缺少了某个 S 中元素”的 A 值，再从全集中减去它们；(3) 在实际 DBMS 中除法没有直接对应的 SQL 语法，需要用 NOT EXISTS 双嵌套子查询来模拟。

掌握除法运算有助于理解全称量词查询的设计思路，这也是数据库理论中一个重要的抽象工具。知识点：第二章——关系代数除法运算的定义与”全部”语义表达

(3) 创建 sc 表，属性的数据类型和长度自行给定，并定义该表的主码、外码和 grade 值在 0-100 分之间的约束。(SQL)

参考答案

参考 SQL:

```
1 CREATE TABLE sc (  
2     sno    CHAR(5),  
3     cno    CHAR(8),  
4     grade INT,  
5     CONSTRAINT pk_sc PRIMARY KEY (sno, cno),  
6     CONSTRAINT fk_sc1 FOREIGN KEY (sno) REFERENCES s(sno),  
7     CONSTRAINT fk_sc2 FOREIGN KEY (cno) REFERENCES c(cno),  
8     CONSTRAINT ck_sc CHECK (grade BETWEEN 0 AND 100)  
9 );
```

说明：sno CHAR(5)、cno CHAR(8)、grade INT 为自行指定的数据类型和长度；pk_sc 定义学号和课程号联合主码；fk_sc1/fk_sc2 分别定义对 s 表和 c 表的外码参照；ck_sc 限定了成绩的取值区间。

解析与知识点

本题考察 SQL 中 CREATE TABLE 语句对各种完整性约束的综合定义。该建表语句同时实现了关系模型中的三类完整性约束：(1) 实体完整性——通过 PRIMARY KEY (sno, cno) 定义联合主码，由 sno 和 cno 共同唯一标识每个元组，DBMS 自动创建唯一索引并保证非空。

为什么选联合主码？

因为单个学生可以选多门课、单门课可以被多个学生选，只有 (学号, 课程号) 的

组合才能唯一标识一条选课记录。

(2) 参照完整性——通过 FOREIGN KEY 子句分别定义 sno 参照 s(sno) 和 cno 参照 c(cno)，保证 sc 表中的学号必须存在于学生表中、课程号必须存在于课程表中。参照完整性还支持 ON DELETE 和 ON UPDATE 级联选项（如 CASCADE 表示级联删除/更新），本题虽未明确要求但实际设计时应予考虑。

(3) 用户定义完整性——通过 CHECK (grade BETWEEN 0 AND 100) 将成绩限制在合理区间内，超出此范围的插入或更新将被 DBMS 拒绝。数据类型的选择原则：学号和课程号使用定长 CHAR 类型便于建立索引和快速匹配；成绩使用 INT 整数型简单高效。

这道题的实战价值很高，建表语句是所有数据库应用开发的基础，理解约束的定义方式对于保证数据质量和维护数据间一致性至关重要。

命名约束（如 pk_sc）也是一种良好的 SQL 编程实践，便于后续通过 ALTER TABLE 修改或删除约束。知识点：第三章——SQL 数据定义 (CREATE TABLE)；

第五章——三类完整性约束的 SQL 实现

(4) 用 EXISTS 查询所有未选课程的学生姓名和所在系。(SQL)

参考答案

参考 SQL:

```
1 SELECT sname, dept
2 FROM s
3 WHERE NOT EXISTS (
4     SELECT *
5     FROM sc
6     WHERE s.sno = sc.sno
7 );
```

相关子查询：外层遍历 s 表的每个学生，内层查询检查 sc 表中是否有该学生的选课记录；若不存在（NOT EXISTS 为真），则输出该学生的姓名和所在系。

解析与知识点

本题考察 SQL 中 EXISTS 谓词用于“不存在”语义的实现。EXISTS 是 SQL 判断子查询结果是否非空的谓词，NOT EXISTS 判断子查询结果是否为空。

这里的执行逻辑是相关子查询 (Correlated Subquery) 的经典模式：外层 SELECT 每取出 s 表的一个学生元组，将当前学号 s.sno 传入内层子查询，内层查询在 sc 表中搜索该学号的选课记录。

若内层查询找到任何记录, EXISTS 返回 TRUE (NOT EXISTS 返回 FALSE), 该学生被过滤掉; 若内层查询返回空集, NOT EXISTS 返回 TRUE, 该学生被选中。这种执行方式意味着 DBMS 对 s 表的每个元组都要执行一次内层查询, 时间复杂度为 $O(n*m)$ (n 是 s 表的行数, m 是 sc 表的行数), 在大数据量下效率较低。SQL 也支持等价的 IN/NOT IN 写法: WHERE sno NOT IN (SELECT sno FROM SC), 两种方式在语义上等价。

但两者的行为在子查询结果包含 NULL 时有差异: 若子查询结果包含 NULL, NOT IN 会因 NULL 比较的不确定性而返回空结果集 (因为 $x \text{ NOT IN } (1, 2, \text{NULL})$ 等价于 $x \neq 1 \text{ AND } x \neq 2 \text{ AND } x \neq \text{NULL}$, 而 $x \neq \text{NULL}$ 总是 UNKNOWN), NOT EXISTS 则不受影响。

因此在工程实践中, 处理可能含 NULL 的子查询时应优先使用 NOT EXISTS 以获得更可靠的结果。此题也体现了 SQL 中全称量词与存在量词的转换关系: ”所有学生都选课” 等价于 ”不存在没选课的学生”。知识点: 第三章——SQL 子查询: 相关子查询、EXISTS/NOT EXISTS 谓词的语义与应用

(5) 查询所有课程都及格的学生学号和姓名。(SQL)

参考答案

参考 SQL:

```
1 SELECT sno, sname
2 FROM s
3 WHERE sno IN (
4     SELECT sno
5     FROM sc
6     GROUP BY sno
7     HAVING MIN(grade) >= 60
8 );
```

思路: 按学号分组后, 用 $\text{MIN}(\text{grade}) \geq 60$ 过滤出最低成绩也不低于 60 分的学生。

解析与知识点

本题考察如何将 ”所有. . .

都. . . ” 的全称量词语义转化为 SQL 中的聚合判断。

”所有课程都及格” 等价于 ”不存在不及格的课程”, 也等价于 ”该学生的最低成绩不低于 60 分”。因此解题策略是在 sc 表中按 sno 分组, 用聚合函数 $\text{MIN}(\text{grade})$

获取每个学生所有课程的最低成绩，再用 $\text{HAVING MIN}(\text{grade}) \geq 60$ 筛选出最低成绩也及格的学号，最后与 s 表连接获取姓名。

这是一种非常常见且高效的写法。等价写法还有：(1) 使用 NOT EXISTS : $\text{SELECT sno, sname FROM s WHERE NOT EXISTS (SELECT * FROM sc WHERE s. sno = sc. sno AND grade} < 60)$ ，语义更直观——“不存在不及格成绩”；(2) 使用 NOT IN : $\text{SELECT sno, sname FROM s WHERE sno NOT IN (SELECT sno FROM sc WHERE grade} < 60)$ 。

三种写法语义等价，但执行计划和性能可能存在差异，取决于 DBMS 的查询优化器和表上的索引设计。

聚合方式 ($\text{GROUP BY} + \text{HAVING}$) 通常需要全表扫描，而 EXISTS 可以利用索引进行快速匹配。选择哪种写法需要结合实际数据量、索引布局和可读性需求综合考虑。这道题的关键在于理解“全部及格”与“最低分 ≥ 60 ”的逻辑等价性，这种全称量词到存在量词和聚合表达的逻辑转换是 SQL 查询设计的核心能力。知识点：第三章——SQL 聚合查询 ($\text{GROUP BY} + \text{HAVING}$)：全称量词语义的实现

(6) 查询至少有 30 名学生选修的、课程号以 5 开头的课程号、课程名和课程的平均成绩，并按平均成绩的降序排序输出。(SQL)

参考答案

参考 SQL:

```
1 SELECT c.cno, c.cname, AVG(sc.grade) AS avgc
2 FROM c, sc
3 WHERE c.cno = sc.cno
4       AND c.cno LIKE '5%'
5 GROUP BY c.cno, c.cname
6 HAVING COUNT(*) >= 30
7 ORDER BY avgc DESC;
```

要点：(1) $\text{c.cno LIKE '5\%'}$ 筛选以 5 开头的课程号；(2) $\text{GROUP BY c.cno, c.cname}$ 按课程分组；(3) $\text{HAVING COUNT(*)} \geq 30$ 保证选修人数不少于 30；(4) AVG(sc.grade) 计算平均成绩；(5) $\text{ORDER BY avgc DESC}$ 按均值降序。

解析与知识点

本题是一道综合性的 SQL 查询设计题，几乎涵盖了 SELECT 语句的所有核心子句。逐步分析如下：(1) FROM 子句使用隐式连接（逗号分隔）将 c 表与 sc 表做笛卡儿积，再由 WHERE 子句中的连接条件 c .

$cno = sc.cno$ 转为等值连接, 获取每门课程对应的所有选课记录。(2) WHERE 子句中的 $c.cno \text{ LIKE '5\%'}$ 使用字符串模式匹配筛选以 5 开头的课程号——注意 % 在 LIKE 中代表零个或多个字符。

(3) GROUP BY c .

$cno, c.cname$ 将选课记录按课程分组, 同一门课程的 record 聚集到一个组中。根据 SQL 标准, SELECT 中出现的非聚合列必须出现在 GROUP BY 中, 这里同时按 cno 和 $cname$ 分组 (因为 $cname$ 虽然没有直接参与聚合, 但与 cno 是一一对应的)。(4) HAVING $COUNT(*) \geq 30$ 在分组后过滤掉选修人数少于 30 的课程——注意 WHERE 不能带聚合函数, 这是 WHERE 与 HAVING 最重要的区别。

(5) SELECT 子句使用 $AVG(sc.grade) \text{ AS } avgc$ 计算每组的平均成绩, 并给列起别名。

(6) ORDER BY $avgc \text{ DESC}$ 按平均成绩降序排列输出结果。SQL 的逻辑执行顺序为 FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY, 理解这一顺序有助于正确构造查询语句。

此题还体现了连接查询中使用表别名 (c, sc) 区分不同表中同名属性的良好实践。知识点: 第三章——SQL 综合查询: 连接、聚合、分组、过滤与排序的组合使用

1.4 四、数据库分析、设计 (共 38 分)

1. 有关系模式 $R(A, B, C, D, E)$, 其上的函数依赖集 $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow B, D \rightarrow E\}$ 。(18 分)

(1) 写出关系 R 所有的候选码。

参考答案

候选码为: **AB、AC、AD。**

计算过程:

- 属性分类: A 仅出现在函数依赖左部 (L 类), 必属于候选码; B、C、D 左右均出现 (LR 类); E 仅出现在右部 (R 类), 不属于候选码。
- 计算 A_F^+ : A 的闭包仅为 $\{A\}$, 无法覆盖 U 。
- 计算 $(AB)_F^+$: $AB \rightarrow C \rightarrow D \rightarrow B, D \rightarrow E$, 得 $\{A, B, C, D, E\} = U$, 故 AB 为候选码。
- 计算 $(AC)_F^+$: $AC \rightarrow D$ (由 $C \rightarrow D$ 经增广律得 $AC \rightarrow D$) $\rightarrow B$ (由 $D \rightarrow B$ 传递) $\rightarrow E$ (由 $D \rightarrow E$ 传递), 得 $\{A, C, D, B, E\} = U$, 故 AC 为候选码。
- 计算 $(AD)_F^+$: $AD \rightarrow B$ (由 $D \rightarrow B$ 增广) $\rightarrow C$ (由 $AB \rightarrow C$, 已得 B, 再

得 $A, B \rightarrow E$ (由 $D \rightarrow E$ 增广), 得 $\{A, D, B, C, E\} = U$, 故 AD 为候选码。

解析与知识点

本题考察利用属性分类法和闭包计算求解关系模式所有候选码的系统性方法。第一步, 属性分类: 将 $U = \{A, B, C, D, E\}$ 中每个属性按其出现在函数依赖集中的位置分为 L 类 (仅左部)、R 类 (仅右部)、LR 类 (左右均出现)、N 类 (左右均不出)。

A 只在左部出现 ($AB \rightarrow C$ 中与 B 共同决定 C), 为 L 类, 必须包含在任意候选码中。

E 只出现在 $D \rightarrow E$ 的右部, 为 R 类, 不可能进入候选码。

B、C、D 均在多个依赖中轮流出现于左右部, 为 LR 类, 是否属于候选码需要进一步验证。第二步, 以 L 类属性 A 为基础, 逐一组合 LR 类属性并计算闭包: A 单独 ($A_F^+ = \{A\}$, 不足); AB 组合 ($AB_F^+ = \{A, B, C, D, E\} = U$, 找到 AB 为候选码); AC 组合 (AC_F^+ 通过 $C \rightarrow D \rightarrow B \rightarrow E$ 导出全属性, 找到 AC 为候选码); AD 组合 (AD_F^+ 通过 $D \rightarrow B$ 与 $AB \rightarrow C$ 联合导出全属性, 找到 AD 为候选码)。

第三步, 验证最小性: 每个候选码去掉任何属性后闭包都不再等于 U, 证明 AB、AC、AD 都是最小化的。R 有三个候选码, 这在实际数据库设计中意味着可以选择其中任意一个作为主码。

另外需要注意一个技术细节: 当存在多个候选码时, 所有候选码中的属性都是主属性, 即 $\{A, B, C, D\}$ 都是主属性, 只有 E 是非主属性。

这一结论对后续 3NF 判定至关重要。知识点: 第六章——候选码求解方法: 属性分类法与闭包计算

(2) 指出所有违反 3NF 条件的函数依赖。

参考答案

非主属性仅有 E。违反 3NF 条件的函数依赖如下:

- 基本违反依赖: $D \rightarrow E$ —— D 不是超码, E 是非主属性, 且 E 不包含在任何候选码中, 故 $D \rightarrow E$ 违反 3NF。
- 推导出的违反依赖 (传递依赖):
 - $AB \rightarrow E$: 由 $AB \rightarrow C, C \rightarrow D, D \rightarrow E$ 传递导出。AB 是候选码 (因而是超码), 此依赖本身不违反 3NF; 但它揭示了 E 通过 C 和 D 传递依赖于 AB。

- $AC \rightarrow E$: 由 $C \rightarrow D, D \rightarrow E$ 传递导出。
- $AD \rightarrow E$: 由 $D \rightarrow E$ 直接导出 (通过增广律)。

严格按参考答案, 违反 3NF 条件的函数依赖为: $AB \rightarrow E$ (经 $AB \rightarrow C, C \rightarrow E$ 推导), $AC \rightarrow E$ (经 $AC \rightarrow D, D \rightarrow E$ 推导), $AD \rightarrow E$ (经 $AD \rightarrow C, C \rightarrow E$ 推导)。这些传递依赖表明非主属性 E 通过非候选码属性传递依赖于候选码, 违反了 3NF 定义。

解析与知识点

本题考察 3NF 的严格判定过程——这是范式理论学习的核心环节。3NF 的形式化定义为: 对于 F^+ 中的每个非平凡函数依赖 $X \rightarrow Y$, 要么 X 是超码, 要么 Y 中的每个属性都是主属性。

在此关系 R 中, 根据上一问结论, 候选码为 AB, AC, AD , 主属性集为 $\{A, B, C, D\}$, 非主属性为 $\{E\}$ 。

逐条检查 F 中的函数依赖: (1) $AB \rightarrow C$: 决定因素 AB 是候选码 (超码), 满足 3NF。

(2) $C \rightarrow D$: C 不是超码, 但 D 是主属性 (属于候选码 CD, AD), 满足 3NF。

(3) $D \rightarrow B$: D 不是超码, 但 B 是主属性 (属于候选码 AB), 满足 3NF。

(4) $D \rightarrow E$: D 不是超码, 且 E 是非主属性, 违反 3NF。参考答案进一步列出了 F^+ 中由基本依赖推导出的传递依赖 ($AB \rightarrow E, AC \rightarrow E, AD \rightarrow E$), 这些依赖揭示了 E 通过中间属性 C 和 D 传递依赖于各候选码的完整路径。

理解 3NF 判定的关键在于: (1) 正确识别主属性/非主属性; (2) 若决定因素本身是超码, 依赖永远合格; (3) 若决定因素不是超码, 检查依赖项是否全是主属性——如果是主属性 (如 $C \rightarrow D$), 即使决定因素非超码也不违反 3NF。

这个“主属性豁免”是 3NF 与 BCNF 的最本质区别, 也是为什么存在“全主属性只保底 3NF 不一定 BCNF”结论的原因。知识点: 第六章——3NF 判定: 主属性/非主属性区分与传递依赖识别

(3) 将 R 分解为具有无损连接性的 BCNF 的关系集合。

参考答案

分解结果: $\rho = \{CD, CB, CE, AC\}$ 。

具体分解步骤:

1. 初始: $R(A, B, C, D, E), F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow B, D \rightarrow E\}$ 。

2. $C \rightarrow D$ 违反 BCNF (C 不是超码)。分解 $R(ABCDE)$ 为:

$$R_1: CD \quad (\text{关键字 } C, F_1 = \{C \rightarrow D\})$$

$$R_2: CBAE \quad (\text{关键字? } F_2 = \{AB \rightarrow C, C \rightarrow B, C \rightarrow E\})$$

保留函数依赖: 从 $C \rightarrow D$ 和 $D \rightarrow B$ 得到 $C \rightarrow B$; 从 $C \rightarrow D$ 和 $D \rightarrow E$ 得到 $C \rightarrow E$ 。

3. $R_1(CD)$: 仅有 $C \rightarrow D$, C 是超码, 满足 BCNF。
 4. $R_2(CBAE)$: $C \rightarrow B$ 违反 BCNF (C 不是超码, $AB \rightarrow C$ 但 $C \not\rightarrow AB$ 不完整)。分解为:

$$R_{21}: CB \quad (\text{关键字 } C, F_{21} = \{C \rightarrow B\})$$

$$R_{22}: CAE \quad (\text{关键字 } AB/AC, F_{22} = \{AB \rightarrow C, C \rightarrow E\})$$

5. $R_{22}(CAE)$: $C \rightarrow E$ 违反 BCNF (C 不是超码, 虽然 AB 可以决定 C , 但 C 自身不能决定 A)。分解为:

$$R_{221}: CE \quad (\text{关键字 } C, F_{221} = \{C \rightarrow E\})$$

$$R_{222}: AC \quad (\text{关键字 } AC, F_{222} = \{AB \rightarrow C\} - \text{仅保留在 } AC \text{ 上的 } AB \rightarrow C)$$

最终分解: $\rho = \{\underline{CD}, \underline{CB}, \underline{CE}, \underline{AC}\}$, 其中下划线标注主码。

解析与知识点

本题考察运用逐步分解算法将关系模式规范化为 BCNF。BCNF 分解的通用算法: 对当前关系模式 R_i 及其上投影的函数依赖集, 检查是否存在非平凡函数依赖 $X \rightarrow Y$ 违反 BCNF (即 X 不是超码)。

若存在, 将 R_i 分解为 $R_{i1} = XY$ 和 $R_{i2} = R_i - (Y - X)$, 即 $X \cup (R_i - Y)$ 。

此分解保证无损连接性 (因为 X 是 $R_{i1} \cap R_{i2}$ 且 $X \rightarrow (R_{i1} - R_{i2})$ 成立)。具体到本题: 第一轮, 选择 $C \rightarrow D$ (C 不是超码) 分解 R 为 CD 和 $CBAE$ 。

$CBAE$ 上的函数依赖为 F 在 $\{A, B, C, E\}$ 上的投影: 由 $AB \rightarrow C$ 保留原依赖; 由 $C \rightarrow D$ 和 $D \rightarrow B$ 传递得 $C \rightarrow B$; 由 $C \rightarrow D$ 和 $D \rightarrow E$ 传递得 $C \rightarrow E$ 。

第二轮, 在 $CBAE$ 中 $C \rightarrow B$ 违反 BCNF (因为 $C_{CBAE}^+ = \{C, B, E\}$, 不是超码), 分解为 CB 和 CAE 。第三轮, 在 CAE 中 $C \rightarrow E$ 违反 BCNF, 分解为 CE 和 AC 。最终得 4 个关系模式均满足 BCNF。

整个分解过程中每次都满足无损连接性条件, 因此最终分解也不丢失信息。需要注意两个关键点: (1) BCNF 分解不一定保持函数依赖——本题最终确实丢失了原有的 $AB \rightarrow C$ 跨关系成立的完整约束 (AC 关系中虽然保留了 AB 的信息, 但 AB 跨关系还原需要连接操作); (2) 分解的选择次序可能影响最终结果的模式数

量和质量，不同选择策略可能产生不同的 BCNF 规范覆盖。知识点：第六章——BCNF 分解算法与无损连接性保证

2. 设要建立一个有多个子公司的总公司数据库。子公司 (C) 有许多职员 (E)，但一个职员仅属于某一个子公司；每个职员可参加多项工程 (J) 或负责管理，每项工程可有多名职员参加，但只有一个负责管理者；有若干供应商 (S) 同时为多个不同工程供应各种机械产品 (P)；一种产品又可由其它若干种产品组装而成，或用来组成其它多种产品。(20 分)

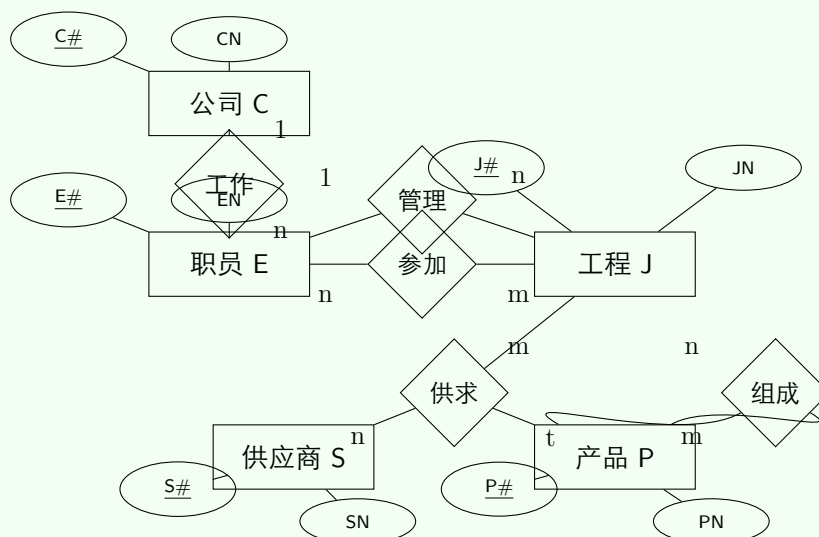
(1) 给出各实体的 2~3 个属性，设计该 DB 的基本 E-R 图。

参考答案

各实体的属性：

- 公司 C：公司编号 (C#)、公司名称 (CN)
- 职员 E：职工编号 (E#)、职工姓名 (EN)
- 工程 J：工程编号 (J#)、工程名称 (JN)
- 供应商 S：供应商编号 (S#)、供应商名称 (SN)
- 产品 P：产品编号 (P#)、产品名称 (PN)

E-R 图如下 (TikZ 绘制)：



图谱说明：

- 公司-职员通过”工作”联系，为 1:N（一个公司有多名职员，一个职员只属一个公司）。
- 职员-工程通过”参加”联系，为 N:M（职员参加多项工程，工程有多名职员

参与)。

- 职员-工程通过”管理”联系,为 1:N (每个工程只有一名负责管理者,一个职员可管理多项工程)。
- 供应商-工程-产品通过”供求”三元联系,为 N:M:T (供应商向工程供应产品)。
- 产品-产品通过”组成”自联系,为 M:N (产品可由多种其他产品组成,也可参与组成其他多种产品)。

解析与知识点

本题考察根据复杂的业务描述设计完整的 E-R 模型,是数据库概念设计的核心技能。正确绘制的关键在于依次从需求描述中提取出实体、属性和联系。

实体识别方面,从描述中可清晰提取五个实体:公司(C)、职员(E)、工程(J)、供应商(S)和产品(P),每个实体至少包含编号和名称两个基本属性。

联系识别是本题的难点:第一个联系”工作”——”子公司有许多职员,一个职员仅属于某一个子公司”,这是公司到职员的 1:N 二元联系。

第二个联系”参加”——”每个职员可参加多项工程,每项工程可有多个职员参加”,这是职员到工程的 M:N 二元联系。第三个联系”管理”——需要特别注意:”每项工程只有一个负责管理者,一个职员可负责管理多项工程”,但管理者和参加者是不同类型的职员角色,在 E-R 图中实际应描述为职员与工程的另一个独立 1:N 联系(一个工程对应一个管理者,一个职员可管理多个工程)。

第四个联系”供求”——”供应商同时为多个不同工程供应各种机械产品”,这是供应商-工程-产品的三元 N:M:T 联系。第五个联系”组成”——”一种产品可由其它若干种产品组装而成”,这是产品自身的递归 M:N 自联系。

常见设计错误包括:(1)将管理联系误认为 M:N (未注意到”只有一个负责管理者”的约束);(2)遗漏了产品的自联系;(3)错误地将三元联系分解为两个二元联系(会丢失语义信息)。

E-R 模型是数据库逻辑设计和物理设计的蓝图,其质量直接决定后续所有阶段的成败。知识点:第七章——E-R 模型设计:实体、属性与联系的识别与绘制

(2) 将上题的 E-R 图转为尽可能少的等价的关系模式,指出各模式的主码。

参考答案

关系模式集合(8 个关系,下划线表示主码):

1. 公司: C(C#, CN) ——主码为公司编号。
2. 职员: E(E#, EN, C#) ——主码为职工编号, C# 为外码参照 C。

3. 工程: $J(\underline{J\#}, JN, M_E\#)$ ——主码为工程编号, $M_E\#$ 为外码参照 E (表示负责管理者)。
4. 供应商: $S(\underline{S\#}, SN)$ ——主码为供应商编号。
5. 产品: $P(\underline{P\#}, PN)$ ——主码为产品编号。
6. 参加: $E_J(\underline{E\#}, \underline{J\#})$ ——主码为 (职工编号, 工程编号)。
7. 供求: $S_J_P(\underline{S\#}, \underline{J\#}, \underline{P\#})$ ——主码为 (供应商编号, 工程编号, 产品编号)。
8. 组成: $PP(\underline{P\#}, \underline{Sub_P\#})$ ——主码为 (产品编号, 子产品编号)。

转换说明:

- 1:N 的”工作”联系: 将 1 端 (公司 C) 的主码 $C\#$ 纳入 N 端 (职员 E) 的表作为外码。
- 1:N 的”管理”联系: 将 1 端 (职员 E, 作为管理者) 的 $E\#$ 纳入 N 端 (工程 J) 的表, 命名为 $M_E\#$ 。
- N:M 的”参加”联系: 转换为独立的关系 E_J , 属性为两个实体的主码。
- 三元 N:M:T 的”供求”联系: 转换为独立关系 S_J_P , 属性为三个实体的主码。
- M:N 的”组成”自联系: 转换为独立关系 PP , 两个属性均为 $P\#$ (需重命名以区分, 如 $P\#$ 和 $Sub_P\#$)。

解析与知识点

本题考察 E-R 图向关系模式的系统转换规则, 是数据库逻辑设计阶段的核心任务。转换需要遵循以下基本规则: (1) 实体转换——每个实体类型转换为一个关系模式, 实体的属性即为关系的属性, 实体的标识符 (码) 即为关系模式的主码。

本题 5 个实体产生 5 个关系。

(2) 1:N 二元联系的转换——可以与 N 端实体合并: 将 1 端实体的主码纳入 N 端实体所对应的关系中作为外码。本题”工作”联系 (公司 1:N 职员) 通过将 $C\#$ 加入 E 表实现; ”管理”联系 (职员 1:N 工程) 通过将 $E\#$ (管理者编号) 加入 J 表实现。

合并方式可以最大限度地减少关系数量, 这是”尽可能少的等价关系模式”的关键。

(3) M:N 二元联系的转换——必须转换为独立的关系模式, 其属性包括参与实体的主码, 而联系本身的主码由这些主码联合构成。本题”参加” (M:N) 产生独立表 E_J 。(4) 三元及以上联系的转换——同样必须单独建表, 主码为参与实体的全部主码的联合。

本题”供求”三元联系产生 S_J_P 表。(5) 自联系 (递归联系) 的转换——根据联系类型按上述规则处理, 但同表属性需要重命名以区分角色, 本题”组成”自联

系 (M:N) 产生 PP 表, 两个属性分别为 P# 和 Sub_P#。最终得到 8 个关系模式 (5 实体表 + 3 联系表), 这是合并了两个 1:N 联系 (工作、管理) 后最精简的方案, 如果所有联系都独立建表则会有 $5+5=10$ 个关系。知识点: 第七章——E-R 图向关系模式的转换规则与主码确定

(3) 为公司的工程数据表, 用 SQL 语句定义一个允许全体用户查询、插入的安全性控制功能。

参考答案

参考 SQL:

```
1 GRANT SELECT, INSERT
2 ON TABLE J
3 TO PUBLIC;
```

该语句授予所有数据库用户 (PUBLIC) 对工程表 J 的查询 (SELECT) 权限和插入 (INSERT) 权限。

解析与知识点

本题考察 SQL 的自主存取控制 (DAC) 机制——GRANT 语句的基本用法。GRANT 是 SQL 数据控制语言 (DCL) 的核心语句, 用于向用户或角色授予数据库对象的操作权限。

本题中 SELECT 权限允许用户执行查询操作读取 J 表中的数据, INSERT 权限允许用户向 J 表中添加新记录。

TO PUBLIC 是特殊关键词, 表示将权限授予当前数据库系统中的所有用户 (包括已存在的和将来新创建的用户), 是最广范围的权限授予。

自主存取控制的核心思想是“资源所有者自主决定授权”, 通常由数据库管理员 (DBA) 或表的创建者执行 GRANT 语句。与 GRANT 配套的还有 REVOKE 语句用于收回权限, 以及 WITH GRANT OPTION 子句允许被授权者将权限再授予他人 (权限的传播)。

需要注意的是, 本题仅授予了 SELECT 和 INSERT, 未授予 UPDATE 和 DELETE 权限, 这意味着普通用户无法修改或删除 J 表中的现有数据——这是一种最小权限原则的体现。

在工程实践中, 通常不推荐直接对 PUBLIC 授予写权限 (INSERT/UPDATE/DELETE), 因为这意味着所有用户可以无差别地修改数据, 可能导致数据污染。更安全的做法是通过角色 (ROLE) 机制进行精细化的

权限管理。知识点：第四章——SQL 自主存取控制：GRANT/REVOKE 语句与 PUBLIC 授权

1.5 五、简答题（共 2 小题，每小题 6 分，共 12 分）

1. 关系表有哪几种类型？说明它们之间的联系与区别。

参考答案

关系表的类型：基本表、查询表、视图表。

联系与区别：

- 基本表：实际存储数据的表，是数据库中数据物理存在的唯一形式，描述了全局数据的逻辑结构。所有数据操作的基础都源于基本表。
- 视图表：从基本表或其他视图导出的虚表，数据库中只存储其定义而不存储对应的数据。视图描述了局部数据的逻辑结构，提供了数据的逻辑独立性，可作为安全机制限制用户的数据访问范围。
- 查询表：通过查询操作生成的临时结果表，是 SQL 查询语句的直接输出，存在于查询执行期间，不持久化存储。

三者的联系：视图表和查询表都从基本表衍生而来；视图可看作被命名并持久化定义的查询；查询表在执行时动态生成，是视图定义的执行结果实例。

（注：此题为开放性题，参考答案明确说明“答案多样化”。另一种常见分类是按用途划分：基本表、辅助表、临时表等，如实陈述并合理辨析即可得分。）

解析与知识点

本题考察对关系数据库中“表”概念的全面理解，要求区分基本表、查询表和视图表三类不同的表结构。基本表（Base Table）是数据在数据库中的物理存在形式，它是通过 CREATE TABLE 语句定义的、实际占用存储空间的数据集合。

基本表是关系模型中数据存储的最小单位，也是所有数据查询和更新的直接或间接来源。

SQL 中的 INSERT、UPDATE、DELETE 等数据操纵语句都直接作用于基本表。视图表（View）是从一个或多个基本表（或其他视图）导出的虚表——数据库中仅存储视图的定义（即对应的 SELECT 查询语句），而不实际存储视图的数据。

这一设计的核心价值在于：(1) 简化用户操作，将复杂的多表连接查询封装为简单的单表查询；(2) 保护数据安全，通过视图向不同用户展示数据的裁剪子集；(3) 提供逻辑数据独立性，当基本表的结构改变时，只需调整视图定义以保持用户接口不变。

视图的更新受到严格限制，不是所有视图都支持更新操作。查询表（Query Table）是 SQL 查询语句执行过程中产生的临时中间结果，其生命周期仅限于查询执行期间，查询结束后即被释放。它在 SQL 查询处理中扮演中间数据载体的角色，在连接、分组、排序等操作之间传递数据。

三者之间的本质关系可以概括为：基本表是数据的源头（物理存在），视图是数据的窗口（逻辑定义），查询表是数据的管道（过程产物）。深入理解这三类表有助于从数据存储、数据访问和数据流动三个维度全面把握关系数据库的工作机制。知识点：第一章——基本表、查询表与视图的概念区分；第三章——视图的定义机制与作用

2. 简述关系数据库提高查询效率的主要方法；并说明利用并发控制技术能否提高某 SQL 查询效率？

参考答案

（一）提高查询效率的主要方法：

1. 建立索引：包括逻辑索引（如 B+ 树索引、哈希索引）和聚簇索引（将数据按索引键值物理排序），可大幅减少 I/O 开销。
2. 查询代数优化：选择运算尽可能先做（ σ 下推），在执行连接前对关系做适当预处理（排序或建立哈希表），避免不必要的笛卡儿积。
3. 关系分割：包括水平分割（按元组划分）和垂直分割（按属性划分），减少每次查询需要扫描的数据量。
4. 查询物理优化：基于代价估算选择最优的连接算法（如嵌套循环、排序合并、哈希连接）和访问路径。
5. 查询重写：将用户查询等价转换为执行效率更高的形式（如视图展开、子查询去相关等）。
6. 缓存与物化：物化视图将频繁使用的查询结果预计算并存储，避免重复计算。

（二）并发控制技术能否提高某 SQL 查询效率？

不能直接提高。并发控制技术的主要目的是解决多事务并发执行时可能出现的数据不一致性问题（丢失修改、不可重复读、读脏数据），确保事务的隔离性和调度的可串行化，而非提升单个 SQL 查询的执行速度。

两种视角的分析：

- 不支持提高的观点：并发控制的基本单位是事务，通常包含一个或多个 SQL 操作。封锁、时间戳等并发控制机制本身会引入额外的开销（如加锁/解锁、等待、死锁检测等），反而可能降低系统吞吐量。这些机制的价值在于维护数据一致性，与单条 SQL 的查询效率无直接关系。

- 可间接提高整体系统效率的观点（开放性讨论）：良好的并发控制策略可使多个查询任务并行执行而互不干扰，提高了系统整体吞吐量和资源利用率。从这个角度看，并发控制间接提升了系统级别的整体 SQL 执行效率。但此“效率”指的是系统吞吐量而非单条 SQL 的响应时间，需注意概念区分。

（注：参考答案明确指出“利用并发控制技术不能提高某 SQL 查询效率。因为它主要是解决并发操作带来的数据不一致性问题”。同时也认可“利用并发控制技术能间接提高整体 SQL 查询程序效率”的开放讨论视角，两方分析均给出。）

解析与知识点

本题是一道对比分析型简答题，考察对数据库两大优化领域——查询优化和并发控制的边界认知。查询效率优化是使单个查询（或事务）执行得最快的技术体系，分为代数优化和物理优化两个层次。

代数优化（启发式优化）基于关系代数的等价变换规则，遵循“选择先做、投影下推、避免笛卡儿积”等启发式策略，对查询语法树进行等价变换以缩小中间结果规模。

物理优化基于代价估算（Cost-based Optimization），利用统计信息估算每种候选执行计划的 I/O 代价和 CPU 代价，选择总代价最小的执行计划，涉及连接算法的选择（嵌套循环、排序合并、哈希连接）、索引的利用和访问路径的确定。此外，索引是连接物理设计与查询执行的桥梁，B+ 树索引适合范围查询，Hash 索引适合等值查询，聚簇索引将相关数据物理相邻以减少磁盘随机 I/O。

水平分片和垂直分片通过数据分布策略减少每次查询的 I/O 数据量。

并发控制则是解决多事务并发执行时的一致性问题的，其核心是保证调度的可串行化和事务的隔离性。封锁机制（包括共享锁、排他锁和意向锁）虽然保证了正确性，但引入了加锁/解锁的开销、锁等待和可能的死锁。MVCC（多版本并发控制）通过维护数据的多个版本来避免读写冲突，在性能和一致性之间取得了较好的平衡。

两部分的结论可以总结为：查询优化让“跑得快”，并发控制让“一起跑不出错”——前者关注速度，后者关注正确性。两者协同工作：优化的查询持有锁的时间更短，减少了并发冲突的可能性；良好的并发调度虽然不直接加速单个查询，但能使系统在单位时间内处理更多查询（提高吞吐量）。知识点：第九章——查询优化方法（代数优化与物理优化）；第十章——并发控制的范畴与技术目标

2 2008–2009 学年《数据库原理》期末试卷 (A)

2.1 一、单项选择题 (每小题 1 分, 共 15 分)

1. 关系模式的设计任务是在 ____ 阶段进行的。

参考答案

A. 逻辑设计

解析与知识点

本题考察数据库设计的阶段划分及各阶段核心任务的区别。

数据库的生命周期包含需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施和运行维护六个阶段。

概念设计阶段的任务是将需求分析得到的用户需求抽象为独立于任何具体 DBMS 的概念模型, 通常使用 E-R 图作为建模工具, 这一阶段关注的是“现实世界的数据含义”而非“如何在计算机中存储”。

逻辑设计阶段是本题的正确答案——该阶段将概念模型 (E-R 图) 转换为特定 DBMS 支持的数据模型, 在关系数据库语境下就是将 E-R 图转换为关系模式, 包括确定每个关系模式的属性、主码、外码以及各个关系之间的联系。物理设计阶段为逻辑数据模型选择最合适的物理存储结构和存取方法 (如索引策略、聚簇方案、文件组织方式等), 关注存储效率和存取速度。

实施阶段则是在实际 DBMS 上建库建表、加载数据、编写应用程序。

常见误区是将概念设计阶段 (绘制 E-R 图) 错误地理解为关系模式的设计任务, 实际上概念设计产出的是与具体数据模型无关的概念模型, 还不是关系模式——只有到了逻辑设计阶段, 概念模型才被具体化为关系 (表) 的结构定义。知识点: 第七章——数据库设计各阶段的任务划分: 逻辑设计阶段的关系模式设计

2. E-R 图是数据库设计的工具之一, 它一般适用于建立数据库的 ____。

参考答案

A. 概念模型

解析与知识点

本题考察 E-R 图在数据库设计方法论中的正确定位。E-R 模型 (Entity-Relationship Model) 由 P.

P. Chen 于 1976 年提出, 是描述现实世界信息结构的概念建模工具。

它使用三个基本抽象元素——实体（Entity，客观存在并可相互区分的事物）、属性（Attribute，实体的特征描述）和联系（Relationship，实体之间的关联）——来对现实世界的数据进行形式化表达。

E-R 图属于概念模型层面的建模工具，因为它完全独立于任何具体的 DBMS 软件和硬件环境，只关心“现实世界的数据结构是什么”，而不涉及“如何在计算机中组织存储”。这与逻辑模型（如关系模型中的二维表、层次模型中的树形结构）、物理模型（如文件组织、磁盘布局）形成层次分明的抽象关系。E-R 图的绘制通常发生在数据库设计的第二阶段——概念结构设计，其后的逻辑设计阶段将 E-R 图转换为关系模式。

常见错误是将 E-R 图归类为“逻辑模型”——虽然 E-R 图最终会被转换为逻辑模型，但其本身的性质和设计初衷是充当现实世界与机器世界之间的概念桥梁。掌握 E-R 图的概念层次定位，是理解数据库三层抽象（概念层-逻辑层-物理层）的关键。知识点：第七章——E-R 模型的概念层次定位：概念模型的建模工具

3. 当局部 E-R 图合并成全局 E-R 图时，可能出现冲突，下列不属于这种冲突的是。

参考答案

B. 语法冲突

解析与知识点

本题考察在数据库概念设计中，将多个局部 E-R 图集成为全局 E-R 图时可能出现的三类冲突。

属性冲突是最常见的一类，包括：(1) 属性域冲突——同一属性在不同局部应用中使用了不同的数据类型或长度（如学号在某处用 CHAR(8)，另处用 INT）；(2) 属性取值单位冲突——同一属性的度量单位不一致（如产品的重量在某一局部应用中用“公斤”、另一应用用“克”）。

命名冲突包括：(1) 同名异义（Homonyms）——不同局部应用中对不同含义的对象使用了相同的名称；(2) 异名同义（Synonyms）——不同局部应用中对相同含义的对象使用了不同的名称。

结构冲突包括：(1) 同一对象在不同局部应用中被视为实体或属性（如“部门”在一个应用中作为实体，在另一应用中作为职员的一个属性）；(2) 同一实体在不同局部应用中的属性组成不同；(3) 实体间的联系类型在不同局部应用中不一致（如某应用中是 1:N，另一应用中是 M:N）。选项 B“语法冲突”并不属于 E-R 合并中的这三类冲突，是一个无中生有的干扰项。

这类考题要求对三种冲突的子类型有全面、精确的记忆，通常以选择题形式出现并考察排除干扰项的能力。知识点：第七章——局部 E-R 图合并为全局 E-R 图时的三类冲突：属性冲突、命名冲突、结构冲突

4. SQL 语言提供用于实现数据存取安全性的语句是_____。

参考答案

C. GRANT、REVOKE

解析与知识点

本题考察 SQL 语言各语句的功能归属，尤其是数据控制语言（DCL）与数据定义语言（DDL）和事务控制语句的区分。SQL 语言按功能可分为四大类：数据定义语言（DDL，含 CREATE、ALTER、DROP）、数据查询语言（DQL，主要是 SELECT）、数据操纵语言（DML，含 INSERT、UPDATE、DELETE）和数据控制语言（DCL，含 GRANT、REVOKE）。

GRANT 用于向数据库用户或角色授予特定数据库对象（表、视图、存储过程等）的操作权限（如 SELECT、INSERT、UPDATE、DELETE 等），REVOKE 则用于收回已授予的权限。

这两者构成了 SQL 自主存取控制（Discretionary Access Control, DAC）的核心机制——数据的所有者可以自主决定谁有权访问自己的数据。

选项 A（CREATE TABLE）属于 DDL，用于定义关系模式而非控制访问。选项 B（COMMIT）和选项 D（ROLLBACK）属于事务控制语句，用于管理事务的提交和回滚，确保事务的原子性和一致性，与安全性控制无关。

除了自主存取控制外，数据库安全性还包括强制存取控制（MAC）、基于角色的存取控制（RBAC）、视图机制（隐藏敏感列）、审计跟踪和数据加密等分层防护体系。在工程实践中，通常先通过角色（ROLE）定义权限模板，再将角色授予用户，实现权限的集中管理和分层分发，避免直接对个体用户逐一授权的繁琐。知识点：第四章——SQL 自主存取控制：GRANT 与 REVOKE 的权限管理机制

5. 关系规范化中所介绍的删除操作异常是指_____。

参考答案

A. 不应该删除数据被删除

解析与知识点

本题考察关系规范化理论中操作异常概念的精确定义。不规范的关系模式存在三类操作异常，它们是非主属性对码的不当函数依赖造成的后果。

删除异常 (Deletion Anomaly) 指在删除某些元组时，无意中连带删除了其他本应保留的有用信息。

典型例子是在一个将学生信息和选课信息混合的关系模式中，若某学生只选修了一门课程，当删除这门课的选课记录时，该学生的全部基本信息（学号、姓名、出生日期等）也随之丢失——这些学生基本信息本不应该因为选课记录被删除而消失。

插入异常 (Insertion Anomaly) 指某些数据因缺乏其他数据的支持而无法被插入——例如在混合关系模式中，一个尚未选修任何课程的新生无法被记录，因为选课记录要求有课程号，而该新生还没有选课。更新异常 (Update Anomaly) 指当修改某条数据的值（如某课程信息）时，需要同时修改表中多处出现该数据的元组，若遗漏某处则导致数据不一致。

这些操作异常产生的根本原因是关系模式中不存在不恰当的函数依赖，尤其是非主属性对候选码的部分函数依赖（导致 1NF 但仍不满足 2NF）和传递函数依赖（导致 2NF 但仍不满足 3NF）。规范化的目标就是通过逐步模式分解消除这些不当依赖，从而根除操作异常。知识点：第六章——规范化动机：删除异常、插入异常和更新异常的定义与成因

6. 若关系模式 R 中的属性全部是主属性，则 R 的最高范式必定是_____。

参考答案

C. 3NF

解析与知识点

本题考察对全主属性关系范式上限的准确判断，是区分 3NF 与 BCNF 的关键考点。当关系模式 R 中的所有属性都是主属性（即每个属性都至少属于某个候选码）时，不存在任何非主属性。

在此前提下，2NF 的条件（非主属性不能部分依赖候选码）自然成立，因为没有非主属性需要检查。

3NF 的条件（非主属性不能传递依赖候选码）同样自然成立。因此全主属性关系至少达到 3NF。

是否一定达到 BCNF 呢？

BCNF 要求每个非平凡函数依赖的决定因素都是超码。在全主属性关系中，虽然

不存在非主属性，但主属性之间仍可能存在函数依赖 $X \rightarrow Y$ 其中 X 不是超码（如 $A \rightarrow B$, B 是某个候选码的一部分但不是全部, A 也不是超码），这违反 BCNF 但不违反 3NF（因为 3NF 允许“决定因素非超码但依赖项是主属性”的情况）。因此全主属性关系的最高范式只保证是 3NF，不一定是 BCNF。

典型反例： $R(A, B, C)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$ ，候选码为 A 、 B 、 C ，全属性都是主属性，但 $A \rightarrow B$ 中 A 已是候选码，满足 BCNF。

更精确的反例需要构造 A 和 B 均为非超码主属性且存在 $A \rightarrow B$ 的情形，如 $R(A, B, C, D)$, $F = \{AB \rightarrow CD, C \rightarrow D\}$ 。常见混淆点是将“全主属性”与“全码”混为一谈——全码关系必定同时满足 3NF 和 BCNF，但全主属性关系只保底 3NF。

这个选择题集中体现了 3NF 与 BCNF 之间最本质的差异。知识点：第六章——全主属性关系的范式上限：3NF 与 BCNF 的判定差异

7. 当 B 属性函数依赖于 A 属性，则属性 A 与 B 的联系为_____。

参考答案

B. 多对 1

解析与知识点

本题考察函数依赖与实体间联系类型的对应关系，要求从数学定义的角度理解函数依赖的语义。函数依赖 $A \rightarrow B$ 的数学含义为：对于关系中的任意两个元组 t_1 和 t_2 ，若 $t_1[A] = t_2[A]$ ，则 $t_1[B] = t_2[B]$ 。

这意味着一个 A 值唯一确定一个 B 值，但反过来一个 B 值可能对应多个 A 值——即多个不同的 A 值映射到同一个 B 值。

从实体间联系的角度看，这是典型的“多对一”联系：多个 A 实体对应一个 B 实体。

若类比到现实场景：学号 $\rightarrow$ 系别——一个学号唯一确定一个系别（单值映射），但一个系别可以有多个学生（多个学号值对应同一个系别值），因此学号与系别之间就是多对一的关系。常见混淆方向是误答为“一对多”——若写成 $B \leftarrow A$ 的形式，从 A 的视角看向 B ，确实是 A 中多个值对应 B 中一个值（多对一），而不是一个 A 值对应多个 B 值（一对多）。

函数依赖的方向性理解至关重要： $A \rightarrow B$ 不是对称关系， A 决定 B 意味着对于给定的 A ， B 被唯一确定，这是多对一而非一对多的映射。在 E-R 图中，这种关系体现为实体集 A 到实体集 B 的 N:1 联系。知识点：第六章——函数依赖与实体间联系的对应关系： $A \rightarrow B$ 表示多对一

8. 建立索引的目的是_____。

参考答案

D. 提高存取速度

解析与知识点

本题考察索引机制在数据库中的核心定位——加速数据检索。

索引类似于书籍的目录，是建立在表上的一种辅助数据结构，其本质是通过预制的有序数据结构（如 B+ 树、哈希表）来避免全表扫描，使 DBMS 能够以 $O(\log n)$ 或 $O(1)$ 的时间复杂度定位目标元组，而不是逐行检查 $O(n)$ 。

建立索引虽然显著提升了查询速度，但也存在不容忽视的副作用：第一，增加存储空间开销，索引本身需要占用额外的磁盘空间；第二，降低写入性能，每次对表进行 INSERT、DELETE 或 UPDATE 操作时，DBMS 需要同时维护表上所有相关索引的数据结构更新，索引越多写入越慢。

因此是否建索引、在哪列上建索引、建哪种类型索引，都需要基于实际工作负载（读多还是写多）进行权衡。在管理类考试中这是一个基础常识点，每个选项虽然都是数据库需要实现的目标，但应清楚区分不同机制的主攻方向：视图用于安全性和逻辑独立性，外码约束用于参照完整性，CHECK 约束用于用户定义完整性，索引的核心功能就是加速查询。

另外，索引也是物理设计阶段的重要决策内容，与查询优化中的访问路径选择直接相关——DBMS 的查询优化器会基于代价估算决定是否使用索引以及使用哪个索引来执行查询。知识点：第三章——索引的定义：以空间换时间的存取加速数据结构

9. 数据模型的三要素是_____。

参考答案

D. 数据结构、数据操作和完整性约束

解析与知识点

本题考察数据库领域最基础的概念之一——数据模型的三大构成要素。数据模型是对现实世界数据的抽象表示，其三个要素分别刻画了数据的不同维度。

数据结构（Data Structure）定义数据库的静态特性，描述了数据对象的类型、内容、属性以及对象之间的联系方式。

在关系模型中，数据结构就是二维表（关系），一行代表一个元组，一列代表一种属性。

数据操作 (Data Operation) 定义数据库的动态特性, 指对数据库中的各种对象允许执行的操作及其有关规则, 关系模型中的操作包括查询 (Query) 和更新 (Insert、Delete、Update) 两大类。完整性约束 (Integrity Constraints) 定义数据的语义特性, 是保证数据正确性、有效性和相容性的规则集合, 关系模型提供实体完整性、参照完整性和用户定义完整性三级约束体系。

选项 A” 外模式、模式和内模式” 是数据库系统的三级模式体系结构, 属于系统架构层次而非数据模型层次。选项 B” 层次模型、网状模型、关系模型” 是对数据模型按逻辑组织方式进行的分类, 不是模型的构成要素。

选项 C” 实体、属性和联系” 是 E-R 模型的三要素而非数据模型的三要素。

这道题不仅考察记忆, 更在于理解: 任何一个数据模型都可以从这三要素的角度来分析和定义, 无论是经典的关系模型、面向对象模型还是现代的文档模型 (如 MongoDB)。知识点: 第一章——数据模型的定义与三要素: 数据结构、数据操作和完整性约束

10. 在关系 $R(R\#, RN, S\#)$ 和 $S(S\#, SN, SD)$ 中, R 的主码是 $R\#$, S 的主码是 $S\#$, 则 $S\#$ 在 R 中称为_____。

参考答案

A. 外码

解析与知识点

本题考察外码 (Foreign Key) 定义的精确认知。外码是关系数据库中实现表与表之间关联的核心机制。

外码的定义有三个必要元素: (1) 某个关系 R 中的一个属性 (或属性组); (2) 该属性不是 R 的主码; (3) 该属性是另一个关系 S 的主码。

本题中 $S\#$ 在表 S 中是主码, 在表 R 中不是主码 (R 的主码是 $R\#$), 且 $S\#$ 引用了 S 表的主码值, 完全满足外码定义。外码实现了参照完整性 (Referential Integrity) ——外码的值要么取空值 (NULL), 要么必须等于被参照表中某个元组的主码值, 否则 DBMS 拒绝该操作。

这种约束保证了跨表数据的一致性: R 表中每条记录的 $S\#$ 值必须对应 S 表中一个实际存在的实体。

外码除了对应主码外还可以对应候选码 (UNIQUE 约束定义的属性), 但最典型的情形仍是引用主码。在 SQL 中, 外码通过 FOREIGN KEY . .

. REFERENCES 子句定义, 并可指定 ON DELETE 和 ON UPDATE 的级联策略 (CASCADE、SET NULL、NO ACTION)。外码机制与规范化理论中的模式

分解密切相关：将一个存在冗余的大关系模式分解为多个小关系模式后，需通过外码来维护原关系模式中各属性之间的关联信息。知识点：第二章——参照完整性与外码的定义：非本关系主码但引用另一关系主码的属性

11. 数据独立性是指_____。

参考答案

C. DB 的数据结构改变时，不影响应用程序

解析与知识点

本题考察数据库系统最核心的设计目标之一——数据独立性的精确含义。

数据独立性 (Data Independence) 是指应用程序与数据库中存储的数据之间相互独立，当数据库一方的结构发生变化时，另一方无需修改。

数据独立性分为两个层次：(1) 物理数据独立性——当数据库的物理存储结构（内模式）发生变化时（如增加索引、更换存储设备、改变数据文件组织方式），由于模式/内模式映像的隔离作用，逻辑模式（模式）和应用程序保持不变；(2) 逻辑数据独立性——当数据库的全局逻辑结构（模式）变化时（如添加新的数据项、扩展表的定义、增加新的关系），通过外模式/模式映像的调整，使得外模式（用户视图）暂时保持不变，从而应用程序无需立即修改。

数据库的三级模式体系结构（外模式-模式-内模式）和两级映像机制正是为了实现这种分层隔离而设计的架构基础。数据独立性是现代数据库系统与早期文件系统最重要的区别之一：在文件系统中，应用数据的存储结构变化必然导致应用程序的重写或重新编译，而数据库系统通过映像机制将这种耦合关系解耦。

选项 A 和 D 虽与数据库相关但未准确描述独立性；选项 B “DB 结构修改不影响用户” 过于宽泛，未强调 “应用程序不受影响” 这一核心关切。知识点：第一章——数据独立性：物理独立性与逻辑独立性的定义与实现机制

12. 在第一个事务以 S 封锁方式读数据 A 时，第二个事务对数据 A 的读方式会遭到失败的是_____。

参考答案

A. 实现 X 封锁的读

解析与知识点

本题考察封锁机制中最基础的相容性规则——共享锁(S锁)与排他锁(X锁)的相容判断。

封锁机制使用两种基本锁类型: S锁(Shared Lock, 共享锁/读锁)用于读取操作, 允许多个事务同时持有同一数据对象的S锁——S锁与S锁是相容的, 这保证了读操作的并发性。

X锁(Exclusive Lock, 排他锁/写锁)用于写操作, 与任何其他锁(包括S锁和X锁)都不相容——一旦某事务对数据A加了X锁, 任何其他事务都不能再对A加任何类型的锁。

本题场景分析: 第一个事务已对A加了S锁, 此时第二个事务若试图对A加X锁进行读(选项A), 由于S锁与X锁不相容, 该加锁请求将被阻塞或拒绝, 这就是“遭到失败”的情形。选项B(S封锁读)因为S锁与S锁相容而不会被拒绝; 选项C(不加封锁读)不涉及锁请求自然不冲突(但这种脏读违反隔离性); 选项D(共享封锁读)与S锁相容也不会失败。

深入理解锁相容矩阵对分析并发事务的等待关系和死锁场景至关重要。

此外, 当S锁与X锁冲突时, 第二个事务进入等待队列, 若等待超时或形成循环等待(死锁), DBMS将采取相应的解除策略(回滚代价最小的事务)。知识点: 第十章——S锁与X锁的相容矩阵: S与S相容, S与X不相容, X与一切不相容

13. 已知 $A \rightarrow C$, $B \rightarrow D$, 那么下列函数依赖不成立的是_____。

参考答案

C. $A \rightarrow CD$

解析与知识点

本题考察利用 Armstrong 公理系统进行函数依赖的逻辑推导。已知前提仅有 $A \rightarrow C$ 和 $B \rightarrow D$ 两个依赖。

逐项推理: $A \rightarrow AC$ (由 $A \rightarrow C$ 通过增广律: 两边同时加 A, 得 $AA \rightarrow AC$ 即 $A \rightarrow AC$, 选项 D 成立)。

$AB \rightarrow AD$ (由 $B \rightarrow D$ 通过增广律加 A, 选项 A 成立)。

有了 $AB \rightarrow AD$, 且由 $A \rightarrow C$ 增广得 $AB \rightarrow CB = BC$, 联合 $AB \rightarrow AD$ 使用合并规则得 $AB \rightarrow ACD$, 投影去掉 A 得 $AB \rightarrow CD$ (选项 B 成立)。对于选项 C $A \rightarrow CD$: 已知只有 $A \rightarrow C$, 要得到 $A \rightarrow CD$ 还需要 $A \rightarrow D$ 。

但从给定的函数依赖集 $\{A \rightarrow C, B \rightarrow D\}$ 中, 没有途径能从 A 推导出 D——A 和 D 之间也没有通过中间属性 B 的传递路径(因为 A 不能决定 B)。因此 $A \rightarrow CD$

不能由已知依赖逻辑蕴含，选项 C 不成立。

本题的关键在于：(1) 不凭空假设属性间的依赖关系；(2) 每一步推导都必须严格依据 Armstrong 公理三条推理规则（自反律、增广律、传递律）或三条推论规则（合并规则、伪传递规则、分解规则）；(3) 合并规则的正确使用需要在同左部下合并右部，不能跨左部合并。

这类逻辑推理题反映了关系规范化理论中函数依赖集的逻辑蕴含计算，是候选码求解、最小覆盖和范式判定的基础技能。知识点：第六章——Armstrong 公理系统：增广律、传递律与合并规则的逻辑推导

14. 数据库中只存放视图的_____。

参考答案

A. 结构定义

解析与知识点

本题考察视图的物理存储机制——这是区分视图与基本表最关键的技术特征。

视图是从一个或多个基本表（或其他视图）导出的虚拟表，数据库中只存储视图的定义（即对应的 SELECT 查询语句和列结构描述），而不实际存储视图对应的数据行。

每次用户查询视图时，DBMS 先取出视图定义中的 SELECT 语句，将其与用户的查询请求合并改写为一个对基本表的复合查询，然后执行该复合查询并动态返回结果。

这种“定义存储、数据动态生成”的机制带来了若干重要特性：(1) 视图占用极少的磁盘空间（仅存储定义文本）；(2) 视图始终反映基本表的最新数据（因为数据是从基本表实时计算出来的）；(3) 视图查询比直接查询基本表多了一层查询重写的额外开销；(4) 视图的更新受到严格限制，不是所有视图都可以进行 INSERT/UPDATE/DELETE。与视图形成对比的是物化视图（Materialized View），后者将查询结果预先计算并物理存储，定期刷新，以存储空间换查询速度。

但标准 SQL 视图是纯逻辑定义，物理数据始终只存在于基本表中。

这一设计是数据库三层模式体系中外模式的实现——每个用户可以拥有自己的一组视图（局部外模式），通过视图看到数据的不同裁剪和汇总视角，而底层的数据主体存储在共享的模式层基本表中不变。知识点：第三章——视图的存储机制：数据库仅存放视图定义，不存储视图数据

15. 事务的隔离性是由 DBMS 的_____实现的。

参考答案

C. 并发控制子系统

解析与知识点

本题考察事务 ACID 特性在 DBMS 内部由哪些子系统负责实现，要求对数据库系统的功能模块划分有清晰认知。

事务具有四个基本特性 (ACID): (1) 原子性 (Atomicity) ——事务中的所有操作要么全部执行成功, 要么全部撤销回滚, 如同一个不可分割的操作单元, 由事务管理子系统实现; (2) 一致性 (Consistency) ——事务的执行必须使数据库从一个一致性状态转变为另一个一致性状态, 需要应用程序遵守完整性约束, DBMS 的完整性检查机制也参与保障; (3) 隔离性 (Isolation) ——多个并发事务执行时相互隔离, 一个事务的执行不应感知到其他并发事务的存在, 这由并发控制子系统 (Concurrency Control Subsystem) 实现, 核心技术包括封锁 (Locking)、时间戳排序 (Timestamp Ordering) 和多版本并发控制 (MVCC); (4) 持久性 (Durability) ——事务一旦提交, 其对数据库的修改就永久保存在磁盘上, 即使随后发生系统崩溃也不丢失, 由恢复管理子系统 (Recovery Management Subsystem) 通过日志机制保证。

并发控制子系统的核心目标就是通过可串行化调度来确保事务的隔离性: 任何并发调度的执行结果必须等价于这些事务按某种顺序串行执行的结果。

SQL 标准定义了四个隔离级别 (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE), 提供不同强度的隔离保证, 隔离级别越高并发性能越低, 形成了一致性与性能的经典交换。知识点: 第十章——事务 ACID 特性中各子系统的分工: 并发控制子系统负责隔离性

2.2 二、判断题 (正确的打“√”, 错误的打“×”。每小题 1 分, 共 10 分)

1. 若 $X \rightarrow A_i$ ($i=1,2,\dots,k$) 成立, 则 $X \rightarrow A_1 A_2 \dots A_k$ 不一定成立。 ()

参考答案

× (错误)

解析与知识点

本题考察 Armstrong 公理系统中合并规则 (Union Rule) 的正确应用。合并规则可表述为：若 $X \rightarrow Y$ 且 $X \rightarrow Z$ ，则 $X \rightarrow YZ$ 。

其证明过程：由 $X \rightarrow Y$ 通过增广律（两边同时加 X ）得 $X \rightarrow XY$ ；由 $X \rightarrow Z$ 通过增广律（两边同时加 Y ）得 $XY \rightarrow YZ$ ；由 $X \rightarrow XY$ 和 $XY \rightarrow YZ$ 通过传递律（若 $\alpha \rightarrow \beta$ 且 $\beta \rightarrow \gamma$ 则 $\alpha \rightarrow \gamma$ ）得 $X \rightarrow YZ$ ，证毕。

本题中已知 $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_k$ 均成立，反复应用合并规则：先合并 A_1 和 A_2 得 $X \rightarrow A_1A_2$ ，再合并 A_3 得 $X \rightarrow A_1A_2A_3$ ，依此类推，最终必然得到 $X \rightarrow A_1A_2 \dots A_k$ ，这是逻辑必然的结论而非“不一定成立”。

因此该判断表述错误。合并规则在日常使用中极为常见：只要同一个左部决定了多个右部属性，就可以将这些右部属性合并为一个属性组。

反之亦有分解规则 (Decomposition Rule)：若 $X \rightarrow YZ$ 则 $X \rightarrow Y$ 且 $X \rightarrow Z$ ，两者互逆。合并规则与分解规则共同构成了属性组之间的“组合/拆分”推导能力，与增广律、传递律一起支撑了函数依赖集的逻辑蕴含计算体系。知识点：第六章——Armstrong 公理推论：合并规则 (Union Rule) 的证明与应用

2. 两个关系在没有公共属性时，其自然连接操作结果为空关系。（____）

参考答案

√（正确）

解析与知识点

本题考察关系代数中自然连接语义的一个边界情形，也是教学中常见的理论分歧点。自然连接 (Natural Join) 的操作定义是：对两个关系 R 和 S 中的所有同名属性做等值连接，结果关系中同名属性只保留一列。

当 R 和 S 没有任何公共属性（即同名属性集合为空）时，存在两种不同的理论解释：(1) 按部分教材的定义，无公共属性时自然连接的结果为空关系（因为不存在可用于等值连接的属性作为匹配依据，连接条件为空导致所有元组都无法匹配）；(2) 按标准关系代数理论，无公共属性时的自然连接应退化为笛卡儿积（即没有任何连接约束条件，每条 R 元组与每条 S 元组两两配对）。本题所在课程的教学约定采用第一种解释，因此判断为正确。考生需要注意：不同教材和不同教师对此可能存在理解分歧，考试时应以本课程的教学约定为准。在 SQL 标准中，NATURAL JOIN 在没有公共列时的行为也是退化为笛卡儿积。

这一边界情形提醒我们：理论上的精确定义必须在所有边界和退化情形下都给出统一的语义约定，这是关系代数作为形式化查询语言的基本原则。在实际使用中，

建议使用显式的 JOIN . . . ON .

. . 语法明确连接条件，避免依赖自然连接在不同 DBMS 中可能存在的语义差异。

知识点：第二章——自然连接的语义边界：无公共属性时按课程约定为空关系

3. 关系的性质表明，同一个关系的任意两个元组值不能完全相同。 (____)

参考答案

√ (正确)

解析与知识点

本题考察关系数据库中最基本的数学性质——二维表中不存在重复元组。从集合论角度看，关系被定义为一组域上笛卡儿积的有限子集，关系的本质是一个元组的集合。

集合的基本性质就是元素的互异性——同一元素不能在集合中出现两次。因此同一个关系中不允许存在两个值完全相同的元组（即属性值逐列相同），这是关系模型不能触碰的基本底线。

在实际 DBMS 中，通过主码（Primary Key）的 UNIQUE 约束来物理强制执行这一性质——主码值的唯一性保证了任何两个元组不会完全相同（因为至少主码值不同）。如果表上没有定义主码，大多数 DBMS 仍然允许插入重复行（实际破坏了关系的集合语义），但好的数据库设计实践总会为每个关系定义主码，以确保关系在数学上的纯正性。这个性质看似简单，却有深远的理论影响：正因为关系是集合，其操作结果也是集合，关系代数才能拥有良好的代数封闭性和优化变换能力。

如果允许重复元组存在（如 SQL 中允许的多重集语义），则需要引入消除重复的机制（如 DISTINCT 关键字），增加查询处理复杂度。关系的这一性质是区分关系数据库与传统文件系统的重要特征之一。知识点：第二章——关系的基本性质：元组互异性（集合语义）与主码唯一性约束

4. SQL 不仅能由联机终端用户在交互环境下使用，而且也可以嵌入到主语言中使用。(____)

参考答案

√ (正确)

解析与知识点

本题考察 SQL 语言的两种使用方式——这是区分数据库系统专业人员与普通用户的关键知识点。

交互式 SQL (Interactive SQL) 允许用户在数据库客户端终端 (如 mysql 命令行、pgAdmin 查询窗口) 直接输入 SELECT、INSERT 等 SQL 语句, DBMS 立即编译执行并返回结果。

这种方式适合数据查询、快速开发和测试, 但无法实现复杂的程序逻辑 (如循环、条件分支、函数调用)。

嵌入式 SQL (Embedded SQL) 将 SQL 语句嵌入到高级程序设计语言 (如 C、C++、Java、Python、COBOL 等, 称为“主语言”) 的代码中, 通过预编译器将嵌有 SQL 的程序转换为纯主语言代码 (SQL 语句被转换为主语言的函数调用), 再经主语言编译器生成可执行程序。嵌入式 SQL 需要解决几个技术问题: (1) 主变量 (Host Variable) ——在 SQL 语句中通过冒号前缀引用主语言变量, 实现 SQL 与主语言的数据交换; (2) 游标 (Cursor) ——当 SQL 查询返回多行结果时, 主语言无法一次性接收, 需要通过游标逐行获取; (3) SQL 通信区 (SQLCA) ——存储 SQL 语句执行后的状态信息 (成功/失败/行数等); (4) 数据类型匹配——SQL 数据类型与主语言数据类型之间的映射转换。

嵌入式 SQL 极大地拓展了数据库在事务处理程序、ERP 系统、Web 后端等复杂应用场景中的使用范围, 使数据库操作成为通用编程逻辑的有机组成部分。

知识点: 第八章——SQL 的两种使用方式: 交互式 SQL 与嵌入式 SQL (含主变量、游标机制)

5. 若 $X \twoheadrightarrow Y$, 其中 $Z = U - X - Y = \emptyset$, 则称 $X \twoheadrightarrow Y$ 为非平凡的多值依赖。 ()

参考答案

× (错误)

解析与知识点

本题考察多值依赖平凡性的判定条件, 是一个容易因记忆混淆而出错的考点。多值依赖 $X \twoheadrightarrow Y$ 的平凡性标准有两个: (1) $Y \subseteq X$ ——即 Y 是 X 的子集, 此时多值依赖未提供任何有意义的约束; (2) $X \cup Y = U$ ——即 X 与 Y 的并等于全属性集 U , 等价于 $Z = U - X - Y = \emptyset$ 。

当满足上述两个条件中的任意一个时, 多值依赖是平凡的。

本题给出的条件是 $Z = \emptyset$, 这正是第二个平凡条件 ($X \cup Y = U$), 因此 $X \twoheadrightarrow Y$ 是平凡的多值依赖而非非平凡的。

题目将平凡与非平凡的判定条件说反了，因此判断为错误。区分平凡与非平凡多值依赖的多态意义在于：只有非平凡多值依赖才可能导致数据冗余和操作异常，才有必要通过分解来达到 4NF。

平凡多值依赖是自动满足的，不产生冗余。

多值依赖的直观理解：若一个关系存储了课程-教师-教材的信息，且教师的集合与教材的集合相对于课程是相互独立的（即同一门课的教师与教材存在所有可能的两两组合），则存在非平凡多值依赖（ $Cno \twoheadrightarrow Tname$ 且 $Cno \twoheadrightarrow Bname$ ），需要通过分解为两个联系表达到 4NF。在函数依赖中平凡性的判断条件仅有一个（ $Y \subseteq X$ ），而多值依赖增加了 $X \cup Y = U$ 的条件，这是二者的重要区别。知识点：第六章——多值依赖的平凡性判定： $Y \subseteq X$ 或 $X \cup Y = U$ 则为平凡

6. 建立检查点是数据库安全性控制采取的措施之一。（____）

参考答案

×（错误）

解析与知识点

本题考察数据库两大功能模块——安全性控制与恢复技术——之间的正确归属，是一个典型的“张冠李戴”型判断题。检查点（Checkpoint）是数据库恢复技术中的一种性能优化手段，而非安全性措施。

其工作机制为：DBMS 周期性地执行检查点操作——将当前内存中的所有日志缓冲记录和数据缓冲页强制刷写到磁盘，并在日志文件中写入一条检查点记录。

当系统故障后启动恢复流程时，恢复管理子系统只需从最近的检查点位置开始扫描日志文件，而不必从头到尾扫描整个日志，因为在检查点之前的已提交事务数据已经安全地持久化到磁盘上（无需重做），未提交事务在检查点的处理中也被处理。

这种机制大大缩短了故障恢复的时间，因此检查点是效率优化措施，归属于“如何让系统在故障后快速恢复”的恢复技术领域。数据库安全性控制是指防止非法用户对数据库的未授权访问和恶意破坏，其措施包括：用户标识与鉴定（用户名/密码）、自主存取控制（GRANT/REVOKE）、强制存取控制（安全标签）、基于角色的存取控制（RBAC）、视图机制（隐藏敏感数据）、审计跟踪（记录用户操作日志）和数据加密存储等。

两者虽都属于数据库系统管理范畴，但目标和手段完全不同。判断题中常见的一种命制套路就是将恢复技术的术语误置于安全性控制中，或者反过来将锁机制误置于恢复技术中。知识点：第十一章——检查点机制属于恢复技术而非安全性控

制

7. 突然停电属于事务故障，它不破坏数据库。 (____)

参考答案

× (错误)

解析与知识点

本题考察数据库故障三类分类中系统故障与事务故障的精确认知，且包含“是否破坏数据库”这一易错子判断。数据库故障严格分为三类：事务故障、系统故障和介质故障。

事务故障是指由事务内部的逻辑错误（如整数除以零、违反完整性约束、数据类型转换错误）或系统因死锁而强制回滚某事务引起的情况——事务故障只影响单个事务自身，其他事务和整个数据库不受影响，恢复时只需做事务撤销（Undo）。

系统故障（也称软故障）是指导致系统停止运转、需要重新启动的事件——包括操作系统崩溃、DBMS 异常终止、以及本题中的突然停电。

系统故障导致内存中所有数据（包括数据缓冲区和日志缓冲区）丢失，使所有正在运行的事务非正常中断。系统故障可能破坏数据库——因为故障发生时，已提交事务的更新可能尚未从内存缓冲区写入磁盘，而部分未提交事务的更新却可能已经被写入了磁盘，导致数据库处于不一致状态。

“它不破坏数据库”这一表述是题目的第二个错误点——虽然系统故障不破坏物理存储介质（磁盘完好），但它确实破坏了数据库的逻辑一致性（已提交丢失、未提交残留），需要通过日志进行 Redo 和 Undo 恢复。

介质故障（硬故障）如磁盘损坏、强磁场干扰等，直接破坏磁盘物理存储介质上的数据甚至日志文件，是最严重的一类故障。精确区分这三类故障是掌握恢复策略选择的前提。知识点：第十一章——故障分类：突然停电属系统故障（软故障），影响内存数据并可能破坏数据库逻辑一致性

8. 登记日志文件必须先写日志文件，后写数据库。 (____)

参考答案

√ (正确)

解析与知识点

本题考察数据库恢复技术的基石——先写日志原则 (Write-Ahead Logging, WAL)。WAL 的核心规定是：在数据库的任何数据修改被真正写入磁盘之前，对应的日志记录必须先被安全地写入稳定存储（外存）上的日志文件中。

这一原则被几乎所有现代 DBMS 严格遵循，它保证了“无论系统何时发生故障，日志中都不缺少任何已发生修改的记录”。

如果破坏这一原则（先写数据库，后写日志），将产生严重的灾难性后果：假设先修改了数据库中的某行数据，然后在准备写日志时系统崩溃——此时数据库已经修改（可能是不一致的状态），但日志中没有关于这次修改的任何记录。

故障恢复时，恢复子系统通过日志来判断哪些事务需要重做、哪些需要撤销——但因为日志中没有这条记录，这个修改既无法被撤销 (Undo)，也无法被正确重做 (Redo)，数据库将永远滞留在一个不可恢复的不一致状态。遵循 WAL 原则则完全不同：日志先写盘意味着即使后续数据库写盘失败，日志中已记录了“打算做什么”的完整信息，恢复时可以据此重做或撤销，将数据库还原到一致性状态。

WAL 与检查点机制配合使恢复高效进行，与数据转储备份配合使系统能够应对介质故障。这一原则体现了数据库理论在工程实现中的一个关键设计决策——在等多一道写开销（先写日志）和可恢复性之间毫不犹豫地选择后者。知识点：第十一章——先写日志原则 (WAL)：数据库修改前必须先将日志记录写入稳定存储

9. 若事务 T1 对数据对象 A 上了 IS 锁，则事务 T2 可以对数据对象 A 上 IX 锁。
()

参考答案

√ (正确)

解析与知识点

本题考察多粒度封锁协议中意向锁 (Intention Lock) 的相容性规则。多粒度封锁设计了五种锁类型：IS (意向共享锁)、IX (意向排他锁)、S (共享锁)、SIX (共享意向排他锁) 和 X (排他锁)。

其中意向锁 (IS 和 IX) 的特殊性质是：所有意向锁之间都是相容的。

IS 与 IX 相容的原因在于它们表达的只是“意图”而非实际锁定——IS 表示“我打算在子结点上加 S 锁进行读”，IX 表示“我打算在子结点上加 X 锁进行写”。

这两种意图本身在当前粒度层面不直接冲突——因为还没有对当前结点本身加实际的读锁或写锁。实际的冲突检测发生在子结点层面：当两个事务都向下到达具体需要锁定的数据项（如一条记录或一个元组）时，如果 T1 加 S 锁而 T2 也试

图加 S 锁则相容，如果 T2 试图加 X 锁则不相容（进入等待队列）。

意向锁的相容矩阵可以总结为：IS 与除 X 以外的所有锁都相容；IX 与 IS 和 IX 相容，但与 S、SIX、X 都不相容；S 只与 IS 和 S 相容；SIX 只与 IS 相容；X 与一切都不相容。

理解这个相容矩阵是分析多粒度封锁场景下并发事务等待关系的基础，也是并发控制理论部分的常考内容。意向锁的设计精髓在于：通过在高层结点上加意图锁来表示低层结点的访问意图，避免每次访问数据时都要从根到叶逐层加锁，从而在保证正确性的前提下降低了封锁粒度管理的开销。知识点：第十章——多粒度封锁：意向锁 IS/IX 的相容性与锁相容矩阵

10. 对并发操作若不加以控制，可能会带来死锁问题。（____）

参考答案

√（正确）

解析与知识点

本题考察并发控制中死锁问题的产生根源。

数据库并发操作系统若不施加有效的控制，会产生两类不同层面的问题：第一类是数据不一致问题（丢失修改、不可重复读、读脏数据），这类问题直接破坏数据库的正确性；第二类是并发控制机制本身的副作用——当使用封锁机制来防止数据不一致时，各事务在加锁/等待/释放锁的过程中可能相互等待形成循环，从而产生死锁。

典型场景：事务 T1 锁住数据 A 并请求数据 B 的锁，事务 T2 锁住数据 B 并请求数据 A 的锁——两者相互等待对方释放自己所需的资源，形成僵局，若无外力介入将永远等待下去。

解决死锁问题有三条路线：(1) 死锁预防（Deadlock Prevention）——使用一次封锁法（事务一开始就锁定所有所需数据，但严重降低并发度）或顺序封锁法（所有事务按预定顺序请求锁，避循环等待）；(2) 死锁避免（Deadlock Avoidance）——如基于时间戳的 Wait-Die 和 Wound-Wait 策略，在加锁前判断是否可能导致死锁，若可能则不分配锁；(3) 死锁检测与解除（Deadlock Detection and Resolution）——允许死锁发生，通过事务等待图（Wait-for Graph）周期性检测是否存在有向环，一旦发现死锁，选择代价最小的事务作为牺牲者回滚以打破环。

死锁问题提醒我们：并发控制不是完美无代价的，封锁机制在保障数据一致性的同时引入了新的管理问题，系统设计需要在正确性、性能和复杂性之间进行综合权衡。知识点：第十章——并发操作的死锁问题：成因、预防策略与检测恢复机制

2.3 三、查询设计题（每小题 5 分，共 25 分）

已知一个公司的职工-社团数据库有三个基础表：

- 职工：E(Eno, Ename, Age, Sex) 属性依次是：职工号，职工名，年龄，性别。
- 社团：C(Cno, Cname, Manager, Address) 属性依次是：社团号，社团名，负责人职工号，社团地址。
- 参加：EC(Eno, Cno, Date) 属性依次是：职工号，社团号，参加社团的日期。

2.4 1. 用关系代数表示下列查询

(1) 查找张力所参加社团的社团名和参加日期。

参考答案

$$\pi_{\text{Cname, Date}}(\sigma_{\text{Ename}='张力'}(E) \bowtie EC \bowtie C)$$

解析与知识点

本题考察关系代数中多表连接查询的典型模式——选择-连接-投影三步操作。第一步，通过选择运算 $\sigma_{\text{Ename}='张力'}(E)$ 从职工表 E 中过滤出名称为“张力”的元组。选择运算是按条件提取行的操作，结果是 E 表的一个水平子集（仅包含张力这一行）。

第二步，将选择结果与参加表 EC 做自然连接：自然连接基于两个关系中的所有同名属性（此处为 Eno）进行等值匹配。由于选择结果中只有张力的记录（含 Eno），与 EC 自然连接后得到张力参加各社团的全体记录（含 Eno、Cno 和 Date）。

第三步，将上一步结果与社团表 C 做自然连接：基于同名属性 Cno 匹配，将各社团的名称（Cname）关联到每条参加记录。

第四步也是最外层操作，通过投影 $\pi_{\text{Cname, Date}}$ 提取用户最终需要的两列，过滤掉不相关的中间列（如 Eno、Cno 等）。为什么投影放在最后？关系代数中投影可以随时做，但放在最后最直观——先获得所有关联数据，再去除不需要的列。

实用中有时提前做投影以缩小中间结果的宽度，这是代数优化中“投影下推”的思想体现。理解并精确表达多表查询的关系代数表达式是数据库理论的核心基本技能，也是理解 SQL 查询执行计划的基础。知识点：第二章——关系代数：选择、投影与自然连接的组合使用流程

(2) 查找参加了全部社团的职工号和职工名。

参考答案

$$\pi_{\text{Eno}, \text{Ename}} \left(E \bowtie \left(\pi_{\text{Eno}, \text{Cno}}(\text{EC}) \div \pi_{\text{Cno}}(\text{C}) \right) \right)$$

解析与知识点

本题考察关系代数中除法运算 (Division) 在“全部/所有”语义下的标准应用模式。“参加了全部社团”对应全称量词 ($\forall$) 语义：对于每个社团，该职工都有对应的参加记录。

除法运算正是为此设计的——设 $R(A, B)$ 和 $S(B)$ 是两个关系, $R \div S$ 返回那些“A 值中关联了 S 中所有 B 值”的 A 值。

具体到本题：(1) $\pi_{\text{Cno}}(\text{C})$ 获得所有社团编号的集合 (除数 S)；(2) $\pi_{\text{Eno}, \text{Cno}}(\text{EC})$ 获得职工参加社团的二元关系 (被除数 R , 属性组为职工号-社团号)；(3) 执行除法 $\pi_{\text{Eno}, \text{Cno}}(\text{EC}) \div \pi_{\text{Cno}}(\text{C})$ 得到满足条件的职工号——这些职工在 EC 表中与 C 表所有社团号都构成了参与记录。

除法运算可以等价地用基本运算表达为 $R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$, 其思路是：先找出所有可能的 (职工号, 社团号) 组合, 减去实际存在的参加记录, 剩下的 (职工号, 社团号) 就是“某职工缺少的社团”记录, 投影出这些职工号, 再从全体职工号中排除它们——最终得到的就是“不缺少任何社团”的职工号。(4) 最后与 E 表自然连接获取职工姓名 Ename, 再投影出最终需要的列。

除法运算是处理“全部/所有/均”语义的形式化方法, 在 SQL 中通常用双重 NOT EXISTS 子查询来等价实现。掌握除法有助于系统化地处理各种复杂的查询需求, 尤其是涉及全称量词的数据检索。知识点：第二章——关系代数除法运算： $\div$ 处理“全部/所有”语义的形式化表达

2.5 2. 用 SQL 语句表示下列查询

(1) 查找社团负责人的信息：社团名、负责人名字、负责人性别。

参考答案

```
1 SELECT Cname, Ename, Sex
2 FROM E, C
3 WHERE E.Eno = C.Manager;
```


解析与知识点

本题考察 SQL 中外码-主码连接查询的基本模式。社团表 C 的 Manager 字段存储的是负责人的职工号 (Eno)，它是一个外码，参照职工表 E 的主码 Eno。负责人本身的姓名 (Ename) 和性别 (Sex) 信息存储在职工表 E 中，不在 C 表中。

因此必须通过连接操作将两张表关联起来：连接条件为 $E. Eno = C. Manager$ ，即“职工表的主码等于社团表的外码”。SQL 支持两种连接写法：本题使用的逗号分隔 + WHERE 条件的隐式连接（也称旧式连接或 SQL-89 风格），以及更推荐的显式 JOIN ...

. ON ... 语法 (SQL-92 风格：FROM E JOIN C ON E. Eno = C.

Manager)。两种写法在功能上等价，但显式 JOIN 语法更清晰地将“连接关系”与“过滤条件”分离，避免了 WHERE 子句中混合连接条件和业务过滤条件带来的可读性问题。本题查询结果的每一行输出包含：社团名（来自 C 表）、负责人名和性别（来自 E 表），直观地展示了多表查询如何将不同表的信息通过外码-主码关系关联在一个结果集中。这也是实际开发中最常见的查询类型——任何涉及一对多或多对多关联实体的信息提取，都需要通过 JOIN 来组装不同表中的列。知识点：第三章——SQL 多表连接查询：外码-主码等值连接的基本写法

(2) 查找没有参加任何社团的职工号、职工名和年龄。

参考答案

方法一 (NOT IN):

```
1 SELECT Eno, Ename, Age
2 FROM E
3 WHERE Eno NOT IN (SELECT Eno FROM EC);
```

方法二 (NOT EXISTS):

```
1 SELECT Eno, Ename, Age
2 FROM E
3 WHERE NOT EXISTS (SELECT * FROM EC WHERE E.Eno = EC.Eno);
```

解析与知识点

本题考察 SQL 中实现“否定/不存在/排除”语义的标准技术。“没有参加任何社团”等价于“该职工的职工号不在参加表 EC 中”，也等价于“不存在该职工的参加记录”。

SQL 为此提供了两种标准写法。方法一：NOT IN 子查询。子查询 SELECT Eno

FROM EC 获取所有参加了社团的职工号（无论参加了几个社团，每个职工号只出现一次或多次），外层 WHERE Eno NOT IN (子查询结果集) 筛选出那些职工号不在该集合中的职工。

这种方法直观易懂，但有一个潜在的坑：若子查询结果中包含 NULL 值，NOT IN 的表现可能会出乎意料——因为 $x \text{ NOT IN } (1, 2, \text{NULL})$ 逻辑上等价于 $x <> 1 \text{ AND } x <> 2 \text{ AND } x <> \text{NULL}$ ，而 $x <> \text{NULL}$ 永远返回 UNKNOWN（非 TRUE），导致整个 WHERE 条件永远不会为 TRUE，最终返回空结果集。方法二：NOT EXISTS 相关子查询。外层逐行读取 E 表的学生元组，每读一行时将当前行 Eno 值传递给内层子查询，内层子查询检查 EC 表中是否存在该 Eno 的参加记录。若子查询返回空集（即没有找到该职工的任何参加记录），NOT EXISTS 为 TRUE，该学生被选中。

NOT EXISTS 不受 NULL 值影响，语义更加健壮。在实际工程中，处理可能涉及 NULL 值的排除查询时建议优先使用 NOT EXISTS。此外，无论使用哪种写法，都要注意条件表达应准确对应题目要求——排除所有参加了任何社团的职工，而非仅排除参加了特定社团的职工。知识点：第三章——SQL 子查询：NOT IN 与 NOT EXISTS 实现“不存在”语义及 NULL 值行为差异

(3) 查找参加人数最多的社团号、社团名称和参加人数。

参考答案

```
1 SELECT C.Cno, Cname, COUNT(*) AS JoinCount
2 FROM C, EC
3 WHERE C.Cno = EC.Cno
4 GROUP BY C.Cno
5 HAVING COUNT(*) >= ALL (SELECT COUNT(*)
6                          FROM EC
7                          GROUP BY Cno);
```

解析与知识点

本题考察“分组统计 + 极值筛选”的 SQL 查询设计，是聚合查询中的经典题型。该查询需要分两步完成：第一步统计每个社团的参加人数，第二步从中筛选出人数最多的社团。

具体执行逻辑：将 C 表和 EC 表按 Cno 连接（WHERE C.Cno = EC.Cno），关联出每条参加记录对应的社团名称。

然后按 C.

Cno (社团号) 分组——注意 GROUP BY 中出现的非聚合列 Cname 虽不在聚合函数内, 但因 Cname 函数依赖于 Cno (一个 Cno 唯一对应一个 Cname), 在 SQL 标准中是允许的 (若 DBMS 严格遵循 SQL 标准中 GROUP BY 的规则, 可能需要 GROUP BY C. Cno, Cname)。每组的 COUNT(*) 即该社团的参加总人数。关键在 HAVING 子句: 其子查询 SELECT COUNT(*) FROM EC GROUP BY Cno 重新统计每个社团的人数 (返回一个数值列表), 外层用 COUNT(*) >= ALL (该列表) 筛选出人数不小于所有其他社团的组 (即人数最多者)。

注意这里不能简单地使用 MAX 聚合函数替代, 因为全称量词 ALL 直接比较的是每个组的人数与所有组人数集合的关系。

如果有多个社团并列第一 (人数相同), >= ALL 会将它们都输出, 这正是符合预期的行为。另一种等价写法是用子查询先找出最大人数值: SELECT MAX(cnt) FROM (SELECT COUNT(*) AS cnt FROM EC GROUP BY Cno) AS T, 然后将它放在 HAVING 条件中比较。

考虑到可读性和复用性, 这种方法在实际工程中更常见, 但这道题考察的是 ALL 量词在极值比较中的应用。知识点: 第三章——SQL 聚合查询: GROUP BY + HAVING + ALL 量词实现分组极值筛选

2.6 四、分析与设计题 (共 50 分)

2.7 1. 指出下列各关系模式的候选码、判断其属于第几范式(1NF~BCNF)。(9 分)

(1) R1(W,X,Y,Z) F1={WX→Y, WY→Z, Y→X}

参考答案

候选码: WX, WY 最高范式: 3NF

解析与知识点

本题考察候选码求解与范式判定的完整流程, 是一道综合型计算分析题。第一步: 求候选码。计算各属性及组合在函数依赖集下的属性闭包。

WX⁺: 由 WX→Y 得 WXY, 再由 WY→Z 得 WXYZ = U, WX 是候选码。

WY⁺: 由 Y→X 得 WYX, 再由 WY→Z (注意这里 WY 已在闭包中) 得 WYXZ = U, WY 也是候选码。检查最小性: W⁺ = W, X⁺ = X, Y⁺ = YX (仅 X, 因 Y→X), Z⁺ = Z, 均不能覆盖全属性, 故 WX 和 WY 都是最小的候选码。

候选码集合为 $\{WX, WY\}$ ，主属性集为 $\{W, X, Y\}$ ，非主属性集为 $\{Z\}$ （只有 Z 不属于任何候选码）。第二步：逐级检查范式。首先检查 2NF 的条件——是否存在非主属性 Z 对候选码的部分依赖？

候选码 WX 的真子集 W 或 X 的闭包中是否包含 Z ？ $W^+ = W$ ， $X^+ = X$ ，均不包含 Z ，因此 Z 不对候选码 WX 部分依赖。

候选码 WY 的真子集 W 或 Y 的闭包中是否包含 Z ？

$Y^+ = YX$ ，不包含 Z ； $W^+ = W$ ，不包含 Z ，因此也不存在部分依赖。2NF 满足。

第三步：检查 3NF——是否存在非主属性 Z 对候选码的传递依赖？

需要检查是否存在 $K \rightarrow A \rightarrow Z$ 的模式（ K 是候选码， A 不是超码）。从 F 中可以找到 WX （候选码） $\rightarrow Y$ （通过 $WX \rightarrow Y$ ），而 Y 不是超码，但从 Y 得出 Z 没有直接的函数依赖（注意 $WY \rightarrow Z$ ，不是 $Y \rightarrow Z$ ）。而从 WY （候选码）直接有 $WY \rightarrow Z$ ，这是对候选码的直接依赖而非传递。

因此 Z 不存在对候选码的传递依赖，3NF 满足。第四步：检查 BCNF——对 F 中每个非平凡函数依赖，其决定因素是否都是超码？

检查 $Y \rightarrow X$ ： Y 是不是超码？

$Y^+ = YX$ ，不等于全属性 U ，因此 Y 不是超码。而 $Y \rightarrow X$ 是非平凡依赖（ X 不是 Y 的子集），违反 BCNF。所以最高范式为 3NF。

这个分析过程的关键在于：3NF 允许“决定因素非超码但依赖项是主属性”的依赖（ $Y \rightarrow X$ 中 X 是主属性），而 BCNF 不允许，这就是 3NF 与 BCNF 的核心差异在具体案例中的体现。知识点：第六章——候选码求解（闭包法）与范式逐级判定（1NF 到 BCNF）的完整流程

(2) $R_2(W, X, Y, Z)$ $F_2 = \{W \rightarrow X, WY \rightarrow Z\}$

参考答案

候选码： WY 最高范式：1NF

解析与知识点

本题考察 2NF 违反条件“部分依赖”的典型识别。第一步：求候选码。

WY^+ ：由 $W \rightarrow X$ 将 X 加入闭包得 WYX ，再由 $WY \rightarrow Z$ 将 Z 加入闭包得 $WYXZ = U$ 。

$W^+ = WX \neq U$ ， $Y^+ = Y \neq U$ ，因此唯一候选码为 WY 。主属性集为 $\{W, Y\}$ ，非主属性集为 $\{X, Z\}$ 。

第二步：检查 2NF——是否存在非主属性对候选码 WY 的部分依赖？

所谓部分依赖,指某个非主属性函数依赖于候选码的某个真子集(即不需要完整的候选码就能被决定)。检查 W (候选码 WY 的真子集): $W \rightarrow X$ 在 F 中直接给出,而 X 是非主属性,因此非主属性 X 对候选码 WY 存在部分依赖,2NF 被破坏,最高范式为 1NF。 X 的部分依赖会产生严重的数据冗余和操作异常: W 值在表中可能会重复出现,每次重复都会携带相同的 X 值;如果想单独修改某个 W 对应的 X 值,需要更新所有包含该 W 的元组;如果某 W 暂时没有任何 Y 值与之关联(例如没有对应的 Z 记录),则根本无法插入 W 的基本信息。

注意 Z 是否有部分依赖?

Z 依赖于 WY (完整的候选码,非部分),也依赖于 $WY \rightarrow Z$ 中的左部 WY (完整候选码),因此 Z 对候选码是完全函数依赖,不违反 2NF。但只要存在一个部分依赖,整个关系模式就不满足 2NF。

要解决此问题需将关系分解为 $R_{21}(W, X)$ 和 $R_{22}(W, Y, Z)$,将部分依赖 $W \rightarrow X$ 分离到一个独立的关系中去,使两个子关系都满足 2NF 以上。知识点:第六章——部分函数依赖的识别与 2NF 违反判定

(3) $R_3(W, X, Y, Z)$ $F_3 = \{WX \rightarrow YZ, Y \rightarrow WX\}$

参考答案

候选码: WX, Y 最高范式: BCNF

解析与知识点

本题考察一个达到最高范式等级(BCNF)的理想情况。第一步:求候选码。 WX^+ : 由 $WX \rightarrow YZ$ 直接得 $WX \cup YZ = WXYZ = U$, WX 是候选码。

Y^+ : 由 $Y \rightarrow WX$ 得 $Y \cup WX = WXY$;再由 $WX \rightarrow YZ$ 将 Z 加入(Y 已在闭包中)得 $WXYZ = U$, Y 也是候选码。检查最小性: $W^+ = W \neq U$, $X^+ = X \neq U$, WX 最小; W^+ 不包含全部; X^+ 不包含全部; $Z^+ = Z \neq U$ 。因此候选码为 WX 和 Y 。

主属性集为 $\{W, X, Y\}$,非主属性仅有 Z (仅出现在右部 $WX \rightarrow YZ$ 中,不在任何候选码中)。第二步:逐级检查范式。

2NF——检查是否存在非主属性 Z 对候选码的部分依赖。

对于候选码 WX ,检查 W 的闭包: $W^+ = W$,不含 Z ;检查 X 的闭包: $X^+ = X$,不含 Z ,无部分依赖,2NF 满足。对于候选码 Y (单属性候选码,没有真子集),不存在部分依赖的问题。3NF——检查是否存在非主属性 Z 对候选码的传递依赖。

Z 通过 $WX \rightarrow YZ$ 直接依赖于候选码 WX ,也通过 $Y \rightarrow WX$ 和 $WX \rightarrow YZ$ 间接依赖于候选码 Y ,这两种情况都是对候选码的直接或间接函数依赖,不存在“非候选

码的中间属性决定 Z”的传递依赖模式——实际从依赖图看，Z 直接由候选码 WX 决定，而候选码 Y 通过决定 WX 间接决定 Z，结构上没有单独的传递链。3NF 满足。BCNF——检查 F_3 中每个非平凡函数依赖的决定因素是否都是超码。

$WX \rightarrow YZ$ 的决定因素 WX 是候选码（超码），满足； $Y \rightarrow WX$ 的决定因素 Y 也是候选码（超码），满足。所有依赖的决定因素都是超码，BCNF 完全满足。因此该关系模式无需进一步分解，已是最优化形式。

这个例子展示了范式判定的最佳可能结果：从函数依赖集本身就能直接判定候选码，且所有依赖的左部恰好都是候选码，构成了 BCNF 的最优条件。知识点：第六章——BCNF 的判定条件：所有非平凡函数依赖的决定因素均为超码

2.8 2. 关系模式分解及判断（12 分）

(1) 设有关系模式 $R(A,B,C,D,E,G)$ ，R 的函数依赖集

$F=\{AB \rightarrow C, C \rightarrow D, CA \rightarrow E, E \rightarrow A, BD \rightarrow A, B \rightarrow C\}$ ，求 R 的候选码，将 R 分解为具有无损连接且函数依赖保持性的 3NF。

参考答案

候选码：BG

最小函数依赖集 F_m ： $\{B \rightarrow C, C \rightarrow D, CA \rightarrow E, E \rightarrow A, B \rightarrow A\}$

3NF 分解结果 ρ ： $\{CD, ACE, ABC, BG\}$

解析与知识点

本题是一道完整的规范化分析大题的典型代表，要求依次完成候选码求解、最小覆盖计算和保持依赖的 3NF 分解三步骤。步骤一（候选码求解）：计算各属性及组合的闭包。

$B^+ = BCD$ ($B \rightarrow C, C \rightarrow D$)，无法继续。

BG^+ ： $B \rightarrow C$ 加入 C， $C \rightarrow D$ 加入 D，得 BCDG，再由 $BD \rightarrow A$ （已有 B 和 D）加入 A，得 ABCDEG = U，故 BG 是候选码。验证 BG 的最小性： $B^+ = BCD \neq U$ ， $G^+ = G \neq U$ ，故 BG 最小。步骤二（最小函数依赖集 F_m ）：使用最小覆盖算法：(i) 将右部拆分为单属性——原 F 中右部均已是单属性，无需处理。

(ii) 消除左部冗余属性—— $AB \rightarrow C$ 中，因 F 中已有 $B \rightarrow C$ （已知），A 是冗余属性，可简化为 $B \rightarrow C$ 。

$BD \rightarrow A$ 中，由 $B \rightarrow C$ 和 $C \rightarrow D$ 传递可得 $B \rightarrow D$ ，即 D 实际上可由 B 推导，D 在 $BD \rightarrow A$ 的左部冗余，简化为 $B \rightarrow A$ 。 $CA \rightarrow E$ 中，C 和 A 均不可去除。

(iii) 消除冗余函数依赖——逐一检查每个依赖能否由其余依赖推导。最终得 $F_m = B \rightarrow C, C \rightarrow D, CA \rightarrow E, E \rightarrow A, B \rightarrow A$ 。步骤三（保持依赖的 3NF 分解）：按 F_m 中函数依赖的左部相同者合并分组： $C \rightarrow D$ 产生关系 CD （属性集 C, D ）； $CA \rightarrow E$ 产生关系 ACE （属性集 C, A, E ）； $B \rightarrow C$ 和 $B \rightarrow A$ 的左部相同（均为 B ），合并产生关系 ABC （属性集 B, A, C ）。

检查无损连接性：候选码 BG 未被以上任何关系包含，根据 3NF 分解算法，需增加一个仅包含候选码 BG 的关系 BG （属性集 B, G ）。

最终分解结果为 $\rho = CD, ACE, ABC, BG$ 。这个分解既保持了所有函数依赖（每个 F_m 中依赖都在某个子关系模式上完整保留），又具有无损连接性（通过添加候选码关系 BG 保证）。

该题涵盖了规范化技术从问题分析到解决方案的全部关键步骤，是期末考试中的核心考察内容。知识点：第六章——候选码求解、最小覆盖算法与保持依赖的 3NF 无损分解的完整流程

(2) 已知： $R(A,B,C)$, $F=\{A \rightarrow B, B \rightarrow C\}$, R 的分解 $\rho=\{AC, BC\}$, 判断该分解 ρ 是否具有无损连接性？

参考答案

该分解 ρ 不是无损连接分解。

解析与知识点

本题考察用二分判定法（简化 Chase 算法）判断二元分解的无损连接性。当将一个关系模式 R 分解为两个子模式 R_1 和 R_2 时，无损连接性的二分判定条件为： $(R_1 \cap R_2) \rightarrow (R_1 - R_2)$ 或 $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$ 至少有一个成立，其中函数依赖必须在 F^+ 中（即由已知依赖集 F 逻辑蕴含）。

代入本题： $R_1 = AC, R_2 = BC$, 则 $R_1 \cap R_2 = \{C\}, R_1 - R_2 = \{A\}, R_2 - R_1 = \{B\}$ 。

现在检查两个条件：(1) $C \rightarrow A$ 是否由 $F = \{A \rightarrow B, B \rightarrow C\}$ 逻辑蕴含？计算 C_F^+ ：由 F 中依赖， C 的闭包没有可推导的右部（因为 F 中没有以 C 为左部的依赖），仅 $C^+ = \{C\}$ ，不包含 A 。因此 $C \rightarrow A$ 不成立。

(2) $C \rightarrow B$ 是否由 F 逻辑蕴含？同样 $C^+ = \{C\}$ ，不包含 B 。因此 $C \rightarrow B$ 也不成立。

两个条件均不满足，该分解不是无损连接分解。这意味着：如果按 ρ 将原关系模式分解为 AC 和 BC 两个子表，然后通过自然连接将两个子表重新合起来，无法保证得到的结果等价于原数据——可能会产生多余的“虚假元组”（悬空元组）或

者丢失原有的数据，造成信息丢失。

举一个反例来说：原关系中有元组 $(a1, b1, c1)$ 和 $(a2, b2, c1)$ ，分解到 AC 表为 $(a1, c1)$, $(a2, c1)$ ，BC 表为 $(b1, c1)$, $(b2, c1)$ ，自然连接后的结果为 $(a1, b1, c1)$, $(a1, b2, c1)$, $(a2, b1, c1)$, $(a2, b2, c1)$ ——多出了两条原关系中不存在的元组！

这就是信息丢失的直观展现。正确的无损分解应考虑依赖传递链，如分解为 AB, BC 时，交集 B 的决定闭包包含 C，满足 $B \rightarrow C \in F^+$ ，为无损分解。知识点：第六章——无损连接性判定：二元分解的二分 Chase 算法

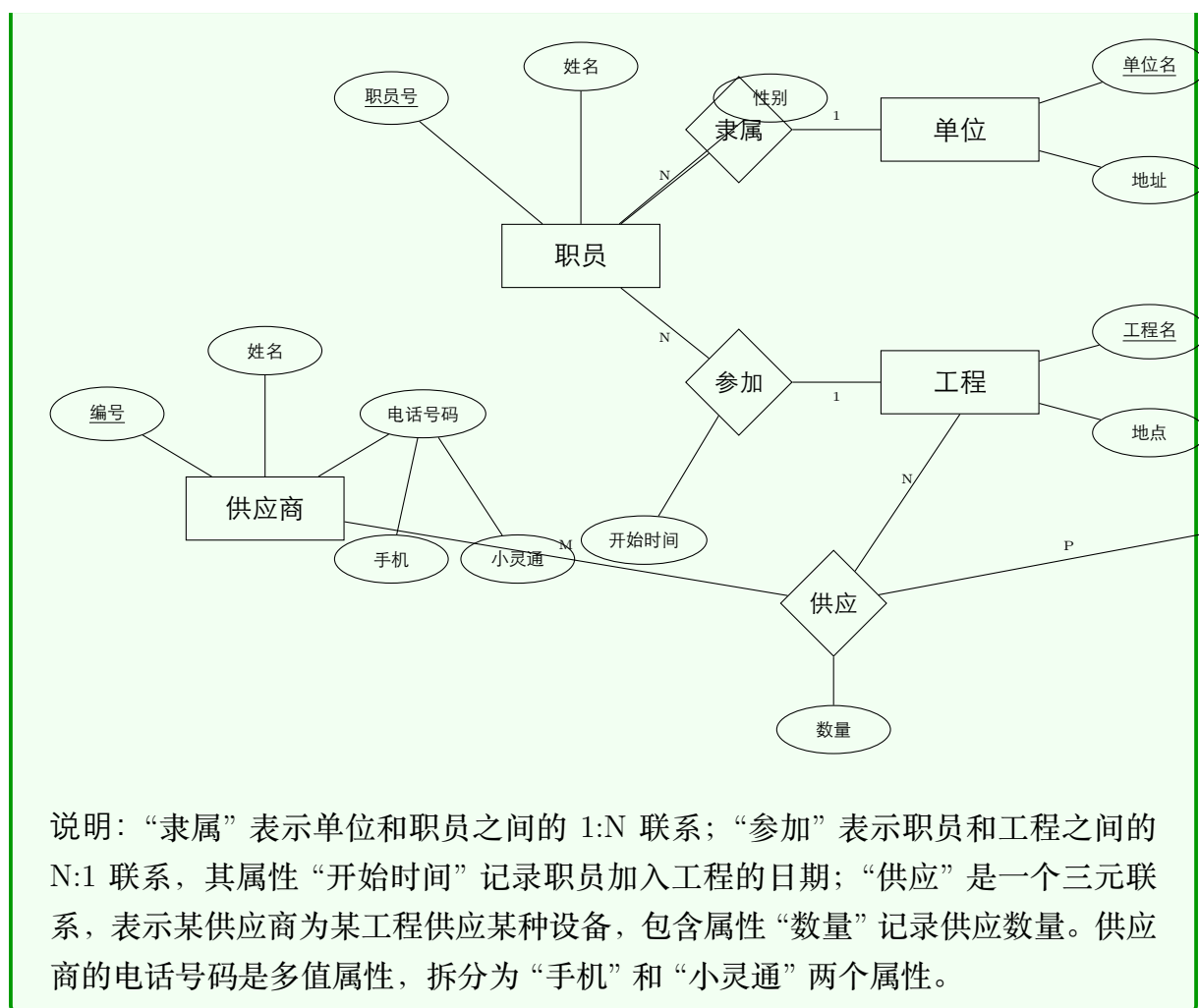
2.9 3. 企业数据库综合设计（29 分）

该企业有多个下属单位，每一单位有多个职员，一个职员仅属于一个单位，且一个职员仅在一个工程中工作，但一个工程中有很多职员参加建设，有多个供应商为各个工程供应不同设备。单位的属性有：单位名、地址。职员的属性有：职员号、姓名、性别。设备的属性有：设备号、设备名、产地。供应商的属性有：编号、姓名、电话号码（分手机和小灵通）。工程的属性有：工程名、地点。此外，需考虑职工参加某工程的开始时间，供应商为各个工程供应不同设备的数量。

(1) 画出该数据库的基本 E-R 图。

参考答案

基本 E-R 图如下：



解析与知识点

本题考察根据复杂的业务描述绘制完整 E-R 图的综合能力。正确绘制的关键在于依次从需求文本中系统性提取实体、属性与联系，并从参与约束中准确判断联系的类型（1:1、1:N、N:M）。

实体识别：文本中可提取五个核心实体——单位（有单位名和地址）、职员（有职员号、姓名和性别）、工程（有工程名和地点）、供应商（有编号、姓名和电话号码，其中电话号码进一步拆分为手机和小灵通两个子属性）和设备（有设备号、设备名和产地）。

联系识别与分析：(1) “隶属”联系——“每一单位有多个职员，一个职员仅属于一个单位”，单位到职员的 1:N 二元联系，联系不需要独立属性。

(2) “参加”联系——“一个职员仅在一个工程中工作，一个工程中有很多职员参加建设”，职员到工程的 N:1 二元联系，且携带“开始时间”作为联系的属性。这里需要注意题目中职员的参与约束很强——“仅在一个工程中工作”，说明从职员到工程是 N:1（而非 M:N），这与常规“一个职员参与多个工程”的场景不同。

(3) “供应”联系——“有多个供应商为各个工程供应不同设备”，这是供应商-工

程-设备的三元联系，三者之间构成 M:N:P 的关系模式。注意到“不同设备”隐含了一个供应商可向一个工程供应多种设备，一个工程可从不同供应商采购同种设备，种种组合构成了三元联系的必要性（如果仅是二元联系会丢失部分语义信息），且供应联系携带“数量”属性。

(4) 复合属性处理——供应商的电话号码分为“手机”和“小灵通”两子项，在 E-R 图中以复合属性的方式表示（主属性“电话号码”连接两个子属性）。

该 E-R 模型准确反映了业务语义中的全部约束和关联关系。知识点：第七章——E-R 图设计：从业务需求描述中提取实体、属性和联系并确定联系方式（1:N, N:1, M:N, 三元联系、自联系）

(2) 将该 E-R 图转换为尽可能少的等价的关系模式，并指出主码。

参考答案

转换得到以下 6 个关系模式（主码以下划线标出）：

- (1) 单位：C(单位名, 地址)
- (2) 职员：E(职员号, 姓名, 性别, 单位名, 工程名, 开始时间)
- (3) 工程：J(工程名, 地点)
- (4) 供应商：S(编号, 姓名, 手机, 小灵通)
- (5) 设备：P(设备号, 设备名, 产地)
- (6) 供应：SPJ(编号, 工程名, 设备号, 数量)

也可用英文缩写表达：

- (1) C(C#, CN, Addr)
- (2) E(E#, EN, Sex, C#, Date, J#)
- (3) J(J#, JN, Jaddr)
- (4) S(S#, SN, Tel1, Tel2)
- (5) P(P#, PN, Paddr)
- (6) SPJ(S#, J#, P#, Q)

解析与知识点

本题考察 E-R 图向关系模式转换的系统性规则应用。根据转换规则，逐一处理各类实体和联系。实体转换（5 个关系模式）：每个实体直接转为一个关系模式，实体的属性变为关系的属性，实体的主码变为关系的主码。

得到 C（单位）、E（职员）、J（工程）、S（供应商）、P（设备）五个关系模式。对于供应商 S 中的电话号码复合属性，拆分为手机和小灵通两个独立列。联系转换（合并与独立建表）：(1) 1:N 联系”隶属”（单位 1:N 职员）——将 1 端（单位）的主码”单位名”纳入 N 端（职员）的关系中作为外码，无需为”隶属”单独建表。

”单位名”作为外码添加到职员表 E，同时反映了该职员所属的单位。这一合并使关系数量减少一个。

(2) N:1 联系”参加”（职员 N:1 工程）——将 1 端（工程）的主码”工程名”纳入 N 端（职员）的关系中作为外码，同时将联系的属性”开始时间”也一并纳入职员关系。

这样职员表 E 就包含了单位名（来自隶属联系合并）、工程名和开始时间（来自参加联系合并）。由于”一个职员仅在一个工程中工作”的约束，每个职员只有一条参加记录，”开始时间”不会产生冗余。这一合并又减少了一个关系。

(3) 三元 M:N:P 联系”供应”——三元联系必须转换为独立的关系模式，属性包括三个参与实体的主码（编号、工程名、设备号）构成联合主码，再加上联系本身的属性”数量”。产生关系 SPJ。

三元联系不能通过将外码合并到某个实体中来处理，因为三元联系的信息量可能大于分别合并到三个实体中各自能表达的联系信息之和。

(4) 最终得到 6 个关系模式（5 个实体表 + 1 个三元联系表）。由于两个 1:N/N:1 联系被成功合并到 N 端实体，这是”尽可能少”的最优转换方案。中英文两种表达方式等价，实际考试中任选一种即可。知识点：第七章——E-R 图向关系模式的转换：实体转换、1:N/N:1 联系合并、三元联系独立建表

(3) 列出几种利用 SQL 实现该数据库三类完整性约束的方法。

参考答案

三类完整性约束的 SQL 实现方法：

- (1) 实体完整性：通过定义主码（PRIMARY KEY）实现。在建表时或修改表结构时，使用 PRIMARY KEY 子句指定表的主码，DBMS 自动保证主码值唯一且非空。

```
1 CREATE TABLE E (
```

```
2      Eno      CHAR(10) PRIMARY KEY ,
3      Ename    VARCHAR(20) NOT NULL ,
4      ...
5  );
```

(2) 参照完整性：通过定义外码（**FOREIGN KEY**）实现。在创建表时使用 **FOREIGN KEY ... REFERENCES** 子句指定外码及其引用的父表和属性。

```
1  CREATE TABLE E (
2      Eno      CHAR(10) PRIMARY KEY ,
3      Ename    VARCHAR(20) ,
4      Cno      CHAR(10) ,
5      Jno      CHAR(10) ,
6      FOREIGN KEY (Cno) REFERENCES C(Cno) ,
7      FOREIGN KEY (Jno) REFERENCES J(Jno)
8  );
```

(3) 用户定义完整性：可通过以下多种机制实现：

- 域约束：使用 **NOT NULL**、**UNIQUE**、**DEFAULT** 等列级约束。
- 检查约束（**CHECK**）：定义属性取值范围约束，如 **CHECK (Sex IN ('M','F'))**。
- 断言（**ASSERTION**）：定义跨表的全局约束条件。
- 触发器（**TRIGGER**）：定义复杂的、主动触发的完整性规则，如下一小题所示。

解析与知识点

本题考察 SQL 中三类完整性约束的实现机制及其对应关系，是数据库完整性保障体系的核心内容。数据库完整性约束分为三个层次，各自由 SQL 的不同语法机制来实现：第一层实体完整性（Entity Integrity）要求每个关系的主码值必须唯一且非空。

SQL 通过 **PRIMARY KEY** 子句在列级或表级定义主码——列级定义适用于单属性主码（直接在列定义后写 **PRIMARY KEY**），表级定义适用于多属性联合主码或需要命名的约束（使用 **CONSTRAINT** 子句）。

一旦定义了 **PRIMARY KEY**，DBMS 自动为该列创建唯一索引，并在插入或更新时检查值的唯一性和非空性，违反约束的操作将被拒绝并返回错误信息。第二层参照完整性（Referential Integrity）要求外码的值要么取空值（**NULL**），要么等于

被参照关系中某元组的主码值。

SQL 通过 FOREIGN KEY . . . REFERENCES 子句定义外码，DBMS 在插入或更新外码值、删除或更新父表主码值时触发检查。

可通过 ON DELETE/ON UPDATE 指定级联行为：CASCADE（级联删除/更新）、SET NULL（置空）、NO ACTION（拒绝操作，默认行为）。

第三层用户定义完整性（User-Defined Integrity）涵盖除实体完整性和参照完整性之外的所有语义约束。SQL 提供了从简到繁的多层机制：(1) 列级约束——NOT NULL（非空）、UNIQUE（唯一）、DEFAULT（默认值）、CHECK（检查约束），用于单列范围的简单约束；(2) 表级约束——表级的 CHECK 子句可以引用同一表的多列（如 CHECK (start_date <= end_date)）；(3) 断言（ASSERTION）——独立于表的全局约束对象，可跨表定义约束条件（如“供应表中的设备号必须在设备表中存在”——但外码已能处理此需求）；(4) 触发器（TRIGGER）——最灵活但也最复杂的机制，由数据库事件（INSERT/UPDATE/DELETE）自动触发执行的程序块，可实现任意复杂的业务规则。

了解各类约束的适用层级及其 SQL 实现，是数据库应用设计中保证数据质量的基石。知识点：第五章——三类完整性约束的定义与 SQL 实现：PRIMARY KEY、FOREIGN KEY、CHECK、断言与触发器

(4) 为了提高该数据库的查询效率，可以采取哪些措施？

参考答案

可以从以下几个方面采取措施提高查询效率：

- (1) 建立索引：为经常出现在 WHERE 子句、连接条件中的属性建立索引（特别是 B+ 树索引）。例如在 E 表的 C# 和 J# 上建索引可加速连接操作；在 SPJ 表的 S#、J#、P# 上建索引可加速三元关系的查询。
- (2) 建立聚簇：对经常一起查询的数据，在物理存储上将它们聚集存放，减少 I/O 次数。例如将 E 表按 C# 聚簇，使同一单位的职员记录物理上相邻。
- (3) 选择运算尽可能先做：在查询执行时，优先执行选择 (σ) 操作，缩小中间结果规模，降低后续连接操作的开销。
- (4) 在执行连接前对关系适当预处理：对连接属性先排序或建立 Hash 索引，为归并连接或 Hash 连接算法做准备。
- (5) 关系的水平分解和垂直分解：根据数据访问频率将关系进行水平分片（按行拆分）或垂直分片（按列拆分），将频繁访问的数据与不常访问的数据分开

存储，减少每次查询的数据扫描量。

- (6) 合理设计查询语句：避免使用 `SELECT *`，只选取需要的列；合理使用 `EXISTS` 代替 `IN`；充分利用连接条件而非子查询等。

解析与知识点

本题考察数据库查询性能优化的多维度知识体系，要求从物理设计优化和查询执行优化两个层面系统性地回答。物理设计优化层面：索引是查询加速的首要手段。B+ 树索引适合等值查询和范围查询（如 `=`, `>`, `<`, `BETWEEN`），Hash 索引仅适合等值查询（`=`, `IN`）但查找速度更快（ $O(1)$ vs $O(\log n)$ ）。

索引列的选择应优先考虑 `WHERE` 子句中的过滤列、`JOIN` 连接列和 `ORDER BY` 排序列。但索引不是“越多越好”——每个索引都会增加 `INSERT/UPDATE/DELETE` 的维护开销，需要在查询加速与写入性能之间取得平衡。

聚簇索引将索引键值相邻的元组物理存储在邻近的磁盘块中，对范围查询和排序有显著提速效果，因为减少了随机磁盘 I/O。

关系分割策略包括水平分片（按某种条件将行分布到不同物理存储，如按年份分区）和垂直分片（将表中常用列与不常用列分离存储，减少 I/O 传输的不需要列数据）。查询执行优化层面：代数优化是关系代数的等价变换，核心原则是“先选择后连接”——因为选择操作可以大幅削减行数（从 $O(n)$ 缩小到 $O(s)$ ，其中 $s < n$ ），先筛选再连接可以极大减少后续连接操作的输入规模。物理优化基于代价估算进行三个层面的决策：访问路径选择（是否使用索引、使用哪个索引）、连接算法选择（嵌套循环、排序合并、哈希连接）和连接顺序选择（多表连接时最优的连接先后次序）。

查询重写层面的优化包括视图展开、子查询去相关（将相关子查询转换为 `JOIN`）、谓词化简等。

此外，物化视图可以将频繁使用的聚合查询结果预计算并持久化存储，以空间换时间。SQL 语句的写法也对性能影响巨大：`SELECT *` 传输不必要的列数据；`NOT IN` 在子查询含 `NULL` 值时语义异常并可能导致性能退化；合理使用 `EXISTS` 替代 `IN` 可实现半连接优化。

几乎所有现代 DBMS 都内置了查询优化器来自动完成大部分优化决策，但 DBA 和开发人员仍需理解优化原理以便在必要时通过索引设计、查询重构和系统配置提供帮助。知识点：第九章——查询优化的多层面策略：索引选择、聚簇设计、代数优化、物理优化、关系分割与查询重写

- (5) 为该数据库的某个表自行设计一个触发器并注明其作用。

参考答案

触发器功能说明：在供应表 SPJ 中，当更新某条供应记录的设备数量 (Q) 时，要求修改后的数量不得低于修改前的数量。若新数量低于原数量，则拒绝修改（将数量恢复为原值）。

```
1 CREATE TRIGGER Trig_Check_Qty
2 AFTER UPDATE OF Q ON SPJ
3 REFERENCING OLD ROW AS oldg, NEW ROW AS newg
4 FOR EACH ROW
5 WHEN (oldg.Q > newg.Q)
6 BEGIN
7     UPDATE SPJ
8     SET Q = oldg.Q
9     WHERE S# = newg.S#
10         AND J# = newg.J#
11         AND P# = newg.P#;
12 END;
```

触发器说明：

- 触发时机：AFTER（在 UPDATE 操作执行之后触发）。
- 触发事件：UPDATE OF Q ON SPJ（对供应表中 Q 列的更新操作）。
- 触发粒度：FOR EACH ROW（行级触发器，每更新一行触发一次）。
- 触发条件：oldg.Q > newg.Q（当新数量小于旧数量时）。
- 触发动作：将 Q 值恢复为修改前的数值 oldg.Q，实质上拒绝了对数量的降低修改。

注意

补充说明：上述触发器通过覆盖更新使数量不低于原值。在现代 DBMS 中，也可使用 BEFORE 触发器配合 SIGNAL 语句直接拒绝违规操作，或抛出异常终止事务。这种触发器属于用户定义完整性约束的实现机制，可保证供应数量只增不减的业务规则。

解析与知识点

本题考察数据库编程对象——触发器 (Trigger) 的设计与编写能力, 是从”被动约束”(如 CHECK 只能做简单值判断) 到”主动规则”(触发器可以执行任意复杂的逻辑) 的进阶考察。触发器是一种特殊的存储过程, 它不由用户显式调用, 而是由特定数据库事件 (INSERT、UPDATE、DELETE) 的发生自动触发执行。根据触发时机的不同, 可分为三种类型: BEFORE 触发器在操作执行前触发, 可用于拦截和修改即将执行的操作 (如自动填充字段、校验数据合法性、阻止非法操作); AFTER 触发器在操作执行完成后触发, 适用于级联更新、维护派生数据 (如更新统计汇总表) 和审计日志等后续处理任务; INSTEAD OF 触发器主要用于视图 (因为视图本身不可直接更新), 拦截对视图的更新操作并转化为对基本表的相应操作。

根据触发粒度的不同, 可分为行级触发器 (FOR EACH ROW, 每影响一行触发一次, 可访问 OLD/NEW 行数据) 和语句级触发器 (FOR EACH STATEMENT, 每条 SQL 语句触发一次, 不管影响了多少行)。本题设计的触发器属于 AFTER + FOR EACH ROW 的组合——在更新供应数量 Q 后逐行检查新值是否小于旧值, 若违反则通过 UPDATE 将值改回原值以强制执行”供应数量不可减少”的业务规则。

这是一种变通的”执法”方式 (先允许违规修改, 再覆盖回来)。

更规范的做法是使用 BEFORE 触发器: 在操作执行前用条件判断 $oldg. Q > newg. Q$ 拦截并抛出异常 (如 MySQL 的 SIGNAL 语句) 终止事务, 或者直接 SET $newg. Q = oldg.$

Q 在操作执行前修正新值。需要注意触发器的使用会引入一定的系统开销 (每个触发事件都需要执行额外的触发器代码逻辑), 且多个触发器之间的执行顺序可能存在不确定性, 应谨慎使用——仅在其他声明式约束 (如 CHECK、FOREIGN KEY) 无法满足需求时才求助于触发器。触发器的典型应用场景包括: 复杂的跨表数据校验、自动维护审计日志、自动更新汇总统计字段、数据同步和级联操作等。知识

点: 第五章——触发器机制: 触发事件、触发时机 (BEFORE/AFTER/INSTEAD OF)、粒度 (行级/语句级) 与用户定义完整性的实现

3 2011–2012 学年《数据库原理与设计》期末试卷 (A)

3.1 一、判断题 (每小题 1 分, 共 10 分)

参考答案

1–5. × × × √ ×

6–10. × √ × × √

解析与知识点

本组 10 道判断题覆盖 SQL 语言基础 (通配符、DDL/DML/DCL 分类)、空值语义 (NULL 的三值逻辑)、规范化理论 (模式分解不唯一性、3NF 与 BCNF 区分)、并发控制 (两段锁协议、封锁粒度、意向锁兼容性)、查询优化 (代数等价变换) 及外码定义等核心知识点。

逐题辨析: 第 1 题区分 SQL 通配符与操作系统通配符; 第 2 题理解 SQL 集 DDL、DML、DCL 于一体的完整功能; 第 3 题掌握 NULL 的“未知值”本质及三值逻辑; 第 4 题认识模式分解因依赖集选择不同而结果不唯一; 第 5 题牢记两段锁是可串行化的充分非必要条件; 第 6 题理解封锁粒度与系统开销的正相关关系; 第 7 题掌握查询优化以减少中间结果为核心目标; 第 8 题熟记 IS 与 IX 兼容; 第 9 题区分全主属性 (仅保底 3NF) 与 BCNF; 第 10 题理解外码数量由参照关系决定。

1. 错误。SQL 中的通配符是下划线 ‘_’ (匹配单个字符) 和百分号 ‘%’ (匹配零个或多个字符), 而非 ‘*’。‘*’ 是某些操作系统或文件搜索中的通配符, 与 SQL 无关。这道题考察的是 SQL 标准语法的基本细节, 考生常将操作系统通配符与 SQL 通配符混淆。LIKE 模式匹配是 SQL 查询中常用的模糊匹配手段, 应当熟记正确的通配符符号。在 SQL 中, ‘_’ 需要转义时可以使用 ESCAPE 关键字指定转义字符。
2. 错误。SQL 语言不仅可以定义和查询基本表, 还可以创建视图 (VIEW)、建立索引 (INDEX)、授予权限 (GRANT/REVOKE)、定义完整性约束等, 数据定义与数据操纵功能远不止基本表的定义与查询。SQL 作为关系数据库的标准语言, 集数据定义语言 (DDL)、数据操纵语言 (DML)、数据控制语言 (DCL) 于一体。

视图是虚表, 基于基本表但不存储数据, 可以简化复杂查询并提高安全性。索引是物理存储层面的优化手段, 而权限控制则关系到数据库的安全性管理。这道题的考点在于区分 SQL 的功能边界, 考生容易将 SQL 仅理解为“查询语言”而忽略其完整的数据库管理能力。
3. 错误。NULL 表示“未知值”或“不存在”, 而不是空格。空格 (‘ ’) 是一个

具体的字符值，与 NULL 有本质区别。NULL 在运算和比较中有特殊语义，不可与空字符串或空格混为一谈。在 SQL 中任何与 NULL 的比较运算（如 =、<、>）结果都是 UNKNOWN，而非 TRUE 或 FALSE。NULL 值在聚合函数中通常被忽略（COUNT(*) 除外），在 GROUP BY 中所有 NULL 被视为同一组。理解空值的三值逻辑是正确处理数据库查询的基础，也是考试中常考的概念辨析题。

4. 正确。关系模式的分解策略有多种选择（如按函数依赖分解、按多值依赖分解、3NF 分解、BCNF 分解等），分解结果不唯一，不同分解方式可能在无损连接性和依赖保持性上有差异。规范化理论提供了分解算法的指导，但实际分解的结果因依赖集的最小化方式、分组策略的不同而有多种可能。例如，选择不同的候选码或不同的分组方式都会产生不同的分解结果。这种不唯一性要求设计者根据实际应用场景（查询性能、更新频度等）权衡选择最优的分解方案。因此，关系模式分解既是科学也是艺术。
5. 错误。事务遵守两段锁协议是可串行化调度的充分条件，而非必要条件。
即：若所有事务均遵守两段锁协议，则调度一定是可串行化的；但可串行化的调度不一定要所有事务都遵守两段锁协议。这一定理的核心在于理解充分条件与必要条件的逻辑区别。两段锁协议通过规定事务分为加锁阶段和解锁阶段来防止循环等待，但其他并发控制机制（如时间戳排序、可串行化快照隔离等）也能实现可串行化。考生容易误以为两段锁是可串行化的等价条件，从而在分析和解题时得出错误结论。
6. 错误。封锁粒度越小（如行级锁），并发度越高，但系统需要管理更多锁资源，锁定/解锁操作次数增多，系统开销反而增大。封锁粒度越大（如表级锁），并发度降低，但系统开销较小。封锁粒度与系统开销之间存在典型的权衡关系，类似于计算机体系结构中的时间-空间权衡。多粒度封锁协议（如意向锁机制）正是为了解决这一矛盾而设计的——通过在不同粒度级别上设置不同模式的锁，在保证数据一致性的前提下最大化并发度。考生常被“粒度小则开销小”的直觉误导，需理解锁管理本身也是一个系统资源消耗。
7. 正确。查询优化的核心目标之一是减少查询处理的中间结果规模，通过先执行选择/投影等一元运算、合理安排连接顺序等手段，降低后续操作的数据量，从而减少整体执行代价。关系代数表达式的等价变换（如选择下推、投影下推）是查询优化的基本技术。数据库系统通过基于代价的优化器（CBO），利用统计信息（如关系大小、属性值分布）估算各种执行计划的代价，选择代价最小的方案。查询优化的等价变换规则（如选择串接律、投影串接律、选择和投影的交换律等）是关系代数的核心推导手段。

8. 错误。IS（意向共享锁）与 IX（意向排他锁）之间是兼容的。意向锁的兼容性规则是：IS 与 IS、IS 与 IX、IX 与 IX 均兼容；只有当涉及 S 或 X 锁时才会有不兼容的情况。意向锁的设计目的是在表级表达较低粒度上的加锁意图，使得其他事务在请求表级锁时无需逐行检查锁冲突。例如，当事务要在某行加 X 锁时，先在表上加 IX 锁表示意图，另一个要加 S 锁的事务看到 IX 后就知道存在冲突。理解各类锁的兼容性矩阵是分析封锁调度正确性的基础。
9. 错误。所有属性都是主属性仅保证 $R \in 3NF$ ，但不保证 BCNF。
- 3NF 允许“主属性对候选码的传递函数依赖”，而 BCNF 不允许任何非平凡的函数依赖的决定因素不是超码。例如 $R(A, B, C), F = \{AB \rightarrow C, C \rightarrow B\}$ ，所有属性都是主属性（候选码为 AB 和 AC），但 $C \rightarrow B$ 的决定因素 C 不是超码，故属于 3NF 而不属于 BCNF。这道题是区分 3NF 与 BCNF 的经典考点——两者的唯一区别在于 BCNF 对主属性参与的函数依赖也施加约束。全主属性关系可排除非主属性的传递依赖问题，但主属性之间的决定关系仍可能导致 BCNF 违例。
10. 正确。一个关系可以有零个、一个或多个外码。外码用于参照其他关系（或自身）的主码，只要存在参照关系约束即可定义外码，数量无限制。例如学生选课关系可同时参照学生表和课程表，拥有两个外码。外码的数量取决于关系之间的参照关系数量，一个关系可能参与多个一对多联系的“多”端。此外，自参照关系的外码参照自身主码（如雇员表中的经理编号参照雇员编号），在递归关联中广泛应用。零个外码对应的是独立实体关系（不参照其他关系），也是最常见的情况。

常见错误包括将操作系统通配符 '*' 与 SQL 通配符 '_'/'%' 混淆、将 SQL 仅理解为查询语言而忽略 DDL/DCL、将两段锁协议误判为可串行化必要条件、认为封锁粒度越小系统开销越小、将全主属性关系误判为必定满足 BCNF。

知识点： 1.1 SQL 语言概述（通配符）； 3.2 SQL 数据定义与操纵功能（DDL/DML/DCL）； 2.3 空值语义（NULL 的三值逻辑）； 6.3 关系模式分解的不唯一性； 10.3 两段锁协议与可串行化（充分条件与必要条件）； 10.2 封锁粒度与系统开销的权衡； 9.2 查询优化基本原则（代数等价变换与代价估算）； 10.3 意向锁兼容性矩阵（IS/IX/S/X）； 6.4 BCNF 与 3NF 的关系（主属性与范式判定）； 2.2 外码定义与自参照约束。

3.2 二、单项选择题（每小题 1 分，共 10 分）

参考答案

（选择题答案由解析者补充，原答案仅提供判断题答案）

1. A
2. C
3. D
4. A
5. D
6. C
7. B
8. D
9. B
10. D

解析与知识点

本组 10 道选择题覆盖数据模型层次（概念模型）、关系代数（并相容性）、完整性约束（参照完整性）、三级模式结构（数据独立性）、E-R 建模（冲突类型与弱实体）、规范化理论（候选码、3NF 与 BCNF）、嵌入式 SQL（主变量）以及并发控制（脏读、隔离级别）等核心知识点。

逐题分析：第 1 题区分数据模型的四个抽象层次及概念模型的定位；第 2 题掌握关系代数集合运算的并相容性条件；第 3 题理解参照完整性对外码操作的约束；第 4 题识记三级模式两级映像实现数据独立性；第 5 题区分 E-R 图合并的三种冲突类型；第 6 题掌握弱实体关系转换时主码的构成方式；第 7 题通过函数依赖推导候选码并判定 BCNF；第 8 题理解嵌入式 SQL 中主变量的接口作用；第 9 题掌握 3NF 与 BCNF 在插入/删除异常上的区别；第 10 题识记脏读的定义及隔离级别的保护作用。

1. **A. 概念模型。**数据模型从低到高分为：物理模型（最底层，描述数据在存储介质上的结构）、逻辑模型（中间层，如关系模型）、外部模型（用户视图层）、概念模型（最高层，独立于 DBMS，描述现实世界的整体信息结构）。概念模型抽象级别最高，与具体数据库管理系统无关。数据模型的层次划分反映的是抽象级别的递增——从物理字节到用户可理解的概念，每一层都屏

蔽了下层的实现细节。概念模型的典型代表是 E-R 模型，它面向现实世界建模，不涉及任何 DBMS 相关概念。这种层次抽象是实现数据独立性的理论基础，也是数据库设计的起点。

2. C. $R \cup S$ 。并运算要求两个关系具有相同的属性集（同目且对应属性域相同）。 $R(A, B, C)$ 与 $S(B, C, D)$ 结构不同，因此 $R \cup S$ 不成立。A 为两个单列关系的笛卡尔积，合法；B 为在公共属性 B, C 上的自然连接，合法；D 为两个单列关系的交运算，合法。

关系代数的集合运算（并、交、差）都要求关系具有“并相容性”——相同的属性个数和对应的属性域。考生在答题时需要首先检查属性的数量和类型是否匹配，这是判断运算合法性的第一步。自然连接、笛卡尔积、选择和投影等运算则无此限制，可以在结构不同的关系之间进行。

3. D. 将 ('4004', '2') 插入选课表。学生选课表的外码 Sno 参照学生表的主码 Sno。选项 B 和 D 都涉及学号 4004，若该学号在学生表中不存在，则 D 向选课表插入记录时违反参照完整性约束，无法执行。因原卷表格数据缺失，解析者推断 4004 不在学生表中，故 D 不可执行。参照完整性是关系模型三大完整性约束之一，保证子表中的外码值必须全部或部分出现在父表的主码中。违反参照完整性的操作包括子表的插入/修改（外码值不存在）以及父表的删除/修改（被参照的主码被删除）。考生需要逐行检查每个操作对完整性的影响，这是数据库约束理解的基础训练。
4. A. 数据独立性。数据库三级模式结构（外模式-模式-内模式）的核心目的就是在模式与模式之间提供映像，当底层存储结构（内模式）或整体逻辑结构（模式）发生变化时，通过调整映像而保持上层不变，从而实现物理数据独立性和逻辑数据独立性。物理独立性是指内模式变化不影响模式和外模式（如更换存储设备、改变索引结构），逻辑独立性是指模式变化不影响外模式（如增加新的数据项）。三级模式两级映像是数据库管理系统的核心架构设计，它将数据的物理存储、逻辑组织和用户视图完全解耦。这也是为什么应用程序可以独立于数据库物理结构编写的原因。
5. D. 事务冲突。局部 E-R 图合并为全局 E-R 图时，可能出现的冲突有三类：属性冲突（属性类型、取值范围、单位等不一致）、命名冲突（同名异义、异名同义）、结构冲突（同一对象在不同视图中抽象层次不同）。"事务冲突"是并发控制中的概念，与 E-R 图合并无关。

E-R 图合并是数据库需求分析阶段的关键步骤，在设计多用户视角的大型系统时尤其重要。属性冲突需要通过协商统一属性定义，命名冲突需要建立统

一的命名规范，结构冲突则需要重新审视实体和联系的抽象层级。这三类冲突的解决质量直接影响最终数据库模式的设计质量。

6. C. 由标识实体的主码和自身的关键属性组成。弱实体依赖于标识实体而存在，不能单独被标识。转换成关系模式时，弱实体的主码由其标识实体 (owner entity) 的主码加上弱实体自身的部分键 (partial key, 即关键属性) 共同构成，同时标识实体的主码也是参照标识实体关系的外码。弱实体是 E-R 模型中的重要概念，表示那些存在依赖于其他实体的实体类型。例如“订单明细”依赖于“订单”而存在，“房间”依赖于“大楼”而存在。在关系模式转换中，弱实体对应的关系必须以标识实体的主码为外码并参与主码的构成，这既表达了存在依赖关系，也保证了参照完整性。
7. B. 只有 (学号, 课程号) 能作为候选码。由题意可知：学生 + 课程 $\rightarrow$ 名次、课程 + 名次 $\rightarrow$ 学生 (每门课程每一名次只有一名学生)。因此 (学号, 课程号) 和 (课程号, 名次) 都可以唯一确定元组的两组候选码，选 B 说“只有”是错误的。此外该关系模式不存在非主属性对候选码的传递依赖，且所有决定因素都是超码，故属于 BCNF。候选码的求解是关系规范化理论的基础步骤，需要从给定的函数依赖集出发，利用闭包算法或直观推理。本题中的“课程 + 名次 $\rightarrow$ 学生”说明每门课中名次唯一确定学生，这是从题目的语义中推导出来的隐含约束，而非显式给出的依赖。考生需要具备从题目场景中提取函数依赖关系的能力。
8. D. 主变量。在嵌入式 SQL 中，SQL 语句与高级语言 (主语言) 之间的数据传递通过主变量 (host variable) 实现。主变量由主语言声明，在 SQL 语句中以冒号前缀引用 (如 :var)，用于接收查询结果或向 SQL 语句传递参数值。DBMS、OS、DML 均是其他相关概念而非接口。嵌入式 SQL 解决了非过程性的 SQL 语言与过程性的高级语言 (如 C、Java) 之间的阻抗不匹配问题。通过游标 (Cursor) 逐行获取查询结果并存入主变量，再由主语言程序进行处理。主变量是嵌入式 SQL 编程中最核心的接口概念，理解其声明、使用和作用域是编写数据库应用程序的基础。
9. B. 仍存在一定的插入和删除异常。

3NF 消除了非主属性对候选码的传递函数依赖，但仍可能存在主属性对候选码的部分或传递函数依赖，以及非平凡函数依赖的决定因素不是超码的情况，因此 3NF 不能完全消除插入和删除异常。只有达到 BCNF 级别才能消除由函数依赖引起的所有异常。插入异常是指无法独立插入某个实体的信息 (因缺少另一实体的信息)，删除异常是指删除某一实体时丢失了另一实体的信息。这些异常的根本原因是关系模式中存在不恰当的属性关联，而规

范化的目标正是通过分解来消除这些不良依赖。3NF 与 BCNF 的差异在于 BCNF 对主属性参与的依赖也施加了超码要求，因此是更强的范式。

10. D. 该操作读”脏数据”。T1 将 A 修改为 200 后，T2 读到 $A = 200$ 。随后 T1 执行 ROLLBACK 将 A 恢复为 100，T2 读到的 200 就是一个从未提交的”脏数据”。这是典型的脏读（Dirty Read）问题：一个事务读取了另一个未提交事务的修改数据，而该修改最终被回滚。并发操作带来的数据不一致问题主要有三类：脏读、不可重复读和丢失修改（或幻读）。脏读的危害在于事务基于了一个从未被确认的数据做了后续处理，一旦该数据被回滚，事务的处理基础就坍塌了。SQL 标准的四个隔离级别（读未提交、提交读、可重复读、可串行化）中，只有读未提交允许脏读，而提交读及以上级别均可避免脏读。

常见错误包括混淆数据模型层次与三级模式结构、忽略关系代数并相容性条件、将 E-R 图冲突类型与并发事务概念混淆、将弱实体与普通实体混淆、遗漏隐含函数依赖导致候选码求解不全、认为 3NF 可完全消除插入/删除异常、以及混淆脏读与其他并发不一致问题。

知识点：1.2 数据模型的四个层次（物理/逻辑/外部/概念模型）；2.4 关系代数运算（并、交、连接兼容性与集合运算条件）；3.3 参照完整性约束（外码插入修改的违规判定）；1.3 数据库三级模式结构与数据独立性；7.2 E-R 图合并冲突类型（属性/命名/结构冲突）；7.3 弱实体的关系模式转换规则（标识实体主码 + 部分键）；6.3 候选码求解与 BCNF 判定（隐式依赖推导）；8.1 嵌入式 SQL 与主变量（游标与过程化接口）；6.4 3NF 的插入删除异常与 BCNF 对比；10.2 并发操作问题（脏读、不可重复读、丢失修改）与隔离级别。

3.3 三、查询设计题（共 6 小题，每小题 5 分，共 30 分）

已知一个公司的职工-社团数据库有三个基础表：

- 职工：E(Eno, Ename, Age, Sex)
- 社团：C(Cno, Cname, Manager, Address)
- 参加：EC(Eno, Cno, DateOfAttend)

其中：Eno 为职工号，Ename 为职工名，Age 为年龄，Sex 为性别；Cno 为社团号，Cname 为社团名，Manager 为负责人职工号，Address 为社团地址；DateOfAttend 为参加日期。

1. 用关系代数表示下列查询

参考答案

$$\pi_{\text{Eno}}(E) - \pi_{\text{Eno}}(EC) \bowtie E$$

或等价的写法: $E \bowtie (\pi_{\text{Eno}}(E) - \pi_{\text{Eno}}(EC))$

(原答案提供: $E \bowtie (\pi_{\text{Eno}}(E) - \pi_{\text{Eno}}(EC))$)

解析与知识点

首先用投影取出所有职工的工号集合 $\pi_{\text{Eno}}(E)$, 再用投影取出所有已参加社团的职工工号集合 $\pi_{\text{Eno}}(EC)$, 二者做差集得到未参加任何社团的职工工号; 最后与职工关系 E 做自然连接, 补全职工的全部属性信息。

差集要求两个关系同目, 因此 $\pi_{\text{Eno}}(E)$ 与 $\pi_{\text{Eno}}(EC)$ 均投影到单属性才可做差。

这道题考察的是关系代数中“否定”语义(未参加)的表达方式——由于关系代数中没有直接表示“不存在”的运算符, 需要通过集合差运算间接实现。

另一种等价的思考方式是用左外连接配合 NULL 值判断, 但在标准关系代数中差集加连接是更直接的写法。考生常犯的错误是直接对两个不同结构的关系做差集(如 $E - EC$), 这违反了集合运算的并相容性要求。

关系代数查询设计的核心思维在于将自然语言条件分解为基本运算的组合, 差集处理否定、除法处理全称量词、连接处理关联。

掌握了这些基本转换模式后, 复杂查询的代数表达就能系统化地完成。知识点:

2.4 关系代数(投影、差集、自然连接); 集合运算的并相容性要求; 否定语义的差集表达模式。

参考答案

$$\pi_{\text{Cno}, \text{Cname}}(C \bowtie (\pi_{\text{Cno}, \text{Eno}}(EC) \div \pi_{\text{Eno}}(E)))$$

(原答案提供: $\pi_{\text{Cno}, \text{Cname}}(C) \bowtie (\pi_{\text{Cno}, \text{Eno}}(EC) \div \pi_{\text{Eno}}(E))$)

解析与知识点

这是一个典型的“全部/所有”语义的除法运算。

首先构造被除数 $\pi_{\text{Cno}, \text{Eno}}(EC)$, 列出“哪个职工参加了哪个社团”的二元关系; 除数 $\pi_{\text{Eno}}(E)$ 是所有职工的集合; 两者做除法 $\div$ 得到的结果是那些“被所有职工都参加了”的社团号; 最后与社团关系 C 连接以获取社团名称。

除法运算是处理“所有/全部/每一个”这类全局量词语义的关键代数工具。

理解除法的本质有助于正确构建被除数和除数: 被除数必须是包含“主体”和“客体”两列的二元关系, 除数必须是“全部主体”的单列关系。除法结果中的每个值对应于被除数中在除数列上“完整覆盖”的所有分组。

考生容易将除法的左右参数搞反——切记是“被除数 ÷ 除数”，除数确定了“全部”的范围。

除法运算可以与选择、投影、连接灵活组合使用，是表达复杂查询条件（如“选修了所有课程的学生”、“被所有供应商供应的零件”）的通用范式。知识点：2.4 关系代数——除法运算处理“所有/全部”语义；被除数与除数的构造规则；投影与连接的组合使用。

2. 用 SQL 语句表示下列查询

参考答案

（答案由解析者补充）

```
1 SELECT C.Cname, E.Ename, E.Sex
2 FROM C JOIN E ON C.Manager = E.Eno;
```

解析与知识点

社团表 C 中 Manager 字段存储的是负责人的职工号，职工表 E 中存储职工详细信息。二者在 C.Manager 和 E.Eno 上做等值连接即可获取负责人的名字和性别。若使用隐式连接写法等价于 FROM C, E WHERE C.Manager = E.Eno。

由于每名社团负责人必然在职（参照完整性约束），内连接足以满足需求。这道题考察的是 SQL 中最基础的多表连接查询，核心在于识别连接条件——外码 Manager 参照主码 Eno。在 SQL 中，多表连接分为内连接（INNER JOIN）、外连接（LEFT/RIGHT/FULL OUTER JOIN）和交叉连接（CROSS JOIN）三种。内连接只返回两表中满足连接条件的行，在参照完整性约束的保证下不会丢失数据。如果某些社团的负责人字段为 NULL（表示暂时空缺），则需要使用外连接以保留这些社团的信息。此外，在 SELECT 子句中明确列出需要的列名（而非使用 SELECT *）是一种良好的编程习惯，有助于提高查询可读性和可维护性。知识点：3.3 SQL 多表连接查询（JOIN ON 与隐式连接）；连接条件中外码与主码的对应关系；内连接与外连接的选择依据。

参考答案

（原答案提供）

```
1 SELECT Ename
2 FROM E
3 WHERE Eno IN (SELECT Eno FROM EC WHERE Cno = 'C1')
```



```
4 AND Eno IN (SELECT Eno FROM EC WHERE Cno = 'C2');
```

解析与知识点

”至少参加 C1 和 C2”意味着该职工号在 EC 表中既要有 Cno = 'C1' 的记录，也要有 Cno = 'C2' 的记录。使用两个 IN 子查询并取逻辑与（AND）联接即可表达该条件。

等价写法也可用交集：查找参加了 C1 的职工号与参加了 C2 的职工号取交集，或者使用自连接将 EC 表在工号上连接，筛选两端社团号分别为 C1 和 C2 的行。

这道题的关键在于理解”至少同时满足两个条件”与”满足其中之一”在 SQL 中的表达差异——前者需要逻辑与 (AND) 或交集 (INTERSECT)，后者需要逻辑或 (OR) 或并集 (UNION)。

常见错误是使用 WHERE Cno = 'C1' OR Cno = 'C2',这会选出参加了 C1 或 C2 任一社团的职工(包括只参加了一方的),不符合题意。另一种错误是使用单条 IN 子查询: WHERE Cno IN ('C1', 'C2'),效果同样等价于 OR。

正确的处理方式是利用嵌套子查询的交集语义或自连接的分组筛选。在 SQL 中表达“至少满足全部 N 个条件”的模式通常需要 N 次子查询的交集，或者用 GROUP BY + HAVING COUNT 的方式实现（前提是每个职工在每个社团最多有一条记录）。知识点： 3.4 SQL 嵌套子查询与 IN 操作符；交查询逻辑 (AND 联接多个条件与 INTERSECT)；“至少满足全部”语义的表达模式。

参考答案

(原答案提供)

[illegible]

解析与知识点

查询分三步：最内层子查询通过姓名”李明”从职工表中找到其职工号；中间层子查询根据该职工号从社团表中找到其所负责的社团号；外层查询将职工表 E 和参加表 EC 在职工号上连接，用 IN 筛选出属于”李明负责的社团”的记录，按社团号分组并用 AVG() 聚合函数计算每个社团的平均年龄。注意 GROUP BY Cno 是必需的，因为 AVG() 是分组聚合函数。

这道题展示的是多层嵌套子查询与聚合查询的组合使用，是从业务问题到 SQL 语句的典型转换路径。

多层嵌套子查询的解析按照”由内向外”的顺序进行：最内层先执行，结果传递给外层，以此类推。

在聚合函数的使用中，SELECT 子句中出现的非聚合列必须包含在 GROUP BY 子句中，否则会触发语法错误。AVG() 函数在计算平均值时会自动忽略 NULL 值，对于整型列的除法结果可能是小数。

AS 关键字用于给计算结果列起别名，使得输出更加清晰。整体查询逻辑体现了 SQL 中”筛选-连接-分组-聚合-排序”的标准处理流程。知识点：3.4 嵌套子查询（多层 IN 与相关子查询）；3.5 聚合函数 AVG() 与 GROUP BY 分组查询；非聚合列在 GROUP BY 中的要求；NULL 值在聚合中的行为。

参考答案

（答案由解析者补充）

```
1 SELECT DISTINCT E.Eno, E.Ename
2 FROM E JOIN EC ON E.Eno = EC.Eno
3 WHERE E.Eno NOT IN (SELECT Eno
4                      FROM EC
5                      WHERE DateOfAttend >= '2012-01-01');
```

或等价的 NOT EXISTS 写法：

```
1 SELECT DISTINCT E.Eno, E.Ename
2 FROM E, EC
3 WHERE E.Eno = EC.Eno
4       AND NOT EXISTS (SELECT *
5                       FROM EC AS EC2
6                       WHERE EC2.Eno = E.Eno
7                       AND EC2.DateOfAttend >= '2012-01-01');
```

解析与知识点

”参加的每个社团的参加日期都在 2012-1-1 以前”等价于”不存在某个社团,其参加日期在 2012-1-1 或之后”。使用 NOT IN 子查询找出那些在 EC 表中有 DateOfAttend $\geq$ '2012-01-01' 记录的职工号,再从外层排除这些职工,剩下的即为全部参加日期均早于 2012-1-1 的职工。

使用 DISTINCT 避免因一个职工参加多个社团而产生重复行。

NOT EXISTS 写法语义相同,利用相关子查询逐行检查。这道题的核心技巧在于全称量词”所有/每个”在 SQL 中的双重否定转换:”所有参加日期都在某日之前”等价于”不存在任何参加日期在某日或之后”。

这是离散数学中谓词逻辑在 SQL 中的应用—— $\forall x P(x) \equiv \neg \exists x \neg P(x)$ 。

NOT IN 的写法简洁但在子查询结果包含 NULL 时可能产生意外行为(整个 NOT IN 结果为空),因此更安全的是 NOT EXISTS 写法。相关子查询中的变量绑定 (EC2. Eno = E.

Eno)使得内层查询在每一行外层数据上重新执行,实现了逐行检查的效果。这种双表别名加 NOT EXISTS 的模式是 SQL 中表达全称量词的标准范式。知识点:

3.4 SQL 嵌套子查询 (NOT IN、NOT EXISTS 相关子查询); 全称量词”所有/每个”在 SQL 中转化为 NOT EXISTS 的双重否定模式; DISTINCT 的消重作用。

3.4 四、数据库分析、设计 (共 38 分)

1. 关系模式规范化

参考答案

(原答案提供)

候选码: $ABCD$ 。

$R \in 1NF$, 因为 R 中存在非主属性部分函数依赖于候选码。

解析与知识点

通过闭包求解法:从 $ABCD$ 出发, $B \rightarrow PI$ 得 $ABCDPI$, $AB \rightarrow E$ 得 $ABCDEPI$, $ABE \rightarrow GP$ 得 $ABCDEGPI$, $C \rightarrow J$ 得全属性集, 故 $(ABCD)_F^+ = U$, $ABCD$ 为候选码。

由于 $B \rightarrow PI$ 中 B 是候选码 $ABCD$ 的一部分(真子集), 存在非主属性 (P 、 I) 部分函数依赖于候选码, 因此不满足 2NF, 只能属于 1NF。

属性闭包算法是求解候选码的标准方法, 其核心是迭代地将能由已有属性集推导出的属性加入闭包中, 直到不再增加或已达到全属性集。

一个关系模式的候选码可能不止一个, 但求得的第一个闭包达到全集的属性集即

可确认为候选码。部分函数依赖是指非主属性依赖于候选码的真子集,这是 2NF 要消除的冗余依赖类型。

判断范式等级的步骤是:先求候选码、再列出所有依赖、检查非主属性是否完全函数依赖于候选码(2NF 判定)、检查是否存在非主属性对候选码的传递依赖(3NF 判定)、检查所有非平凡依赖的决定因素是否都是超码(BCNF 判定)。

这个自底向上的判定流程是规范化分析的基本方法论。知识点: 6.3 候选码求解(属性闭包算法——迭代推导); 6.2 部分函数依赖的定义与识别; 6.4 1NF/2NF/3NF/BCNF 判定准则与检验流程。

参考答案

(原答案提供)

$$F_m = \{AB \rightarrow E, AB \rightarrow G, B \rightarrow P, B \rightarrow I, C \rightarrow J, C \rightarrow I, G \rightarrow H\}$$

解析与知识点

求最小依赖集的步骤:(a) 将所有依赖右部单属性化: $AB \rightarrow E$ (不变), $ABE \rightarrow GP$ 拆为 $ABE \rightarrow G$ 和 $ABE \rightarrow P$, $B \rightarrow PI$ 拆为 $B \rightarrow P$ 和 $B \rightarrow I$, $C \rightarrow J$ (不变), $CJ \rightarrow I$ (不变), $G \rightarrow H$ (不变);(b) 逐个检查左部冗余属性: $ABE \rightarrow G$ 中 E 冗余(因 $AB \rightarrow E$ 且 AB 可决定 G), $ABE \rightarrow P$ 中 E 和 P 均可去掉($B \rightarrow P$ 已存在), $CJ \rightarrow I$ 中 J 冗余($C \rightarrow J$ 且 C 可决定 I);(c) 去掉冗余依赖: $ABE \rightarrow P$ 可由 $B \rightarrow P$ 推出而冗余, $ABE \rightarrow G$ 简化为 $AB \rightarrow G$, $CJ \rightarrow I$ 简化为 $C \rightarrow I$ 。最终得 7 个最小依赖。

最小函数依赖集是模式分解的前提——它消除了原始依赖集中的冗余信息,使得每个依赖都是“最小”的(右部单属性、左部无冗余属性、无冗余依赖)。

左部冗余属性的检测方法是:对于依赖 $X \rightarrow Y$,若 X 中存在属性 A 使 $(X - \{A\})_F^+$ 包含 Y ,则 A 在左部冗余。冗余依赖的检测方法是:对于依赖 f ,若 $F - \{f\}$ 的逻辑闭包仍包含 f ,则 f 冗余。

Armstrong 公理体系(自反律、增广律、传递律)是判断依赖蕴含关系的理论工具,而最小覆盖的求解过程本质上是对依赖集应用这些公理进行等价化简。知识

点: 6.3 最小函数依赖集求解算法(右部单属性化、去除左部冗余属性、去除冗余依赖); Armstrong 公理推理体系; 依赖集等价与最小覆盖概念。

参考答案

(原答案提供)

$$\rho = \{ABEG, BPI, CJI, GH, ABCD\}$$

解析与知识点

3NF 分解算法：将 F_m 中的依赖按相同左部分组： $AB \rightarrow E$ 、 $AB \rightarrow G$ 合并为关系 $ABEG$ （含 ABE 得 $ABEG$ ，码 AB ）； $B \rightarrow P$ 、 $B \rightarrow I$ 合并为 BPI （码 B ）； $C \rightarrow J$ 、 $C \rightarrow I$ 合并为 CJI （码 C ）； $G \rightarrow H$ 单独为 GH （码 G ）。最后加入关系 $ABCD$ （包含候选码 $ABCD$ ）以保证无损连接性。

该分解保持了所有函数依赖，且每个子关系均满足 3NF。

3NF 分解算法有两个目标：保持无损连接性和保持函数依赖。

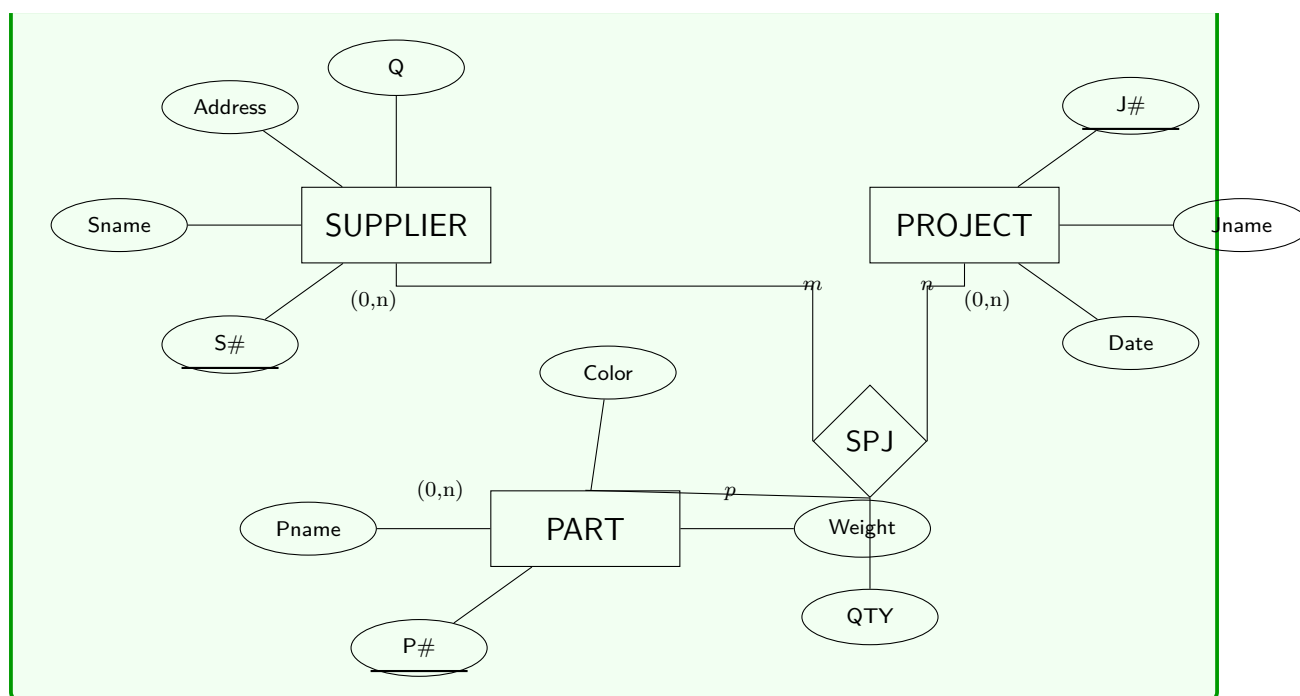
按左部等价类分组保证了依赖保持性，而额外添加候选码关系则保证了无损连接性（任何分解都可以通过补充一个包含原始候选码的子关系来确保无损连接性）。注意 CJI 中 C 决定 I 而 C 又是该子关系的码，故满足 BCNF； $ABEG$ 中 AB 是码且决定 E 和 G ，也满足 BCNF。

但并非所有 3NF 分解都能达到 BCNF，有时由于依赖保持性的要求，不得不保留某些不满足 BCNF 的子关系。这种权衡正是规范化理论与实践应用的结合点——理论上 BCNF 更优，但实际中 3NF 加依赖保持往往更实用。知识点：6.4 3NF 分解算法（保持无损连接与依赖保持性）；按左部等价类分组策略；候选码关系对无损连接性的保障作用；3NF 与 BCNF 的实践权衡。

2. E-R 图设计

参考答案

基本 E-R 图如下（以 TikZ 绘制）：



解析与知识点

本设计涉及三个实体 (SUPPLIER、PROJECT、PART) 和一个三元联系 SPJ。三元联系表示“供应商为工程项目提供零件”这一业务事实，每个实体对之间的多对多关系通过三元联系统一表达。

QTY 是联系 SPJ 的描述属性 (既不属于任何一个实体，而属于多对多联系的附加信息)。

供应商的 Q 属性是供应商本身的属性 (供货总量)，属于实体属性的范畴。

使用 TikZ 绘制时按 entity/relationship/attribute 样式统一渲染。在 E-R 建模中，判断某个属性属于实体还是联系是关键决策：如果属性的值由参与联系的所有实体共同决定，则它属于联系 (描述属性)；如果只由单一实体标识，则属于实体。

三元联系和三个二元联系的语义区别在于：三个二元联系无法表达“某供应商供应某零件给某项目”的三方关联完整性约束 (如供应量 QTY 由三方共同确定)，而三元联系天然支持这种约束。

考生常犯的错误是将三元联系错误地分解为三个二元联系，这会导致语义丢失——例如单独记录“供应”、“提供”、“承担”三个二元关系后，无法重建“哪个供应商向哪个项目提供了哪个零件”的完整信息。此外，基数的标注也有要求在本设计中三个实体均以 (0,n) 参与三元联系，表示允许存在不参与 SPJ 关联的独立实体。知识点：7.2 E-R 模型设计 (实体、联系、属性识别)；三元联系与二元联系的语义对比；联系的描述属性与实体属性的区分；TikZ 绘图基础知识。

参考答案

(原答案提供)

- PART(P#, Pname, Color, Weight)
- PROJECT(J#, Jname, Date)
- SUPPLIER(S#, Sname, Address)
- SPJ(S#, P#, J#, QTY)

其中 SPJ 关系的主码由三个实体的主码联合构成 (S#, P#, J#), QTY 为联系的描述属性; SPJ 中 S#、P#、J# 同时为外码, 分别参照 SUPPLIER、PART、PROJECT 的主码。

解析与知识点

转换规则: 每个实体转换为一个独立关系模式, 实体的属性即为关系的属性, 实体的主码即为关系的主码; 三元联系 SPJ 转换为一个独立关系, 其属性包括所有参与实体的主码 (S#、P#、J#) 以及联系自身的描述属性 (QTY), 该关系的主码为三个外码的组合。

特别注意, 由于三个实体均为多对多参与三元联系, 转换后的 SPJ 关系的主码必须包含全部三个实体的主码属性。

在 E-R 图向关系模式的转换中, 联系的映射方式由联系的基数类型决定: 对于 1:1 联系, 可与任一端实体关系合并; 对于 1:N 联系, 与 N 端实体关系合并 (添加 1 端的主码作为外码); 对于 M:N 联系 (包括三元多对多), 必须转为独立的关系模式。

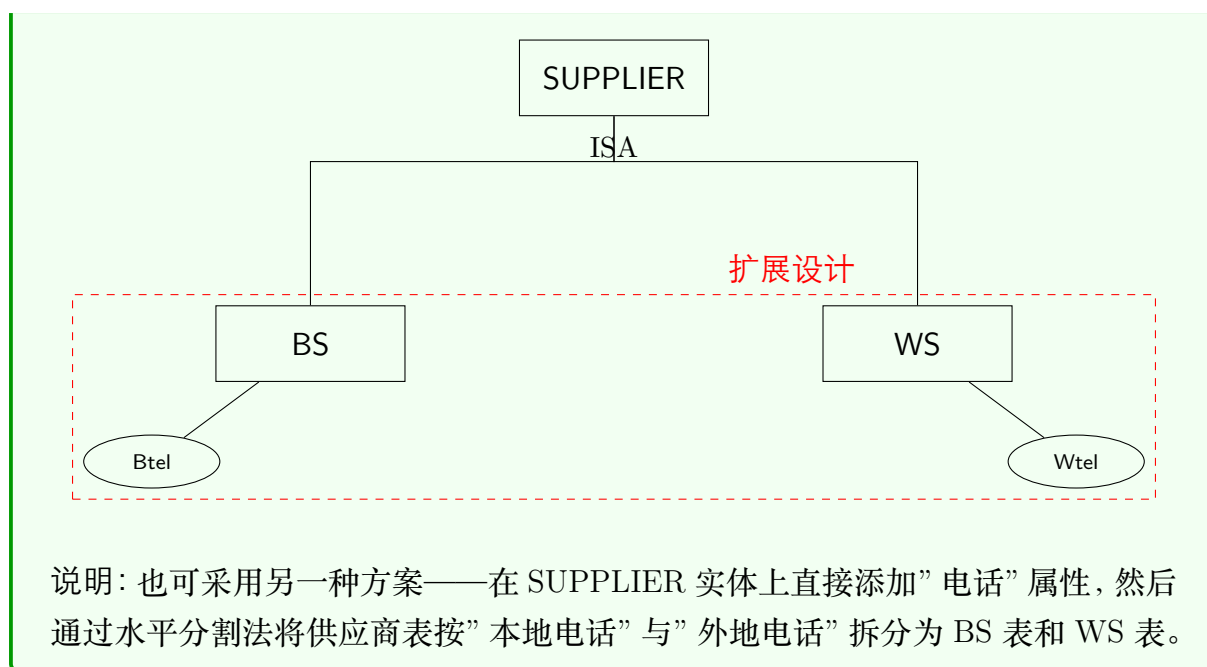
此处三元联系的三端都是多对多参与, 因此 SPJ 必须独立为一个关系模式。三个外码在 SPJ 中既是主码的一部分又是外码, 体现了主码与外码的角色重叠——这在联系关系中是常见情况。

除主码外码外, 每条 SPJ 元组还包含 QTY (供应数量), 这是联系特有的描述属性, 存储的是“某供应商向某项目供应某零件的数量”这一业务数据。知识点:

7.3 E-R 图向关系模式转换规则; 实体转换 (一对一); 三元多对多联系转换 (独立关系); 外码与主码的组合定义; 联系基数对转换策略的影响。

参考答案

在基本 E-R 图上, 通过子类 (特殊化) 方法扩展: SUPPLIER 作为超类, BS (本地供应商) 和 WS (外地供应商) 作为子类。扩展部分 E-R 图如下:



解析与知识点

原答案给出了两种扩展方法：(1) 子类/超类（ISA）建模——SUPPLIER 作为超类，BS 和 WS 作为子类，各自附加特有属性 Btel 和 Wtel，子类继承超类的所有属性；(2) 水平分割法——在 SUPPLIER 关系中增加电话属性，然后按本地/外地电话将数据水平分割为两张表。E-R 图中使用 ISA 联系（带三角形的超类-子类连线）表示继承关系更为直观和符合建模规范，是数据库设计中处理实体分类的经典方法。

ISA 联系（也称为特殊化/泛化）是扩展 E-R 模型的核心机制，表达了分类层次（如供应商分为本地供应商和外地供应商）和继承语义（子类自动拥有超类的所有属性和联系）。

ISA 联系可以是互斥的（一个供应商要么是本地要么是外地）或重叠的、全部的（每个供应商必须是某个子类的成员）或部分的（有些供应商不归属任何子类）。本题中 BS 和 WS 显然是互斥且完全的划分。转换为关系模式时，子类/超类有两种常见策略：一是所有子类放在同一张表中（用类型字段区分），二是每个子类各自一张表（水平分割法的本质）。

选择哪种策略取决于子类特有属性的差异性、查询模式和数据库性能需求。掌握 ISA 建模对于设计复杂的分类层次结构（如人员可分为教师、学生、职员，再进一步细分）至关重要。知识点：7.2 E-R 模型扩展（超类/子类 ISA 联系、特殊化/泛化机制）；属性继承语义；水平分割法与子类表的转换策略；互斥/重叠、全部/部分的分类约束。

3.5 五、简答题（共 2 小题，每小题 6 分，共 12 分）

参考答案

（答案由解析者补充，原答案未提供）

解析与知识点

聚集索引与非聚集索引的区别：(1) 聚集索引决定表中数据行的物理存储顺序，索引的叶节点就是实际的数据页，因此一张表只能有一个聚集索引；非聚集索引的叶节点存储的是指向数据行的指针（或聚集索引键），物理顺序与索引顺序无关，一张表可以有多个非聚集索引。(2) 聚集索引在范围查询和排序操作上效率极高（因为数据物理相邻），但插入/更新可能导致数据行物理移动（页分裂）；非聚集索引在精确查找和连接操作上效果较好。

(3) 建表时若定义了主键，默认在主键上建立聚集索引。

聚集索引的底层结构通常是 B+ 树，叶子节点按索引键顺序存储全部数据行，因此范围扫描（如 BETWEEN 或 ORDER BY）时磁盘 I/O 是顺序的。

非聚集索引的叶子节点存储的是索引键值加上指向数据行的书签（Bookmark，若存在聚集索引则为聚集索引键，否则为物理行标识符 RID），因此通过非聚集索引找到索引条目后，还需要一次书签查找（Key Lookup）才能获取完整数据行。建立索引的一般原则：(a) 在 WHERE 子句中频繁使用的列上建立索引；(b) 在 JOIN 连接条件的列（通常为外码）上建立索引；(c) 在 ORDER BY、GROUP BY 涉及的列上建立索引；(d) 选择列值分布较分散（基数高）的列建索引，如性别等低基数列不适合建索引；(e) 避免在频繁更新的列上建立过多索引，以降低维护代价；(f) 权衡查询需求与系统开销，合理控制索引数量。

索引的选择性（Selectivity）——即不同值的数量占总行数的比例——是决定索引效果的关键因素：高选择性的列（如主键、身份证号）索引效率极高，低选择性的列（如性别只有男/女两个值）索引几乎无用。复合索引（多列联合索引）遵循最左前缀原则，列的顺序应按照查询条件中的使用频率和选择性高低来排列。

此外，覆盖索引（Covering Index）——索引包含了查询所需的所有列——可以避免回表操作，是重要的查询优化手段。

在实际数据库运维中，定期分析和重建索引碎片也是保证索引效率的重要工作。

知识点：3.2 索引（聚集索引与非聚集索引的 B+ 树结构与物理存储区别）；索引建立的一般原则（选择性、基数、查询频率、更新开销的权衡）；复合索引的最左前缀原则与覆盖索引优化技术。

参考答案

（答案由解析者补充，原答案未提供）

解析与知识点

主要故障类型：(1) 事务内部故障（事务故障）：事务在执行过程中因逻辑错误、违反完整性约束、死锁被选中撤销等原因未能正常完成，需要通过事务回滚（UNDO）恢复到事务开始前的状态。这类故障的影响范围仅限于单个事务本身，DBMS 通过读取该事务在日志中的 UNDO 记录（旧值/Before Image）来撤销其所有修改。(2) 系统故障（软故障）：因断电、操作系统崩溃等导致内存数据丢失，已提交事务的修改可能未写入磁盘（需 REDO），未提交事务的修改可能已写入磁盘（需 UNDO）。

系统故障的核心特征是磁盘数据未被破坏（数据文件完好），仅仅是内存中的缓冲数据丢失，因此恢复过程需要基于日志重建内存状态。(3) 介质故障（硬故障）：磁盘损坏导致数据库数据永久丢失，是最严重的故障类型。

硬盘物理坏道、磁头损坏等都会导致部分或全部数据文件不可读，恢复只能依赖后备副本和日志的组合。

恢复技术和方法：(a) 数据转储（备份）：定期将数据库完整或增量地备份到其他存储介质上，是介质故障恢复的基础。完整转储备份全部数据，增量转储只备份自上次转储以来的变化数据，前者恢复快但备份慢，后者相反。(b) 日志文件（Log）：记录事务对数据库的所有更新操作（包括事务标识、操作类型、修改前的旧值 Before Image 和修改后的新值 After Image），是事务故障和系统故障恢复的核心依据，遵循“先写日志后写数据”（WAL, Write-Ahead Logging）原则——在对数据页进行实际修改之前，必须先将对应的日志记录写入稳定存储。

(c) 检查点（Checkpoint）：定期将日志缓冲区和数据缓冲区的内容强制写入磁盘，记录当前活跃事务列表，系统故障恢复时只需从最近的检查点开始扫描日志，大幅缩短恢复时间。(d) 数据库镜像：通过实时复制保持数据库的冗余副本（主数据库与镜像数据库），介质故障时可快速切换到镜像数据库，提供最高级别的可用性保障。上述技术形成了从预防（镜像）到检测（检查点）到恢复（转储 + 日志）的完整容灾体系，是现代数据库高可用性架构的基石。知识点：11.1 事务故障类型（事务内部故障、系统故障、介质故障）的特征与影响范围；11.2 数据转储与恢复（完整转储与增量转储）；11.3 日志文件与 WAL 原则（先写日志后写数据）；11.4 检查点机制（缩短恢复时间）；11.5 数据库镜像与高可用性。

4 2022–2023 学年《数据库系统》期末试卷 (A 卷)

本卷为武汉大学国家网络安全学院 2022–2023 学年第 2 学期《数据库系统》期末考试 A 卷（闭卷）。试卷注重数据库核心理论基础与系统级概念，涵盖数据独立性、完整性约束、SQL 编程、关系代数、E-R 建模、规范化理论、事务并发控制和数据库恢复等主题。

4.1 一、单项选择题（共 10 小题，每小题 2 分，共 20 分）

参考答案

C

解析与知识点

本题考察物理数据独立性（Physical Data Independence）的概念辨析，要求区分物理独立性与逻辑独立性、系统模块独立性的本质差异。物理独立性是指当数据库的物理存储结构（如索引结构、文件组织方式、存储设备等）发生变化时，应用程序不受影响，即用户应用程序与磁盘上物理存储的数据之间是解耦的。

解题关键是以“物理独立性对应存储层面、逻辑独立性对应逻辑结构层面”为判断基准逐一排除错误选项。选项 A、B 描述的是系统模块间的独立性，并非数据独立性的核心含义；选项 D 描述的是逻辑独立性（Logical Data Independence），而非物理独立性。数据库三级模式结构中的“内模式/物理模式”与外模式之间的映射正是通过 DBMS 实现这种物理独立性。

数据库系统的三级模式——外模式（用户视图）、概念模式（全局逻辑结构）和内模式（物理存储结构）——通过两级映像机制保障了数据独立性。概念模式与内模式之间的映像实现了物理独立性，外模式与概念模式之间的映像实现了逻辑独立性。这种层次抽象设计是数据库管理系统区别于文件系统的根本特征之一，使得数据库应用可以独立于底层硬件和存储组织的变更而长期稳定运行。

考生在区分物理独立性与逻辑独立性时，关键记忆点是：物理独立性对应的是存储层面的变化（硬件、索引、文件组织），逻辑独立性对应的是全局逻辑结构的变化（增加新字段、修改字段类型等）。在实际的数据库运维中，DBA 可以对索引进行添加、删除或重建而无需修改任何应用程序代码，这正是物理独立性的具体体现。

知识点：1.2 数据库系统的三级模式结构（外模式、概念模式、内模式）与两级映像机制；物理数据独立性与逻辑数据独立性的区别与联系。

参考答案

C

解析与知识点

本题考察数据库完整性约束（Integrity Constraints）的核心概念与分类，要求区分完整性约束与数据库安全性措施的本质边界。完整性约束用于保证数据库中数据的正确性和相容性，主要包括：(1) 实体完整性（主码约束，Primary Key）；(2) 参照完整性（外码约束，Foreign Key）；(3) 用户定义完整性（如数据类型约束、

CHECK 约束等)。

解题关键是以“完整性保障数据正确性、安全性防止非法访问”为判别准则。数据加密属于数据库安全性 (Security) 的范畴,其目的是防止未经授权的访问和数据泄露,而非保证数据自身的正确性。因此选项 C 不属于完整性约束。数据库的“安全”与“完整”是两个既相关又独立的概念:安全性保护数据免受非法用户的恶意破坏和泄露(通过存取控制、审计、加密等手段),完整性保护数据免受合法用户的非恶意错误(通过约束、触发器、断言等手段)。

数据加密是防御外部攻击的典型安全措施,将明文数据转换为不可读的密文,保障数据传输和存储过程中的机密性。实体完整性通过主码的非空、唯一特性确保每行可被唯一标识;参照完整性通过外码的取值约束确保关联数据的一致性;用户定义完整性则允许设计者根据业务规则自定义约束条件。三者共同构成了关系数据库的数据质量保障体系。

考生常见的混淆点是将所有“保护数据”的机制都归为完整性约束,忽略了安全性和完整性的本质区别——安全性针对“人”,完整性针对“数据本身”。在考试中遇到“加密”“审计”“权限控制”等关键词,应立即判断其属于安全性范畴而非完整性范畴。

知识点: 5.1 数据库完整性概述(实体完整性、参照完整性、用户定义完整性); 4.1 数据库安全性概述(存取控制、加密、审计); 安全性与完整性的概念辨析。

参考答案

C

解析与知识点

本题考察自然连接 (Natural Join) 运算的结果规模分析,包括自然连接的列数计算公式和元组数量的上下界估计。自然连接基于公共属性进行等值连接并消除重复列,其结果的列数和元组数都有确定的数学公式约束。

自然连接基于公共属性 B, C 进行等值连接并消除重复列,结果的列数为 $3+3-2=4$ (即 A, B, C, D), 选项 A 的 6 列错误。元组数量最多情况: r 中所有元组的 (B, C) 值相同,且 s 中所有元组的 (B, C) 值也相同,此时形成笛卡尔积效应,最多 $N_1 \times N_2$ 个元组。最少情况: 若 r 和 s 在公共属性 (B, C) 上无任何匹配值,则自然连接结果可为 0 个元组,不存在“最少 N_2 个”的保证。因此选项 C 正确。

这道题的难点在于同时考察了自然连接的列数变化和元组数量的上下界。自然连接的核心操作步骤是: (1) 找出两个关系的公共属性 (相同属性名); (2) 在这些公共属性上做等值连接; (3) 去掉重复的公共属性列。在计算列数时,公式为 $|R_{cols}| + |S_{cols}| - |common_cols|$ 。在估算元组数量上下界时,需要考虑公共属性值的分布情况——最坏情况下每个元组的公共属性值都相同,退化为笛卡尔积;最

好情况下完全无匹配，结果为 0。掌握自然连接的结果规模估算对于查询优化中的代价估计非常重要，数据库的查询优化器正是通过类似的推理来评估不同执行计划的代价。

考生常犯的错误是忘记去除重复列或将结果列数简单相加，以及误以为自然连接的元组数量下界至少为某一方关系的元组数。实际上自然连接的元组数范围是 $[0, N_1 \times N_2]$ ，下界为零（无匹配时可能为空）。

知识点：2.4 关系代数——自然连接运算的定义、列数计算公式、元组数量的上下界估计；查询优化中的结果规模估算。

参考答案

C

解析与知识点

本题考察关系代数中除法运算 (Division) 的语义理解和具体应用。除法运算专门用于回答“哪些对象满足全部某类条件”（全称量词）的查询，是关系代数中最抽象但也最强大的运算。

除法运算 $r_1 \div r_2$ 的含义是：找出 r_1 中那些在 r_2 的公共属性上与 r_2 全部元组都匹配的元组（在其余属性上的投影）。 r_1 包含（学院，课程）， r_2 包含（课程），公共属性为“课程”。 $r_1 \div r_2$ 即查询“选修了 r_2 中所有课程的学院”。信息学院同时拥有“数据结构与算法”和“数据库系统”两门课程，满足全部匹配，而历史学院和文学院各自只有一门不相关的课程，不满足条件。因此结果为 {信息学院}。注意 $r_2 \div r_1$ 的定义域不匹配（除关系的属性集必须是被除关系属性集的真子集），选项 D 无意义。

除法运算的运算过程可以理解为：从被除数中按非公共属性列分组，保留那些在每个分组中公共属性列的取值集完全包含除数关系公共属性列全部取值元组的分组。在数学上， $R \div S = \pi_{R-S}(R) - \pi_{R-S}((\pi_{R-S}(R) \times S) - R)$ ，这个等价的非除法定义揭示了除法的本质——通过差集、笛卡尔积和投影的组合来表达“全部”语义。除法运算在选课系统（“选修了所有课程的学生”）、供应链系统（“被所有供应商供应的零件”）等场景中频繁出现，是关系代数表达完备性的重要组成部分。

考生在除法运算中常见的错误包括：混淆除数与被除数的角色（ $r_1 \div r_2$ 与 $r_2 \div r_1$ 语义完全不同），以及忽略除法对属性集包含关系的要求（除关系的属性集必须是被除关系属性集的真子集）。解题时应先明确公共属性，再逐一分组验证是否“包含全部”。

知识点：2.4 关系代数——除法运算的语义定义（全称量词表达）；除法的非除法定义；除法在“全部/所有”查询场景中的应用模式。

参考答案

A

解析与知识点

本题考察 SQL INSERT 语句中列缺省值的处理规则, 涉及 DEFAULT 约束、NULL 约束与 AUTO_INCREMENT (自动编号) 列的交互机制。重点考察当 INSERT 语句未显式指定某列的值时, 系统如何根据列的约束定义自动填充该列。

INSERT 语句中未指定 itemtype 字段, 且该字段无 DEFAULT 约束、允许为 NULL, 因此该列被自动赋予 NULL 值。itemid 是自动编号字段 (AUTO_INCREMENT), 由系统自动生成编号值, 无需在 INSERT 中指定。该插入语句仅指定了两个列 (unitprice 和 itemcount), 其余未指定的可空列均取 NULL。插入操作本身是合法且能成功执行的, 不会因为列缺省而导致失败。

这道题考察的是 SQL INSERT 语句中列缺省值的处理规则: 如果列有 DEFAULT 约束, 则使用默认值; 如果列无 DEFAULT 但允许 NULL, 则自动填充 NULL; 如果列既无 DEFAULT 又不允许 NULL, 则必须在 INSERT 中显式提供值, 否则插入失败。AUTO_INCREMENT (或 IDENTITY/序列) 列是特殊的列类型, 系统在每次插入时自动生成递增的唯一值, 用户通常不在 INSERT 语句中为它指定值 (指定也可以, 但不推荐, 因为可能破坏自增序列的连续性)。掌握列的缺省行为对于正确编写 INSERT 语句至关重要, 也需要理解 NOT NULL 约束和 DEFAULT 约束的交互作用。

考生常见的错误是认为“未指定列的 INSERT 一定会失败”或混淆 DEFAULT 和 NULL 的行为。实际上, 只有当列既无 DEFAULT 又不允许 NULL 时, INSERT 才会因缺省而失败; 允许 NULL 的列在缺省时会被自动填充为 NULL, 这是完全合法的操作。

知识点: 3.5 SQL 数据更新——INSERT 语句中列缺省值的处理规则; 3.1 SQL 数据定义——DEFAULT 约束、NULL 约束与 AUTO_INCREMENT 的交互机制。

参考答案

D

解析与知识点

本题考察数据库索引的全方位特性, 包括索引对查询速度的提升作用、对增删改操作的影响、存储空间开销及维护需求。重点考察对索引优劣的全面理解, 而非仅停留在“索引加速查询”的表面认知。

索引是一种加快数据查询 (SELECT) 速度的物理存储结构, 选项 A 正确。创建索引和在对数据进行增/删/改操作时维护索引都需要额外的时间开销, 选项 B 正

确。索引本身需要额外的物理存储空间来存储索引项（如 B+ 树的节点数据），选项 C 正确。然而，索引会降低增/删/改（INSERT、DELETE、UPDATE）的执行效率，因为每次数据变更都需同步更新相关索引结构。选项 D 的说法正好与事实相反，故不正确。这道题考察的是索引的全方位特性理解，而非仅知道“索引加速查询”这个常识。

索引的性能影响具有非对称性：对查询是正面的（减少磁盘 I/O 和比较次数），对修改是负面的（每次修改需要更新索引树结构）。在 B+ 树索引中，插入可能导致节点分裂（Node Split），删除可能导致节点合并（Node Merge），这些都是时间开销较大的操作。在工程实践中，DBA 需要在查询性能和修改性能之间权衡，过度的索引化（Over-Indexing）是导致数据库写入性能下降的常见原因。

此外，索引也需要定期维护（如重建索引、更新统计信息）以保持其效率，因为频繁的增删改会导致索引碎片化（Fragmentation）。考生常误以为索引只会带来正面效果，忽略了其对写入性能的负面影响和维护需求。

知识点： 3.4 SQL 索引——索引的概念、优缺点与性能权衡；B+ 树索引的节点分裂与合并机制；索引碎片化与维护策略；查询优化与物理存储的关联。

参考答案

C

解析与知识点

本题综合考察数据库物理设计中的多个关键决策点，包括规范化的适度性、BCNF 的目标设定、聚簇（Clustering）的物理优化效果以及索引数量的权衡。涉及从逻辑设计到物理设计的全流程优化思维。

规范化并非程度越高越好——过度规范化会导致过多的表连接操作，严重影响查询性能，有时需要适当反规范化（Denormalization）以平衡性能，故选项 A 错误。逻辑设计阶段的目标通常是达到 3NF 或 BCNF，但并非绝对要求满足 BCNF；某些实际场景下可能因性能考虑而不分解到 BCNF，故选项 B 错误。聚簇（Clustering）将具有相同或相近属性值的元组物理上存储在相邻位置，可极大提高基于该属性范围查询的效率，选项 C 正确。索引并非越多越好：每个额外索引都会增加存储开销并降低增/删/改操作的速度，需要根据实际查询需求权衡，故选项 D 错误。反规范化是一种有意的冗余设计——将已经规范化的表格进行合并或添加冗余列，以用空间换取查询性能。聚簇的设计思想是将经常一起被访问的数据物理上放在临近的磁盘块中，减少随机 I/O。例如，将同一部门的员工记录物理上聚集在一起，查询部门员工时只需顺序读取连续的磁盘块。聚簇索引（Clustered Index）和非聚簇索引（Non-Clustered Index）的区别也是物理设计的重要决策点。

数据库设计是一个从需求分析、概念设计（E-R 图）、逻辑设计（规范化）到物理设计（索引、聚簇、分区）的完整流程，每个阶段都有各自的优化目标和约束条件。考生易犯的错误是将规范化追求到极致而忽略性能代价，或者在物理设计中盲目添加索引和聚簇而不权衡维护成本。

知识点： 7.4 数据库物理结构设计（聚簇的概念与范围查询优化）； 7.3 数据库逻辑结构设计（反规范化与性能权衡）； 6.2 规范化——范式的权衡（过度规范化的代价）。

参考答案

D

解析与知识点

本题考察第一范式（1NF）的原子性要求及其判定方法。1NF 要求关系的每一个分量都是不可再分的原子数据项，即不允许“表中套表”，是所有关系模式必须满足的最低范式要求。

选项 D 中的关系模式 R_4 在“电话”属性中嵌套了一个子表结构 {办公室电话, 手机号码}，使得“电话”列的值不是原子性的，违背了 1NF 的基本要求。选项 A、B、C 的所有属性值都是单一标量值，均满足 1NF。选项 B 将两个电话号码拆分为独立的原子属性，这正是规范化到 1NF 的正确做法。

1NF 是关系模型定义的基础——关系的每个属性值必须是原子值（不可分割的数据单元）。原子性的判断标准与具体的应用场景有关：在某个场景中被视为原子的数据，在另一场景中可能需要拆分。例如“地址”字段在某些场景作为单个字符串即可，但在需要按省/市/区进行统计时则需要拆分为多个属性。常见的 1NF 违例包括：属性中嵌套集合（A, B, C）、属性中嵌套表结构、一行中有“列 1_1, 列 1_2, 列 1_3”这样的重复组。将 1NF 违例纠正为满足 1NF 的标准方法有两种：一是将嵌套部分拆分为独立的属性（如选项 B），二是将重复组提升为独立的行或表。

考生需要具备识别属性是否原子性的判断能力，以及将不符合 1NF 的结构正确规范化的能力。常见的判断错误是忽略了原子性与应用场景的相关性，将“看起来是一串”的数据一律视为非原子。

知识点： 6.2 规范化——第一范式（1NF）的原子性要求； 1NF 违例的常见模式（嵌套集合、嵌套表结构、重复组）；将 1NF 违例纠正的标准方法。

参考答案

B

解析与知识点

本题考察事务 (Transaction) 的基本概念, 包括事务作为恢复和并发控制的基本逻辑单位、ACID 四大特性、COMMIT/ROLLBACK 的事务结束语义, 以及事务的定义方式 (显式与隐式)。

事务是数据库系统中恢复 (Recovery) 和并发控制 (Concurrency Control) 的基本逻辑单位, 选项 A 正确。事务具有 ACID 四大特性——原子性 (Atomicity)、一致性 (Consistency)、隔离性 (Isolation) 和持久性 (Durability), 选项 C 正确。COMMIT (提交) 表示事务正常结束, 将所有更新持久化; ROLLBACK (回滚) 表示事务异常终止, 撤销所有更新, 两者都标志事务的结束, 选项 D 正确。然而, 事务并非必须由用户显式定义: DBMS 中存在隐式事务 (如单条 DML 语句自动构成一个事务) 和自动提交模式, 因此选项 B 的说法是错误的。

事务可以由用户显式定义 (BEGIN TRANSACTION ... COMMIT/ROLLBACK), 也可以由系统隐式管理 (如 MySQL 的 autocommit 模式下每条语句都是一个独立事务)。ACID 特性中: (a) 原子性通过日志 (Undo 操作) 和回滚机制实现; (b) 一致性是事务的目标, 由其他三个特性共同保障; (c) 隔离性通过并发控制机制 (封锁、时间戳、多版本等) 实现; (d) 持久性通过先写日志和检查点机制实现。事务的“全部或全不”执行理念是数据库区别于文件系统的核心特征之一, 它使应用开发者无需考虑中途失败导致的数据部分更新问题。

理解事务的定义方式 (显式/隐式) 和结束方式 (COMMIT/ROLLBACK) 对于编写正确的数据库应用程序至关重要。考生常见的错误是认为所有事务都必须显式地以 BEGIN TRANSACTION 开始, 忽略了隐式事务和自动提交模式的存在。

知识点: 10.1 事务的基本概念——ACID 特性 (原子性、一致性、隔离性、持久性) 与事务定义方式 (显式事务与隐式事务); COMMIT 与 ROLLBACK 的事务结束语义。

参考答案

C

解析与知识点

本题考察数据库恢复 (Recovery) 技术中的两类核心“冗余数据”: 日志文件 (Log File) 和数据库后备副本 (Backup/Dump)。要求区分哪些数据属于恢复机制中的冗余数据, 哪些属于其他系统组件 (数据字典、审计档案、应用程序代码)。

数据库恢复的“冗余数据”是指在正常运行之外额外存储的可用于恢复的数据。主要包括两类: (1) 日志文件 (Log File) ——记录每次数据更新操作的前像 (Before Image) 和后像 (After Image), 用于故障后重做 (Redo) 或撤销 (Undo); (2) 数据库后备副本 (Backup/Dump) ——数据库在某一时刻的完整静态拷贝, 用于介

质故障后的恢复。数据字典存储元数据而非“转储冗余数据”；审计档案用于安全审计而非恢复；应用程序是代码而非数据。因此正确答案为 C。

日志文件是数据库恢复的核心基础设施，记录的内容包括事务标识符（Transaction ID）、操作类型（插入/删除/修改）、被修改的数据项标识（如页面 ID 和偏移量）、修改前的旧值（Undo 所需）和修改后的新值（Redo 所需）。日志文件通常以追加（Append-only）方式写入，以保证性能。后备副本（也称备份或转储）是数据库在某一时刻的快照，逻辑备份导出 SQL 语句，物理备份复制数据文件。恢复时首先从后备副本恢复到备份时间点的状态，然后从日志文件重放备份时间点之后的已提交事务操作，以恢复到故障发生前的最新一致状态。

这两类冗余数据共同构成了数据库的“安全网”，是数据库系统保障数据“永不丢失”承诺的技术基础。考生容易混淆数据字典（存储元数据，用于系统目录管理）和日志文件（存储操作记录，用于恢复），或将审计档案（记录访问操作，用于安全审查）误归为恢复冗余数据。

知识点：11.2 数据库恢复技术——日志文件（Undo/Redo 记录）与后备副本（完整转储与增量转储）；恢复的基本流程（先用副本恢复快照，再用日志回放后续操作）。

4.2 二、判断题（正确的打“√”，错误的打“×”，共 10 小题，每小题 2 分，共 20 分）

参考答案

×（错误）

解析与知识点

本题考察数据库系统三级模式结构中各级模式的功能区分，重点是外模式（External Schema）与概念模式（Conceptual Schema）各自的作用范围。

数据库系统的三级模式结构中，(1) 外模式（External Schema）描述的是特定用户可见的局部数据逻辑结构和特征，是用户的数据视图；(2) 模式/概念模式（Conceptual Schema）描述全体数据的逻辑结构和特征，是所有用户的公共数据视图；(3) 内模式（Internal Schema）描述数据的物理存储结构和存取路径。题目将外模式与概念模式的功能混淆，描述全体数据逻辑结构的应为概念模式而非外模式。

外模式是概念模式的子集，一个数据库系统可以有多个外模式，分别服务于不同用户或应用的需求（如财务人员看到的外模式只包含薪资相关的表和列，人事看到的外模式包含员工信息但隐藏了薪资列）。概念模式是唯一的，它定义了数据库的全局逻辑结构——有哪些表、表有哪些列、列的类型和约束是什么。内模式也

通常是唯一的（在单一 DBMS 实例中），定义了数据在磁盘上的存储细节——使用什么文件组织、哪些列建了索引、数据是否压缩和加密等。这种“三层隔离、两级映射”的架构使得数据库系统能够同时支持数据独立性（当内模式或概念模式变化时不需修改外模式和应用程序）和数据安全性（用户只能看到其外模式授权范围内的数据）。

牢记三级模式的定义和功能分工是数据库系统架构学习的最基础也是最核心的内容。考生常见错误是将外模式与概念模式的功能互换，误认为外模式描述全体数据——实际上外模式只是面向特定用户的局部视图。

知识点：1.2 数据库系统的三级模式结构——外模式（用户视图）、概念模式（全局逻辑结构）、内模式（物理存储结构）的功能定义与区分；两级映像与数据独立性的关系。

参考答案

×（错误）

解析与知识点

本题考察参照完整性（Referential Integrity）的约束条件及可能破坏参照完整性的操作类型，要求精确区分哪些操作在哪些关系上会破坏参照完整性。

参照完整性要求外码的值要么为空，要么等于被参照关系中某个元组的主码值。可能破坏参照完整性的操作包括：(1) 在参照关系上插入或修改元组（新外码值可能不存在于被参照关系中）；(2) 在被参照关系上删除或修改主码值（原有的参照元组可能“悬空”）。但被参照关系上的插入操作——向被参照关系中新增一个主码——不会破坏参照完整性，因为新插入的主码值与任何参照关系中的外码值都不冲突。因此题目说“插入”也可能破坏是错误的。

参照关系的插入/修改可能破坏参照完整性（因为新外码值必须存在），被参照关系的删除/修改也可能破坏参照完整性（因为原主码值被删除或修改后参照它的外码变得“悬空”），但被参照关系的插入是安全的——新加入的主码值不与任何现有外码冲突，它只是没有被参照而已。DBMS 处理参照完整性违规的策略包括：拒绝违规操作（RESTRICT/NO ACTION）、级联更新/删除（CASCADE，自动传播修改）、置空处理（SET NULL，将外码设为 NULL）。不同的处理策略适用于不同的业务场景，例如 ORDER 删除时其明细应级联删除（CASCADE），但供应商被删除时产品的外码可置空（SET NULL）。

这道题考察的是对破坏参照完整性的操作进行精确分类和判断的能力。考生常见的错误是将“被参照关系上的修改/删除可能破坏”泛化为“所有修改被参照关系的操作都可能破坏”，忽略了插入操作的安全性。

知识点： 5.2 参照完整性——破坏参照完整性的操作类型（参照关系插入/修改 vs 被参照关系删除/修改）；DBMS 对参照完整性违规的处理策略（RESTRICT、CASCADE、SET NULL）。

参考答案

×（错误）

解析与知识点

本题考察关系代数中“并”（Union）运算的“并相容性”（Union Compatibility）条件，要求区分 Schema 层面的兼容性约束与 Instance 层面的元组数量差异。

关系的“并”运算要求两个关系具有相同的属性个数，且对应属性取值于相同的域（即“相容”或“同类”关系），但不要求具有相同的元组数。例如 r 有 3 个元组、 s 有 5 个元组，只要两者结构相容即可进行并运算，结果为去重后的全部元组（最多 8 个）。“并相容”的核心条件是属性（列）的数量和类型一致，而非元组（行）的数量一致。

并运算、交运算和差运算统称为关系的集合运算，它们共同的前提条件是“并相容性”（Union Compatibility）——两个关系必须具有相同的度（Degree，列数）且对应列的域相同。这个条件约束的是关系模式（Schema），而非关系实例（Instance）。元组数量的不同只影响运算结果的大小，但不影响运算本身的合法性。这与数学中的集合并运算概念完全一致：两个集合的元素数量可以不同，只要它们的元素类型相同即可进行并运算。在实际 SQL 中，UNION（去重）和 UNION ALL（保留重复）实现了关系的并运算，系统会自动检查两个 SELECT 语句的列数和类型是否兼容。

理解并相容性是理解更复杂的关系代数运算（如差集、交集）的前提，也是避免代数表达式书写错误的必要条件。考生常见的错误是将“并相容”误解为要求元组数相同，或将 Schema 层面的约束与 Instance 层面的差异混淆。

知识点： 2.4 关系代数——并运算的并相容性条件（同度、同域）；Schema 兼容性判断与 Instance 无关；SQL 中的 UNION 与 UNION ALL。

参考答案

√（正确）

解析与知识点

本题考察 SQL 子查询中 ANY 和 ALL 量词谓词的语法约束及其与比较运算符的绑定关系。ANY 和 ALL 本身不能独立使用，必须与比较运算符（ $>$, $<$, $=$, $<>$,

$>=, <=$ 搭配使用, 构成如 $> \text{ANY}(\text{subquery})$ 、 $= \text{ALL}(\text{subquery})$ 等形式。

例如 `WHERE salary > ALL (SELECT salary FROM employee WHERE dept='Sales')` 表示比销售部门所有人工资都高; `WHERE age > ANY (SELECT age FROM s WHERE sdept='信息学院')` 表示比信息学院任意一个学生年龄大。ANY/ALL 是子查询比较的修饰符, 离开比较运算符将无法表达具体的比较语义, 因此题目的说法是正确的。

ANY 和 ALL 在语义上有着精确的逻辑对应关系: $> \text{ANY}$ 等价于大于子查询结果中的最小值 ($> \text{MIN}$), $> \text{ALL}$ 等价于大于子查询结果中的最大值 ($> \text{MAX}$); 同理, $< \text{ANY}$ 等价于小于最大值, $< \text{ALL}$ 等价于小于最小值。 $= \text{ANY}$ 等价于 IN, $<> \text{ALL}$ 等价于 NOT IN (但需注意 NULL 的陷阱——NOT IN 在子查询结果含 NULL 时整体返回空集)。这些逻辑等价关系可以帮助理解和优化子查询表达式。在实际的 SQL 编程中, ANY 和 ALL 配合子查询使用, 是表达“比所有...都...”和“比任意一个...”这两种量化条件的标准语法。

考生需要特别注意 $> \text{ANY}$ (比最小的还大即可) 和 $> \text{ALL}$ (必须比所有都大) 的语义差异, 这是子查询比较中容易混淆但在考试中频繁出现的考点。

注: 流出原答案标记为“×”, 但根据 SQL 标准语法, 该说法正确, 此处订正为“√”。

知识点: 3.4 SQL 数据查询——子查询中的 ANY/ALL 谓词 (量词修饰符); ANY/ALL 与比较运算符的绑定语义; ANY/ALL 与 MIN/MAX、IN/NOT IN 的逻辑等价关系。

参考答案

√ (正确)

解析与知识点

本题考察数据库安全中存取谓词 (Access Predicate) 的间接实现方式——通过视图机制实现行级安全 (Row-Level Security)。存取谓词是指对数据访问附加条件限制的授权方式, 如“只允许查看本部门的员工信息”。

SQL 标准的 GRANT 语句本身不支持带有 WHERE 条件的授权 (不支持行级权限控制), 但通过视图机制可以间接实现: 先创建一个只包含授权范围数据的视图 (如 `CREATE VIEW my_dept AS SELECT * FROM emp WHERE dept_id = user_dept`), 再将此视图的查询权限授予用户。这样, 视图充当了存取谓词的角色, 配合权限机制实现了细粒度的访问控制。这是数据库安全中的一种常用间接授权手段。

标准 SQL 的 GRANT 语句只能授予对表或视图的整体访问权限, 无法实现“行级安全” (Row-Level Security)。视图机制填补了这一空白——通过在视图中预设 WHERE 条件, 视图定义本身就成为了存取谓词的载体。更高级的数据库系统 (如

Oracle、PostgreSQL) 还提供了策略 (Policy) 和行级安全 (RLS) 原生支持, 但视图方式仍然是通用的跨平台标准方法。此外, 视图还可以隐藏列级敏感信息——创建不包含敏感字段 (如薪资、密码) 的视图, 即使授权用户查询该视图也无法获取这些敏感数据。

这种“视图 + 权限”的组合模式是数据库安全中实现最小权限原则 (Principle of Least Privilege) 的技术手段, 也是实现寓安全于设计 (Security by Design) 理念的重要实践。考生容易忽略视图在安全控制中的作用, 误认为只有 GRANT/REVOKE 才能实现访问控制。

知识点: 4.4 视图机制与安全性 (存取谓词的间接实现); 4.2 存取控制 (GRANT/REVOKE 的权限粒度限制); 行级安全与列级安全的实现策略; 最小权限原则。

参考答案

√ (正确)

解析与知识点

本题考察完全函数依赖 (Full Functional Dependency) 的定义及其与部分函数依赖 (Partial Functional Dependency) 的区分, 特别关注单属性左部的函数依赖性质。

完全函数依赖的定义是: 在函数依赖 $X \rightarrow Y$ 中, 若对 X 的任何真子集 X' 都不存在 $X' \rightarrow Y$, 则称 Y 完全函数依赖于 X 。当左部 X 是单属性时, 其唯一的真子集是空集 $\emptyset$, 而空集不可能函数决定非平凡的 Y 。因此单属性左部的函数依赖必然是完全函数依赖。部分函数依赖的定义前提是左部为属性组 (含有至少两个属性), 单属性左部不存在“部分”的可能。

这个结论的证明依赖于对完全函数依赖定义中“真子集”概念的正确理解。单属性集合 $X = \{A\}$ 的真子集只有 $\emptyset$, 而根据函数依赖的定义, 空集无法函数确定任何非空属性集 (除非是平凡函数依赖 $Y \subseteq \emptyset$, 即 $Y = \emptyset$)。因此, 不存在任何真子集能推导出 Y , 说明 Y 完全函数依赖于单属性 X 。

这个结论在规范化判定中有着重要的实用价值——当我们看到一个函数依赖的左边是单属性时, 不需要进一步分析就可以判定它是完全函数依赖, 从而无需担心部分依赖导致的 2NF 违例。相反, 对于组合候选码 (由多个属性组成的候选码), 需要逐一检查每个非主属性是否完全函数依赖于整个候选码, 以判断是否满足 2NF 的条件。

知识点: 6.1 函数依赖——完全函数依赖与部分函数依赖的定义; 单属性左部必然是完全函数依赖的证明; 组合候选码的 2NF 判定方法。

参考答案

× (错误)

解析与知识点

本题考察主属性 (Prime Attribute) 的准确定义, 及其与候选码 (Candidate Key)、主码 (Primary Key) 三个核心概念的区分。重点辨析“包含在候选码中”与“包含在主码中”的本质差异。

主属性的定义是: 包含在任意一个候选码中的属性。一个关系模式可能有多个候选码, 其中被选为主码的只是其中之一。因此, 一个属性即使不在主码中, 只要它属于某个候选码, 就是主属性。例如关系模式 $R(A, B, C)$, 若 $\{A, B\}$ 为主码, $\{A, C\}$ 为另一个候选码, 则 C 虽不在主码中但仍是主属性。非主属性 (Non-Prime Attribute) 才是不包含在任何候选码中的属性。

候选码 (Candidate Key) 是能唯一标识元组的最小属性集, 一个关系可以有多个候选码; 主码 (Primary Key) 是从候选码中选出的一个, 用于实际标识元组 (其他候选码称为“替代码”, Alternate Key); 主属性 (Prime Attribute) 是指包含在任何一个候选码中的属性, 而非只包含在主码中的属性。

这个区分在范式判定 (特别是 3NF 和 BCNF 的判定) 中非常关键——3NF 只要求消除非主属性对候选码的传递依赖, 但如果误将某个候选码中的属性当作非主属性处理 (因为不在主码中), 就会错误地得出不满足 3NF 的结论。因此, 在进行规范化分析时, 首先必须完整地找出所有候选码, 再区分主属性和非主属性, 这是正确判定范式等级的前提。

知识点: 6.1 函数依赖——候选码与主码的区别; 主属性 (包含在任意候选码中) 与非主属性的定义; 多候选码关系中的主属性识别对范式判定的影响。

参考答案

√ (正确)

解析与知识点

本题考察事务 (Transaction) 在数据库管理系统中的双重角色——事务既是恢复的基本单位, 也是并发控制的基本单位。

事务具有原子性 (Atomicity), 即事务中的操作序列要么全部执行, 要么全部不执行, 是一个不可分割的逻辑工作单元。事务同时也是并发控制的基本单位——并发控制机制 (如封锁、时间戳、多版本并发控制等) 以事务为粒度来决定操作之间的调度顺序, 保障多个事务并发执行时的隔离性和一致性。因此, 事务既是恢复的基本单位, 也是并发控制的基本单位。该判断题陈述正确。

事务的这一双重角色使其成为数据库管理系统中最核心的逻辑概念。作为恢复的基本单位，当事务执行过程中发生故障时，系统必须保证事务的原子性——已执行的操作通过 Undo 回滚，未完成的操作不产生持久效果。作为并发控制的基本单位，系统通过隔离性保证事务在并发执行时互不干扰——每个事务看到的数据要么是其他已完成事务的全部结果，要么什么也看不到，不会看到中间状态。这两种角色的统一体现了事务概念的设计精妙之处：将复杂的“故障恢复”问题和“并发调度”问题都统一到“事务”这个抽象概念之下。

这种抽象显著降低了数据库应用开发的复杂度，是数据库系统最重要的设计思想之一。考生容易误将事务视为仅与恢复或仅与并发控制相关，忽略了事务将两者统一的本质。

知识点：10.1 事务的基本概念——事务作为恢复和并发控制双重基本单位；原子性和隔离性在恢复和并发控制中的各自体现；事务抽象对应用开发的简化作用。

参考答案

×（错误）

解析与知识点

本题考察死锁检测手段——等待图（Wait-for Graph）方法——与软件工程中数据流图（Data Flow Diagram, DFD）的本质区别，要求区分并发控制中的运行时工具与系统分析中的设计时工具。

检测数据库系统中是否存在死锁（Deadlock）通常采用等待图（Wait-for Graph）方法，这是一种有向图，节点代表事务，有向边 $T_i \rightarrow T_j$ 表示事务 T_i 正在等待事务 T_j 释放锁资源。若等待图中出现环，则表明系统存在死锁。而数据流图（Data Flow Diagram, DFD）是结构化分析方法中用于描述系统中数据的流动、处理和存储过程的工具，用于需求分析和系统设计，与死锁检测无关。

死锁是并发控制中需要处理的经典问题，处理死锁通常有两种策略：一是死锁预防（Deadlock Prevention）——通过破坏死锁必要条件（如一次封锁法、顺序封锁法）来主动避免死锁的发生；二是死锁检测与解除（Deadlock Detection and Resolution）——周期性构建等待图检测是否存在环，若检测到死锁则选择一个或多个事务进行回滚以打破死锁循环。在实际系统中，死锁检测的频率和开销需要权衡——检测太频繁会增加系统开销，检测太稀疏会导致死锁事务等待时间过长。数据流图（DFD）则是软件工程中用于系统分析的结构化工具，由外部实体、数据流、处理过程和数据存储四部分组成，用于描述系统中数据从何而来、如何流动和处理、最终存储到哪里。两者虽然都涉及“图”的概念，但一个是运行时的动态事务状态图，一个是设计时的静态系统结构图。考生常见的错误是将这两种“图”混为一谈。

知识点： 10.3 并发控制——死锁检测（等待图法）与死锁预防/解除策略；7.2 数据库需求分析——数据流图（DFD）的定义与应用场景；等待图与数据流图的本质区别。

参考答案

×（错误）

解析与知识点

本题考察检查点（Checkpoint）机制的核心作用——缩短故障恢复时间，避免从日志起始位置全量扫描。重点考察对检查点机制如何提升恢复效率的精确理解。

检查点机制的核心优势就是无需从头扫描日志文件。引入检查点后，恢复时只需从最近的检查点开始向前扫描日志，确定需要 Redo 和 Undo 的事务列表。检查点之前的已提交事务的数据已被写入磁盘（通过检查点时的脏页刷新），无需再处理；检查点之前的未提交事务由 Undo 列表管理。这大幅缩短了恢复时间，避免了从日志起始位置全量扫描的开销。

引入检查点后，恢复算法变为：(1) 从最近的检查点记录开始正向扫描日志，维护两个事务列表——UNDO 列表（初始化为检查点时的活跃事务列表）和 REDO 列表（初始为空）；(2) 扫描过程中，遇到事务的 COMMIT 记录则将其从 UNDO 列表移到 REDO 列表；(3) 扫描结束后，对 UNDO 列表中的事务执行反向撤销操作，对 REDO 列表中的事务执行正向重做操作。检查点的设置频率也是一个设计权衡：频繁的检查点可缩短故障恢复时间，但检查点过程中的脏页刷写会增加运行时的 I/O 开销；稀疏的检查点则相反。在大型数据库系统中，通常结合已提交事务在日志中产生的新增数据量（如每写入 N MB 日志就设置一个检查点）来动态决定检查点频率。

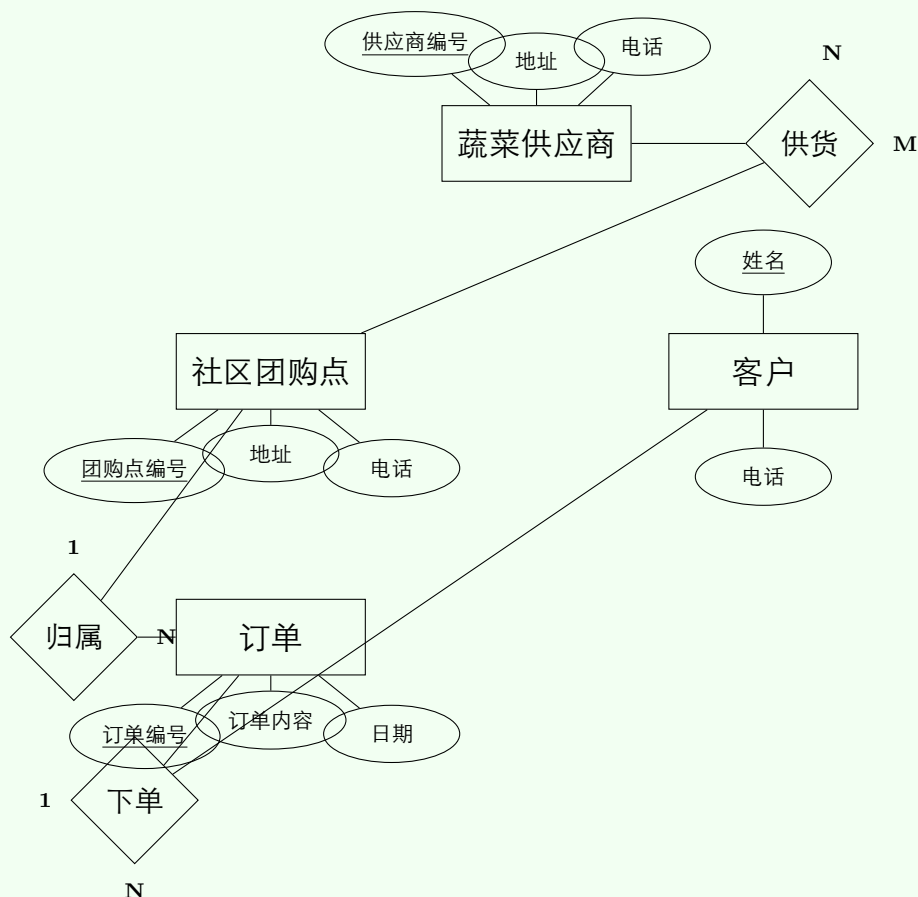
如果没有检查点机制，数据库重启时必须从头扫描整个日志文件，当数据库已运行数周时，恢复可能需要数小时——这在实际生产环境中是不可接受的。考生容易误认为检查点后仍需要全量扫描日志，不理解检查点如何将恢复范围限定在最近的检查点之后。

知识点： 11.3 具有检查点的数据库恢复技术（从最近检查点正向扫描、UNDO/REDO 事务列表的动态维护）；检查点机制对恢复效率的显著提升；检查点频率的设计权衡（恢复时间 vs 运行时开销）。

4.3 三、实体联系（E-R）模型设计及关系模式转化（共 10 分）

参考答案

(1) E-R 图



说明：供货是“蔬菜供应商”与“社区团购点”之间的多对多（ $M:N$ ）联系；下单是“客户”与“订单”之间的一对多（ $1:N$ ）联系；归属是“社区团购点”与“订单”之间的一对多（ $1:N$ ）联系。团购点编号和客户电话作为订单中的联系属性体现。

(2) 关系模式转换

（下划线 表示主码，斜体表示外码）

1. 蔬菜供应商 (供应商编号, 地址, 电话)
2. 社区团购点 (团购点编号, 地址, 电话)
3. 供货 (供应商编号, *团购点编号*)
 供应商编号参照蔬菜供应商，团购点编号参照社区团购点
4. 客户 (姓名, 电话)

5. 订单 (订单编号, 订单内容, 日期, 团购点编号, 客户电话)

团购点编号参照社区团购点, 客户电话参照客户

注: 订单中的“客户电话”作为外码参照客户表, 用于标识下订单的客户。供货关系将多对多联系 $M:N$ 转换为独立的关系模式。

解析与知识点

本题考察从自然语言需求描述到 E-R 图设计, 再到关系模式转换的完整数据库设计流程。包括实体与联系的识别、基数约束的确定、以及不同联系类型 ($1:1$, $1:N$, $M:N$) 向关系模式的映射规则。

需求中共有 4 个实体——蔬菜供应商、社区团购点、客户、订单。供应商和团购点之间是“供货”联系: 一个供应商可向多个团购点供货, 一个团购点可从多个供应商进货 ($M:N$)。客户通过在不同团购点“下单”来购买蔬菜, 不与供应商直接交互: 一个客户可下多个订单 ($1:N$), 每个订单属于一个团购点 ($1:N$)。订单虽然有独立编号, 但通过团购点编号和客户电话与团购点、客户关联。

实体直接转换为独立关系模式, 实体的属性即为关系的属性。 $M:N$ 联系“供货”需转换为独立关系模式, 其属性为两端实体的主码组合。 $1:N$ 联系“下单”和“归属”通过与“ N ”端 (订单) 合并实现——在订单关系中加入团购点编号 (归属联系) 和客户电话 (下单联系) 作为外码。这一转换过程遵循 E-R 模型向关系模型的标准映射规则, 其中联系类型是转换策略的核心决定因素。供货关系表以联合主码的形式存在, 既保证了数据的唯一性, 又为可能的附加属性预留了扩展空间。这道题的核心挑战是通过阅读自然语言需求描述正确地识别出实体、联系和基数约束。实体识别需要关注“哪些名词在系统中是独立可标识的”, 联系的识别则需要关注“实体之间发生了什么样的业务动作”。基数的确定依赖于对业务规则的量化理解。构建 E-R 图本身也是一个迭代的过程, 随着对业务理解的深入, 实体和联系可能被重新设计或细化。实际系统设计中, 在满足范式要求的前提下, 还需要综合考虑查询性能和数据维护的便利性。知识点: 7.3 E-R 模型设计——从业务需求描述中识别实体、联系和基数约束; 属性的识别与归属决策; 7.4 E-R 图向关系模式的转换规则 (实体转换、 $1:N$ 联系合并、 $M:N$ 联系独立关系)。

4.4 四、数据操作（共 30 分）

4.5 四.1 学生-学院-俱乐部管理系统（共 9 分）

参考答案

(1) 创建学生表

```
1 CREATE TABLE student (  
2     stuno    CHAR(11) PRIMARY KEY,           -- 实体完整性：主  
        码约束  
3     stuname  VARCHAR(10) NOT NULL,           -- 用户定义完整  
        性：姓名不可为空  
4     stuage   SMALLINT,  
5     stugen   CHAR(1) CHECK (stugen IN ('F', 'M')), -- 用户定义完  
        整性：性别取值约束  
6     schno    CHAR(3),  
7     FOREIGN KEY (schno) REFERENCES school(schno)  
8         ON DELETE CASCADE                     -- 参照完整性：级  
        联删除  
9 );
```

(2) 创建俱乐部人数视图

```
1 CREATE VIEW club_stu_count (clubno, clubname, clubstunum) AS  
2     SELECT jc.clubno, MIN(c.clubname), COUNT(jc.stuno)  
3     FROM joinclub jc, club c  
4     WHERE jc.clubno = c.clubno  
5     GROUP BY jc.clubno  
6     UNION  
7     SELECT clubno, clubname, 0  
8     FROM club  
9     WHERE clubno NOT IN (  
10         SELECT DISTINCT clubno FROM joinclub  
11     );
```

(3) 创建触发器

```
1 CREATE TRIGGER stu_num_trg  
2 AFTER INSERT OR DELETE ON student  
3 REFERENCING NEW ROW AS nrow, OLD ROW AS orow  
4 FOR EACH ROW  
5 BEGIN
```

```
6      IF INSERTING THEN
7          UPDATE school
8          SET schstunum = schstunum + 1
9          WHERE school.schno = nrow.schno;
10     END IF;
11     IF DELETING THEN
12         UPDATE school
13         SET schstunum = schstunum - 1
14         WHERE school.schno = orow.schno;
15     END IF;
16 END;
```

注：UPDATING 分支也可依实际需求补充。上述触发器在 INSERT 和 DELETE 操作后分别对相关学院人数进行 +1 或 -1 操作，以保持数据一致性。

解析与知识点

本题综合考察 SQL 数据定义中的三类完整性约束（实体完整性、参照完整性、用户定义完整性）、视图创建（含 UNION 与 NOT IN 补全技巧）以及行级触发器（FOR EACH ROW）的编写，涵盖数据库编程的核心技能。

(1) 建表语句需完整体现三类完整性约束。实体完整性通过 PRIMARY KEY 声明主码 stuno 实现——主码值是唯一的且不为 NULL，是标识每一行数据的依据。参照完整性通过 FOREIGN KEY 引用 school(schno) 并指定 ON DELETE CASCADE 实现级联删除——当学院被删除时，该学院的所有学生记录也自动被删除，防止出现孤儿记录。用户定义完整性通过 CHECK 约束限制 stugen 只能取 'F' 或 'M'（性别枚举），并通过 NOT NULL 确保姓名不为空。

(2) 视图包含两部分。上半段计算有学生的俱乐部人数，使用 GROUP BY 和 COUNT(stuno) 统计；MIN(c.clubname) 或 MAX(c.clubname) 作为聚合函数满足 GROUP BY 语法要求——在 GROUP BY 查询中 SELECT 子句的非聚合列必须出现在 GROUP BY 子句中。下半段通过 UNION 和 NOT IN 子查询补充无学生参加的俱乐部，人数固定为 0，确保视图完整性。这种“有数据的 UNION 空数据的”模式是 SQL 中处理统计类视图的常见技巧。视图是虚表，不存储实际数据，每次查询时动态计算结果，这保证了数据的实时性。

(3) 行级触发器 FOR EACH ROW 在每次 INSERT 或 DELETE 后自动执行。INSERTING 分支更新对应学院人数加 1，DELETING 分支减 1。使用 REFERENCING NEW ROW AS nrow, OLD ROW AS orow 区分新值和旧值——INSERT 时 new 为插入行的值，DELETE 时 old 为删除行的值。触发器自动在 DML 操作发生时由系统触发调用，无需应用层显式执行学院人数更新，这样保证了 schstunum

与 student 表中实际学生数的一致性。

触发器的优势在于它将业务逻辑移到数据库端，保证了逻辑在任何应用下都能被执行。但触发器也可能带来调试困难、递归触发（如学生表触发器更新学院表，学院表触发器又更新其他表）等风险，需要谨慎设计。合理使用约束可以显著减少应用层的校验代码量，将数据质量控制的责任内嵌到数据库系统中。

知识点： 3.1 SQL 数据定义（CREATE TABLE 及三类完整性约束——PRIMARY KEY、FOREIGN KEY REFERENCES ... ON DELETE CASCADE、CHECK/NOT NULL）； 3.7 SQL 视图（含 UNION 与 NOT IN 子查询补全技巧）； 5.5 触发器（行级触发器 FOR EACH ROW 与条件分支 INSERTING/DELETING/UPDATING）。

4.6 四.2 学生选课系统——关系代数与 SQL 综合（共 21 分）

参考答案

(1) 关系代数——信息学院学生信息

$$\sigma_{\text{sdept}=' \text{信息学院}'}(s)$$

(2) 关系代数——选修“数据库系统”的学生姓名

$$\pi_{\text{sname}}(\sigma_{\text{cname}=' \text{数据库系统}'}(s \bowtie sc \bowtie c))$$

或分步表达：

$$\pi_{\text{sname}}(s \bowtie sc \bowtie \sigma_{\text{cname}=' \text{数据库系统}'}(c))$$

(3) SQL——选修“数据库系统”的学生姓名

```
1 SELECT sname
2 FROM s, c, sc
3 WHERE s.sno = sc.sno
4       AND c.cno = sc.cno
5       AND cname = '数据库系统';
```

(4) SQL——选修三门及以上课程的学生学号

```
1 SELECT sno
2 FROM sc
3 GROUP BY sno
4 HAVING COUNT(*) >= 3;
```

注：HAVING COUNT(*) > 2 与 HAVING COUNT(*) >= 3 等价，均可。

(5) SQL——学生学号及平均成绩视图

```
1 CREATE VIEW s_avg (sno, avg_score) AS
2     SELECT sno, AVG(score)
3     FROM sc
4     GROUP BY sno;
```

(6) SQL——比信息学院任意一个学生年龄都大的外院学生

```
1 SELECT sno, sname, sage
2 FROM s
3 WHERE sage > ALL (
4     SELECT sage
5     FROM s
6     WHERE sdept = '信息学院'
7 )
8 AND sdept <> '信息学院';
```

或等价写法:

```
1 SELECT sno, sname, sage
2 FROM s
3 WHERE sage > (SELECT MAX(sage) FROM s WHERE sdept = '信息学院')
4 AND sdept <> '信息学院';
```

(7) SQL——授予 U1 查询 C1 成绩的权限

```
1 CREATE VIEW sc_c1 AS
2     SELECT sno, score
3     FROM sc
4     WHERE cno = 'C1';
5
6 GRANT SELECT ON sc_c1 TO U1;
```

注: 先创建只包含课程 C1 成绩的视图, 再对视图授予查询权限。若直接 GRANT SELECT ON sc TO U1 则会暴露所有课程的成绩信息, 违背最小权限原则。这里通过视图与权限机制配合实现了细粒度的存取控制。

解析与知识点

本题综合考察关系代数与 SQL 的核心操作, 涵盖选择/投影/连接运算、GROUP BY/HAVING 分组筛选、视图 (CREATE VIEW)、量词谓词 ($> \text{ALL}$ / $> \text{ANY}$) 的语义辨析, 以及基于视图的细粒度权限控制 (GRANT)。

(1) 关系代数中, 选择运算 σ 按条件筛选元组, 投影运算 π 选取指定列。查询“信息学院”学生仅需对 s 表作选择即可得到全部属性。选择运算是最基础的一元关系代数运算, 其作用是水平方向(行维度)上过滤数据, 根据谓词条件保留满足条件的元组。

(2) 此查询涉及三个表。三表自然连接 $\bowtie$ 利用公共属性 ($s.sno = sc.sno$ 、 $c.cno = sc.cno$) 关联, 再选择课程名为“数据库系统”的行, 最后投影出姓名列。由于查询只需学生姓名, 先连接再选择再投影的书写顺序与逻辑执行顺序一致。自然连接基于同名的公共属性自动进行等值连接, 是关系模型中最优雅的表达方式之一。

(3) SQL 语句中三表连接通过 WHERE 子句的等值条件实现。 $cname = '数据库系统'$ 直接对课程名进行过滤。等值连接在 SQL 中有两种等价写法: INNER JOIN ... ON ... (显式连接) 和 FROM ... WHERE ... (隐式连接)。

(4) GROUP BY sno 将选课记录按学号分组, HAVING COUNT(*) ≥ 3 筛选出选课数不少于 3 的学生。注意 HAVING 对分组结果进行筛选(在 GROUP BY 之后执行), WHERE 对原始行筛选(在 GROUP BY 之前执行)。这是 SQL 执行逻辑顺序中的关键差别。

(5) 使用 CREATE VIEW 创建虚拟表, AVG(score) 计算每个学生的平均成绩, GROUP BY sno 按学号分组。视图不存储数据, 每次查询时动态计算结果。视图提供了一种逻辑数据独立性的机制——当底层表结构变化时, 只需修改视图定义, 基于视图的查询和应用无需修改。

(6) $> ALL$ (子查询) 表示比子查询结果集中的所有值都大, 即比信息学院年龄最大的学生还要大。同时需排除信息学院自身的学生。 $> ANY$ 则是比任意一个大即可(即比最小的还大), 与 $> ALL$ 的语义完全不同, 是考试中频繁考察的辨析点。MAX 聚合函数写法在语义上与 $> ALL$ 等价, 且执行效率通常更高。

(7) SQL 标准中 GRANT SELECT 只能授予整个表或视图的查询权限, 不能直接在 GRANT 中带 WHERE 条件来限制可见的行。因此通过创建视图 sc_c1 只包含 C1 课程的数据, 再将视图的查询权限授予 U1 来间接实现细粒度的行级访问控制。

在多表连接中, sc 表作为连接桥梁, 连接了 s 表和 c 表。这种“桥接表”(Junction Table)模式在多对多关系的数据库设计中非常常见。隐式连接是更早期的写法, ANSI SQL-92 标准推荐使用显式 JOIN 语法, 因为它将连接条件与过滤条件分离, 提高了可读性。“视图 + 权限”的模式是数据库安全中的标准技术——先通过视图定义来限制数据的可见范围, 再通过 GRANT 来授予对该视图的访问权限, 符合最小权限原则。

考生容易混淆 WHERE 和 HAVING 的执行时机——WHERE 在分组前过滤行, HAVING 在分组后过滤组。 $> ALL$ (比所有都大) 与 $> ANY$ (比任意一个大) 的语义差异也是高频易错点: $> ALL$ 等价于 $> MAX$, $> ANY$ 等价于 $> MIN$ 。此

外, GRANT 本身不支持行级条件限制, 需借助视图间接实现。

知识点: 2.4 关系代数 (选择 σ 、投影 π 、自然连接 $\bowtie$ 的定义与组合使用); 3.4 SQL 数据查询 (多表连接、GROUP BY/HAVING 的执行顺序、 $> ALL$ 与 $> ANY$ 的语义辨析、子查询等价转换); 3.7 SQL 视图定义 (CREATE VIEW 与逻辑数据独立性); 4.2 存取控制 (GRANT 与视图配合实现行级权限)。

4.7 五、关系模式规范化 (共 10 分)

参考答案

(1) 快递投递——候选码与 3NF 判定

候选码: {快递编号}

是否为 3NF: 否, 不满足 3NF。

理由: 该关系模式存在如下函数依赖:

- 快递编号 $\rightarrow$ {揽件站点编号, 快递员编号, 发件人姓名, 发件人身份证号, 发件人电话号码, 发件人地址编号}
- 发件人身份证号 $\rightarrow$ {发件人姓名, 发件人电话号码, 发件人地址编号}

因此存在传递函数依赖: 快递编号 $\rightarrow$ 发件人身份证号 $\rightarrow$ {发件人姓名, 发件人电话号码, 发件人地址编号}, 即非主属性“发件人姓名”“发件人电话号码”“发件人地址编号”通过非主属性“发件人身份证号”传递依赖于候选码“快递编号”, 违反了 3NF 的要求 (3NF 要求无非主属性对码的传递函数依赖)。

(2) 快递跟踪——候选码与 2NF 判定

候选码: {(快递编号, 当前负责人编号)}

是否为 2NF: 否, 不满足 2NF。

理由: 该关系模式的函数依赖包括:

- (快递编号, 当前负责人编号) $\rightarrow$ {前一负责人编号, 当前状态, 收到时间, 当前站点编号}
- 当前负责人编号 $\rightarrow$ 当前站点编号

候选码为 (快递编号, 当前负责人编号)。非主属性“当前站点编号”仅依赖于码的一部分——“当前负责人编号”, 即存在部分函数依赖 (非主属性部分依赖于候选码), 违反了 2NF 的要求 (2NF 在 1NF 基础上消除非主属性对码的部分函数依赖)。

(3) 新增“快递费支付”关系模式

快递费支付 (支付编号, 快递编号, 支付金额, 支付方式, 收费快递员编号, 支付人身份证号, 支付时间)

属性说明:

- 支付编号——主码, 唯一标识每次支付记录
- 快递编号——外码, 参照“快递”关系, 标识所支付的快递
- 支付金额——该笔快递的费用金额
- 支付方式——取值为“预支付”或“到付”
- 收费快递员编号——外码, 参照“快递员”关系, 负责收款的快递员
- 支付人身份证号——标识实际付款人 (可能是发件人或收件人)
- 支付时间——支付发生的具体时间

注: 一个快递对应一笔支付 (1:1), 快递编号作为外码确保关联性。支付方式字段区分“预支付”和“到付”。支付人身份证号和支付时间满足统计需求。

解析与知识点

本题综合考察关系模式的规范化理论, 包括候选码的求解、第二范式 (2NF) 与第三范式 (3NF) 的判定标准、传递函数依赖与部分函数依赖的识别, 以及从业务需求自底向上设计关系模式的方法。

(1) 3NF 的判定标准是: 在 2NF 基础上, 消除非主属性对候选码的传递函数依赖。“快递投递”关系中, 发件人的个人信息 (姓名、电话、地址) 取决于发件人身份证号, 而发件人身份证号又取决于快递编号, 形成传递依赖, 故不满足 3NF。传递函数依赖的存在意味着关系模式中混合了属于不同实体的属性——“快递”实体和“发件人”实体的属性被错误地合并到同一个关系中。改进方法是将发件人信息独立为“发件人”关系模式, 在快递关系中只保留发件人身份证号作为外码。

(2) 2NF 的判定标准是: 在 1NF 基础上, 消除非主属性对候选码的部分函数依赖。“快递跟踪”的组合候选码包含两个属性——快递编号和当前负责人编号, 但“当前站点编号”只由“当前负责人编号”决定, 与“快递编号”无关——这是典型的部分函数依赖。因为在业务上, 一个快递员在某时刻只能在一个站点工作, 所以当前站点编号函数依赖于当前负责人编号, 而不依赖于完整的候选码。

(3) 设计关系模式时应考虑: 支付与快递的一一对应关系 (快递编号为外码, 因为一笔快递只有一次支付)、支付方式枚举 (预支付或到付)、支付金额和时间等统计信息, 以及付款人身份 (发件人或收件人, 通过支付人身份证号记录)。主码采

用独立的支付编号，确保唯一性——即使未来业务扩展为分期支付或退款产生多条支付记录，主码也无需更改。

部分依赖导致的 2NF 违例会产生三种典型异常：(a) 插入异常——无法单独插入一个新快递员及其站点信息（因为没有快递编号就无法形成完整的候选码值）；(b) 删除异常——删除最后一个快递跟踪记录时，快递员的站点信息也随之丢失；(c) 更新异常——如果快递员换了站点，需要更新所有跟踪记录中的站点编号，容易造成数据不一致。解决方法是将其分解为两个关系：快递跟踪（快递编号, 当前负责人编号, ...）和快递员站点（当前负责人编号, 当前站点编号）。

设计时要兼顾当前需求（如一个快递对应一笔支付的一对一关系）和未来扩展的可能（如主码使用独立的支付编号而非快递编号，为可能的多次支付预留空间）。同时 CHECK 约束可以用于限制支付方式取值的合法性。在进行规范化分析时，必须完整找出所有候选码再区分主属性和非主属性，这是正确判定范式等级的前提。

知识点：6.1 函数依赖（完全函数依赖与部分函数依赖的判定）；6.2 规范化——第二范式（2NF，消除部分依赖）与第三范式（3NF，消除传递依赖）的判定标准；传递依赖导致的插入/删除/更新异常分析；从业务需求自底向上设计关系模式的方法论。

4.8 六、事务、并发控制与恢复（共 10 分）

4.9 六.1 银行账务系统——基于封锁的并发控制

参考答案

(1) 补全空缺

- (a) 等待——104 账户已被转账事务加上排它锁 LX(104)，汇总事务试图读取 104 余额时无法获取锁，进入等待状态。
- (b) UX(101), UX(104) ——转账事务提交前释放对 101 和 104 账户的排它锁。解锁后汇总事务得以继续执行（T8 读 104 余额）。

(2) 汇总得到的总余额

汇总事务的读取序列：T1 → 101 余额 = 500（旧值），T3 → 102 余额 = 350，T4 → 103 余额 = 550，T8 → 104 余额 = 600（转账后的新值）。

$$\text{汇总余额} = 500 + 350 + 550 + 600 = \mathbf{2000}$$

(3) 数据是否正确？

不正确。

原因：这是典型的不可重复读（Unrepeatable Read）问题。汇总事务在 T1 时刻读取 101 账户余额为 500（转账前），而转账事务在 T2-T5 完成了将 400 元从 101 转至 104 的操作（101 变为 100，104 变为 600）。汇总事务读到的 101 为 500（旧值）而 104 为 600（新值），导致总余额虚增 400。正确的总余额应为： $100 + 350 + 550 + 600 = 1600$ 。虽然提交读隔离级别下仍可能出现不可重复读，这正是本题要展示的并发问题。

(4) 补全死锁调度中的空缺

- (c) LX(104)，更新 104 账户余额——转账事务先锁定并更新 104 账户（转入方）。
- (d) LX(101)，更新 101 账户余额——转账事务再尝试锁定并更新 101 账户（转出方）。

死锁形成过程：

1. T1：汇总事务对 101 加共享锁 LS(101) 并读取
2. T2：转账事务对 104 加排它锁 LX(104) 并更新
3. T3-T4：汇总事务依次对 102、103 加共享锁并读取
4. T5：转账事务尝试对 101 加排它锁 LX(101)，但 101 已被汇总事务持有共享锁，须等待
5. T6：汇总事务尝试对 104 加共享锁 LS(104)，但 104 已被转账事务持有排它锁，须等待
6. T7：两个事务互相等待对方释放锁，形成死锁

死锁的根本原因是两个事务以不同顺序请求锁资源（汇总：101→102→103→104；转账：104→101），违反了死锁预防中的顺序加锁策略。

解析与知识点

本题综合考察基于封锁的并发控制机制，包括排它锁（X 锁）与共享锁（S 锁）的兼容性规则、两段锁协议（2PL）的加锁与解锁阶段、不可重复读（Unrepeatable Read）现象，以及死锁的形成条件与预防策略。

(1) 排它锁（X 锁）的特性是：持有 X 锁期间，其他事务无法读取或修改该数据项——即 X 锁与任何其他锁（包括共享锁 S 和排它锁 X）都不兼容。转账事务在

更新 101 和 104 账户余额前分别锁定它们，汇总事务在 T6 试图读 104 时发现被锁，只能等待转账事务释放锁（在 T7 处解锁）。(a) 填写“等待”反映了汇总事务被阻塞的状态，(b) 填写 UX(101), UX(104) 表示转账事务提交前释放了两个账户的锁，这是两段锁协议中“解锁阶段”的操作。

(2)–(3) 汇总事务读取的是“混合”状态的数据——101 账户是转账前的旧值 (500)，104 账户是转账后的新值 (600)，而 102 和 103 未参与转账保持不变。这种不一致源于两段锁协议下的不可重复读现象：汇总事务在 T1 读取 101 后未对 101 保持锁，随后转账事务修改了 101 并将差额转入 104。在“提交读” (Read Committed) 隔离级别下，不可重复读是允许发生的并发问题。正确的总余额应为 $100+350+550+600=1600$ ，虚增的 400 正好等于转账金额。要避免此问题，需要升级到“可重复读” (Repeatable Read) 隔离级别。

(4) 死锁需要四个必要条件同时满足：互斥、持有并等待、不可剥夺、循环等待。通过将转账事务的加锁顺序改为先锁 104 后锁 101（与正常的转出再转入的业务顺序相反），而汇总事务按正常顺序 101->102->103->104 加锁，人为制造了 101 和 104 上的循环等待条件。

两段锁协议要求事务分为加锁阶段 (Growing Phase, 只加锁不解锁) 和解锁阶段 (Shrinking Phase, 只解锁不加锁)，这种分段设计是防止死锁的有效手段——虽然不能完全消除死锁，但能保证一旦发生则可检测。死锁预防中的顺序加锁策略可以避免此问题——要求所有事务按相同的全局顺序请求锁资源，这样就无法形成相互等待的循环。死锁解除则通过回滚其中一个事务（通常是代价最小的）来打破循环依赖。

死锁的根本原因是两个事务以不同顺序请求锁资源（汇总：101->102->103->104；转账：104->101），违反了死锁预防中的顺序加锁策略。考生常见的错误是忽略了不同隔离级别下锁的保持时长差异——提交读下锁在读取后立即释放，可重复读下锁保持到事务结束。

知识点：10.2 封锁机制（排它锁 X 与共享锁 S 的兼容性规则）；10.3 两段锁协议（加锁阶段与解锁阶段）与死锁的形成条件和预防策略（顺序加锁法）；10.4 事务隔离级别（提交读下的不可重复读问题）与锁保持时长对隔离性的影响。

4.10 六.2 基于日志和检查点的数据库恢复

参考答案

(1) Redo 与 Undo 事务列表

恢复过程：从最近的检查点 LSN11 开始，向前扫描日志至末尾 (LSN13 崩溃点)。

- 检查点时的活跃事务： T_2 (LSN3 启动)、 T_3 (LSN5 启动)、 T_7 (LSN7 启动)

- T_1 在检查点之前已提交 (LSN8)，其修改已通过检查点写入磁盘，无需处理
- 扫描检查点之后的日志：
 - LSN12: T_2 提交 $\implies$ 加入 Redo 列表
 - T_3 、 T_7 均未提交 $\implies$ 加入 Undo 列表

Redo 列表: $\{T_2\}$; Undo 列表: $\{T_3, T_7\}$

(2) 恢复后的数据项值

- T_2 Redo: 回放 T_2 的所有更新操作 (从 START 到 COMMIT)。
 - LSN4: $L = 37$ (L 非查询对象, 忽略)
 - LSN6: J 从 45 更新为 5 $\implies$ 恢复后 $J = 5$
- T_3 Undo: 反向回滚 T_3 的所有更新操作。
 - LSN10: K 从 9 更新为 11, 撤销后 $K = 9$
 - LSN9: M 从 53 更新为 15, 撤销后 $M = 53$ (M 非查询对象, 忽略)
- T_7 Undo: T_7 仅有 START 记录, 无数据修改操作, 无需回滚
- T_1 在检查点前提交, 数据 $I = 3$ 已由检查点刷新到磁盘。但是, 检查点仅记录了活跃事务列表, T_1 不在其中, 恢复时不再处理 T_1 。然而在某些教材的恢复模型中, 检查点前的脏页不一定全部落盘, T_1 也不在 Redo 列表内, 因此 I 可能保持其初始磁盘值 22。

$$I = 22, \quad J = 5, \quad K = 9$$

注: 根据原卷参考答案, I 恢复为 22。此结果对应一种恢复模型: 检查点仅记录活跃事务标识和日志缓冲刷盘, 不一定强制所有脏数据页落盘。由于 T_1 已提交且不在检查点活跃列表中, 不在 Redo 范围内, 故 I 保留检查点前的磁盘值 22。

解析与知识点

本题考察具有检查点的数据库恢复技术, 包括从检查点正向扫描日志以动态维护 Redo/Undo 事务列表的算法, 以及基于日志记录的 Undo (逆序撤销) 与 Redo (正向重做) 操作恢复数据项的具体过程。

(1) 具有检查点的恢复算法核心步骤：(i) 从最后一个检查点正向扫描日志至末尾，将检查点活跃事务中已提交的加入 Redo 列表，未提交的加入 Undo 列表；(ii) 正向 Redo Redo 列表中所有事务的全部操作；(iii) 反向 Undo Undo 列表中所有事务的全部操作。 T_1 在检查点前已提交 (LSN8)，不在活跃事务列表中，既不需要 Redo 也不需要 Undo。 T_2 在检查点时为活跃状态，检查点后提交 (LSN12)，需 Redo。 T_3 和 T_7 在检查点为活跃状态，检查点后均未提交，需 Undo。

(2) J ： T_2 的更新操作 (LSN6: 45->5) 被 Redo，故恢复后 $J = 5$ 。Redo 操作是幂等的——即可以安全地重复执行多次。 K ： T_3 的更新操作 (LSN10: 9->11) 被 Undo 反向恢复到初始值，故 $K = 9$ 。Undo 操作需要反向扫描日志，按逆序撤销更新。 I ： T_1 在 LSN2 将其从 22 改为 3 并提交于检查点前的 LSN8。按原卷答案 $I = 22$ ，对应一种恢复模型：检查点只写日志不强制刷所有脏数据页，且 T_1 不在 Redo 范围，故 I 仅有磁盘旧值 22。

检查点是日志中的一个特殊记录点，它将恢复的起点从“日志的开头”移到“最近的检查点”，大幅缩短恢复时间。Redo 列表中的事务之所以需要重做，是因为它们的修改可能在崩溃时尚未写入磁盘（数据缓冲区中的脏页丢失），但其提交状态已确认，必须保证持久性。Undo 列表中的事务之所以需要撤销，是因为它们的修改可能已经在缓冲区替换策略下写入了磁盘，但其未提交，必须保证原子性。

这反映了不同的检查点实现策略对恢复结果的影响——“完整检查点” (Full Checkpoint) 会刷所有脏页到磁盘，而“增量检查点” (Incremental Checkpoint) 只记录检查点信息但允许脏页继续保持缓冲状态。在实际的数据库系统中，后一种策略更为常见，因为全量刷盘对运行时的 I/O 性能影响太大。恢复的最终目标是将数据库恢复到一个“一致性状态”——所有已提交事务的效果都被保留（持久性），所有未提交事务的效果都被清除（原子性）。

知识点：11.3 具有检查点的数据库恢复技术 (Redo 与 Undo 事务判定——检查点活跃事务列表的动态维护)；11.2 日志文件与恢复原理 (Undo 的逆序撤销与 Redo 的幂等性)；检查点的不同实现策略 (完整检查点 vs 增量检查点) 及其对恢复结果的影响；WAL 原则保障恢复正确性。

5 2002–2003 学年《数据库原理》期末试卷 (A 卷)

注意

试卷说明：本卷为武汉大学计算机学院 2002–2003 学年度第一学期 2000 级 A 卷，满分 100 分（含选做题 5 分附加）。原卷无官方参考答案，本解析所有答案均由解析者根据数据库理论体系自行解答，已标注（答案由解析者提供）。

5.1 一、填空题（每小题 2 分，共 10 分）

1. 关系模型的三种完整性约束为_____。

参考答案

（答案由解析者提供）

实体完整性（Entity Integrity）、参照完整性（Referential Integrity）、用户定义完整性（User-Defined Integrity）。

解析与知识点

本题考察的是关系数据库理论中最基础的完整性约束分类知识。关系模型定义了三种完整性约束，它们共同构成了数据库中数据正确性、有效性和相容性的保护体系。

第一，实体完整性（Entity Integrity）要求每个关系的主码值必须唯一且非空，这保证了关系中的每一行（每一个实体实例）都能被唯一标识，是数据库记录可追溯性的基础。第二，参照完整性（Referential Integrity）确保外码的值要么为空，要么必须等于被参照关系中某个元组的主码值，从而维护表与表之间关联的正确性，杜绝“悬空引用”——即子表中的外码指向一个不存在的父表记录。

第三，用户定义完整性（User-Defined Integrity）是针对具体应用场景由用户自定义的约束条件，如年龄必须在 0 至 150 之间、性别只能是“男”或“女”等，体现了特定业务规则对数据取值的限制。

这三类约束从不同层面保护数据：实体完整性确保行标识，参照完整性确保表间关联，用户定义完整性确保列取值。在 SQL DDL 中，它们分别通过 PRIMARY KEY、FOREIGN KEY 以及 CHECK / NOT NULL / UNIQUE 等子句来声明和执行。理解这三类完整性约束不仅是掌握关系模型的基础，也是后续学习数据库完整性和安全性章节的前提。

知识点：第二章：关系数据库——2.3 关系完整性约束（实体完整性、参照完整性、用户定义完整性）

2. 数据库中常用的数据模型有_____、_____、_____、_____。

参考答案

（答案由解析者提供）

层次模型（Hierarchical Model）、网状模型（Network Model）、关系模型（Relational Model）、面向对象模型（Object-Oriented Model）。

解析与知识点

本题考察对数据库发展史上四种基本数据模型的认识。数据模型是数据库系统的核心和基础，是对现实世界数据特征的抽象，描述了数据的结构、操作和约束。

层次模型 (Hierarchical Model) 是最早出现的数据模型之一，用树形结构组织数据 (如 IBM 的 IMS 系统)，适合具有明显父子关系的数据，但无法直接表达多对多关系。网状模型 (Network Model) 用图结构表示实体间的复杂联系 (如 DBTG 系统)，允许节点有多个双亲，能够表达多对多关系，但结构复杂、导航式操作对程序员要求高。

关系模型 (Relational Model) 由 E.F. Codd 于 1970 年提出，用二维表 (关系) 表示数据，以数学化的关系代数为基础，具有严格的数学理论、简洁的数据结构和非过程化的查询语言，是当前最主流、应用最广泛的数据模型。面向对象模型 (Object-Oriented Model) 将面向对象编程思想 (封装、继承、多态) 引入数据库领域，支持复杂对象、类层次和方法的定义，适用于 CAD/CAM、多媒体等复杂应用领域。

此外，随着技术发展，后续还出现了对象-关系模型 (ORDB)、XML 半结构化模型和 NoSQL 等多种新型数据模型，但上述四种是数据库课程中需要掌握的基本数据模型类型。

知识点：第一章：绪论——1.2 数据模型 (层次模型、网状模型、关系模型、面向对象模型的特点与发展)

3. 数据库的三级模式结构是指_____；提供的两个独立性是指_____。

参考答案

(答案由解析者提供)

三级模式结构：外模式 (External Schema，也称子模式或用户模式)、概念模式 (Conceptual Schema，也称模式)、内模式 (Internal Schema，也称存储模式)。

两个独立性：逻辑数据独立性 (Logical Data Independence) 和物理数据独立性 (Physical Data Independence)。

解析与知识点

本题考察数据库系统体系结构的核心概念——三级模式结构与两级映射机制。三级模式结构是 ANSI/SPARC 于 1975 年提出的标准化数据库体系架构。

外模式 (External Schema) 是用户视角的局部数据描述，每个用户可以有不同的外模式，它定义了用户所能看到和使用的数据视图。

概念模式 (Conceptual Schema) 是数据库全局逻辑结构的完整描述, 位于中间层, 定义了所有数据实体、属性和关系, 是整个数据库的单一逻辑蓝图。内模式 (Internal Schema) 是最底层的数据描述, 定义数据在存储介质上的物理结构和存取方式 (如文件组织方式、索引结构等)。两级映射为数据库提供了两个层面的独立性保障: 外模式/概念模式映射保障逻辑数据独立性——当概念模式发生变化时, 只需调整这两层之间的映射关系, 外模式 (用户视图) 无需修改; 概念模式/内模式映射保障物理数据独立性——当物理存储结构改变时, 只需调整概念模式与内模式之间的映射, 概念模式和应用均可保持不变。

这种分层架构思想是数据库系统区别于传统文件系统的关键特征, 也是现代大型数据库系统能够灵活演进而不影响上层应用的根本原因。

知识点: 第一章: 绪论——1.3 数据库系统的体系结构 (三级模式结构与两级映射, 数据独立性的实现机制)

4. SQL 的集合处理方式与宿主语言单记录处理方式之间通过_____进行协调。

参考答案

(答案由解析者提供)

通过游标 (Cursor) 进行协调。

解析与知识点

本题考察嵌入式 SQL 中游标 (Cursor) 机制的理解, 这是数据库编程中连接集合操作与逐记录处理的桥梁。SQL 语言本质上是面向集合的——一条 SELECT 语句返回的结果是一个元组的集合 (结果表), 操作的对象和结果都是关系的集合。而宿主编程语言 (如 C、Java、COBOL、Python 等) 通常是面向单记录的, 每次只能通过变量接收和处理一条数据记录。

游标机制在这两种不同的数据处理范式之间架设了桥梁: 它将 SQL 查询返回的结果集看作一个内存中的缓冲区 (游标区), 通过定义游标 (DECLARE CURSOR)、打开游标 (OPEN)、逐行推进提取 (FETCH) 和关闭游标 (CLOSE) 四个标准操作, 将集合中的元组逐一读取并赋值给宿主语言的主变量 (Host Variable), 实现逐行处理。

游标本质上是一个指向结果集中当前行的指针, 通过 FETCH 语句可以移动指针到下一行并获取该行的数据。

理解游标机制对于掌握嵌入式 SQL 编程和数据库应用程序开发至关重要, 它解决了”一次一集合”的数据库操作与”一次一记录”的过程化编程之间的”阻抗失配”

问题。

知识点：第八章：数据库编程——8.1 嵌入式 SQL 与游标机制（定义、打开、推进、关闭的四个操作步骤）

5. 数据库恢复的基本原理是_____。

参考答案

（答案由解析者提供）

数据库恢复的基本原理是利用存储在系统其他地方的冗余数据（如日志文件、数据库后备副本等）来重建或恢复被破坏的数据库，其核心思想是数据的冗余存储。

解析与知识点

本题考察数据库恢复技术基本原理的深刻理解。数据库恢复机制是保证事务原子性（Atomicity）和持久性（Durability）的关键技术，其基本原理可以概括为两个字——“冗余”。

当系统发生故障时，根据不同故障类型采取不同恢复策略：事务故障（单个事务的逻辑错误或死锁）通过日志的 UNDO 操作撤销未完成事务的所有修改；系统故障（如断电导致内存数据丢失）需要 UNDO 未完成事务并 REDO 已提交但未写入磁盘的事务；介质故障（磁盘物理损坏）则需要使用最近的后备副本加上该副本之后的所有日志来进行完全恢复。

系统通过两种主要的冗余手段来应对各种可能的故障：第一是日志文件（Log File），以日志记录的形式记录每一次数据修改操作的详细信息，包括事务标识、修改前后的数据值（前像 Before Image 和后像 After Image）、数据项标识等，遵循“先写日志后写数据库”（WAL）原则；第二是数据库后备副本（Dump/Backup），定期将整个数据库或增量变化复制到独立的存储介质上。日志文件用于应对事务故障和系统故障，而后备副本则是应对介质故障的最后防线。

冗余是恢复的代价——用空间换安全，只要冗余数据不丢失，就可以重建丢失或被破坏的数据。

知识点：第十一章：数据库恢复——11.1 故障分类与恢复原理（事务故障 UNDO、系统故障 UNDO+REDO、介质故障后备副本 +REDO）、11.2 日志与恢复技术（WAL 原则）

5.2 二、单项选择题（每小题 1 分，共 10 分）

1. 通常所说的数据库系统（DBS）、数据库管理系统（DBMS）、和数据库（DB）三者之间的关系是（ ）。

参考答案

（答案由解析者提供） C

解析与知识点

本题考察对数据库领域三个基本概念——DB、DBMS、DBS 之间的包含关系的理解，这是数据库课程的入门知识点。数据库（DB, Database）是长期存储在计算机内、有组织、可共享的大量数据的集合，是整个系统中最核心的“数据容器”。

数据库管理系统（DBMS, Database Management System）是位于用户与操作系统之间的一层系统软件，负责对 DB 进行统一管理和控制，提供数据定义、数据操纵、数据库运行管理和维护等功能，是管理 DB 的“数据引擎”。

数据库系统（DBS, Database System）是引入数据库技术后的完整计算机系统，其外延最大，包含数据库（DB）、数据库管理系统（DBMS）、应用程序（Application）、数据库管理员（DBA）和最终用户（User）等多个组成部分。从概念的外延范围来看，三者之间存在严格的大小包含关系：DBS 包含 DBMS 和 DB，而 DBMS 负责管理 DB。因此，DBS 的外延 > DBMS 的外延 > DB 的外延，从概念包含关系看，DBS 包含 DBMS，DBMS 管理 DB，DBS 包含 DB。

理解这三个概念的层次关系，有助于在宏观上把握数据库课程的知识体系骨架。

知识点：第一章：绪论——1.1 数据库基本概念（DB 是数据容器、DBMS 是管理软件、DBS 是完整系统；三者外延关系为 DBS 包含 DBMS 和 DB）

2. 数据库三级模式体系结构的划分，有利于保持数据库的（ ）。

参考答案

（答案由解析者提供） A

解析与知识点

本题考察对三级模式体系结构核心价值理解。三级模式结构（外模式、概念模式、内模式）的最重要贡献就是通过两级映射机制实现了数据独立性（Data Independence），这是数据库系统区别于传统文件系统的根本特征。

外模式/概念模式之间的映射保证了逻辑数据独立性——当数据库全局逻辑结构（概念模式）发生变化时（如增加新的数据项、修改数据项的定义等），只需调整

该映射关系，外模式（用户视图）无需改变，因而基于外模式的应用程序不受影响。概念模式/内模式之间的映射保证了物理数据独立性——当数据库的物理存储结构改变时（如更换存储设备、改变索引结构、重组文件等），只需修改概念模式/内模式映射，概念模式和应用均可保持不变。

数据安全性主要通过存取控制（GRANT/REVOKE）、视图机制和审计等技术来保障，结构规范化是通过关系理论的范式分析来实现，操作可行性则是一个更为宽泛的概念。

因此，三级模式结构的根本目的和最大价值在于提供数据独立性，使得数据库系统的各个层面可以相对独立地演进而互不影响。

知识点：第一章：绪论——1.3 数据库系统体系结构（三级模式结构与两级映射保障逻辑数据独立性和物理数据独立性）

3. 设关系 R 和 S 的属性个数为 2 和 3，那么 $R \bowtie_{2>1} S$ 与下列（ ）等价。

参考答案

（答案由解析者提供） B

解析与知识点

本题考察 θ -连接运算的精确定义及其与笛卡尔积加选择运算之间的等价关系。 θ -连接的标准定义为 $R \bowtie_{i\theta_j} S = \sigma_{i\theta(j+n)}(R \times S)$ ，其中 n 为 R 的元数（即属性个数）， i 和 j 分别是在 R 和 S 中的属性位置编号。

在本题中， R 的属性个数 $n = 2$ ， S 的属性个数为 3。条件 $2 > 1$ 表示将 R 的第 2 个属性与 S 的第 1 个属性进行比较。在笛卡尔积 $R \times S$ 的结果中，前 $n = 2$ 个位置（位置 1 和 2）对应 R 的属性，后 3 个位置（位置 3、4、5）对应 S 的属性。因此 R 的第 2 个属性在笛卡尔积中对应位置 2， S 的第 1 个属性在笛卡尔积中对应位置 $2 + 1 = 3$ 。

将 θ -连接定义代入， $R \bowtie_{2>1} S = \sigma_{2>(1+2)}(R \times S) = \sigma_{2>3}(R \times S)$ 。选项 B 恰好是 $\sigma_{2>3}(R \times S)$ ，正确。

选项 A 使用等号（=）而非大于号，错误；选项 C 比较的是 $R \times S$ 中的位置 1 和 2，错误；选项 D 的比较条件和角标位置也都不正确。 θ -连接的定义是理解关系代数中各种连接运算的基础。

知识点：第二章：关系数据库——2.4 关系代数（ θ -连接运算的定义： $R \bowtie_{i\theta_j} S = \sigma_{i\theta(j+n)}(R \times S)$ ，属性位置编码规则）

4. 关系数据库系统进行（ ）的处理，是为了提高效率。

参考答案

(答案由解析者提供) D

解析与知识点

本题考察对关系数据库系统关键功能模块及其各自目的的理解，尤其需要区分查询优化与规范化、可串行化调度以及视图定义的不同目的。关系数据库系统通过查询优化 (Query Optimization) 处理来提高 SQL 查询的执行效率。

由于关系数据库使用非过程化的查询语言 (如 SQL)，用户只需声明“要什么” (What)，而无需指定“怎么做” (How)，因此系统必须自动选择一个最优的执行策略。

查询优化包括两个层面：代数优化——基于关系代数的等价变换规则对查询树进行优化 (例如尽早执行选择与投影操作、推迟笛卡尔积)；物理优化——选择最优的存取方法 (全表扫描还是索引扫描) 和连接算法 (嵌套循环连接、散列连接还是归并连接)。而视图定义的目标是简化用户操作和提供安全保护 (隐藏敏感数据)；规范化处理的目的是消除数据冗余和操作异常 (插入/删除/修改异常)；可串行化调度的目的是保证事务并发执行的正确性，防止丢失更新、脏读和不可重复读等问题。

这四种机制各有其专门目的，但唯一直接面向效率提升的是查询优化。

知识点：第九章：查询处理与优化——9.1 查询优化概述 (非过程化语言的必然需求)、9.2 代数优化 (启发式规则与等价变换)

5. SQL 中，谓词 EXISTS 用来测试一个结果集合是否（ ）。

参考答案

(答案由解析者提供) A

解析与知识点

本题考察 SQL 中 EXISTS 谓词的语义理解。EXISTS 是 SQL 中的一个存在量词谓词 (Existential Quantifier)，用于测试子查询返回的结果集是否非空。

当子查询返回至少一行数据 (即结果集不为空) 时，EXISTS 返回 TRUE；当子查询返回空集 (即没有任何行符合条件) 时，EXISTS 返回 FALSE。例如查询“至少有一次供应记录的供应商”可以写为：SELECT * FROM S WHERE EXISTS (SELECT * FROM SPJ WHERE SPJ.SNO = S.SNO)。

EXISTS 的典型使用场景是相关子查询 (Correlated Subquery)，其中内层子查询引用外层查询的属性列。EXISTS 只关心子查询结果集中是否存在满足条件的元组，而不关心具体返回了多少行或具体内容是什么，因此 SELECT 子句中通常使用通配符 * 或常量 (如 SELECT 1)。与 EXISTS 相对的是 NOT EXISTS，用于判断子查询结果集合是否为空。EXISTS 和 NOT EXISTS 经常配合使用来模拟关系代数中的除法运算 (全称量词“所有”可以通过双重否定 EXISTS 来间接表达)，这也是解答“全部”类 SQL 查询的重要技巧。

初学者容易将 EXISTS 的“存在量词”语义与 IN 运算混淆——EXISTS 只关心子查询是否有返回行而不关心值本身，而 IN 则需要子查询返回具体的值列表进行比较。当子查询结果包含 NULL 时，二者的行为有重要差异。

知识点：第三章：SQL 语言——3.4 数据查询 (EXISTS 谓词的语义——测试子查询结果集是否非空；相关子查询的典型用法)

6. SQL 和宿主语言的接口是 ()。

参考答案

(答案由解析者提供) D

解析与知识点

本题考察嵌入式 SQL 中 SQL 与宿主语言之间数据交换机制的理解。在嵌入式 SQL 编程中，SQL 语句被嵌入到宿主语言 (如 C、COBOL、Java 等) 的程序代码中执行。

SQL 和宿主语言之间通过主变量 (Host Variable，也称共享变量 Shared Variable) 进行数据交换。

主变量是宿主语言程序中的普通变量，但在 SQL 语句中以冒号 (:) 前缀加以引用——这样 SQL 解析器就能区分哪些标识符是数据库对象名、哪些是来自宿主程序的变量。主变量的功能是双向的：在 SELECT 语句的 INTO 子句中，主变量用于接收 SQL 查询返回的结果值；在 WHERE 或 VALUES 子句中，主变量用于将宿主程序中的变量值传递给 SQL 语句。DBMS 是数据库管理系统整体，不是接口机制；OS 是操作系统，不属于 SQL/宿主语言的接口层；DML 是数据操纵语言本身，是 SQL 的一个功能类别，而非连接 SQL 与宿主语言的具体接口。

因此正确选项是主变量 (D)。理解主变量的使用是掌握嵌入式 SQL 编程的关键第一步。

知识点：第八章：数据库编程——8.1 嵌入式 SQL (主变量的定义、以冒号前缀引用、作为 SQL 与宿主语言之间的数据交换接口)

7. 已知关系模式 $R = \{A, B, C, D, E\}$, 函数依赖集为 $\{A \rightarrow D, B \rightarrow C, E \rightarrow A\}$, 则该关系模式的候选码是 ()。

参考答案

(答案由解析者提供) B

解析与知识点

本题考察候选码求解的系统方法——基于属性集闭包计算和“不在右部出现的属性”启发式策略。候选码求解的一般步骤是：首先找出所有不出现在任何函数依赖右部的属性，因为这些属性不可能由其他属性推导出来，必须包含在所有候选码中。

给定的 FD 集为 $\{A \rightarrow D, B \rightarrow C, E \rightarrow A\}$, 观察右部出现的属性有 D (在 $A \rightarrow D$ 中)、C (在 $B \rightarrow C$ 中)、A (在 $E \rightarrow A$ 中)。B 和 E 均未出现在任何 FD 的右部，因此 B 和 E 必须属于任何候选码。然后逐一验证各个选项：对选项 B (BE)，计算 $(BE)^+ = \{B, E\} \xrightarrow{E \rightarrow A} \{B, E, A\} \xrightarrow{A \rightarrow D} \{B, E, A, D\} \xrightarrow{B \rightarrow C} \{B, E, A, D, C\} = \{A, B, C, D, E\} = R$, 故 BE 是超码。再检验极小性： $B^+ = \{B, C\} \neq R$, $E^+ = \{E, A, D\} \neq R$, 因此 BE 是极小候选码。

其他选项的闭包计算不能覆盖全属性集 R: $(AB)^+ = \{A, B, C, D\} \neq R$, $(CD)^+ = \{C, D\} \neq R$, $(DE)^+ = \{D, E, A\} \neq R$ 。

因此正确答案是 BE (选项 B)。

知识点：第六章：关系数据理论——6.2 函数依赖与候选码求解（基于闭包计算的候选码求解算法：先找出不在右部出现的属性集，再逐步扩展并验证极小性）

8. 事务的原子性是指 ()。

参考答案

(答案由解析者提供) A

解析与知识点

本题考察对事务 ACID 特性之一——原子性 (Atomicity) 的准确理解。事务的 ACID 特性是数据库事务处理理论的核心概念，各特性之间有明确的定义和区分。原子性 (Atomicity) 指的是事务中包括的所有操作要么全部成功执行并提交 (COMMIT)，要么由于故障全部撤销 (UNDO / ROLLBACK)，数据库回滚到事务开始前的状态。这就像化学中的原子一样不可再分——事务是最小的逻辑工作单元。选项 A 的描述“事务中包括的所有操作要么都做，要么都不做”正是原子性的精

准定义。选项 B 描述的是持久性 (Durability) ——事务一旦提交, 其对数据库的修改就是永久性的, 即使随后发生系统故障也不会丢失。选项 C 描述的是隔离性 (Isolation) ——并发执行的事务之间互不干扰, 每个事务都感觉不到其他并发事务的存在。

选项 D 描述的是一致性 (Consistency) ——事务的执行保证数据库从一个一致性状态转换到另一个一致性状态 (这通常是事务编程者的责任, 而原子性、隔离性和持久性是 DBMS 保障一致性的手段)。ACID 是事务正确性的四个保障维度, 理解各特性的精确定义是掌握事务管理和并发控制理论的基础。

知识点: 第十章: 事务与并发控制——10.1 事务的 ACID 特性 (原子性 Atomicity、一致性 Consistency、隔离性 Isolation、持久性 Durability 的定义与区别)

9. 下面哪种不属于数据库安全技术 ()。

参考答案

(答案由解析者提供) C

解析与知识点

本题考察对数据库安全技术与数据库恢复技术的概念区分能力。数据库安全技术关注的是“谁能访问什么数据”, 其目标是在保证合法用户正常访问的前提下, 防止非法用户的越权访问和数据泄露, 确保数据的保密性、完整性和可用性。存取控制 (Access Control) 通过 GRANT 和 REVOKE 等机制授予或回收不同用户对数据库对象的操作权限, 是安全性的核心手段。

视图机制 (View) 可以隐藏敏感数据列或行, 使用户只能看到其被授权的数据子集 (如只看到本部门的职工信息), 属于安全性中的“视图隔离”策略。

审计 (Audit) 通过记录用户对数据库的所有操作 (谁、什么时间、做了什么) 形成不可篡改的操作日志, 用于事后追查和责任认定, 也属于数据库安全性的范畴。而数据库镜像 (Mirroring) 是指数据从一台存储设备实时复制到另一台设备的技术, 其主要目的是提高系统的可用性——当主设备故障时立即切换到镜像设备继续服务, 同时也间接支持数据恢复 (防范介质故障) ——但它并不涉及“谁能访问”的安全授权和控制问题。因此镜像属于数据库恢复和高可用性技术, 而非安全技术。

知识点: 第四章: 数据库安全性——4.1 安全性概述、4.2 存取控制、4.3 视图与审计 (安全性关注“谁能访问”); 第十一章: 数据库恢复——11.3 数据镜像 (镜像关注“数据如何不丢失”和“服务不中断”)

10. 在关系模式 $R(C, S, Z)$ 中, 有函数依赖集 $F = \{(C, S) \rightarrow Z, Z \rightarrow C\}$, 则 R 能达到 ()。

参考答案

(答案由解析者提供) C

解析与知识点

本题考察关系模式范式等级的判定过程, 是规范化理论中 BCNF 判定的经典题型。范式判定的标准流程是: 先求出所有候选码, 再逐一检查各层级范式条件是否满足。

首先计算候选码: $(C, S)^+ = \{C, S, Z\} = R$, 故 (C, S) 是候选码。由 $Z \rightarrow C$, $(Z, S)^+ = \{Z, S, C\} = R$, 故 (Z, S) 也是候选码。

因此 R 有两个候选码 (C, S) 和 (Z, S) , 三个属性 C, S, Z 全部属于某个候选码, 全部是主属性——这意味着不存在任何非主属性, 自然也就不存在非主属性对码的部分函数依赖 (满足 2NF), 也不存在非主属性对码的传递函数依赖 (满足 3NF)。因此 R 必定满足 3NF。接下来检查 BCNF: BCNF 的定义要求关系中成立的每一个非平凡函数依赖 $X \rightarrow Y$, X 都必须是超码。

检查 FD $Z \rightarrow C$: $Z^+ = \{Z, C\} \neq R$, Z 不是超码 (尽管 C 是主属性, 但决定因素 Z 不是超码), 违反了 BCNF 条件。因此 R 不满足 BCNF。最终结论: R 的最高范式是 3NF。这个例子同时也是“全主属性关系不一定是 BCNF”的经典反例——虽然所有属性都是主属性, 但由于存在主属性对码的传递依赖 (或 BCNF 违规), 关系仍可能达不到 BCNF。

知识点: 第六章: 关系数据理论——6.3 范式 (1NF、2NF、3NF、BCNF 的定义与判定流程: 求候选码 $\rightarrow$ 检查部分依赖 $\rightarrow$ 检查传递依赖 $\rightarrow$ 检查 BCNF 违规)

5.3 三、判断改错题 (每小题 2 分, 共 10 分)

1. 在关系数据库中, 用户只能通过基本关系操作数据库中的数据。()

参考答案

(答案由解析者提供) 错误 (×)

解析与知识点

本题考察对数据库用户访问数据路径多样性的认识。题干“用户只能通过基本关系 (基表) 操作数据”的说法是错误的。

用户访问数据库中的数据有多种途径，基本关系（基表 Base Table）只是其中之一。

数据库系统提供了外模式（视图 View）机制，用户可以通过视图来查询数据，在某些满足可更新视图条件的情况下甚至可以通过视图来更新数据。视图是从一个或多个基表导出的虚拟表，它向用户呈现的是经过过滤、重组和抽象的数据视图，一方面简化了复杂查询的表达，另一方面也提供了重要的安全隔离层——用户只能看到被授权的数据子集。此外，数据库编程中的存储过程（Stored Procedure）封装了复杂的业务逻辑，用户可以通过调用存储过程来间接操作数据；触发器（Trigger）则在特定事件发生时自动执行预定义操作。因此，用户的数据库访问方式远比“仅通过基本关系”丰富得多。

该判断题混淆了数据的实际存储载体（基表）与用户的操作界面（可以是基表、视图、存储过程等），没有认识到三级模式架构中“外模式”所扮演的重要角色。

知识点：第三章：SQL 语言——3.5 视图的定义与使用（视图作为外模式给用户提供操作界面）；第一章：绪论——1.3 三级模式结构中外模式的概念与作用

2. 若关系模式 R 中的属性全部是主属性，则 R 必定是 BCNF。（ ）

参考答案

（答案由解析者提供） 错误（×）

解析与知识点

本题考察全主属性关系与范式等级之间的关系，这是规范化理论中一个容易混淆的重要知识点。全主属性的关系模式一定满足 3NF——因为不存在非主属性，所以自然不会有非主属性对码的部分函数依赖（满足 2NF），也不会有非主属性对码的传递函数依赖（满足 3NF）。但全主属性的关系不一定满足 BCNF。反例：设 $R(A, B, C)$ ， $F = \{AB \rightarrow C, C \rightarrow B\}$ 。

首先求候选码： $(AB)^+ = \{A, B, C\} = R$ ， $(AC)^+ = \{A, C, B\} = R$ ，故 (AB) 和 (AC) 均为候选码。三个属性 A 、 B 、 C 全部是主属性（均属于某个候选码）。

因此不存在非主属性， R 满足 3NF。

但检查 BCNF：对于 FD $C \rightarrow B$ ， $C^+ = \{C, B\} \neq R$ （不包含 A ），故 C 不是超码，违反了 BCNF 条件。该反例清楚地说明了“全主属性”只能保证到 3NF，不能保证到 BCNF。BCNF 的突破之处在于它不仅要求非主属性不依赖于码，还要求主属性之间也不能存在不被候选码“掌控”的函数依赖关系。因此该判断题的说法是错误的。

知识点：第六章：关系数据理论——6.3 范式（全主属性关系必满足 3NF，但不一

定满足 BCNF——反例： $F = \{AB \rightarrow C, C \rightarrow B\}$

3. 若事务 T_1 对数据对象 A 上了 IS 锁，则事务 T_2 不能对数据对象 A 上 SIX 锁。()

参考答案

(答案由解析者提供) 正确 (✓)

解析与知识点

本题考察多粒度封锁协议中意向锁相容性矩阵的理解。在多粒度封锁机制中，为了支持在不同粒度级别上灵活加锁，引入了三种意向锁：IS 锁（意向共享锁，Intention Shared）——表示将在下层节点加 S 锁或 IS 锁；IX 锁（意向排他锁，Intention Exclusive）——表示将在下层节点加 X 锁或 IX 锁；SIX 锁（共享意向排他锁）——是 S 锁和 IX 锁的组合，表示持有者对整个对象有读权限，且有对下层节点加排他锁的意向。

根据多粒度封锁的相容性矩阵，IS 锁与 SIX 锁是不相容的——原因在于 SIX 锁隐含有 IX 锁的分量（意向排他），而 SIX 同时包含的 S 锁分量与 IS 的“下层有 S 锁”在下层会产生冲突（S 锁与 X 锁互斥），所以在当前层就需要判定不相容。

在标准相容性矩阵中：IS 与 IX 相容，因为两者都只是“意向”，不存在对当前节点的实际冲突操作——意向下层的 S 和 X 锁在下层另行检查。但 IS 与 SIX 不相容，因为 SIX 中包含了 S 锁分量（在当前节点有实际的读锁），而 IS 锁虽然本身不实际读，但其含义是“下层有 S 锁”，与 SIX 的“下层要加 X 锁”在下层会产生冲突，所以在当前层就不相容。理解意向锁的相容性矩阵，需要区分“当前节点”和“下层节点”两个层面的锁冲突判定。

初学者容易将 IS 与 IX 的相容性（两者都相容）错误地推广到 IS 与 SIX，认为它们也相容。实际上 SIX 同时携带 S 和 IX 的分量，其 S 分量与下层 X 锁意向形成冲突，导致 IS 与 SIX 不相容。因此该判断题的判断是正确的。

知识点：第十章：事务与并发控制——10.3 封锁机制与封锁协议（多粒度封锁的意向锁类型 IS/IX/SIX 及其相容性矩阵；IS 与 SIX 不相容）

4. 规范化过程中，关系的分解不是唯一的。()

参考答案

(答案由解析者提供) 正确 (✓)

解析与知识点

本题考察规范化理论中模式分解不唯一性的理解。关系模式分解的目的是消除数据冗余和操作异常，但分解的路径和结果确实不是唯一的，这是规范化理论中的一个重要事实。

分解的不唯一性来源于多个方面：第一，函数依赖集中存在多个可以被处理的违规依赖时，选择不同的依赖先进行分解可能导致不同的中间结果和最终分解结果。例如，对 $R(A, B, C, D)$ 若同时存在 $A \rightarrow B$ 和 $C \rightarrow D$ ，可以先基于 $A \rightarrow B$ 分解再处理 $C \rightarrow D$ ，也可以先基于 $C \rightarrow D$ 分解再处理 $A \rightarrow B$ ——两种分解路径产生的结果形式可能不同。

第二，在删除冗余函数依赖以求最小覆盖时（如前面判断题所述），选择删除顺序的不同可能导致不同的最小集 F_m ，进而导致不同的 3NF 分解结果。第三，对于一个给定的函数依赖 $X \rightarrow Y$ ，分解时可能选择不同的方式（如将 R 分解为 $\{XY, R - Y\}$ 或 $\{XY, X(R - Y)\}$ 等）。尽管分解结果形式上可能不同，但只要所有分解都满足无损连接性 (Lossless Join) 和依赖保持性 (Dependency Preservation, 对 3NF 分解)，不同的分解在数据意义上彼此等价。因此该判断题的说法正确。

知识点：第六章：关系数据理论——6.4 模式分解（分解的不唯一性来源于依赖处理顺序和最小集的不唯一；无损连接性和依赖保持性是分解等价性的保证）

5. 查询优化时，尽可能先做笛卡尔积。（ ）

参考答案

（答案由解析者提供） 错误（×）

解析与知识点

本题考察查询优化中代数优化启发式规则的核心思想，特别是对笛卡尔积与选择/连接操作执行顺序的理解。题干的说法恰好与查询优化的标准策略完全相反——正确的策略是尽可能推迟笛卡尔积运算，而不是先做。

笛卡尔积是关系代数中代价最高的运算之一，其结果规模是两个参与运算的关系的元组数量之乘积。

如果有三个关系 $R(1000\text{行})$ 、 $S(10000\text{行})$ 和 $T(5000\text{行})$ ，全量笛卡尔积将产生 500 亿行的中间结果，对内存和磁盘空间都是灾难性的。代数优化的核心启发式规则为：（1）尽早执行选择运算——选择运算通常具有很强的过滤效果（高选择性），能大幅缩减元组数量，必须先做；（2）尽早执行投影运算——投影运算减少属性列的数量，缩小每条元组的宽度；（3）将笛卡尔积与选择条件结合，转换为等值连接或自然连接——连接运算可以使用索引、散列等高效实现，远比笛卡尔积后

加选择高效。

概括而言，优化策略是“先过滤、后连接、推迟甚至避免纯粹的笛卡尔积”。该判断题完全倒置了优化顺序，因此错误。

知识点：第九章：查询处理与优化——9.2 代数优化（启发式代数优化规则：先做选择和投影缩减数据规模，后将笛卡尔积与选择结合转为连接，尽可能推迟或避免笛卡尔积）

5.4 四、简答题（每小题 5 分，共 20 分）

1. 数据库设计的几个阶段及其主要任务是什么？

参考答案

（答案由解析者提供）

数据库设计通常分为以下六个阶段：

- (1) 需求分析阶段：调查和分析用户的应用需求，产生数据流图（DFD）和数据字典（DD）等文档。
- (2) 概念结构设计阶段：将需求分析得到的用户需求抽象为信息结构，建立独立于具体 DBMS 的概念模型，常用工具为 E-R 图。
- (3) 逻辑结构设计阶段：将概念模型（E-R 图）转换为选定的 DBMS 所支持的数据模型（如关系模型），并进行优化和规范化处理。
- (4) 物理结构设计阶段：为逻辑数据模型选取最适合应用环境的物理存储结构和存取方法，如确定索引、聚簇等。
- (5) 数据库实施阶段：运用 DBMS 提供的数据库定义语言建立数据库结构，编写和调试应用程序，组织数据入库。
- (6) 数据库运行和维护阶段：对数据库系统进行日常运行管理、性能监测、数据备份与恢复、安全性控制和数据库重构等。

解析与知识点

本题考察对数据库设计结构化生命周期方法的完整理解。数据库设计是一个从现实世界到信息世界再到机器世界的逐步抽象、转化的系统工程过程，遵循结构化生命周期（SDLC）方法。

六个阶段各司其职且有序推进：需求分析是基础——通过调查用户业务活动、分析信息流程，产出数据流图（DFD）和数据字典（DD），明确系统要“做什么”。概念结构设计是关键——将用户需求抽象为独立于任何具体 DBMS 的信息模型（E-R 图），这是从现实世界到信息世界的第一次抽象，关注的是信息的本体结构而非实现细节。逻辑结构设计是转化——将概念模型按照转换规则转化为特定 DBMS 支持的数据模型（关系模型的表结构），并应用规范化理论消除数据冗余和操作异常。物理结构设计是落地——为逻辑模式选择存储结构（如堆文件、散列文件）、设计存取路径（索引设计）和分配存储空间。数据库实施是将设计转化为可运行系统的执行阶段。

运行和维护是时间最长、投入最大的阶段，包含性能调优、备份恢复、安全管理、数据库重构等持续活动。这六个阶段既有序推进（瀑布模型特点），也存在反馈和迭代——后续阶段的发现可能需要回溯修改前序设计的决策。

知识点：第七章：数据库设计——7.1 数据库设计概述（数据库设计的六个阶段：需求分析 → 概念设计 → 逻辑设计 → 物理设计 → 实施 → 运行维护）

2. 登记日志文件的原则是什么？登记日志文件为什么必须先写日志文件，后写数据库？

参考答案

（答案由解析者提供）

登记日志文件的两条原则：

- （1）登记的次序严格按并发事务执行的时间次序。
- （2）必须先写日志文件，后写数据库（Write-Ahead Logging，WAL 原则）。

先写日志后写数据库的原因：如果先修改数据库、再将操作记录写入日志，在“数据库已修改但日志尚未记录”的间隙发生系统故障，则数据库已被修改却无日志可查，恢复程序无法确定应 UNDO 还是 REDO。而遵循 WAL 原则，任何数据修改前都先确保日志记录已持久化存储。即使写数据库时发生故障，恢复程序也可根据日志中的前像（Before Image）撤销未完成操作，或根据后像（After Image）重做已提交但未写入磁盘的操作，从而保证事务原子性和数据库一致性。

解析与知识点

本题考察数据库恢复技术中最核心的 WAL（Write-Ahead Logging）原则的深刻理解。登记日志文件的两条原则——按事务执行的时间次序登记和先写日志后写数

数据库——是保证故障恢复正确性和完备性的基石。

时间次序原则确保恢复程序能够按照操作发生的先后顺序正确地 UNDO 或 REDO，不会打乱事务之间的因果依赖关系。而 WAL 原则的重要性需要从故障安全 (Fail-Safe) 的角度来理解：数据修改操作实际上由两个步骤组成——写入日志记录（描述修改了什么）和写入数据库（实际执行修改）。

在“数据库已修改但日志尚未写入”的危险窗口中，如果系统发生崩溃，数据库已被改变但没有任何关于这次改变的记录——恢复系统既不知道应该 UNDO（因为没有记录表明这属于未提交事务的操作），也不知道应该 REDO（因为记录从未存在过），数据库陷入不可恢复的不一致状态。而如果反过来，先确保日志记录已经持久化到磁盘上的日志文件，再对数据库执行修改：在写数据库之前发生故障——数据库未改变，日志记录无效或标记为未完成，系统重启后忽略即可；写数据库过程中发生故障——可以向日志追加 UNDO 记录来撤销已写入的部分数据，或者依据日志中的后像进行 REDO。

因此 WAL 原则为恢复系统提供了一个“安全网”——在任何故障场景下，都存在日志记录作为依据来判断如何恢复。初学者常误认为“先写数据库后写日志”也是可行的，这忽略了“已修改但无日志”这一危险故障窗口。

知识点：第十一章：数据库恢复——11.2 日志文件与恢复技术（WAL 原则的必然性：保障在任何故障时都有日志记录可依；两条登记原则是恢复正确性的前提）

3. 什么是数据库的完整性？DBMS 的完整性控制机制有哪些功能？

参考答案

（答案由解析者提供）

数据库完整性：指数据库中数据的正确性 (Correctness)、有效性 (Validity) 和相容性 (Consistency)，即数据必须符合现实世界的语义规则和业务逻辑约束。

DBMS 完整性控制机制的三方面功能：

- (1) 定义功能：提供定义完整性约束条件的机制（如 PRIMARY KEY、FOREIGN KEY、NOT NULL、UNIQUE、CHECK 子句以及断言 ASSERTION）。
- (2) 检查功能：在插入、删除、修改操作时，自动检查是否违反已定义的约束。
- (3) 违约处理功能：当检测到违规时，按预定义策略处理（拒绝执行/REJECT、级联操作/CASCADE、设为空值/SET NULL 等）。

解析与知识点

本题考察数据库完整性概念的全面理解以及 DBMS 完整性控制机制的三个功能环节。数据库完整性是衡量数据质量的核心标准——正确性指数据值符合现实世界的客观事实，有效性指数据值在语义上是有意义的（如年龄不能是负数），相容性指同一数据在多处使用时保持一致（不矛盾）。

完整性与安全性是两个不同的概念范畴：安全性关注“谁能访问什么数据”（访问控制），而完整性关注“数据自身的值是否符合规则”（数据质量）。

DBMS 的完整性控制机制构成了一个完整的“定义—检查—处理”的闭环控制系统。首先，定义功能允许 DBA 或应用程序开发者通过 DDL 语句声明约束——实体完整性通过 PRIMARY KEY 声明，参照完整性通过 FOREIGN KEY 声明，用户定义的完整性通过 CHECK、NOT NULL、UNIQUE 以及断言（ASSERTION）声明。其次，检查功能由 DBMS 在每一次数据变更操作时自动触发，无需应用程序额外编码。

最后，违约处理功能提供多种策略——REJECT 拒绝违规操作（最常用），CASCADE 级联传播变更（如删除父表记录时级联删除子表相关记录），SET NULL 将违规外码置为空值。断言和触发器作为更灵活的工具，用于实现跨表、跨操作、带复杂逻辑的约束——当声明式约束不足以表达业务规则时，触发器是强有力的补充手段。

知识点：第五章：数据库完整性——5.1 完整性概念（正确性、有效性、相容性三要素）、5.2 实体完整性、5.3 参照完整性、5.4 用户定义完整性及断言（定义—检查—处理三步闭环机制）

4. 当前数据库技术发展的主要特征是什么？

参考答案

（答案由解析者提供）

当前（2002–2003 年）数据库技术发展的主要特征包括：

- （1）面向对象与对象-关系数据库：面向对象方法融入数据库，对象-关系数据库（ORDB）成为新一代主流方向。
- （2）Web 数据库技术：数据库与 Web 技术融合，实现 Internet 环境下的数据存取与发布。
- （3）数据仓库与联机分析处理（OLAP）：从操作型 OLTP 向分析型 OLAP 拓展，支持多维数据建模与决策。

- (4) 数据挖掘 (Data Mining): 从海量数据中自动发现知识、模式和关联规则。
- (5) 分布式与并行数据库: 数据物理分布、逻辑统一; 利用多处理器并行提高性能。
- (6) 多媒体数据库: 管理图像、音视频等非结构化数据。
- (7) XML 与半结构化数据管理: 支持 Internet 数据交换与半结构化数据的存储查询。

解析与知识点

本题考察对数据库技术发展趋势的宏观认知。2000 年代初, 数据库领域正处于从传统集中式关系数据库向多元化、网络化、智能化方向全面拓展的关键转型期。发展主线可以从四个维度来归纳。

第一, 从”结构化”到”多模态”——传统关系模型擅长处理严格结构化的二维表数据, 但难以高效处理复杂对象、空间数据、多媒体内容等非结构化或半结构化数据, 这催生了面向对象数据库 (OODB) 和对象-关系数据库 (ORDB) 的发展, 它们在关系模型基础上融合了继承、封装和多态等面向对象特性。第二, 从”OLTP”到”OLAP”——数据库不再仅仅是支撑企业日常事务处理的操作型系统, 数据仓库技术和联机分析处理 (OLAP) 将数据库的功能扩展到战略决策支持领域, 支持多维数据建模和复杂分析查询, 形成了”操作型数据库 + 分析型数据仓库”的双层体系。第三, 从”单机集中式”到”网络化分布式”——Internet 的爆炸式增长使 Web 数据库成为必然趋势, 同时分布式数据库技术将数据分布在网络多节点上实现逻辑统一管理。

第四, 从”管理数据”到”发现知识”——数据挖掘和知识发现技术的兴起使数据库从被动的数据存储工具跃升为主动的知识发现平台。

知识点: 第一章: 绪论——1.4 数据库技术的发展历程 (面向对象/ORDB、Web 数据库、数据仓库与 OLAP、数据挖掘、分布式/并行数据库等新技术方向的演进)

5.5 五、查询设计题 (每小题 4 分, 共 24 分)

已知某供应数据库中包含四个基本表:

- S(SNO, SNAME, CITY): 供应商表, SNO: 供应商号, SNAME: 供应商名, CITY: 所在城市
- J(JNO, JNAME, CITY): 工程表, JNO: 工程号, JNAME: 工程名, CITY: 所在城市

- PART(PNO, PNAME, COLOR, WEIGHT): 零件表, PNO: 零件号, PNAME: 零件名, COLOR: 颜色, WEIGHT: 重量
- SPJ(SNO, PNO, JNO, QTY): 供货表, SNO: 供应商号, PNO: 零件号, JNO: 工程号, QTY: 供应数量

1. 关系代数查询

(1) 查询为所有工程供货的供应商名。

参考答案

(答案由解析者提供)

$$\pi_{\text{SNAME}}(S \bowtie (\pi_{\text{SNO}, \text{JNO}}(\text{SPJ}) \div \pi_{\text{JNO}}(\text{J})))$$

即: 先求出所有供应了所有工程的供应商号 (除运算), 再与供应商表自然连接获取供应商名。

解析与知识点

这是关系代数中除法运算 (Division) 的经典应用, 考察”全部”类查询 (全称量词查询 Universal Quantification) 的代数表达。查询要求找出”为所有工程供货的供应商”, 关键词”所有”意味着这是一个典型的全称量词语义问题, 在关系代数中只能通过除法运算 ($\div$) 来直接表达。

解题逻辑分为三步: 第一步, $\pi_{\text{JNO}}(\text{J})$ 通过投影获取所有工程号的完整集合 (即”全集”), 这是除法的除数。第二步, $\pi_{\text{SNO}, \text{JNO}}(\text{SPJ})$ 通过投影获取每个供应商实际供应的 (供应商号, 工程号) 对集合, 这是除法的被除数。第三步, 除法 $\pi_{\text{SNO}, \text{JNO}}(\text{SPJ}) \div \pi_{\text{JNO}}(\text{J})$ 的语义是: 找出那些在 SPJ 中与 J 中所有工程号都有关联的供应商号——即”供应了全部工程”的供应商号。最后与供应商表 S 做自然连接来获取供应商名, 并通过投影输出最终结果。

除法运算的严格定义为 $R \div S = \{t \mid \forall s \in S, \langle t, s \rangle \in R\}$, 即 R 中与 S 的每个值都配对出现的那些属性值。

理解除法运算的语义是掌握关系代数高级查询表达的关键。

知识点: 第二章: 关系数据库——2.4 关系代数 (除法运算 $\div$ 的定义与语义: $R \div S = \{t \mid \forall s \in S, \langle t, s \rangle \in R\}$, 用于表达全称量词语义)

(2) 查询为工程号 J1 供货且与该工程位于同一城市的供应商名。

参考答案

(答案由解析者提供)

$$\pi_{\text{SNAME}}\left(\sigma_{\text{JNO}='J1'\wedge\text{S.CITY}=\text{J.CITY}}(\text{S} \bowtie \text{SPJ} \bowtie \text{J})\right)$$

即：先将 S、SPJ、J 自然连接，选择出 JNO 为 J1 且供应商所在城市与工程所在城市相同 (S.CITY=J.CITY) 的元组，再投影出供应商名。

解析与知识点

该查询需要同时满足两个条件：(1) 供应给工程 J1 (限制供货关系)，(2) 供应商所在城市与该工程所在城市相同 (地理匹配条件)。

首先将三个表 S、SPJ、J 通过自然连接合并为一个大中间表——S 与 SPJ 通过公共属性 SNO 自然连接，SPJ 与 J 通过公共属性 JNO 自然连接，得到包含供应商信息、供货信息和工程信息的完整宽表。然后在连接结果上施加复合选择条件 $\sigma_{\text{JNO}='J1'\wedge\text{S.CITY}=\text{J.CITY}}$ ，最后用投影提取所需的供应商名 (SNAME)。

由于 S 表和 J 表都有 CITY 属性，在选择条件中必须加上关系名前缀 (S.CITY 和 J.CITY) 以消除歧义——这是多表查询中的一个标准做法。此题也可以采用分解策略：先找到供应给 J1 的供应商号集合，获取其所在城市；再检查该城市是否与 J1 所在城市匹配——两种方法等价，但“先连接再选择”方式更为直观和简洁，适合在关系代数表达式中一步完成。

使用自然连接时需确保三个表之间有公共属性可用于连接——S 和 SPJ 通过 SNO，SPJ 和 J 通过 JNO。如果连接属性名不同，则需要使用 θ -连接显式指定连接条件。

知识点：第二章：关系数据库——2.4 关系代数 (多表自然连接 $\bowtie$ 、复合选择条件 σ 、投影 π 的综合运用；歧义属性的前缀处理)

2. SQL 查询

(1) 查询至少为工程号 J1 和 J2 供货的供应商号。

参考答案

(答案由解析者提供)

```
1 SELECT SNO FROM SPJ WHERE JNO = 'J1'
2 INTERSECT
3 SELECT SNO FROM SPJ WHERE JNO = 'J2';
```


或者使用子查询：

```
1 SELECT DISTINCT SNO FROM SPJ
2 WHERE JNO = 'J1'
3    AND SNO IN (SELECT SNO FROM SPJ WHERE JNO = 'J2');
```

解析与知识点

”至少为 J1 和 J2 供货”意味着该供应商号必须同时出现在 JNO='J1' 和 JNO='J2' 的供货记录中，即供应商号的交运算（Intersection）。使用 INTERSECT 集合操作取交集是最直观、最符合语义的写法：分别查询为 J1 供货和为 J2 供货的供应商号集合，然后对两个结果集做交运算找出共有元素。第二种方案通过嵌套子查询来等价实现：外层查询先找出为 J1 供货的供应商号，再通过 IN 子查询进一步限制这些供应商号必须也出现在为 J2 供货的记录中。

两种写法在语义上等价，但在执行效率上可能有所不同——INTERSECT 使查询意图更明确，SQL 优化器可以更有效地选择执行策略。

使用 DISTINCT 是为了避免同一供应商对同一工程有多次供货（如供应了不同零件给同一工程）而产生重复的供应商号影响结果。

这实际上是集合交的”至少”语义——供应商号的集合必须满足同时属于两个工程各自的供应清单。

知识点：第三章：SQL 语言——3.4 数据查询（集合操作 INTERSECT 取交集与嵌套子查询 IN 的等价表达；DISTINCT 消除重复行）

(2) 查询不供应 P2 零件号的供应商号和供应商名。

参考答案

（答案由解析者提供）

```
1 SELECT SNO, SNAME FROM S
2 WHERE SNO NOT IN (
3     SELECT SNO FROM SPJ WHERE PNO = 'P2'
4 );
```

或者使用 NOT EXISTS:

```
1 SELECT SNO, SNAME FROM S
2 WHERE NOT EXISTS (
3     SELECT * FROM SPJ
4     WHERE SPJ.SNO = S.SNO AND SPJ.PNO = 'P2'
```

```
5 );
```

解析与知识点

“不供应 P2 零件”是一个典型的否定查询，对应的是集合的差运算——从全体供应商的集合中排除供应过 P2 的供应商。

第一种方案使用 NOT IN 子查询实现这一差集逻辑：先通过子查询 SELECT SNO FROM SPJ WHERE PNO = 'P2' 获取所有供应过 P2 零件的供应商号集合，然后在主查询中用 NOT IN 排除这些供应商号，选出剩下的供应商。第二种方案使用 NOT EXISTS 关联子查询（Correlated Subquery）表达：对 S 表中的每个供应商（逐行），在 SPJ 表中检查是否存在 SNO 相同且 PNO='P2' 的记录——如果不存在，则该供应商符合条件。

NOT EXISTS 是相关子查询，子查询引用了外层表 S 的 SNO 列，因此对外层查询的每一行都需要执行一次子查询。二者语义等价，但在性能推荐上有所不同：当子查询结果可能包含 NULL 值时，NOT IN 可能返回意想不到的空结果（因为任何值与 NULL 作 IN / NOT IN 比较都返回 UNKNOWN），而 NOT EXISTS 则不受 NULL 影响。因此在实际开发中，NOT EXISTS 更受推荐用于否定类查询。

初学者容易忽略 NULL 值对 NOT IN 语义的影响——只要子查询结果集中包含一个 NULL 值，整个 NOT IN 条件可能始终返回 UNKNOWN，导致外层查询不返回任何行。这不是语法错误，而是 SQL 三值逻辑的内在特性。

知识点：第三章：SQL 语言——3.4 数据查询（NOT IN 集合差与 NOT EXISTS 相关子查询的等价表达；NULL 值对 NOT IN 语义的特殊影响）

(3) 查询重量低于红色螺丝刀的零件号、零件名和颜色。

参考答案

（答案由解析者提供）

```
1 SELECT PNO, PNAME, COLOR FROM PART
2 WHERE WEIGHT < (
3     SELECT WEIGHT FROM PART
4     WHERE PNAME = '螺丝刀' AND COLOR = '红色'
5 );
```

解析与知识点

此题需要使用标量子查询 (Scalar Subquery) 来获取基准值进行逐行比较。标量子查询是一种只返回单个值 (一行一列) 的子查询, 可以出现在比较运算符 (=、<、>、<=、>=、<>) 的右侧。

解题逻辑: 内层子查询 `SELECT WEIGHT FROM PART WHERE PNAME = '螺丝刀' AND COLOR = '红色'` 用于获取“红色螺丝刀”的重量值作为比较基准, 假设表中存在这样的记录且只有一个重量值 (或用 `MAX/MIN` 等聚合函数确保返回单值)。外层查询遍历 `PART` 表中的每一行零件记录, 将其重量 `WEIGHT` 与子查询返回的基准值进行比较, 只保留重量更小的零件, 最后投影输出零件号、零件名和颜色三个属性。

该查询假设 `PART` 表中确实存在“红色螺丝刀”这一记录——如果不存在, 子查询返回空集 (`NULL`), 外层 `WHERE` 子句的条件变成 `WEIGHT < NULL`, 在 SQL 三值逻辑中, 任何与 `NULL` 的比较结果都是 `UNKNOWN` (既不是 `TRUE` 也不是 `FALSE`), `WHERE` 子句只接受 `TRUE`, 因此最终不会返回任何行。

这种标量子查询模式是数值比较类查询中常用的设计模式。

知识点: 第三章: SQL 语言——3.4 数据查询 (标量子查询: 返回单值的子查询可作为比较运算符的操作数; `NULL` 在三值逻辑中的传播特性)

- (4) 查询供应商数超过 4 个的工程号和供应商个数。输出时按供应商个数升序排列, 供应商个数相同时, 按工程号降序排列。

参考答案

(答案由解析者提供)

```
1 SELECT JNO, COUNT(DISTINCT SNO) AS SupplierCount
2 FROM SPJ
3 GROUP BY JNO
4 HAVING COUNT(DISTINCT SNO) > 4
5 ORDER BY SupplierCount ASC, JNO DESC;
```

解析与知识点

该查询综合考察 SQL 中分组 (`GROUP BY`)、聚合函数 (`COUNT`)、分组过滤 (`HAVING`) 和多级排序 (`ORDER BY`) 的联合使用。查询的逻辑分解如下: 首先, `GROUP BY JNO` 将供货记录 `SPJ` 表按工程号分成若干组, 每个组包含该工程的所有供货记录。

其次, COUNT(DISTINCT SNO) 作为聚合函数对每个组内的不同供应商号进行去重计数——注意这里必须使用 DISTINCT, 因为同一供应商可能为该工程供应多种不同零件 (或多次供应同一种零件), 如果不加 DISTINCT, 重复的供应商号会被多次计数, 导致统计偏差。第三, HAVING COUNT(DISTINCT SNO) > 4 对分组结果进行筛选, 只保留供应商数量超过 4 个的工程——这里必须使用 HAVING 而非 WHERE, 因为 COUNT 是聚合函数, WHERE 子句在分组之前执行, 不能包含聚合函数, 而 HAVING 在分组之后执行, 专门用于对分组结果施加条件。

第四, ORDER BY SupplierCount ASC, JNO DESC 实现两级排序——首先按供应商个数升序排列 (个数少的在前), 在供应商个数相同时按工程号降序排列 (编号大的在前)。

多级排序时各排序关键字的优先级从左到右递减。

知识点: 第三章: SQL 语言——3.4 数据查询 (GROUP BY 分组 → COUNT(DISTINCT) 去重计数 → HAVING 分组后条件 → 多级 ORDER BY 排序; WHERE 与 HAVING 的执行时机区别)

5.6 六、数据库设计 (每小题 10 分, 共 20 分)

六-1. 关系模式 BCL 的规范化

关系模式 BCL(BNO, CITY, SSETS, CNO, NAME, ADDR, LNO, AMOUNT), 各属性依次为支行号、支行所在城市、支行总资产、客户号、客户名、客户地址、贷款号、贷款金额。设一个客户可贷多笔贷款, 一笔贷款可由多个客户共同贷款; 贷款由各个支行发出, 一笔贷款只能由一个支行发出, 贷款号在各支行唯一。试分析该关系模式存在的问题并用规范化理论将其分解为合理的关系模式。

参考答案

(答案由解析者提供)

Step 1: 分析语义与函数依赖 根据题意:

- 一个客户可贷多笔贷款, 一笔贷款可由多个客户共同贷款
- 一笔贷款只能由一个支行发出
- 贷款号在各支行唯一 (即 LNO 在各支行内局部唯一)

由此分析函数依赖集 F ：

$BNO \rightarrow CITY, SSETS$ （支行号决定支行所在城市与总资产）

$CNO \rightarrow NAME, ADDR$ （客户号决定客户名与地址）

$LNO \rightarrow BNO, AMOUNT$ （贷款号决定发款支行与贷款金额）

因客户与贷款是多对多的关系，在 BCL 表中，一个合理的候选码可设为 (CNO, LNO)（一笔贷款能被多个客户拥有，一个客户能贷多笔贷款），该码唯一标识一个“某客户参与某贷款”的关联事实。

Step 2：存在的问题分析

- (1) 插入异常 (Insertion Anomaly)：若一个新支行尚未发贷，因 BCL 表中必须有 CNO 和 LNO（主码的一部分），该支行的信息无法插入。
- (2) 删除异常 (Deletion Anomaly)：若某支行发放的唯一一笔贷款被全部结清（删除记录），则该支行的信息也随之丢失。
- (3) 更新异常 (Update Anomaly)：若某支行所在城市变更，需修改所有包含该支行信息的元组，修改过程中可能遗漏导致数据不一致。
- (4) 数据冗余 (Data Redundancy)：支行信息 (CITY, SSETS) 和客户信息 (NAME, ADDR) 在各自的每笔贷款记录中重复存储，浪费空间。
- (5) 范式问题：非主属性 CITY, SSETS 函数依赖于 BNO（主码 (CNO, LNO) 的一部分的传递依赖），违反 2NF；同样，NAME, ADDR 依赖于 CNO（也是部分依赖），也不满足 2NF。

Step 3：规范化分解（至 3NF/BCNF） 采用模式分解消去部分函数依赖：

- (1) 支行关系：Branch(BNO, CITY, SSETS)

码：BNO。函数依赖： $BNO \rightarrow CITY, SSETS$ 。满足 BCNF。

- (2) 客户关系：Customer(CNO, NAME, ADDR)

码：CNO。函数依赖： $CNO \rightarrow NAME, ADDR$ 。满足 BCNF。

- (3) 贷款关系：Loan(LNO, BNO, AMOUNT)

码：LNO。函数依赖： $LNO \rightarrow BNO, AMOUNT$ 。外码：BNO 参照 Branch。满足 BCNF。

(4) 借贷关系: Borrow(CNO, LNO)

码: (CNO, LNO)。外码: CNO 参照 Customer, LNO 参照 Loan。满足 BCNF。

验证: 该分解具有无损连接性, 也保持了所有原始函数依赖。分解后彻底消除了插入异常、删除异常、修改异常和数据冗余问题。

解析与知识点

本题是一个完整的规范化分析综合题, 考察从语义描述中提取函数依赖、识别范式问题并执行规范化分解的全流程能力。解题的关键步骤包括: (1) 从文字语义中准确提取函数依赖关系。

题目语义“一笔贷款只能由一个支行发出”导出 $LNO \rightarrow BNO$, “一个支行位于一个城市、有一定总资产”导出 $BNO \rightarrow CITY, SSETS$, “每个客户有唯一名称和地址”导出 $CNO \rightarrow NAME, ADDR$, “一笔贷款有确定的金额”导出 $LNO \rightarrow AMOUNT$ 。

准确提取所有函数依赖是后续分析的基础。(2) 识别主码——“一个客户可贷多笔贷款, 一笔贷款可由多个客户共同贷款”表明客户与贷款之间是多对多关系, 因此在原始 BCL 宽表中需要用 (CNO, LNO) 的复合码来唯一标识每个“客户-贷款”关联记录。(3) 检测部分依赖——以 (CNO, LNO) 为主码时, CITY 和 SSETS 仅通过 LNO 就可以决定 (而不需要 CNO), NAME 和 ADDR 仅通过 CNO 就可以决定——这些都是主码的部分函数依赖, 导致 BCL 连 2NF 都不满足。(4) 采用“一事一地”原则进行分解——将支行信息 (Branch)、客户信息 (Customer)、贷款信息 (Loan) 和借贷关联 (Borrow) 分别放入独立的关系模式中, 消除了所有部分依赖和传递依赖, 最终得到四个满足 BCNF 的关系模式。

需要注意“贷款号在各支行唯一”这一条件对 LNO 全局唯一性的影响——由于一笔贷款由一个支行发出, LNO 实际上在全局范围内也是唯一的 (贷款与支行一一对应), 因此 Loan 关系可以安全地使用单属性 LNO 作为主码。

知识点: 第六章: 关系数据理论——6.3 范式判定 (从语义提取 FD $\rightarrow$ 识别部分依赖 $\rightarrow$ 确定 2NF 违规)、6.4 模式分解 (基于“一事一地”原则的分解策略, 消除插入/删除/更新异常和数据冗余)

六-2. 舰队管理信息系统 E-R 图设计

设一个海军基地要建立一个舰队管理信息系统, 包含舰队和舰艇两方面信息:

- 舰队方面: 舰队 D (舰队名称, 基地地点, 舰艇数量 Q1); 舰艇 T (编号, 舰艇名称, 舰队名称)

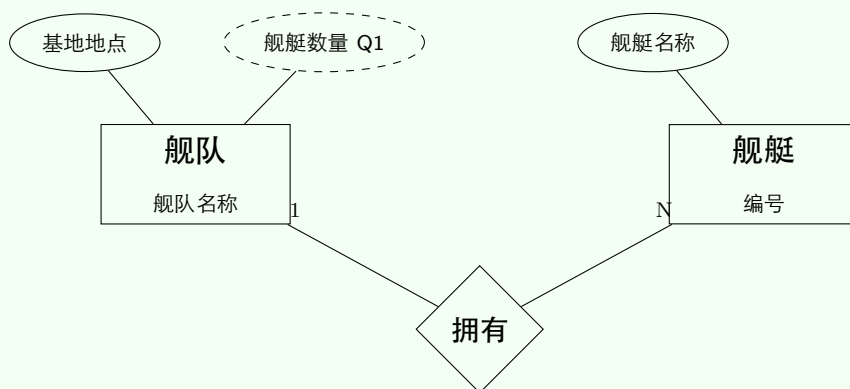
- 舰艇方面：舰艇 T (舰艇编号, 舰艇名, 武器名称); 武器 W (武器名称, 武器生产时间, 舰艇编号); 官兵 G (官兵证号, 姓名, 舰艇编号)

约束规则：一个舰队拥有多艘舰艇，一艘舰艇属于一个舰队；一艘舰艇安装多种武器，一种武器可安装于多艘舰艇上，某舰艇上安装某种武器有具体日期 date 及数量 Q2；一艘舰艇有多个官兵，一个官兵只属于一艘舰艇。请完成：(1) 分别设计舰队和舰艇两个局部 E-R 图，并合并为一个全局 E-R 图；(2) 将该全局 E-R 图转换为关系模式。

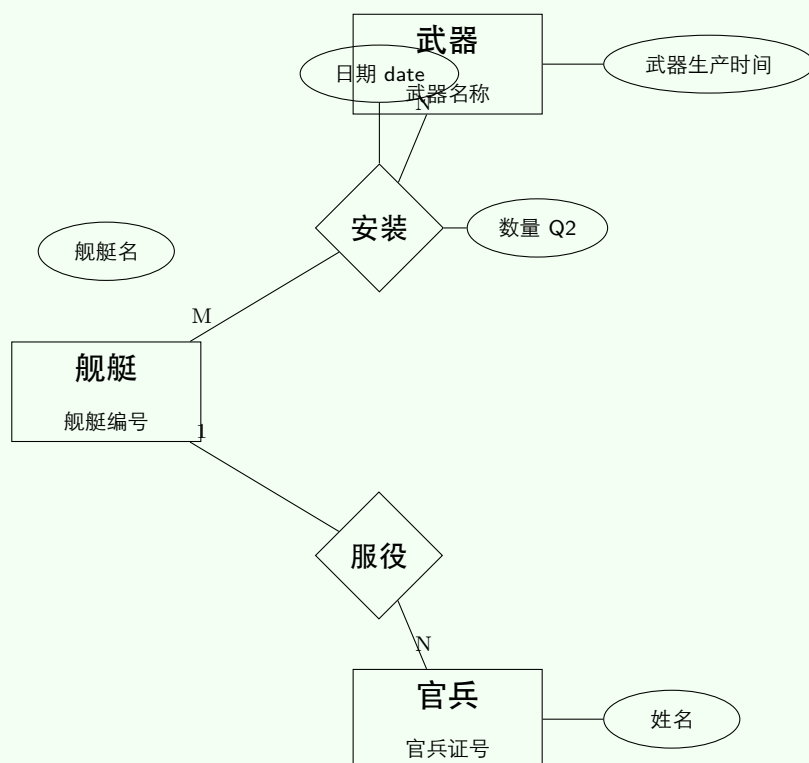
参考答案

(答案由解析者提供)

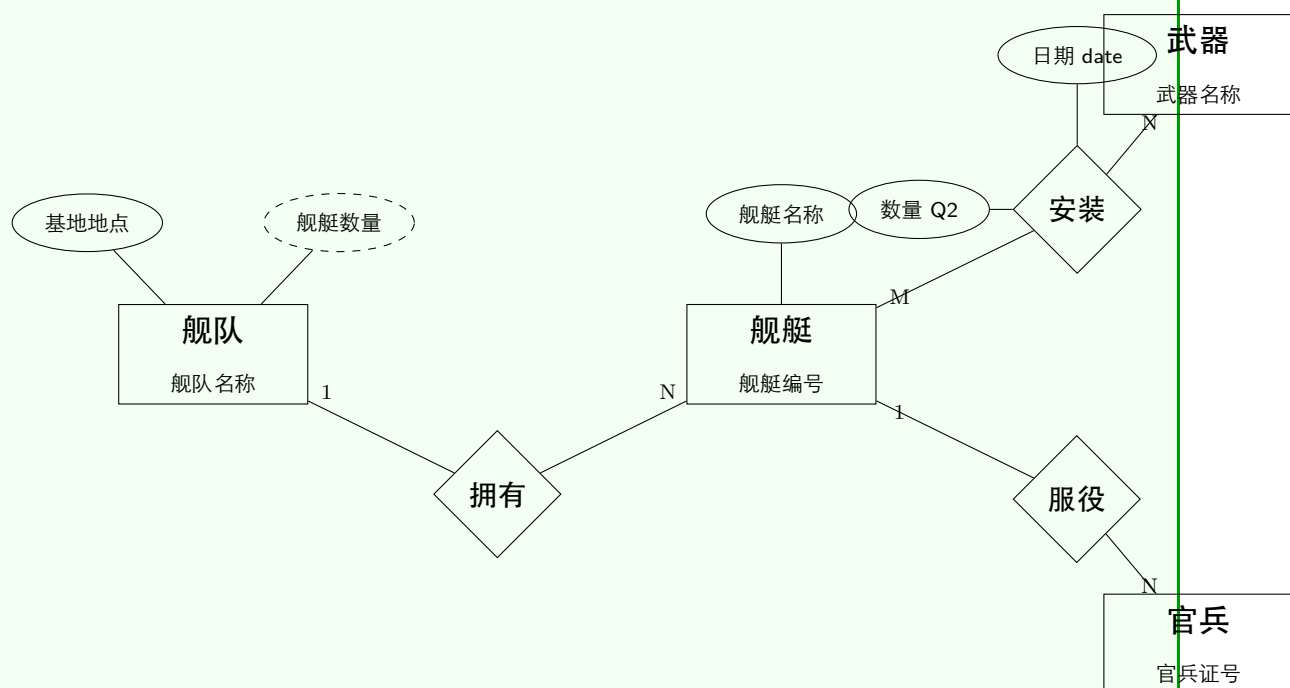
(1) 局部 E-R 图与全局 E-R 图 局部 E-R 图 a 一舰队管理视角：



局部 E-R 图 b 一舰艇管理视角：



全局 E-R 图（合并后）：



(2) 全局 E-R 图转换为关系模式 根据 E-R 图向关系模型的转换规则：

规则 1： 实体 → 关系模式

- Fleet(舰队名称, 基地地点) ——码： 舰队名称
- Ship(舰艇编号, 舰艇名称, 舰队名称) ——码： 舰艇编号；外码： 舰队名称参照 Fleet
- Weapon(武器名称, 武器生产时间) ——码： 武器名称
- Officer(官兵证号, 姓名, 舰艇编号) ——码： 官兵证号；外码： 舰艇编号参照 Ship

规则 2： 1 : N 联系（拥有、服役） → 合并到 N 端

- 联系”拥有” (Fleet–Ship, 1:N)： 在 Ship 模式中增加外码”舰队名称”，已体现在上述 Ship 模式中。
- 联系”服役” (Ship–Officer, 1:N)： 在 Officer 模式中增加外码”舰艇编号”，已体现在上述 Officer 模式中。

规则 3： M : N 联系（安装） → 独立关系模式

- Install(舰艇编号, 武器名称, 安装日期, 数量)

码: (舰艇编号, 武器名称) 的复合主码; 外码: 舰艇编号参照 Ship;
外码: 武器名称参照 Weapon。安装日期和数量为该联系自身的属性。

最终关系模式集合 (共 5 个):

- (1) Fleet(舰队名称, 基地地点)
- (2) Ship(舰艇编号, 舰艇名称, 舰队名称) FK: 舰队名称 → Fleet
- (3) Weapon(武器名称, 武器生产时间)
- (4) Install(舰艇编号, 武器名称, 安装日期, 数量) FK: 舰艇编号 → Ship, 武器名称 → Weapon
- (5) Officer(官兵证号, 姓名, 舰艇编号) FK: 舰艇编号 → Ship

注意: 舰艇数量是派生属性 (可由 Install 表中每个舰艇安装的武器数量汇总 + 对 Ship 记录计数得出), 在 Fleet 模式中不存储, 而通过查询动态计算。

解析与知识点

该题为 E-R 模型的综合设计题, 全面考察了局部 E-R 图设计、全局 E-R 图合并以及 E-R 图向关系模式转换的完整流程。解题的第一步是正确识别实体、属性和联系。

局部视图 a (舰队管理视角) 涉及舰队实体和舰艇实体, 之间的”拥有”联系是 1:N 关系。局部视图 b (舰艇管理视角) 涉及舰艇实体、武器实体和官兵实体, 包含”安装”联系 (M:N) 和”服役”联系 (1:N)。合并局部 E-R 图时, 需要将”舰艇”实体的不同属性和来自两个视图的关联关系统一合并到同一个舰艇实体中。

转换到关系模式时遵循标准的 E-R 转换规则: 每个实体转换为一个关系模式, 实体的属性成为关系的列; 1:N 联系通过将”1”端的主码作为外码加入到”N”端的关系模式中 (如拥有和服役联系); M:N 联系需要单独建立一个关系模式 (如 Install), 其主码由两端实体的主码共同组成, 联系自身的属性 (date 和 Q2) 也包含在内。局部 E-R 图的合并需要识别并消除属性冲突、命名冲突和结构冲突三类冲突。

派生属性 (舰艇数量 Q1) 不存储, 通过 SQL 的 COUNT 聚合函数动态计算。另外, 将 1:N 联系错误地转换为独立关系模式 (而非合并到 N 端) 也是常见的转换错误。

知识点: 第七章: 数据库设计——7.3 概念结构设计 (E-R 模型: 实体、属性、联

系的识别与局部/全局 E-R 图设计方法); 7.4 逻辑结构设计 (E-R 图向关系模式的转换规则: 实体转换、1:N 联系合并到 N 端、M:N 联系建立独立关系、派生属性的处理)

5.7 七、证明题 (共 6 分)

设关系模式 $R(U)$ 的函数依赖集 F 中包含函数依赖: $Y_j \rightarrow Z_j$ ($j = 1, 2, \dots, k$), 试证明: 在 F^+ 中, $Y_1Y_2 \cdots Y_k \rightarrow Z_1Z_2 \cdots Z_k$ 成立。

参考答案

(答案由解析者提供)

证明: 使用 **Armstrong** 公理体系 已知: 对任意 $j \in \{1, 2, \dots, k\}$, 有 $Y_j \rightarrow Z_j \in F$ 。

第 1 步: (自反律 / **Reflexivity**)

由自反律 (若 $Y \subseteq X$, 则 $X \rightarrow Y$), 因为对任意 j 有 $Y_j \subseteq Y_1Y_2 \cdots Y_k$ (Y_j 是整体属性集的子集), 故:

$$Y_1Y_2 \cdots Y_k \rightarrow Y_j \quad \text{对每个 } j = 1, 2, \dots, k$$

这 k 个函数依赖均在 F^+ 中。

第 2 步: (传递律 / **Transitivity**)

已知 $Y_1Y_2 \cdots Y_k \rightarrow Y_j$ (由步骤 1), 且 $Y_j \rightarrow Z_j \in F \subseteq F^+$ 。由传递律 (若 $X \rightarrow Y$ 且 $Y \rightarrow Z$, 则 $X \rightarrow Z$), 可得:

$$Y_1Y_2 \cdots Y_k \rightarrow Z_j \quad \text{对每个 } j = 1, 2, \dots, k$$

至此, 我们证明了属性集 $Y_1Y_2 \cdots Y_k$ 函数决定每一个 Z_j 。

第 3 步: (合并律 / **Union Rule**)

由步骤 2 得到 k 个函数依赖: $Y_1Y_2 \cdots Y_k \rightarrow Z_j$ ($j = 1, 2, \dots, k$)。应用合并律 (若 $X \rightarrow Y$ 且 $X \rightarrow Z$, 则 $X \rightarrow YZ$), 将这 k 个函数依赖逐一合并:

$$Y_1Y_2 \cdots Y_k \rightarrow Z_1, Y_1Y_2 \cdots Y_k \rightarrow Z_2 \implies Y_1Y_2 \cdots Y_k \rightarrow Z_1Z_2$$

$$Y_1Y_2 \cdots Y_k \rightarrow Z_1Z_2, Y_1Y_2 \cdots Y_k \rightarrow Z_3 \implies Y_1Y_2 \cdots Y_k \rightarrow Z_1Z_2Z_3$$

...

$$\implies Y_1Y_2 \cdots Y_k \rightarrow Z_1Z_2 \cdots Z_k$$

综上所述, 根据 Armstrong 公理体系的三条基本公理 (自反律、增广律、传递律) 及其推论 (合并律), 我们严格推导出:

$$Y_1Y_2\cdots Y_k \rightarrow Z_1Z_2\cdots Z_k \in F^+$$

证毕。

□

证明要点小结:

- 自反律保证了左部属性集的任意子集可被左部函数决定。
- 传递律将每个单独的 $Y_j \rightarrow Z_j$ 从 F 中”桥接”到联合左部。
- 合并律将所有 Z_j 汇集为一个属性集右部。

解析与知识点

该证明题考察对 Armstrong 公理体系的深入理解和系统运用能力, 是函数依赖理论的核心证明题。Armstrong 公理体系由三条基本公理构成: 自反律 (Reflexivity: $Y \subseteq X \Rightarrow X \rightarrow Y$)、增广律 (Augmentation: $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$) 和传递律 (Transitivity: $X \rightarrow Y \wedge Y \rightarrow Z \Rightarrow X \rightarrow Z$)。

基于这三条基本公理, 可以推导出多个重要的推论——合并律 (Union: $X \rightarrow Y \wedge X \rightarrow Z \Rightarrow X \rightarrow YZ$)、分解律 (Decomposition: $X \rightarrow YZ \Rightarrow X \rightarrow Y \wedge X \rightarrow Z$) 和伪传递律 (Pseudotransitivity: $X \rightarrow Y \wedge WY \rightarrow Z \Rightarrow WX \rightarrow Z$) 等。

本题的证明严格遵循了”自反律 $\rightarrow$ 传递律 $\rightarrow$ 合并律”的三步推理链: 第一步利用自反律, 从 Y_j 是 $Y_1Y_2\cdots Y_k$ 的子集出发, 建立联合左部对每个单独左部 Y_j 的函数决定关系; 第二步利用传递律, 将上一步得到的函数依赖与原始 FD $Y_j \rightarrow Z_j$ 进行传递, 建立联合左部对每个单独 Z_j 的直接决定关系; 第三步利用合并律, 将 k 个独立的 $Y_1Y_2\cdots Y_k \rightarrow Z_j$ 逐一合并, 最终推导出 $Y_1Y_2\cdots Y_k \rightarrow Z_1Z_2\cdots Z_k$ 。

Armstrong 公理体系的核心价值在于其正确性 (Soundness——公理推导出的 FD 一定被 F 蕴涵) 和完备性 (Completeness——所有被 F 蕴涵的 FD 都可以通过公理推导出来), 这使得它成为函数依赖逻辑推理的完备理论基础。

知识点: 第六章: 关系数据理论——6.1 函数依赖与 Armstrong 公理体系 (三条基本公理: 自反律、增广律、传递律; 重要推论: 合并律、分解律、伪传递律; F^+ 的逻辑蕴涵定义; 公理体系正确性与完备性)

5.8 选做题（参考分：5 分）

试简述数据库技术的新发展。

参考答案

（答案由解析者提供）

数据库技术的新的发展主要包括以下方向：

- (1) 面向对象数据库（OODB）与对象-关系数据库（ORDB）：将面向对象的编程思想（封装、继承、多态）与数据库技术相结合。OODB 从零开始以对象模型构建数据库；ORDB 以关系模型为基础扩展面向对象特性（如用户自定义类型、继承、引用），是当时数据库产品的主流演进方向。
- (2) Web 数据库技术：随着 Internet 的普及，数据库通过 Web 服务器与浏览器连接，支持通过 HTTP/XML 等协议存取数据库，形成三层或多层 B/S 架构的应用系统。
- (3) 数据仓库（Data Warehouse）与联机分析处理（OLAP）：面向主题的、集成的、时变的、非易失的数据集合，支持多维数据建模与复杂分析查询，服务于决策支持系统（DSS）。
- (4) 数据挖掘（Data Mining）与知识发现（KDD）：从海量数据中自动提取隐含的、先前未知的、具有潜在价值的信息和知识（如关联规则、分类、聚类、异常检测）。
- (5) 分布式数据库系统：数据在物理上分布于网络的多个节点，但在逻辑上是一个整体，支持数据分片、分布式查询和分布式事务管理。
- (6) 并行数据库系统：利用多处理器硬件（SMP、MPP 架构）并行执行数据库操作，提高大规模数据处理和事务吞吐量。
- (7) 多媒体数据库：管理文本、图形、图像、音频、视频等多媒体数据类型，支持基于内容的检索。
- (8) XML 数据库：XML 成为互联网数据交换的事实标准，原生 XML 数据库支持半结构化数据的存储与 XQuery 查询。

解析与知识点

该选做题考察学生对数据库技术发展趋势的宏观认知，定位在 2000 年代初的时间节点。这段时期是数据库领域的黄金转型期——传统关系数据库已高度成熟，但

面临新的应用场景和数据类型的挑战，催生了大量新技术方向。

发展主线可归纳为四个维度。

第一，数据模型维度——传统关系模型的表格式结构（适合严格结构化的业务数据）在面对复杂对象、空间数据、多媒体内容等新兴需求时力不从心，因此面向对象数据库（OODB）和对象-关系数据库（ORDB，如 PostgreSQL 的早期版本）应运而生，将面向对象的封装、继承、多态等特性融入数据库核心。第二，应用架构维度——Web 的爆发式增长使“数据库上网”成为必然趋势，多层 B/S 架构取代传统 C/S 架构，数据库通过应用服务器和 Web 服务器向 Internet 开放，推动了数据库连接池、分布式缓存和负载均衡等相关技术的发展。第三，计算模式维度——数据库不再仅仅是事务处理（OLTP）的引擎，向分析处理（OLAP）、数据挖掘和商业智能方向全面拓展，数据仓库技术支撑了从“发生了什么”到“为什么发生”再到“将要发生什么”的分析深度跃升。第四，系统架构维度——从单一服务器走向分布式、并行、集群化部署，以支撑海量数据处理和高并发场景。

XML 数据库则回应了互联网数据交换标准化的需求。这些趋势反映了数据库技术从“封闭的数据管理工具”向“开放的信息基础设施”的根本角色转变。

知识点：第一章：绪论——1.4 数据库技术发展（面向对象/ORDB、Web 数据库、数据仓库与 OLAP、数据挖掘与 KDD、分布式/并行数据库、多媒体数据库、XML 数据库——技术发展的四大维度）

5.9 附：《数据库原理》测试试题

注意

说明：本测试试题为同一时期（2002–2003 学年前后）的《数据库原理》课程测试卷，原卷无官方答案，以下答案与解析均由解析者自行解答，以（答案由解析者提供）标注。

5.10 一、单项选择题（每小题 1 分，共 10 分）

1. 要保证数据库的数据独立性，需要修改的是（ ）。

参考答案

（答案由解析者提供） A

解析与知识点

本题考察数据独立性实现机制的精确理解。数据独立性是通过三级模式结构之间的两级映射来实现的，而非通过修改三层模式本身。

当底层存储结构改变时，只需调整概念模式/内模式映射就可以使概念模式保持不变——这保障了物理数据独立性；当概念模式逻辑结构发生变化时，只需调整外模式/概念模式映射就可以使外模式（用户视图）保持不变——这保障了逻辑数据独立性。

模式本身是数据库结构定义的静态框架，修改它们恰恰会破坏已有的独立性体系。因此，保障数据独立性的关键是修改和维护两级映射关系，而非修改三层模式。选项 B 和 C 各只提及一个方向的映射，不够全面。选项 D 提及修改三层模式更是不正确——三层模式是框架结构，随意修改会直接破坏独立性。

数据独立性是数据库系统区别于文件系统的核心优势，理解其实现机制（通过可调整的映射而非通过修改模式定义）是掌握数据库体系结构的关键。

知识点：第一章：绪论——1.3 数据库系统体系结构（三级模式结构中的两级映射是数据独立性的实现机制；修改映射而非修改模式本身来保持独立性）

2. 下列四项中说法不正确的是（ ）。

参考答案

（答案由解析者提供） C

解析与知识点

本题考察对数据库系统特点的辩证理解，尤其需要识别绝对化表述中的逻辑漏洞。数据库系统通过集中管理和规范化设计可以显著减少不必要的、有害的数据冗余，但不可能做到“避免一切数据的重复”。

在实际数据库设计中，一定程度的受控冗余（Controlled Redundancy）是有意保留的，且是合理的。最典型的例子就是外码（FOREIGN KEY）——子表中存储的外码值是对父表主码的“重复”，但这种重复是维护表间参照完整性所必需的。此外，为了提高查询性能和减少连接操作，有时会采用反规范化（Denormalization）手段故意引入冗余列或汇总表——这是用空间换时间的经典权衡。

选项 A、B、D 的描述都是正确的：数据库系统通过共享机制、统一的存储和管理确实减少了文件系统中广泛存在的“各自为政”式冗余；数据库的多用户共享特性使数据不再是某个应用程序独占的资源；三级模式结构确实提供了远高于文件系统的物理和逻辑独立性。

选项 C 用“避免一切”加以绝对化，不切实际也不正确。

知识点：第一章：绪论——1.1 数据库系统的特点（可控冗余 vs 有害冗余的区分；外码是必要的受控冗余；反规范化的性能权衡）

3. 公司中有多个部门和多名职员，每个职员只能属于一个部门，一个部门可以有多名职员，从职员到部门的联系类型是（ ）。

参考答案

（答案由解析者提供） C

解析与知识点

本题考察 E-R 模型中联系类型的判定，关键在于正确识别“从... 到...”的观察方向。联系类型（1:1、1:N、M:N）是根据实体实例之间的数量对应关系确定的，且具有方向性——从不同方向看，同一联系可以有不同的称谓。

题目描述“每个职员只能属于一个部门，一个部门可以有多名职员”——从职员的视角，多名职员可以映射到同一个部门（多对一，N:1）；从部门的视角，一个部门可以管理多名职员（一对多，1:N）。题目明确要求“从职员到部门的联系类型”，即从职员实体出发观察映射到的部门实体——单个职员只能对应一个部门，而不同职员可以对应同一个部门，因此是 N:1，即多对一。

常见的混淆是将“多对一”写为“一对多”——关键在于确认题目要求的观察方向（谁到谁）。

在职员-部门例子中，如果职员是起点，集合中不同的职员实体实例可以映射到相同的部门实例，起点的多个值对应终点的一个值，所以是多对一。这一细节在 E-R 图设计中直接影响基数的标注位置。

知识点：第七章：数据库设计——7.3 概念结构设计（E-R 模型中联系类型的判定需注意观察方向：“从 A 到 B 的联系类型”以 A 为起点确定 N:1 还是 1:N）

4. 下列数据库中，必须进行查询优化的是（ ）。

参考答案

（答案由解析者提供） A

解析与知识点

本题考察不同类型数据库对查询优化的需求差异，核心在于数据操作语言的本质特征。关系数据库“必须”进行查询优化的根本原因在于其使用非过程化（Non-procedural）的查询语言——SQL。

在关系数据库中，用户只需通过 SQL 声明”要什么数据”（What），并不指定”如何获取数据”（How），具体的执行路径（选择什么索引、按什么顺序连接表、使用哪种连接算法等）完全由 DBMS 的查询优化器自动决定。

同一 SQL 查询往往存在多种等价的实现方式，但执行代价可能相差数十倍甚至数百倍，因此查询优化器成为关系数据库必不可少的核心组件。相比之下，层次数据库和网状数据库采用过程化（Procedural）的导航式查询语言，应用程序员需要显式地指定存取路径（如”从根结点向下遍历”、”沿指针链前进”等），查询效率在很大程度上由程序员的程序设计水平决定，系统层面的自动查询优化并非设计上的必需。

因此，关系数据库对查询优化的依赖是根本性的、结构性的——查询优化是关系数据库非过程化语言特性的必然补偿。

知识点：第九章：查询处理与优化——9.1 查询优化概述（非过程化语言是关系数据库必须查询优化的根本原因；过程化语言的层次/网状数据库由程序员掌控执行路径）

5. 在视图上不能完全实施的操作是（ ）。

参考答案

（答案由解析者提供） C

解析与知识点

本题考察对视图本质及其操作限制的深刻理解。视图（View）是从一个或多个基表（Base Table）导出的虚拟表，它的定义（子查询）存储在数据字典中，但视图本身不独立存储数据——每次访问视图时，DBMS 都会动态执行其定义查询从基表生成数据。

基于这一本质特征，在各选项中分析哪些操作可以在视图上实施。查询（A）是视图最基本、毫无限制的功能——视图就是为查询而设计的。

在视图上定义新视图（B）也是完全可行的——可以基于视图再创建更深层次的视图，形成视图层次。

更新操作——包括 INSERT、UPDATE、DELETE（D）——虽然受限于可更新视图的条件（如不能涉及多表连接、不能包含聚合函数和 GROUP BY 等），但在满足一定条件时是可行的（如单表视图的更新可直接映射到基表）。而”在视图上定义新的基本表”（C）是唯一完全不可能的操作：基本表（Base Table）必须有独立的物理存储来持久化数据，视图只是查询的”窗口”而不具备存储能力，因此不能也不应该作为基表的创建容器或基础。在 SQL 语法中，CREATE TABLE 只能创

建基表，CREATE VIEW 只能创建视图——二者是不同的对象类型，不能通过视图来创建基表。

知识点：第三章：SQL 语言——3.5 视图（视图是虚拟表，不存储数据；可查询、可创建视图、有条件可更新；不能作为基表创建的宿体）

6. 关系数据模型的三个组成部分中，不包括（ ）。

参考答案

（答案由解析者提供） C

解析与知识点

本题考察数据模型三要素的准确界定及其与其他数据库功能概念的区别。

任何一个数据模型（不仅仅是关系模型）都由三个不可分割的要素组成：（1）数据结构——描述数据库的组成对象以及对象之间的联系，在关系模型中就是关系（二维表）的结构定义，包括属性、域、关系模式等概念；（2）数据操作——描述对数据库中数据允许执行的操作及操作规则，在关系模型中就是关系代数（选择、投影、连接、并、交、差、除等）和关系演算，以及在此基础上设计的 SQL DML；（3）数据的完整性约束——描述数据及其联系必须满足的制约和依存规则，关系模型中的三种完整性约束（实体完整性、参照完整性、用户定义完整性）即属于此要素。

而“数据恢复”不属于数据模型本身的构成要素——它是 DBMS 的一个功能模块（属于事务管理和恢复子系统），是一个系统实现层面的功能，而非数据模型的定义层面的组成部分。

将数据恢复归入数据模型三要素，混淆了“数据模型是什么”（定义层面）与“DBMS 能做什么”（实现层面）这两个不同层次的问题。

知识点：第一章：绪论——1.2 数据模型的组成要素（数据结构、数据操作、完整性约束是任何数据模型的三要素；数据恢复是 DBMS 功能模块，非数据模型要素）

7. 关系代数表达式的优化策略中，首先要做的是（ ）。

参考答案

（答案由解析者提供） B

解析与知识点

本题考察代数优化启发式规则中最高优先级策略的理解。关系代数优化的核心启发式策略中，“尽早执行选择运算”（或称“条件下推” Selection Pushdown）是最优先的策略。

选择的优先级最高，原因在于：选择运算通常具有很高的选择性（过滤比例大），能够显著缩减中间结果的元组数量。假设某表有 100 万行，选择条件能过滤掉 99% 的行，那么先执行选择可以将后续所有操作处理的数据量从 100 万行降至 1 万行。如果将选择推迟到连接之后再作，则连接操作需要处理全量的 100 万行数据，产生巨大的中间结果和 I/O 开销。类似地，投影运算也应尽早执行以缩减属性列的数量。而笛卡尔积是代价最高的运算，应该尽可能推迟，或者更理想的是将其与选择条件结合转化为连接运算（连接可以通过高效的索引或散列技术实现）。

选项 A（执行笛卡尔积）与优化原则完全相反；选项 C 和 D（投影优化）虽是优化的一部分，但选择运算（尤其是过滤性强的选择）在优化优先级上高于投影——先大幅减少行数，再考虑减少列数。因此代数优化的首要策略是尽早做选择。

知识点：第九章：查询处理与优化——9.2 代数优化（启发式规则的优先级：选择最早执行（条件下推）→ 投影尽早执行 → 推迟笛卡尔积或转为连接）

8. 下列四项中，不属于关系数据库特点的是（ ）。

参考答案

（答案由解析者提供） D

解析与知识点

本题考察对关系数据库独有特点与通用 DBMS 功能的区分能力。关系数据库的特点是指关系模型相对于其他数据模型（层次模型、网状模型）所特有的优势，而非所有类型数据库管理系统都具备的通用功能。

“多用户访问”是 DBMS 级的功能需求——无论是层次数据库（如 IMS）、网状数据库（如 DBTG）还是关系数据库（如 Oracle、MySQL），都支持多个用户同时访问数据库中的数据。因此“多用户访问”不属于关系数据库的专属特征。

关系数据库的专属特点包括：（1）以二维表为统一的数据结构，简洁直观——层次模型用树形、网状模型用图，都不如关系模型的表格结构统一；（2）通过规范化理论有效控制和减少数据冗余——关系数据库有严格的范式理论体系指导，而层次/网状模型没有同等的系统化冗余消除方法；（3）三级模式结构保证了较高的物理独立性和逻辑独立性——ANSI/SPARC 架构最早就是基于关系模型提出的；（4）具有严格的数学理论基础——关系代数和关系演算为关系模型提供了精

确的数学语义和优化依据。这些特点共同构成了关系数据库的竞争优势。

区分“DBMS 通用功能”与“某类数据库的专属特征”是这类题目的关键。多用户并发访问、数据备份与恢复、安全性控制等都属于 DBMS 层面的通用功能，不应归入关系数据库的专属优势。因此正确答案是 D。

知识点：第一章：绪论——1.1 数据库系统的特点（区分 DBMS 通用功能与关系数据库特有特征——多用户访问是所有 DBMS 的基础功能，非关系数据库专属）

6 2009–2010 学年《数据库原理与设计》期末试卷 (A)

注意

说明：本试卷无官方参考答案，以下全部答案为解析者自行解答，标注（答案由解析者提供）以示区别。

6.1 一、判断题（每小题 1 分，共 10 分）

参考答案

×（答案由解析者提供）

解析与知识点

本题考察对关系数据库系统操作方式的理解。关系数据库系统采用集合操作方式，SQL 查询的结果本身就是元组的集合（结果表），一次可以同时获取和处理多个元组。

通过游标机制固然可以逐行处理，但关系操作本质上是集合导向的，这与层次或网状数据库的逐记录导航方式截然不同。

这是关系模型区别于早期数据模型的核心特征之一——关系代数中的所有运算都遵循闭包性质，操作的输入和输出都是关系的集合。集合操作方式也是 SQL 能够表达复杂查询、支持子查询嵌套和视图定义的理论基础。

许多初学者容易将游标的逐行处理误解为关系数据库的基本操作方式，但实际上游标只是嵌入式 SQL 中协调集合操作与宿主语言单记录处理的桥梁工具，而非关系操作本身。理解这一点，对于区分过程化数据访问（层次/网状）与非过程化数据访问（关系）至关重要。在实际应用中，集合操作方式还使得查询优化器能够对整批数据进行全局优化，选择最优的执行计划，这也是关系数据库高效性的重要来源。因此该判断题的说法是错误的。

知识点：2.1 关系数据结构与关系操作特点——集合操作方式是关系模型的核心

特征

参考答案

✗ (答案由解析者提供)

解析与知识点

本题考察对 SQL 语言功能分类的全面认识。SQL 语言的功能远不限于定义与查询，它是一个综合性的数据库语言。

除数据查询 (SELECT) 和数据定义 (CREATE、ALTER、DROP) 外，SQL 还完整支持数据更新操作 (INSERT、UPDATE、DELETE)，这是数据库日常运维中最频繁的操作类型。

此外，SQL 还提供了数据控制功能 (GRANT、REVOKE) 用于存取权限管理，支持视图定义 (CREATE VIEW) 来构建外模式，支持索引创建 (CREATE INDEX) 来优化查询性能，以及支持触发器定义 (CREATE TRIGGER) 来实现复杂的完整性约束和业务逻辑。将 SQL 的功能仅仅局限于“定义与查询”是严重片面的，忽视了数据操纵这一核心功能模块，也忽略了安全控制和数据库编程方面的扩展能力。从 SQL 标准的演进来看，SQL-92 已经明确划分了 DDL、DML 和 DCL 三大功能类别，SQL:1999 之后又增加了对触发器、存储过程等持久存储模块 (SQL/PSM) 的支持。因此，全面理解 SQL 的功能范畴，对于正确使用数据库语言进行应用开发至关重要。

该判断题的说法是错误的。

知识点：3.1 SQL 概览与功能分类——SQL 涵盖 DDL、DML、DCL 及数据库编程四大功能域

参考答案

✓ (答案由解析者提供)

解析与知识点

本题考察对参照完整性 (Referential Integrity) 核心规则的准确理解。参照完整性是关系模型中三大完整性约束之一，其核心要求正是：外码的取值要么为 NULL (当外码所在属性列允许为空时)，要么必须等于被参照关系中某个元组的主码值。这是保证数据库中表间关联正确性的最基本规则。

DBMS 通过多种机制强制执行此约束：当用户试图插入或更新子表中的外码值时，系统自动检查该值是否存在于父表的主码列中；当用户试图删除父表中的元组或更新父表的主码值时，系统根据预定义的违约处理策略 (如 CASCADE 级联、SET

NULL 置空、NO ACTION 拒绝) 自动维护子表中的外码一致性。这一约束从根本上杜绝了“悬空引用”(Dangling Reference) 问题——即子表中外码指向一个不存在的父表记录。

在实际数据库设计中, 参照完整性约束经常与外码的 ON DELETE 和 ON UPDATE 子句配合使用, 形成一套完整的表间数据一致性保护机制。

理解参照完整性的这一核心规则, 是掌握数据库完整性理论的关键。该判断题的说法是正确的。

知识点: 5.2 参照完整性 (Referential Integrity) ——外码取值规则与 DBMS 的违约处理机制

参考答案

✓ (答案由解析者提供)

解析与知识点

本题考察对函数依赖最小集 (最小覆盖) 性质的理解。对于同一个函数依赖集 F , 其最小覆盖 (最小函数依赖集) F_m 确实不唯一, 这是一个重要的理论事实。

最小化的过程中涉及三个步骤: 右部单属性化、左部极小化、删除冗余函数依赖。这三个步骤的先后执行顺序不同, 可能导致不同的中间结果和最终结果。

尤其是在“删除冗余依赖”这一步骤中, 如果存在多个冗余的函数依赖, 选择删除的顺序不同 (先删哪个、后删哪个), 可能导致不同的最终最小集。

例如, 若有 $A \rightarrow B$ 和 $B \rightarrow A$, 通过传递 $A \rightarrow B \rightarrow A$, 两者中有一个可能是冗余的, 但删除哪一个会得到不同的最小集。然而, 尽管最小集的具体形式可能不同, 所有最小集彼此等价——它们具有完全相同的闭包 $F_m^+ = F^+$, 因此在实际使用中 (如 3NF 分解) 不会产生本质差异。这一特性提醒我们, 规范化的结果虽然形式上可能不唯一, 但在数据语义上是等价的。该判断题的说法是正确的。

知识点: 6.2 函数依赖最小集 (最小覆盖) ——最小集的不唯一性与等价性

参考答案

✗ (答案由解析者提供)

解析与知识点

本题考察对可串行化调度概念的准确理解, 这是一个容易混淆的并发控制核心概念。可串行化调度是指并发事务的执行结果等价于这些事务按某种串行顺序执行的结果, 但调度本身确实允许事务的并发交错执行。

关键区别在于：可串行化是并发调度的正确性准则（Correctness Criterion），而非调度本身的执行方式（是并发的而非串行的）。串行调度是并发度为 1 的极端情况——事务一个接一个顺序执行，虽然绝对正确但系统吞吐量极低。可串行化调度的范围远大于串行调度：它允许事务的指令在时间上交错执行（提高了系统并发度和资源利用率），只要最终结果与某种串行顺序等价即可。

DBMS 的并发控制机制（如两段锁协议 2PL、时间戳排序等）正是为了保证所有并发调度都是可串行化的。

将可串行化等同于“事务串行执行”是对这一重要概念的严重误解。该判断题的说法是错误的。

知识点：10.4 可串行化调度概念——可串行化是并发调度的正确性准则，与串行执行是两个不同的概念

参考答案

×（答案由解析者提供）

解析与知识点

本题考察对封锁粒度与系统开销关系的理解，这是一个典型的“反直觉”知识点。封锁粒度与系统开销的关系恰恰与题干所说相反：封锁粒度越小（如元组级锁），需要的锁数量越多，锁管理开销（锁表维护、锁分配与释放、死锁检测遍历等）反而越大。

例如，若一次更新涉及 10000 条元组，使用元组级锁需要管理 10000 个锁对象；而使用表级锁只需管理 1 个锁对象。封锁粒度大（如表级锁）则管理简单、开销小，但并发度降低（多个事务不能同时访问同一张表）。

多粒度封锁（Multiple Granularity Locking）机制正是为了在两者间折中而设计的：通过引入意向锁（IS、IX、SIX 等），允许在同一数据库对象的不同粒度级别上灵活加锁，既保持了较好的并发度，又控制了锁管理的开销。因此，封锁粒度的选择是并发度与系统开销之间的权衡。

该判断题的说法是错误的。

知识点：10.3 封锁粒度与多粒度封锁——封锁粒度越小管理开销越大，多粒度封锁在并发度与开销间折中

参考答案

✓（答案由解析者提供）

解析与知识点

本题考察查询优化中代数优化核心原则的理解。查询优化的核心启发式策略就是“尽早执行可缩减数据规模的操作”，具体包括“先做选择、投影，后做连接”以及“推迟笛卡尔积”。

选择运算 (σ) 和投影运算 (π) 能够显著缩小中间结果的规模：选择运算通过过滤条件减少元组数量（降低关系的“行数”），投影运算通过删减不需要的属性列减少数据的宽度（降低关系的“列数”）。将这两个操作尽早执行，能够让后续操作（尤其是代价高昂的连接运算）处理更小的数据集，从而显著降低 I/O 访问次数和 CPU 计算开销。

这一优化原则的理论基础是关系代数的等价变换规则，例如选择下推 (Selection Pushdown) ——将选择条件尽可能移到查询树的叶节点附近执行。在实际的 DBMS 查询优化器中，选择操作的“选择性” (Selectivity) 估算是代价估算的关键参数，直接决定了最优执行计划的选择。

因此，这道判断题的说法完全正确，它准确地概括了查询优化的首要原则。

知识点：9.2 代数优化基本原则——先选择投影后连接，尽早缩减数据规模是查询优化的首要策略

参考答案

✓（答案由解析者提供）

解析与知识点

本题考察对数据字典 (Data Dictionary) 功能的理解。数据字典，也称系统目录 (System Catalog)，是 DBMS 中存储数据库元数据 (Metadata，即关于数据的数据) 的系统表集合。

所有 DDL 操作（如 CREATE TABLE 创建表结构、CREATE VIEW 定义视图、CREATE INDEX 建立索引、CREATE ASSERTION 定义断言、GRANT 授权等）产生的数据库对象定义信息，包括表名、列名、数据类型、约束定义 (PRIMARY KEY、FOREIGN KEY、CHECK、NOT NULL、UNIQUE 等)、索引信息、视图定义、用户权限、存储过程定义等，全部持久化保存在数据字典中。数据字典是 DBMS 的“脑”——查询处理器利用它进行查询分析和优化，安全子系统利用它进行权限验证，完整性子系统利用它进行约束检查。

数据字典通常以只读视图（如 Oracle 的 USER_TABLES、ALL_TAB_COLUMNS）的形式向用户开放查询，但不允许用户直接修改。

因此，该判断题的说法是正确的。

知识点： 1.3 数据库系统结构与数据字典——DDL 操作的定义信息全部持久化存储于数据字典

参考答案

×（答案由解析者提供）

解析与知识点

本题考察数据模型三要素与数据库系统其他重要概念之间的区分。数据模型的三要素是：（1）数据结构——描述数据库的组成对象以及对象之间的联系，是数据模型最基础的组成部分；（2）数据操作——对数据库中各种对象的实例和值允许执行的操作及其操作规则；（3）数据的完整性约束——数据及其联系应具有制约和依存规则，以保证数据的正确、有效和相容。这三要素构成了任何一个数据模型的完整定义。而数据独立性（包括物理数据独立性和逻辑数据独立性）是数据库系统的重要特征和设计目标，它是通过 DBMS 的三级模式—两级映像体系结构得以实现的，而非数据模型自身的基本要素。

数据独立性是在数据库系统架构层面解决的问题，属于体系结构的特性，不应与数据模型的组成要素混为一谈。

这种区分对于理解数据库系统的分层设计思想十分重要。

该判断题的说法是错误的。

知识点： 1.2 数据模型三要素与数据独立性——数据模型由数据结构、数据操作和完整性约束三要素构成；数据独立性是体系结构特征

参考答案

×（答案由解析者提供）

解析与知识点

本题考察对三级模式结构中两级映像实现机制的理解。数据库系统的三级模式结构中，外模式/模式映像和模式/内模式映像这两级映像功能是由 DBMS 在系统内部自动完成和管理的，而非由 DBA（数据库管理员）手动实现。当概念模式（全局逻辑结构）发生改变时，DBMS 自动调整外模式/模式映像，使得外模式对用户保持稳定，从而保障逻辑数据独立性；当内模式（物理存储结构）发生改变时，DBMS 自动调整模式/内模式映像，使得概念模式保持稳定，从而保障物理数据独立性。

DBA 的职责主要包括数据库的规划设计、存储空间与存取路径的配置、用户权限的管理、完整性约束的定义、数据库的日常监控与性能调优、数据备份与恢复策

略的制定等。

如果把两级映像的实现错误地归结为 DBA 的职责，就混淆了 DBMS 的自动机制与 DBA 的人工管理职责。

因此该判断题的说法是错误的。

知识点： 1.3 三级模式结构与两级映像——两级映像由 DBMS 自动实现，保障逻辑独立性与物理独立性

6.2 二、单项选择题（每小题 1 分，共 10 分）

参考答案

B（答案由解析者提供）

解析与知识点

本题考察对聚簇索引（Clustered Index）核心目的的理解。聚簇索引的核心目的是提高按该索引键进行范围查询的效率。

聚簇索引的特殊之处在于，它要求数据行按照索引键值的顺序物理地存放在磁盘上，具有相同或相邻索引键值的元组被存储在相同或相邻的数据页中。当执行范围查询（如 WHERE AGE BETWEEN 20 AND 30）时，查询处理器只需扫描连续的物理块，而不必在散布各处的数据页之间随机跳转，从而大幅减少了磁盘 I/O 次数。这一特性对于 OLAP（联机分析处理）和大规模数据扫描查询尤其有效。与之对照，非聚簇索引（Non-clustered Index）的索引条目与数据行是分离存储的，索引叶子节点存储的是指向数据行的指针（RowID），范围查询可能需要大量的随机 I/O。

聚簇索引本身不减少存储空间，反而可能需要额外的存储开销来维护数据的物理有序性；它也不直接减少数据冗余，冗余问题是由规范化理论来解决的。

需要注意的是，一张表通常只能有一个聚簇索引，而可以有多个非聚簇索引，因为数据的物理排序只能有一种方式。

知识点： 3.5 索引（聚簇索引与非聚簇索引）——聚簇索引的核心价值在于范围查询的物理连续性，减少随机 I/O

参考答案

D（答案由解析者提供）

解析与知识点

本题考察对插入异常 (Insertion Anomaly) 概念的准确理解。插入异常是指由于关系模式设计不当 (存在过多的属性耦合、不合适的函数依赖), 导致用户想要插入某一实体或联系的信息时, 由于尚缺另一不相关实体或联系的信息 (如部分主码属性未知或为空) 而无法完成插入操作。

例如典型的选课关系模式中, 将学生信息和课程信息放在同一个宽表中, 若想新增一门尚无学生选修的课程, 因主码需要包含学号属性而无法插入该课程信息。这与题干描述“应该插入的数据无法插入”完全吻合。

插入异常是低范式 (尤其是 1NF 和 2NF) 关系模式的四大问题之一, 另外三个是: 数据冗余、更新异常 (修改异常) 和删除异常。

解决插入异常的根本方法是进行关系模式的规范化分解, 将原本混合在一起的不同实体或联系信息拆分到各自独立的关系模式中, 使得每个关系模式只描述一个主题 (“一事一地” 原则)。在数据库设计的逻辑结构阶段, 消除插入异常是规范化处理的核心目标之一。

知识点: 6.1 数据依赖与操作异常——插入异常的根本原因是属性耦合, 解决方法是通过规范化分解

参考答案

B (答案由解析者提供)

解析与知识点

本题考察对数据库设计各阶段任务和 E-R 图归属的准确理解。数据库设计遵循结构化生命周期方法, 分为六个阶段: 需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施、数据库运行与维护。

E-R 图 (实体—联系图) 是概念结构设计阶段的核心工具和主要产出物。

在概念设计阶段, 设计者将需求分析得到的信息需求抽象为概念模型, 使用 E-R 图来描述实体 (Entity)、属性 (Attribute) 以及实体之间的联系 (Relationship)。概念模型独立于任何具体的 DBMS 和数据模型, 它关注的是“信息世界”的结构表达而非数据在计算机中的存储方式。后续的逻辑结构设计阶段再将 E-R 图按照转换规则转换为特定 DBMS 支持的关系模式 (或层次模式、网状模式等)。

将 E-R 图误归于需求分析阶段 (忽略了概念设计这一关键中间环节) 或物理设计阶段 (混淆了逻辑设计与物理设计的层次) 都是对数据库设计流程的误解。因此正确答案是 B——概念结构设计阶段。

知识点: 7.1 数据库设计阶段及各阶段任务——E-R 图是概念结构设计阶段的核心工具, 独立于具体 DBMS

参考答案

B（答案由解析者提供）

解析与知识点

本题考察范式之间的包含（递进）关系的理解。关系数据库理论中的范式存在严格的递进包含关系： $1NF \supset 2NF \supset 3NF \supset BCNF \supset 4NF$ 。

3NF 的定义是在 1NF 基础上消除了非主属性对码的部分函数依赖（达到 2NF），并进一步消除了非主属性对码的传递函数依赖。

因此，任何满足 3NF 的关系模式自动满足 2NF 的所有要求。

但反过来，3NF 不一定是 BCNF——当存在主属性对码的部分依赖或传递依赖时（即存在一个非平凡函数依赖 $X \rightarrow Y$ ， X 不是超码，但 Y 是主属性），该关系满足 3NF 却不满足 BCNF。同样，3NF 也不一定是 4NF，因为 4NF 排除了非平凡且非函数依赖的多值依赖（MVD），而 3NF 对此没有要求。理解范式之间的这种层次递进关系，有助于在规范化分析中准确判定一个关系模式能够达到的最高范式等级，从而制定合理的分解策略。正确答案是 B——必属于 2NF。

知识点：6.3 范式关系：1NF、2NF、3NF、BCNF、4NF 的递进包含关系——3NF 自动满足 2NF，但不保证 BCNF 或 4NF

参考答案

D（答案由解析者提供）

解析与知识点

本题考察对 E-R 模型中联系（Relationship）可以关联的实体数量的理解。在 E-R 模型中，联系可以关联的实体数量是灵活的，具体取决于参与该联系的实体集的数量。

一元联系（Unary Relationship，或称递归联系 Recursive Relationship）只涉及一个实体集，如职工实体中的“管理”关系（经理和下属都属于同一职工实体）。二元联系（Binary Relationship）涉及两个实体集，这是 E-R 图中最常见的联系形式，如“学生-选课-课程”中的选课联系。三元及更高阶的联系涉及三个或更多实体集，如供应商-零件-工程的供应联系（SPJ）。联系的“元数（Degree）”就是指参与该联系的实体集的数量。因此，联系可以关联的实体个数是 1 个或 1 个以上（可以是 1、2、3 甚至更多），但不能是 0 个。

将联系设计为关联 0 个实体在语义上没有意义，也不符合联系的数学定义。

正确答案是 D——1 个或 1 个以上。

知识点： 7.2 E-R 模型中的联系类型——联系可按元数分为一元、二元、三元及更高阶联系

参考答案

B（答案由解析者提供）

解析与知识点

本题考察对数据库恢复中“先写日志”原则（Write-Ahead Logging, WAL）的深刻理解。数据库恢复技术的基本原则是：在将修改操作实际写入数据库之前，必须先将相应的日志记录写入并持久化到日志文件中。这一原则的提出是为了应对最严重的故障场景——系统在数据修改过程中崩溃（中途故障）。

如果采用“先修改数据库、后写日志”的策略，可能存在这样一个危险窗口：数据库已被修改但日志尚未记录——此时若系统崩溃，数据库处于不一致状态且无法通过日志进行恢复（因为没有日志记录可查，不知该 UNDO 还是 REDO）。

反之，遵循 WAL 原则：若写日志成功但写数据库时系统崩溃，恢复程序可根据日志中的后像（After Image）进行 REDO；若写日志本身失败，则数据库未改变，事务回滚即可。WAL 原则是保证事务原子性（Atomicity）和持久性（Durability）的关键机制，是数据库恢复理论的基石。

所有主流 DBMS（Oracle、MySQL、PostgreSQL、SQL Server）的恢复机制都基于 WAL 原理。正确答案是 B。

知识点： 11.2 日志文件与先写日志（WAL）原则——WAL 是数据库恢复的基石，确保任何故障场景下都可追踪和恢复

参考答案

D（答案由解析者提供）

解析与知识点

本题考察对 E-R 图集成过程中冲突类型的准确识别。

将各局部 E-R 图集成为全局 E-R 图时，需要识别和消除的冲突有三种类型：（1）属性冲突——包括属性值域的冲突（如某部门将学生成绩用百分制，另一部门用五级制）和属性取值单位的冲突（如身高用米或厘米）；（2）命名冲突——包括同名异义（不同意义的对象在不同的局部应用中使用了相同的名字）和异名同义（同一意义的对象在不同的局部应用中使用了不同的名字）；（3）结构冲突——包括同一对象在不同局部 E-R 图中被抽象为不同的层次（在一个视图中作为实体，在另一个视图中作为属性），以及同一实体在不同的局部 E-R 图中包含的属性不完全

相同，或者实体间的联系类型不一致。

”事务冲突”不属于 E-R 图集成中考虑的冲突类型，事务（Transaction）是并发控制和恢复技术中的概念，与数据库设计中的概念结构集成无关。

因此正确答案是 D——事务冲突。

知识点：7.4 E-R 图集成中的三类冲突——属性冲突、命名冲突、结构冲突是集成为必须消除的三类冲突

参考答案

B（答案由解析者提供）

解析与知识点

本题考察对共享锁（S 锁）相容性规则的理解。S 锁（共享锁，Shared Lock）的相容性规则是：S 锁与 S 锁相容（多个事务可以同时持有同一数据对象的 S 锁进行并发读取），但 S 锁与 X 锁（排他锁，Exclusive Lock）互斥。

当事务 T 已对数据 R 加有 S 锁时，意味着 T 拥有对 R 的读权限，其他事务仍然可以对 R 加 S 锁（因为读操作互不干扰），但不能对 R 加 X 锁（因为写操作与读操作冲突）。选项 A 的说法”先加 S 再加 X”在实际并发控制中不能直接实现——事务必须先释放 S 锁才能获取 X 锁，但这违反了传统两段锁协议（2PL）的要求（必须先获取所有锁再开始释放），可能导致可串行化性的丧失。如果确实需要从”读”升级为”写”，通常需要使用 U 锁（更新锁）或直接申请 X 锁。

理解 S 锁和 X 锁的相容性矩阵，是掌握封锁并发控制机制的基础。

正确答案是 B——S 锁允许其他事务加 S 锁进行读取。

知识点：10.2 锁的类型与相容矩阵——S 锁与 S 锁相容（允许并发读），S 锁与 X 锁互斥（读写冲突）

参考答案

B（答案由解析者提供）

解析与知识点

本题考察候选码求解的完整算法应用。候选码求解的一般步骤为：首先找出所有不出现在任何函数依赖右部的属性，这些属性必须包含在任意候选码中。

在给定的 FD 集中， $A \rightarrow X$ 表明 X 在右部； $B \rightarrow C$ 表明 C 在右部； $E \rightarrow A$ 表明 A 在右部。因此 B 和 E 均未出现在任何 FD 的右部，它们必须属于候选码。

然后计算 $(BE)^+ = \{B, E\} \xrightarrow{E \rightarrow A} \{B, E, A\} \xrightarrow{A \rightarrow X} \{B, E, A, X\} \xrightarrow{B \rightarrow C}$

$\{B, E, A, X, C\} = \{A, B, C, E, X\} = R$, 因此 BE 是超码。最后验证极小性: $B^+ = \{B, C\} \neq R$, $E^+ = \{E, A, X\} \neq R$, 故 BE 是极小候选码, 不可再缩减。其他选项: $(CE)^+ = \{C, E, A, X\} \neq R$, $(AB)^+ = \{A, B, C, X\} \neq R$, $(AE)^+ = \{A, E, X\} \neq R$, 均不能覆盖全属性集 R。

因此正确答案是 B——BE。这一求解过程展示了 Armstrong 公理体系中闭包计算的完整应用。

知识点: 6.2 候选码的求解方法与 Armstrong 公理——先找不在右部出现的属性, 再通过闭包计算验证编码极小性

参考答案

C (答案由解析者提供)

解析与知识点

本题考察对数据库完整性保护 (Integrity Protection) 概念范畴的准确界定。数据库完整性保护旨在确保数据库中数据在任何时刻都保持正确性、有效性和相容性, 其约束条件直接针对的是数据本身的值和结构关系。

具体包括: 实体完整性 (主码非空且唯一, 确保每条记录可被唯一标识)、参照完整性 (外码取值约束, 维护表间关联的正确性) 和用户定义的完整性 (如 CHECK 约束, 限制属性的取值范围及其相互关系)。这四个选项中, 将“规定类型”排除在完整性范畴外是不正确的——属性值的类型约束 (如整数型、字符串型) 同样属于完整性中的“值的约束”, 规定了每个属性列的数据必须符合其声明的格式和范围。

选项 A 中的用户标识与口令属于数据库安全性 (Security) 范畴, 关注的是“谁能访问”; 选项 B 中的备份属于数据库恢复 (Recovery) 范畴, 关注的是“数据如何不丢失”; 选项 D 中的并发事务调度属于并发控制 (Concurrency Control) 范畴, 关注的是“多用户同时操作如何不冲突”。

因此正确答案是 C。

知识点: 5.1 数据库完整性与三类完整性约束——完整性关注数据值的正确性和相容性, 与安全性、恢复、并发控制是不同范畴

6.3 三、查询设计题 (共 6 小题, 每小题 5 分, 共 30 分)

已知关系模式:

- 商店表 A(A#, ANAME, WQTY, CITY) ——商店代号、商店名、店员人数、所在城市

- 商品表 $B(B\#, BNAME, PRICE)$ ——商品号、商品名称、价格
- 销售表 $AB(A\#, B\#, QTY)$ ——商店代号、商品号、商品数量

参考答案

$\pi_{B\#, QTY}(\sigma_{CITY='北京'}(A) \bowtie AB)$ (答案由解析者提供)

解析与知识点

本题考察关系代数中选择、投影和自然连接运算的组合运用。查询要求找到北京地区商店销售的商品号和数量，需要分两步完成逻辑：首先用选择运算 $\sigma_{CITY='北京'}(A)$ 从商店表 A 中筛选出所在城市为“北京”的商店记录，这是一个典型的单关系选择操作，其结果是 A 表的一个水平子集。

然后将其与销售表 AB 做自然连接，自然连接会自动匹配两表中同名的属性（这里即 $A\#$ ），将满足匹配条件的元组拼接在一起，得到这些北京商店的所有商品销售记录。最后用投影运算 $\pi_{B\#, QTY}$ 从连接结果中提取所需的商品号和商品数量两个属性列。

这个查询表达式展示了关系代数中“先选择缩小数据集，后连接合并数据，最后投影输出结果”的经典思维模式，也是查询优化器进行代数优化的基本依据。

操作顺序遵循了代数优化的启发式规则：选择运算尽早下推到叶节点附近，减少后续连接操作处理的数据量。

知识点：2.3 关系代数：选择 σ 、投影 π 、连接 $\bowtie$ ——三种运算的组合使用遵循“先选择、后连接、再投影”的优化原则

参考答案

$\pi_{A\#, ANAME, WQTY}(A \bowtie (\pi_{A\#, B\#}(AB) \div \pi_{B\#}(B)))$ (答案由解析者提供)

解析与知识点

本题是关系代数中除法运算 (Division) 的经典应用，考察“全部”类查询的代数表达。查询要求找出“销售了所有商品的商店”，这是一个典型的全称量词 (Universal Quantifier) 查询，在关系代数中只能通过除法运算来直接表达。

首先计算 $\pi_{B\#}(B)$ 获取所有商品号的集合(全集)，这是除法的“除数”； $\pi_{A\#, B\#}(AB)$ 获取每个商店实际销售的(商店号, 商品号)对，这是除法的“被除数”。二者做除法 $\div$ 后，得到那些其销售商品集合覆盖了全部商品集合的商店号。最后将结果与商店表 A 做自然连接获取商店名和店员人数信息，并用投影输出所需属性列。

除法的语义正是“在 R 中找出那些与 S 中所有值都有关联的属性值”。除法运算

的严格定义为 $R \div S = \{t \mid \forall s \in S, (t, s) \in R\}$ ，即 R 中与 S 的每个值都配对出现的那些属性值。除法的标准计算步骤为：先计算被除数在除数的补集上的投影，再计算“应该存在但不存在”的那些配对，最后从被除数的投影中排除它们。除法运算是关系代数中最强大也最不易掌握的运算之一，它是表达“全部”或“所有”语义的唯一关系代数工具。

初学者容易试图用选择或连接来替代除法运算表达“全部”类查询，这在关系代数中是无法直接做到的。另外，在使用除法时需确保除数的所有属性都出现在被除数中，且被除数中多余的属性将成为除法结果的输出属性。

知识点：2.3 关系代数：除法运算 $\div$ ——除法用于表达全称量词语义，是“全部”类查询的标准代数工具

参考答案

（答案由解析者提供）

```
1 GRANT SELECT, INSERT ON TABLE A TO PUBLIC;
```

解析与知识点

本题考察 SQL 中存取控制语句 GRANT 的基本用法。题目要求使全体用户对 A 表只能执行查询和插入操作，而不能做删除或修改。GRANT 语句用于授予数据库操作权限，其基本语法为 GRANT < 权限列表 > ON < 对象 > TO < 用户列表 >。SELECT 权限对应查询操作（读取数据），INSERT 权限对应插入操作（新增数据），两者组合恰好满足“只能查询和插入”的需求。

TO PUBLIC 表示将权限授予所有数据库用户（包括当前和未来的用户），PUBLIC 是 SQL 标准中定义的特殊用户角色。这一授权策略体现了数据库安全性中的“最小权限原则”（Principle of Least Privilege）——仅授予完成任务所必需的最小权限集合。

注意这里没有授予 UPDATE 和 DELETE 权限，也没有授予其他数据定义权限（如 ALTER、DROP、INDEX），确保了 A 表数据的安全性和完整性。

若要撤销已授予的权限，可使用 REVOKE 语句：REVOKE SELECT, INSERT ON TABLE A FROM PUBLIC。

知识点：4.2 存取控制与 SQL 授权语句——GRANT/REVOKE 实现自主存取控制，PUBLIC 代表全体用户

参考答案

(答案由解析者提供)

```
1 SELECT ANAME, WQTY
2 FROM A
3 WHERE CITY = '上海';
```

解析与知识点

本题是最基础的单表 SQL 查询，考察 SELECT-FROM-WHERE 基本结构的使用。查询目标是从商店表 A 中检索所在城市为“上海”的商店名和店员人数，只涉及一张表的两个属性列。

SQL 的三部分各司其职：SELECT 子句指定输出列（ANAME 和 WQTY），对应关系代数中的投影操作；FROM 子句指定查询的数据来源（表 A）；WHERE 子句指定行过滤条件（CITY = '上海'），对应关系代数中的选择操作。

这种查询不需要 GROUP BY 分组、不需要 HAVING 对分组过滤、不需要 ORDER BY 排序、不需要连接（JOIN）其他表，是 SQL 查询中最简单也最基本的形式。尽管简单，它体现了 SQL 声明式查询的核心特征：用户只需描述“要什么数据”（What），而不需指定“怎么获取数据”（How），具体的执行路径（如是否使用索引、使用全表扫描还是索引扫描）由 DBMS 的查询优化器自动决定。

知识点：3.3 SQL 单表查询（SELECT-FROM-WHERE）——SQL 声明式查询的基本结构，SELECT 对应投影，WHERE 对应选择

参考答案

(答案由解析者提供)

```
1 SELECT A.A#, A.ANAME, AB.B#
2 FROM A, AB
3 WHERE A.A# = AB.A#
4       AND A.WQTY <= 100
5       AND A.CITY = '长沙';
```

解析与知识点

本题考察 SQL 中多表连接查询与复合条件的组合使用。查询要求从商店表 A 和销售表 AB 中检索满足“店员人数不超过 100”且“所在城市为长沙”的商店的代号、商店名及其销售的商品号。

在多表查询中，FROM 子句列出所有涉及的表（A 和 AB），它们之间默认产生笛卡尔积；WHERE 子句中的连接条件 $A.A\# = AB.A\#$ （等值连接）将笛卡尔积转化为有意义的连接结果——只有商店代号匹配的元组才会被保留，实现了商店信息与销售信息的关联。另外两个选择条件 $A.WQTY \leq 100$ 和 $A.CITY = '长沙'$ 分别对店员人数和城市进行过滤。

由于 A 表和 AB 表都有 A# 属性，在 SELECT 和 WHERE 子句中使用时必须加上表名前缀（A.A#）以消除歧义——这是多表查询中的一个重要注意事项。

这种“连接条件 + 选择条件”的组合模式是多表查询的标准写法。

知识点：3.4 SQL 连接查询与复合条件——多表查询通过 FROM + WHERE 等值条件实现连接，属性歧义需使用表名前缀

参考答案

（答案由解析者提供）

```
1 SELECT A.ANAME, A.CITY, SUM(AB.QTY) AS TotalQty
2 FROM A, AB
3 WHERE A.A# = AB.A#
4       AND AB.B# IN (SELECT B# FROM AB WHERE A# = 256)
5 GROUP BY A.A#, A.ANAME, A.CITY
6 HAVING A.A# <> 256
7 ORDER BY TotalQty ASC;
```

解析与知识点

本题是 SQL 综合能力题，考察嵌套子查询、分组聚合与排序的组合应用。查询逻辑分为三个层次。第一层是子查询：SELECT B# FROM AB WHERE A# = 256 获取 256 号商店所供应的全部商品号集合，这是一个不相关子查询，只需执行一次，为外层查询提供“商品编号列表”。第二层是外层查询：通过 WHERE AB.B# IN (...) 匹配供应了上述商品集合中至少一种商品的其他商店，并用 $A.A\# \neq 256$ 排除 256 号商店自身，实现“找出其他销售了与 256 号商店相似商品的商店”这一业务需求。

第三层是分组聚合：GROUP BY A.A#, A.ANAME, A.CITY 按商店进行分组，SUM(AB.QTY) 计算每个商店所有相关商品的总销售数量。

ORDER BY TotalQty ASC 按总销售量升序排列结果。

注意这里使用了 HAVING A.A# <> 256 而非 WHERE，这是因为 HAVING 可以对分组结果进行过滤——但此处 HAVING 条件不含聚合函数，实际上也可以放

在 WHERE 子句中。DISTINCT 在处理 B# IN (...) 时通常不需要, 因为 IN 子句隐含了对重复值的去重语义。

知识点: 3.5 SQL 嵌套子查询、聚合函数与分组——子查询提供对比基准, GROUP BY + SUM 实现分组聚合, HAVING 实现分组后过滤

6.4 四、数据库分析、设计 (共 34 分)

参考答案

(答案由解析者提供)

第一步: 求最小函数依赖集 F_m (最小覆盖)

最小化的过程分三步进行。

(1) 右部单属性化。

F 中所有函数依赖的右部已经是单一属性, 无需处理:

$$F = \{E \rightarrow T, C \rightarrow X, T \rightarrow B, E \rightarrow B, B \rightarrow C, EX \rightarrow D, CE \rightarrow D\}$$

(2) 左部极小化。

对左部含多个属性的函数依赖, 逐一检查是否可以删除左部的某个属性而不改变闭包:

- 检查 $EX \rightarrow D$: 令 $F' = F - \{EX \rightarrow D\}$ 。
 - 在 F' 下求 $E_F^+ = \{E, T, B, C, X\}$ (推导链: $E \rightarrow T, T \rightarrow B, B \rightarrow C, C \rightarrow X$), $D \notin E_F^+$, E 不可删除。
 - 在 F' 下求 $X_F^+ = \{X\}$, $D \notin X_F^+$, X 不可删除。
- 检查 $CE \rightarrow D$: 令 $F'' = F - \{CE \rightarrow D\}$ 。
 - 在 F'' 下求 $C_{F''}^+ = \{C, X\}$, $D \notin C_{F''}^+$, C 不可删除。
 - 在 F'' 下求 $E_{F''}^+ = \{E, T, B, C, X\}$, $D \notin E_{F''}^+$, E 不可删除。

结论: 所有左部均已极小化, 无需缩减属性。

(3) 删除冗余函数依赖。

依次检查每个函数依赖是否冗余 (能否由其余 FD 推导):

- $E \rightarrow T$: $F - \{E \rightarrow T\}$ 下, $E^+ = \{E, B, C, X\}$, $T \notin E^+$ 。不冗余, 保留。
- $C \rightarrow X$: $F - \{C \rightarrow X\}$ 下, $C^+ = \{C\}$, $X \notin C^+$ 。不冗余, 保留。
- $T \rightarrow B$: $F - \{T \rightarrow B\}$ 下, $T^+ = \{T\}$, $B \notin T^+$ 。不冗余, 保留。

- $E \rightarrow B$: $F - \{E \rightarrow B\}$ 下, $E^+ = \{E\} \xrightarrow{E \rightarrow T} \{E, T\} \xrightarrow{T \rightarrow B} \{E, T, B\} \xrightarrow{B \rightarrow C} \{E, T, B, C\} \xrightarrow{C \rightarrow X} \{E, T, B, C, X\}$ 。 $B \in E^+$, 故 $E \rightarrow B$ 可由 $E \rightarrow T \rightarrow B$ 推导出, 冗余, 删除。
- $B \rightarrow C$: 当前 F (已删除 $E \rightarrow B$) 下, $F - \{B \rightarrow C\}$ 时 $B^+ = \{B\}$, $C \notin B^+$ 。不冗余, 保留。
- $EX \rightarrow D$: $F - \{EX \rightarrow D\}$ (当前 $F = \{E \rightarrow T, C \rightarrow X, T \rightarrow B, B \rightarrow C, CE \rightarrow D\}$) 下, $(EX)^+ = \{E, X, T, B, C\}$, $D \notin (EX)^+$ 。不冗余, 保留。
- $CE \rightarrow D$: $F - \{CE \rightarrow D\}$ 即 $F' = \{E \rightarrow T, C \rightarrow X, T \rightarrow B, B \rightarrow C, EX \rightarrow D\}$ 下, $(CE)^+ = \{C, E\} \xrightarrow{C \rightarrow X} \{C, E, X\} \xrightarrow{E \rightarrow T} \{C, E, X, T\} \xrightarrow{T \rightarrow B} \{C, E, X, T, B\} \xrightarrow{B \rightarrow C} \{C, E, X, T, B\}$ (已含) $\xrightarrow{EX \rightarrow D} \{C, E, X, T, B, D\}$ 。 $D \in (CE)^+$, 故 $CE \rightarrow D$ 可由 $C \rightarrow X$ 与 $EX \rightarrow D$ 推导出, 冗余, 删除。

由此得到:

$$F_m = \{E \rightarrow T, C \rightarrow X, T \rightarrow B, B \rightarrow C, EX \rightarrow D\}$$

第二步: 求候选码

找出所有未出现在任何函数依赖右部的属性: 在 F_m 中, E 仅出现在左部, F 和 G 不在任何 FD 中出现。因此 $\{E, F, G\}$ 必须包含在任一候选码中。

计算闭包: $(EFG)^+ = \{E, F, G\} \xrightarrow{E \rightarrow T} \{E, F, G, T\} \xrightarrow{T \rightarrow B} \{E, F, G, T, B\} \xrightarrow{B \rightarrow C} \{E, F, G, T, B, C\} \xrightarrow{C \rightarrow X} \{E, F, G, T, B, C, X\} \xrightarrow{EX \rightarrow D} \{E, F, G, T, B, C, X, D\} = R$

$(EFG)^+ = R$, 故 EFG 为超码。检验极小性:

- $(FG)^+ = \{F, G\}$, 不覆盖 R ;
- $(EG)^+ = \{E, G, T, B, C, X, D\}$, 缺少 F , 不覆盖 R ;
- $(EF)^+ = \{E, F, T, B, C, X, D\}$, 缺少 G , 不覆盖 R 。

故 EFG 极小, 为唯一的候选码。

第三步: 3NF 分解 (综合算法)

将 F_m 中的函数依赖按左部等价类分组, 每组生成一个子关系模式:

$$\begin{aligned} E \rightarrow T &\Rightarrow R_1(E, T) && \text{键: } E \\ C \rightarrow X &\Rightarrow R_2(C, X) && \text{键: } C \\ T \rightarrow B &\Rightarrow R_3(T, B) && \text{键: } T \\ B \rightarrow C &\Rightarrow R_4(B, C) && \text{键: } B \\ EX \rightarrow D &\Rightarrow R_5(E, X, D) && \text{键: } EX \end{aligned}$$

检查无子关系包含候选码 EFG ，因此补充关系 $R_6(E, F, G)$ ，键为 EFG 。

检查子集关系：以上六个子关系模式互不为子集关系，全部保留。

最终结果：

$$\rho = \{ R_1(E, T), R_2(C, X), R_3(T, B), R_4(B, C), R_5(E, X, D), R_6(E, F, G) \}$$

其中各关系模式均已满足 3NF（每个非平凡 FD 的左边均为所在关系模式的候选码或超码，或右边为主属性），且分解具有无损连接性（ R_6 含候选码 EFG ）和依赖保持性（ F_m 中每个 FD 都被各自的子关系模式直接保持）。

解析与知识点

本题是关系规范化的经典综合题，全面考察了函数依赖理论中的三个核心算法：求最小函数依赖集、求候选码和 3NF 分解。首先，求最小函数依赖集 F_m （也称为最小覆盖）是规范化的基础步骤，其目的是消除冗余的函数依赖，得到一个等价的、最精简的依赖集合。

最小化过程必须按照“右部单属性化、左部极小化、删除冗余依赖”的标准三步流程执行。在第三步中， $E \rightarrow B$ 被识别为冗余依赖——因为它可以通过 $E \rightarrow T$ 和 $T \rightarrow B$ 的传递推导得出； $CE \rightarrow D$ 也被识别为冗余——因为它可以由 $C \rightarrow X$ 与 $EX \rightarrow D$ 经过伪传递律推导得出。这两个冗余依赖的识别是本题求解的关键难点。

其次，候选码的求解采用“先找出不在任何 FD 右部出现的属性，再计算闭包验证”的方法。E、F、G 作为不在右部出现的属性集合，其闭包恰好覆盖 R，且无真子集能覆盖 R，因此 EFG 是唯一的候选码。

最后，3NF 分解使用综合算法（Synthesis Algorithm）：将 F_m 按左部等价类分组，每个组生成一个子关系模式，再检查是否需要补充候选码的关系。最终得到 6 个满足 3NF 的子关系模式，该分解同时具有无损连接性和依赖保持性。

知识点：6.2 最小函数依赖集（最小覆盖）的求解 | 6.2 候选码与闭包计算 | 6.3 3NF 分解的综合算法（Synthesis Algorithm）——三步算法：求 F_m 、求候选码、按 F_m 分组生成子关系并补充候选码

参考答案

（答案由解析者提供）

证明：设有二元关系 $R(A, B)$ ，其全体属性为 $U = \{A, B\}$ 。

在二元关系中，所有可能的非平凡函数依赖仅有以下两种情形：

- 情形一：只存在 $A \rightarrow B$ 。此时 A 的闭包 $A^+ = \{A, B\} = U$ ，故 A 是 R 的超码（事实上是候选码）。由于 A 是超码，非平凡函数依赖 $A \rightarrow B$ 的决定

因素是超码，满足 BCNF 定义。

- 情形二：只存在 $B \rightarrow A$ 。同理， $B^+ = \{A, B\} = U$ ， B 是 R 的超码， $B \rightarrow A$ 的决定因素是超码，满足 BCNF。
- 情形三：同时存在 $A \rightarrow B$ 和 $B \rightarrow A$ 。此时 $A^+ = B^+ = U$ ， A 和 B 均为候选码。两个非平凡 FD 的决定因素都是候选码，均满足 BCNF。
- 情形四：不存在任何非平凡函数依赖。BCNF 定义的违反条件是：存在非平凡函数依赖 $X \rightarrow Y$ ，其中 X 不是超码。既然不存在非平凡 FD，则 BCNF 条件自然成立（空真）。

上述四种情形穷举了二元关系上所有可能的函数依赖情况。无论哪种情形， $R(A, B)$ 都满足 BCNF 的定义，因此任何二元关系必定属于 BCNF。 ■

解析与知识点

本题考察对 BCNF 定义的本质理解，是一个简洁而深刻的规范化理论证明题。BCNF (Boyce-Codd Normal Form) 的定义是：关系模式 R 属于 BCNF，当且仅当对于 R 上成立的每一个非平凡函数依赖 $X \rightarrow Y$ ， X 都包含 R 的一个候选码（即 X 是超码）。

在仅有两个属性的二元关系 $R(A, B)$ 中，任何非平凡函数依赖必然导致其左部能决定全部属性。

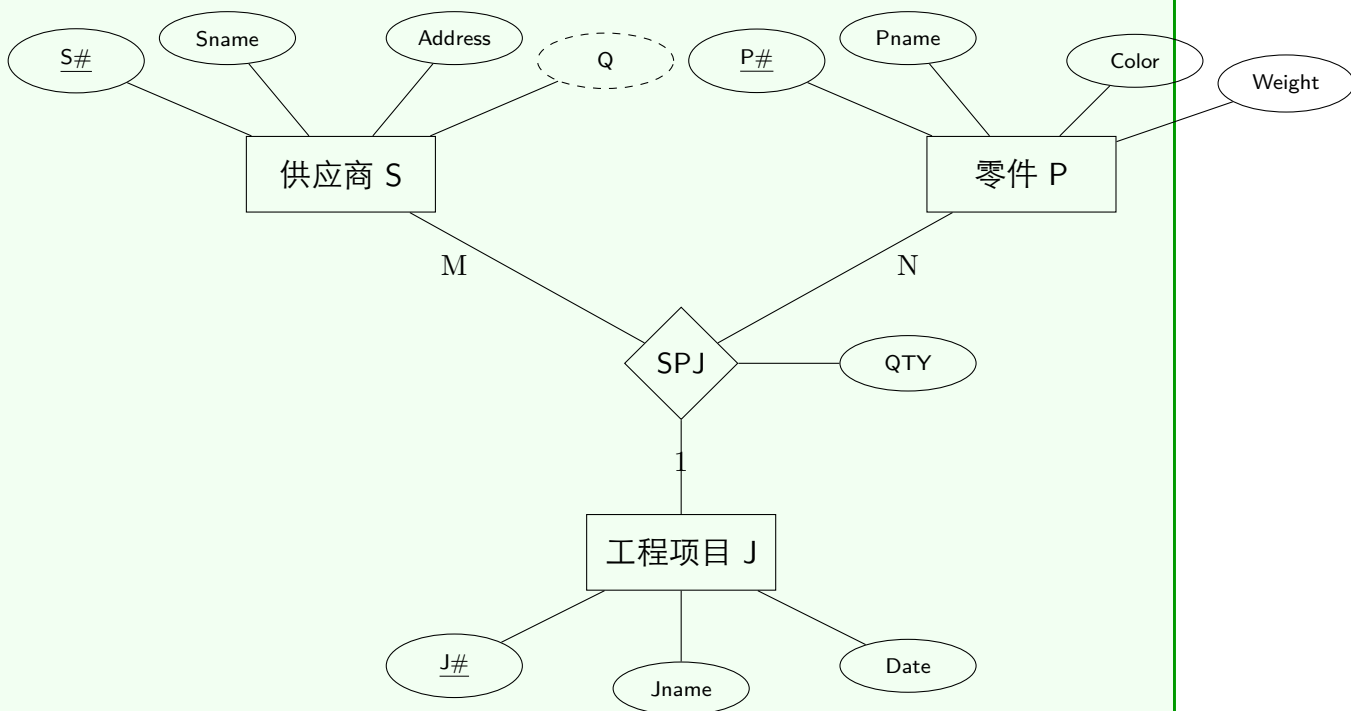
例如 $A \rightarrow B$ 意味着 A 的闭包包含了 A 和 B 的全部，故 A 是超码。这一推理的关键在于二元关系中属性的数量限制——关系模式只有两个属性，任何非平凡 FD 的左部都能借助这个 FD 搭上另一个属性，从而闭包覆盖整个属性集，自然地成为超码。因此二元关系天然满足 BCNF。这一定理揭示了 BCNF 在小规模关系中的天然成立性，也是模式分解可以持续进行到各分量都是二元为止的理论保证。但扩展到三元及以上关系时，BCNF 不再必然满足——这正是数据库规范化的动因所在：随着属性数量的增加，非平凡 FD 的左部可能无法决定全部属性，从而导致各种操作异常。证明的关键在于对二元关系的所有函数依赖情形进行穷举分类讨论，每种情形都严格验证 BCNF 条件成立。

知识点：6.3 BCNF 定义与判定——BCNF 要求每个非平凡 FD 左部都是超码；二元关系因属性数量限制天然满足 BCNF

参考答案

(答案由解析者提供)

(1) 基本 E-R 图



(2) 基本 E-R 图转换为关系模式

将三个实体和三元联系分别转换为关系模式：

- 供应商关系 **S(S#, Sname, Address, Q)**

主码为 S#。Q 为供应商提供零件的总数量（派生属性，可由 SPJ 中的 QTY 聚合计算得出）。

- 零件关系 **P(P#, Pname, Color, Weight)**

主码为 P#。

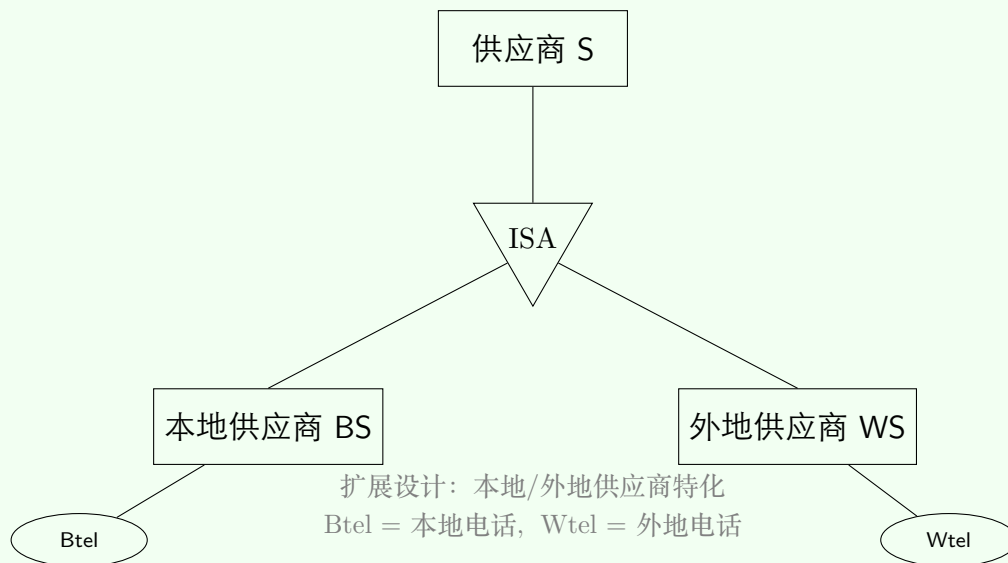
- 工程项目关系 **J(J#, Jname, Date)**

主码为 J#。

- 供应关系 **SPJ(S#, P#, J#, QTY)**

三元联系的转换关系，主码为由三个外码组成的联合主码 (S#, P#, J#)。其中 S# 是引用 S 的外码，P# 是引用 P 的外码，J# 是引用 J 的外码。QTY 为供应数量属性。

(3) 扩展设计——供应商特化 (ISA 层次)



扩展部分的关系模式转换：

使用”父表 + 子表”策略（将超类与子类分别转换为独立关系）：

- 保留原供应商父表 $S(\underline{S\#}, Sname, Address, Q)$
- 本地供应商子表 $BS(\underline{S\#}, Btel)$
主码为 $\underline{S\#}$ ，同时 $S\#$ 为引用 $S(S\#)$ 的外码。
- 外地供应商子表 $WS(\underline{S\#}, Wtel)$
主码为 $\underline{S\#}$ ，同时 $S\#$ 为引用 $S(S\#)$ 的外码。

完整关系模式汇总（含扩展）：

$S(\underline{S\#}, Sname, Address, Q)$

$P(\underline{P\#}, Pname, Color, Weight)$

$J(\underline{J\#}, Jname, Date)$

$SPJ(\underline{S\#}, \underline{P\#}, \underline{J\#}, QTY)$

$BS(\underline{S\#}, Btel)$ ——外码 $S\#$ 引用 S

$WS(\underline{S\#}, Wtel)$ ——外码 $S\#$ 引用 S

解析与知识点

本题是 SPJ（供应商—零件—工程项目）经典案例的完整 E-R 设计与关系模式转换综合题，全面考察了概念结构设计、逻辑结构设计和 IS-A 特化层次的设计能力。三元联系 SPJ 中，供应商 S、零件 P 和工程项目 J 之间是多对多对多（M:N:1 或更准确地说 M:N:P——每个供应商可为多个工程供应多种零件，每项工程可由多个供应商供应多种零件，每种零件可由多个供应商供应给多项工程）的参与关系。

三元联系在转换为关系模式时，需要为其建立一个独立的关系模式，其主码由三个参与实体的主码共同组成，同时还需包含联系自身的属性（QTY 供应数量）。在扩展设计中，IS-A 特化层次（超类/子类关系）用于表达实体内部的分类细化：供应商可分为本地供应商和外地供应商两类，各自拥有特有的属性（Btel 和 Wtel）。转换为关系模式时，采用了“超类表 + 子类表”的分离策略——父表 S 保留供应商的公共属性，子表 BS 和 WS 各自仅存其特有属性，并通过外码引用父表的主码。

这种策略保持了较高的范式等级（BCNF），避免了大量 NULL 值，且支持灵活查询（可根据需求 JOIN 父表和子表或单独访问）。

在设计 E-R 图时，三元联系需要标注每个实体侧的参与基数，ISA 层次则使用 ISA 三角形符号表示泛化/特化关系。

知识点：7.2 E-R 模型（实体、属性、联系、IS-A 层次）| 7.3 E-R 图向关系模式的转换规则（三元联系转为独立关系，1:N 联系合并到 N 端）| 7.4 特化/泛化（IS-A 联系）的关系模式转换策略（父表 + 子表法）

6.5 五、简答题（共 2 小题，每小题 8 分，共 16 分）

参考答案

（答案由解析者提供）

一、DB、DBS、DBMS 三者之间的关系

数据库（DB，Database）是长期存储在计算机内、有组织、可共享的大量数据的集合，是 DBS 的数据基础。数据库管理系统（DBMS，Database Management System）是位于用户与操作系统之间的一层数据管理软件，负责对数据库进行统一管理和控制。数据库系统（DBS，Database System）是引入数据库技术后的计算机系统，其构成可表示为：

$$\text{DBS} = \text{DB} + \text{DBMS} + \text{应用程序} + \text{数据库管理员 (DBA)} + \text{用户}$$

三者关系可概括为：DB 是被管理的对象（数据容器），DBMS 是管理 DB 的核心软件（数据引擎），DBS 是包含 DB、DBMS 以及相关人员和应用的完整系统环

境。DB 在 DBMS 的管控下运行，DBMS 在 DBS 中居于中心枢纽地位，向上服务于应用程序和用户，向下操作 OS 与物理存储。

二、DBMS 在 DBS 中的地位和作用

DBMS 是 DBS 的核心和基石，其地位不可替代。具体功能与作用包括：

- (1) 数据定义功能 (DDL)：提供数据定义语言，供用户定义数据库对象（表、视图、索引、约束等），并将定义信息保存在数据字典中。
- (2) 数据操纵功能 (DML)：提供数据操纵语言，支持用户对数据库进行查询 (SELECT)、插入 (INSERT)、删除 (DELETE) 和修改 (UPDATE) 等基本操作。
- (3) 数据库运行管理功能：这是 DBMS 运行时的核心控制功能，包括并发控制（多用户环境下事务的调度与锁管理）、安全性控制（用户身份认证与存取权限检查）、完整性控制（实体完整性、参照完整性与用户定义完整性的自动检查）以及数据库恢复（日志管理与故障恢复）。
- (4) 数据库建立与维护功能：包括数据的批量加载、数据转储与备份、数据库重组与重构、性能监测与调优等。
- (5) 数据通信接口功能：提供 DBMS 与操作系统、网络软件及其他应用系统之间的通信接口，实现跨系统的数据交换与互操作。

解析与知识点

本题是第一章绪论的核心简答题，全面考察对数据库系统基本架构的三个核心概念 (DB、DBMS、DBS) 及其层次关系的理解。这三个概念构成了数据库课程知识体系的逻辑起点，层层递进、环环相扣。DB (数据库) 是从数据存储的视角定义的概念——它是有组织、可共享、长期存储的数据集合，是整个系统的数据基础和管理对象。DBMS (数据库管理系统) 是从软件系统的视角定义的概念——它是管理 DB 的核心软件层，提供数据定义、操纵、控制、维护和通信五大功能，架起了用户与物理存储之间的桥梁。

DBS (数据库系统) 是从完整系统环境的视角定义的概念——它在 DB 和 DBMS 的基础上，进一步包含应用程序、DBA 和最终用户，构成了一个完整的计算机系统。

三者之间的包含关系可直观理解为：DBS 包含 DBMS，DBMS 管理 DB，DB 承载数据。

理解这三个概念的层次关系和各自定位，有助于从宏观上把握数据库课程的全貌及各章知识之间的内在联系——第二章到第六章 (关系模型和规范化) 聚焦于 DB

的逻辑设计，第七章聚焦于 DBS 的设计过程，第八章到第十一章（编程、优化、事务、恢复）聚焦于 DBMS 的内部机制和实现技术。

知识点： 1.1 数据库系统概述（DB, DBMS, DBS 的定义与层次包含关系）| 1.3 DBMS 的功能与系统结构——DBMS 是 DBS 的核心，其五大功能构成数据库系统的运行基础

参考答案

（答案由解析者提供）

一、存储过程与触发器的相同点

- 两者都是存储在数据库服务器端的预编译 SQL 代码块（命名程序单元），可包含流程控制语句（条件判断、循环等）、变量声明和异常处理。
- 两者均能封装复杂的业务逻辑，减少客户端与服务器之间的网络通信开销，提高执行效率。
- 两者均可被授予或回收执行权限，是数据库安全机制的组成部分。
- 两者在首次执行时被编译并缓存，后续调用/触发时使用编译后的执行计划，性能优于每次解释执行的动态 SQL。

二、存储过程与触发器的不同点

对比维度	存储过程 (Stored Procedure)	触发器 (Trigger)
触发方式	需通过 CALL/EXEC 语句显式调用执行	由特定数据库事件 (INSERT、UPDATE、DELETE) 自动触发, 不需显式调用
参数支持	可定义输入参数 (IN)、输出参数 (OUT) 和输入输出参数 (INOUT)	不能定义参数, 完全依赖于触发事件所涉数据 (通过 NEW/OLD 伪记录访问)
返回值	可通过 OUTPUT 参数或 RETURN 语句返回值	无返回值, 通过修改数据产生效果
与事务的关系	在调用者的事务上下文中执行, 调用者可控制提交与回滚	与触发它的 DML 语句属于同一事务, 触发器失败通常导致触发语句回滚
主要用途	封装通用可复用业务逻辑 (如订单处理、账户转账)、为应用程序提供调用接口	自动维护数据完整性与一致性 (如级联更新/删除)、审计追踪、派生数据自动维护
调用限制	可在过程中执行任意 SQL (包括调用其他存储过程), 但通常不能直接包含 COMMIT/ROLLBACK	触发器中通常禁止显式的事务控制语句 (COMMIT/ROLLBACK), 且不能调用某些可能引起递归触发的操作

三、建立触发器的意义

- (1) 实现复杂的完整性约束: CHECK 约束只能定义同一表中列级或行级的简单条件, 而触发器可以通过查询多张表、执行复杂逻辑来维护跨表、跨行的业务规则, 弥补声明式约束的不足。
- (2) 自动级联维护数据一致性: 当主表数据被修改 (删除或更新主码值) 时, 触发器可自动在子表中执行相应的级联更新或级联删除操作, 保证参照完整性不被动破坏。
- (3) 审计追踪与日志记录: 通过在关键表上设置 BEFORE/AFTER 触发器, 系统可以自动记录对敏感数据的每一次变更操作 (操作者、操作时间、变更前后的值), 形成不可绕过、不可抵赖的审计日志。
- (4) 维护派生数据与冗余字段: 在需要提高查询性能而引入冗余列或汇总表时, 触发器能在基表数据变更时自动同步更新这些派生数据, 保证其始终与基表

保持一致，避免“数据陈旧”问题。

- (5) 实施安全性检查：触发器可在数据操作执行前进行额外的权限验证或操作合法性检查（如禁止在非工作时间修改工资数据），作为 GRANT/REVOKE 静态权限控制的动态补充手段。
- (6) 实现复杂的业务规则：在企业级应用中，许多业务规则涉及多表关联、时序判断和复杂的状态转移逻辑，触发器可将其固化在数据库层面，确保无论通过何种应用程序接口修改数据，核心规则都能被强制执行。

解析与知识点

本题全面考察对数据库编程中两个重要概念——存储过程与触发器的对比理解。存储过程和触发器虽有相同点（都是服务器端预编译的命名程序单元、都能封装业务逻辑、都具有性能优势），但它们在触发方式、参数支持、与事务的关系以及主要用途上存在根本差异。

核心区别可以概括为：存储过程是“按需调用”的主动服务封装，面向的是业务逻辑的复用（如用户主动发起转账操作）；触发器是“事件驱动”的自动响应机制，面向的是数据完整性的自动化保障（如数据变更时自动记录审计日志或级联更新汇总数据）。存储过程支持参数传递和返回值，作为对应用程序的调用接口——这使其天然适合封装通用的、可复用的业务操作。触发器则完全由 DML 事件驱动，无法接收外部参数，只能通过 NEW/OLD 伪记录访问触发事件所涉及的数据行——这使其天然适合实现响应式的数据约束和自动维护。

建立触发器的意义可归纳为：它将数据库从一个被动的“数据容器”升级为能够主动防御数据异常、自动维护一致性和自动记录审计轨迹的“智能数据守护者”。

在实际数据库设计中，CHECK 约束和参照完整性的级联选项能够处理的场景是有限的，触发器作为更灵活、更强大的完整性实现机制，填补了声明式约束无法覆盖的复杂业务规则空间。

知识点：8.2 存储过程——面向业务逻辑复用的主动调用式编程单元 | 8.3 触发器的概念、类型与应用——事件驱动的自动响应机制，用于完整性维护、审计追踪和派生数据同步 | 5.3 完整性约束的实现机制——触发器作为声明式约束的补充，实现复杂业务规则

7 2019–2020 学年《数据库系统》期末试卷 (A 卷, 开卷)

注意

说明：本试卷原无官方参考答案。所有答案和解析均由解析者根据教材内容和专业知识独立完成，并在答案中标注”（答案由解析者提供）”。

一、简答题（每小题 8 分，共 24 分）

参考答案

（答案由解析者提供）

DB（数据库）：是按一定数据模型组织、存储在一起的相互关联的数据集合，具有较小的冗余度和较高的数据独立性，可为多个用户共享。它是长期存储在计算机内的、有组织的、可管理的数据。

DBS（数据库系统）：是由数据库（DB）、数据库管理系统（DBMS）、应用程序、数据库管理员（DBA）和用户共同构成的计算机应用系统。DBS 是安装了数据库软件以后的整体运行环境，涵盖硬件、软件、数据和人员四个层次。

DBMS（数据库管理系统）：是位于用户与操作系统之间的一层数据管理软件，属于系统软件。它是 DBS 中最核心的组成部分，负责数据库的建立、使用和维护。

三者关系：DBS 包含 DBMS 和 DB，即 $DBS = DB + DBMS + \text{应用程序} + DBA + \text{用户}$ 。DB 是数据的物理/逻辑载体，DBMS 是管理 DB 的软件工具，DBS 是包含前两者的完整系统。

DBMS 的地位与作用：DBMS 在 DBS 中处于核心地位，起承上启下的关键作用。具体而言：(1) 数据定义——提供数据定义语言 (DDL) 定义数据库对象；(2) 数据操纵——提供数据操纵语言 (DML) 实现数据增删改查；(3) 数据库运行管理——提供并发控制、安全性检查、完整性约束、日志恢复等统一管理；(4) 数据库建立与维护——提供数据加载、转储、恢复、重组等实用工具。没有 DBMS，用户就无法高效、安全地访问和管理数据库中的数据。

解析与知识点

本题考察数据库系统中最基础也最重要的三个核心概念：DB、DBS 和 DBMS 的内涵及其相互关系。这三个术语是学习整个数据库课程的概念出发点，必须精确区分。

DB（数据库）是数据的有组织集合体，强调”存储什么”——它是长期存储在计算机中的、结构化的、可共享的相关数据集合，具有较小的冗余度和较高的数据独立性。DBS（数据库系统）则是一个完整的应用系统概念，涵盖硬件、软件、数据和人员四个层面，强调”整体运行环境”。DBMS（数据库管理系统）是位于用

户和操作系统之间的系统软件层，是 DBS 中最核心的软件组件，负责数据定义、数据操纵、数据库运行管理和数据库维护四大功能。三者的本质关系是层级包含：DBS 是最大的概念，包含 DBMS 和 DB 以及应用程序、DBA 和用户；DBMS 是管理工具，操作 DB 中的数据；DB 是数据载体。

DBMS 之所以居于核心地位，是因为它屏蔽了数据物理存储的复杂性，向上为用户和应用程序提供统一的逻辑视图和操作接口，向下将逻辑操作翻译为物理存取指令。理解这三个概念的层次关系，有助于在后续学习中把握 SQL（由 DBMS 提供）、数据库设计（为 DB 建立结构）和系统管理（DBA 对 DBS 的运维）等内容的整体框架。

常见的错误是混淆 DB 和 DBS 的边界，或把 DBS 简单等同于装了数据库软件的计算机——实际上 DBS 还需要有应用程序和人员才算完整。

知识点：1.1（数据库系统概述）；1.3（数据库系统的结构）

参考答案

（答案由解析者提供）

可串行化调度：多个事务的并发执行结果是正确的，当且仅当该调度与这些事务按某种顺序串行执行所产生的结果相同。换句话说，若一个并发调度等价于某个串行调度，则称该并发调度是“可串行化”的。可串行化是并发事务正确性的基本准则。

两段锁协议与可串行化调度的关系：两段锁协议（2PL）要求每个事务分为两个阶段：（1）扩展阶段——事务只能获取锁，不能释放任何锁；（2）收缩阶段——事务只能释放锁，不能获取新的锁。两段锁协议是可串行化的充分不必要条件：所有遵守两段锁协议的并发调度一定是可串行化的（充分性），但一个可串行化调度中的事务不一定遵守两段锁协议（非必要性）。

给定调度的可串行化分析：

初始值： $A = 3000$ ， $B = 3000$ 。

并发执行结果：

- t_{1-3} : T_1 读 $A = 3000$ ， $A = 2000$ ，写回 $A = 2000$ 。
- t_{4-7} : T_2 读 $A = 2000$ ，Temp=200， $A = 1800$ ，写回 $A = 1800$ 。
- t_{8-10} : T_2 读 $B = 3000$ ， $B = 3200$ ，写回 $B = 3200$ 。
- t_{11-13} : T_1 读 $B = 3200$ ， $B = 4200$ ，写回 $B = 4200$ 。

并发执行最终结果： $A = 1800$ ， $B = 4200$ 。

串行执行 $T_1 \rightarrow T_2$ (T_1 先全部执行完，再执行 T_2):

- 执行 T_1 : $A = 2000, B = 4000$ 。
- 执行 T_2 : 读 $A = 2000$, $Temp=200$, $A = 1800$; 读 $B = 4000$, $B = 4200$ 。
- 最终结果: $A = 1800, B = 4200$ 。

串行执行 $T_2 \rightarrow T_1$ (T_2 先全部执行完, 再执行 T_1):

- 执行 T_2 : $Temp=3000 \times 0.1 = 300$, $A = 2700$; $B = 3300$ 。
- 执行 T_1 : $A = 2700 - 1000 = 1700$; $B = 3300 + 1000 = 4300$ 。
- 最终结果: $A = 1700, B = 4300$ 。

结论: 该调度是可串行化的。因为并发执行的结果 ($A = 1800, B = 4200$) 与串行执行 $T_1 \rightarrow T_2$ 的结果完全一致, 即该并发调度等价于先执行 T_1 再执行 T_2 的串行调度。

解析与知识点

本题是并发控制的经典综合题型, 从定义、理论关系到实际判断三个层次考察学生对事务并发执行正确性的理解。涵盖可串行化调度的严格定义、两段锁协议 (2PL) 与可串行化的逻辑关系, 以及给定并发调度判断其是否可串行化的实操计算。

第一小问要求给出可串行化调度的严格定义——这是并发控制领域的基石概念: 一个并发调度是正确的, 当且仅当它的执行结果与这些事务按某种顺序串行执行的结果相同。第三小问是实操计算题, 给定了初始值 $A = B = 3000$ 和具体的并发执行步骤, 要求通过计算判断该调度是否可串行化。判断方法为: 先按给定并发调度逐步计算最终结果, 再分别按 $T_1 \rightarrow T_2$ 和 $T_2 \rightarrow T_1$ 两种串行顺序计算最终结果, 若并发结果与任一串行结果一致, 则可判定为可串行化。本题中并发执行结果为 ($A = 1800, B = 4200$), 恰好等于 $T_1 \rightarrow T_2$ 的串行结果, 因此该调度是可串行化的, 且等价于先执行 T_1 再执行 T_2 。

第二小问考察两段锁协议与可串行化的逻辑关系, 这是考试中的高频考点: 2PL 是可串行化的充分但不必要条件——遵守 2PL 的调度一定可串行化 (充分性由两段锁定理保证), 但可串行化调度中的事务不一定遵守 2PL (因为可串行化是语义层面的概念, 2PL 是语法层面的协议, 语义可串行化可能通过非 2PL 的方式实现)。

常见的解题错误包括: 串行执行时忘记 T_1 修改了 B 后会影响到 T_2 对 B 的读取, 或者在并发执行步进时遗漏某一操作的中间值传递。

知识点: 10.4 (调度的可串行化); 10.5 (两段锁协议); 10.2 (事务的基本概念)

参考答案

(答案由解析者提供)

基本表 (Base Table): 是数据库中独立存在的、实际存储在物理存储设备上的关系表。每个基本表对应一个物理存储文件 (或存储区域), 其元组按某种物理顺序存储, 是数据库中最基础的数据组织单元。基本表由 CREATE TABLE 语句定义, 可通过 INSERT、DELETE、UPDATE 直接对其数据进行增删改操作。

查询表 (Query Table): 是由 SELECT 语句执行后动态生成的临时结果集, 表示从一个或多个基本表 (或视图) 中检索出的数据。查询表存放在内存中, 不占用永久存储空间, 会话或查询结束后即被释放。查询表不是数据库的持久对象, 每次查询都重新动态生成。

视图 (View): 是从一个或多个基本表 (或视图) 中导出的虚拟表。数据库中仅存储视图的定义 (即对应的 SELECT 语句), 不存储实际的视图数据。视图一经定义, 可像基本表一样被查询, 且可在此基础上再定义新视图。

联系: 三者均表现为关系 (二维表) 的形式, 都可通过 SELECT 语句查询数据。视图和查询表的数据本质上都来源于基本表——基本表是数据的唯一物理载体, 视图和查询表都是对基本表数据的逻辑映射。

区别: (1) 存储方式不同: 基本表有物理存储; 视图只有逻辑定义 (存定义不存数据); 查询表是临时内存数据。(2) 存在周期不同: 基本表持久存在; 视图定义持久存在但其数据不持久; 查询表是瞬态的。(3) 更新能力不同: 基本表可自由增删改; 视图的更新受限 (只能在有限条件下进行); 查询表不可直接更新。

视图更新受限的原因: 视图不实际存储数据, 对视图的更新最终必须转化为对基本表的更新。然而, 并非所有视图的更新都能唯一确定对应的基本表更新操作。例如: (1) 涉及聚合函数 (SUM、AVG、COUNT 等) 的视图无法逆推出各基本表元组的修改量; (2) 涉及多表连接的视图, 更新操作可能无法唯一确定各基本表的对应元组; (3) 涉及表达式计算的列 (如 $\text{price} \times 1.1$) 无法进行逆向赋值。因此, SQL 标准规定只有行列子集视图 (从单个基本表通过行列选择导出) 可以更新。

数据的有序实现方式:

- **逻辑有序**: 使用 ORDER BY 子句在查询结果中指定排序规则。例如 `SELECT * FROM s ORDER BY sno ASC, city DESC`。ORDER BY 仅影响查询结果的呈现顺序, 不改变数据的物理存储顺序。它可作用于基本表和查询表。
- **物理有序**: 通过创建聚集索引 (Clustered Index) 使基本表中的数据按照索引键值的顺序物理存储。一个基本表最多只能有一个聚集索引。例如 `CREATE CLUSTERED INDEX idx_sno ON s(sno)`。聚集索引决定了数据在磁盘上的物理排列顺序, 能显著提高范围查询的性能。

解析与知识点

本题是关系数据库基础概念的综合考察,涉及基本表、查询表和视图三个密切相关但又本质不同的表类型,并扩展考察了数据“有序”的两种实现方式——逻辑有序 (ORDER BY) 和物理有序 (聚集索引)。

基本表是数据的物理基石——它是实际存储在磁盘上的持久关系,对应 CREATE TABLE 创建的数据库对象,是关系数据库中数据的唯一物理载体。视图是建立在基本表之上的虚拟表,数据库中只保存其定义 (SELECT 语句),执行时从基本表实时计算出虚拟数据。查询表则是 SELECT 语句的即时输出,是内存中的临时结果集,不具持久性。理解三者的区别需要抓住“存储”“持久性”“可更新性”三个维度:基本表在三者中持有全部能力,视图持有查询能力和受限的更新能力,查询表仅有瞬时的读取能力。

视图的核心价值在于提供数据独立性、安全性和简化复杂查询,但代价是删除、插入和更新的受限性——因为对视图的修改必须逆向转化为对基本表的修改,而这种逆向映射对聚合、连接或含表达式的视图而言不具有唯一性。逻辑有序通过 ORDER BY 子句在查询结果层面实现排序,不改变存储,成本是每次查询都需要排序计算;物理有序通过聚集索引 (Clustered Index) 实现,使数据在磁盘上按索引键值顺序排列,适合频繁范围查询的场景。两者的关键区别在于:ORDER BY 是临时的、每查询一次执行一次;聚集索引是永久的、物理层面的排序,一个表最多只能有一个。

常见的误解是认为所有索引都能让数据物理有序,实际上只有聚集索引才有此效果,非聚集索引 (普通索引) 仅建立逻辑索引结构,数据页的物理顺序不变。

知识点: 3.1 (SQL 概述与数据定义); 3.5 (视图); 3.4 (数据查询——ORDER BY); 5.3 (完整性约束与索引); 7.1 (数据库设计概述——基本表的概念)

二、数据操作题 (共 40 分)

注意

已知供应数据库包含四个基本表:

- 供应商: S(sno, sname, status, city)。属性含义: sno——供应商号; sname——供应商名; status——供应商状态; city——供应商所在城市。
- 工程: J(jno, jname, city)。属性含义: jno——工程号; jname——工程名; city——工程所在城市。
- 零件: P(pno, pname, color, price)。属性含义: pno——零件号; pname——零件名; color——颜色; price——价格。

- 供货：SPJ(sno, pno, jno, qty)。属性含义：sno——供应商号；pno——零件号；jno——工程号；qty——供应数量。

语义：某供应商供应某种零件给某个工程；同一零件可有多个供应商提供，同一供应商可提供多种零件；工程不受零件和供应商的限制。

1. 用关系代数表示下列查询（每小题 5 分，共 10 分）

参考答案

（答案由解析者提供）

关系代数表达式：

$$\pi_{jno, pno, qty} \left(\sigma_{sname='张军'} (S \bowtie SPJ) \right)$$

解析：先用自然连接 $\bowtie$ 将 S 表和 SPJ 表按共同属性 sno 连接起来；再用选择 σ 筛选出供应商名为“张军”的元组；最后用投影 π 提取需要的三列（工程号 jno、零件号 pno、供应数量 qty）。

解析与知识点

本题考察关系代数的基本操作组合——自然连接 ($\bowtie$)、选择 (σ) 和投影 (π) 的组合使用方式，是数据库理论考试中的基础题型。

由于最终查询结果需要的是工程号 jno、零件号 pno 和供应数量 qty 三列，而筛选条件“供应商名为张军”依赖于 S 表中的 sname 属性，因此必须通过自然连接将 S 表和 SPJ 表按公共属性 sno 关联起来。操作顺序是从内向外：先做自然连接，再做条件选择，最后做属性投影。自然连接在这里的关键作用是：它自动匹配两张表中 sno 值相等的元组，将供应商的详细信息（包括 sname）附加到每条供货记录上。连接之后再施加选择操作筛选出 sname 为“张军”的元组，最后用投影提取目标列。

这是一种典型的“连接-筛选-投影”三段式关系代数解题模式，几乎适用于所有涉及多表查询的关系代数题目。

常见的错误包括：忘记自然连接会带来额外的属性（需要在投影中去掉），或者混淆了选择和投影的执行顺序——选择必须在投影之前完成，否则 sname 属性已被投影掉，后续无法筛选。

知识点：2.4（关系代数——连接、选择、投影）

参考答案

(答案由解析者提供)

关系代数表达式:

$$\pi_{\text{sno}}(\sigma_{\text{jno}='J1'}(\text{SPJ})) \cap \pi_{\text{sno}}(\sigma_{\text{jno}='J2'}(\text{SPJ}))$$

解析: “至少为 J1 和 J2 供货”意味着该供应商号必须同时出现在向 J1 供货的记录和向 J2 供货的记录中, 即取二者的交集。先分别用选择和投影求出为 J1 供货的供应商号和向 J2 供货的供应商号, 再取交集即可。

等价写法 (使用嵌套除法):

$$\pi_{\text{sno}, \text{jno}}(\text{SPJ}) \div \pi_{\text{jno}}(\sigma_{\text{jno}='J1' \vee \text{jno}='J2'}(\text{J}))$$

其中除法的含义为: 找出那些供应记录中“包含”了全部指定工程 (J1 和 J2) 的供应商号。

解析与知识点

本题考察关系代数中“至少……都……”语义的表达能力, 是交集运算与除法运算的经典应用场景。题目语义为“至少向 J1 和 J2 均供过货的供应商”, 本质上是求两个集合的交集。

解题采用交集法: 分别对 SPJ 表按 jno 条件做选择和投影, 得到向 J1 供货的供应商号集合和向 J2 供货的供应商号集合, 再取二者交集。这种方法的直觉性很强——逐条件拆分后就自然地引出集合运算。备选的除法解法同样正确但通常较难理解: 将 SPJ 按 (sno, jno) 投影后, 除以包含 J1 和 J2 的工程号投影结果, 语义上是找出那些其供应记录中的工程号集合“覆盖”了指定工程集合的供应商。

关系代数中处理“至少……都……”类全称量词语义有三种标准策略: (1) 交集法——将各个条件分别求结果集后取交集, 最直观; (2) 除运算——使用关系代数的除运算, 语义上最“数学”; (3) 利用差运算模拟 NOT EXISTS—— $\pi(A) - \pi(\pi(A) \times \pi(B) - \pi(R))$ 。在应试中, 交集法因其直观性和构造简便性而被推荐为首选解法。常见错误是将条件写成“或”而不是“且”——“至少为 J1 和 J2 供货”要求的是两者都供过货, 故选出向 J1 供货的记录和向 J2 供货的记录后必须取交集而非并集。

知识点: 2.4 (关系代数——选择、投影、交运算、除运算)

2. 用 SQL 语句表示下列操作 (共 30 分)

参考答案

(答案由解析者提供)

```
1  -- 方法一：使用 NOT IN
2  SELECT sno, sname
3  FROM S
4  WHERE sno NOT IN (
5      SELECT DISTINCT SPJ.sno
6      FROM SPJ
7      JOIN P ON SPJ.pno = P.pno
8      WHERE P.color = '红色'
9  );
```

```
1  -- 方法二：使用 NOT EXISTS (更推荐，语义更清晰)
2  SELECT sno, sname
3  FROM S
4  WHERE NOT EXISTS (
5      SELECT *
6      FROM SPJ
7      JOIN P ON SPJ.pno = P.pno
8      WHERE SPJ.sno = S.sno
9          AND P.color = '红色'
10 );
```

解析：思路是找出所有提供了红色零件的供应商号集合，然后用 NOT IN (或 NOT EXISTS) 排除它们，从而得到未提供红色零件的供应商。

解析与知识点

此题考察 SQL 中“不存在型”查询的实现，属于嵌套子查询的经典模式。涵盖 NOT IN 与 NOT EXISTS 两种写法的比较及各自的适用场景和陷阱。

题目语义是“查询没有提供红色零件的供应商号与姓名”，核心思路是两段式：先找出“提供了红色零件”的供应商号集合，再从全体供应商中排除这个集合。方法一使用了 NOT IN 子查询，简洁明了，但有一个重要陷阱——如果子查询结果中包含 NULL 值，NOT IN 会返回空集（因为任何值与 NULL 的比较结果都是 UNKNOWN）。方法二使用了相关子查询 NOT EXISTS，通过外层 S 表的 sno 与内层 SPJ 表的 sno 建立关联，判断是否存在该供应商供应红色零件的记录。NOT EXISTS 的优势在于不受 NULL 值影响——它的判断逻辑是“是否存在满足条件

的行”，而非“值是否在某集合中”。

这里的关键认知是：一个供应商“未提供红色零件”可能的原因有两种——(a) 该供应商根本没有任何供货记录；(b) 该供应商有供货记录但提供的零件都不是红色的。NOT IN 和 NOT EXISTS 都能正确覆盖这两种情形。另一种等效写法是使用 LEFT JOIN ... WHERE ... IS NULL 的“外连接 + 空值判断”模式。

常见的错误是忘记加 DISTINCT（在 NOT IN 中不是必需的但有助于性能），或误用普通子查询代替相关子查询。考生也需要意识到 NOT IN 对 NULL 值的敏感性，以及 NOT EXISTS 在健壮性上的优势。知识点：3.4（数据查询——嵌套查询、NOT IN、NOT EXISTS、连接查询）

参考答案

（答案由解析者提供）

```
1 SELECT DISTINCT SPJ.pno
2 FROM SPJ
3 JOIN S ON SPJ.sno = S.sno
4 JOIN J ON SPJ.jno = J.jno
5 WHERE S.city = J.city;
```

解析：将 SPJ 表与供应商表 S 按 sno 连接，再与工程表 J 按 jno 连接，在 WHERE 中施加条件 S.city = J.city；最后用 DISTINCT 去重，因为同一零件号可能通过多个供应记录出现。

解析与知识点

这是典型的多表连接查询题，考察 SQL 中通过连接将多个表的信息汇总到一条查询中的能力，涉及事实表与维表的连接模式以及 DISTINCT 去重。

SPJ 表作为核心“事实表”（Fact Table），记录了哪个供应商将哪种零件供应给哪个工程。S 表和 J 表作为“维表”（Dimension Table），分别提供供应商和工程的详细属性。查询目标“供应商所在城市与工程所在城市相同的供应记录中的零件号”需要将三张表按外键关系连接起来：SPJ.sno 对应 S.sno（供应商维度关联），SPJ.jno 对应 J.jno（工程维度关联）。连接后的结果集包含三家表的所有属性，此时再施加 WHERE 条件 S.city = J.city 来筛选城市匹配的供应记录。最后的关键操作是 DISTINCT 去重——因为同一个零件可能通过多个供应记录出现。

从查询优化的角度，这个查询的 JOIN 顺序通常由 SQL 优化器自动决定，但理解其执行逻辑有助于写出高效的 SQL：先筛选再连接（如果可能的话），以及优先缩小中间结果集。多表连接是 SQL 最重要也最基础的能力之一。

这道题看似简单，实则隐含了对连接语义、外键关系理解和去重必要性的综合考

察。常见的错误是遗漏 DISTINCT 导致结果中出现重复零件号。

知识点：3.4（数据查询——多表连接、DISTINCT）

参考答案

（答案由解析者提供）

```
1 SELECT sno, pno, SUM(qty) AS total_qty
2 FROM SPJ
3 GROUP BY sno, pno
4 ORDER BY sno ASC, total_qty DESC;
```

解析：按供应商号 sno 和零件号 pno 进行分组（GROUP BY），对每组内所有供应记录的 qty 求和，得到该供应商对该零件供应的总数量。外层用 ORDER BY 实现二级排序：先按 sno 升序（ASC），同 sno 下按聚合结果 total_qty 降序（DESC）。

解析与知识点

本题考察 SQL 分组聚合与多级排序的组合应用，是数据分析类查询的典型代表。核心操作分解为三步骤：分组（GROUP BY）、聚合（SUM）和排序（ORDER BY）。

题目要求“求每个供应商每种零件的总供应量，并排序”。首先，GROUP BY (sno, pno) 将 SPJ 表中的供应记录按供应商号和零件号的组合进行分组——每个 (sno, pno) 组合形成一组。其次，聚合函数 SUM(qty) 对每组内的所有 qty 值求和，得到该供应商对该零件的总供应量。最后，ORDER BY sno ASC, total_qty DESC 实现了两级排序：一级键为 sno 按升序，二级键为 total_qty 按降序。二级排序的执行逻辑是：先按 sno 升序排列各组，当多个组的 sno 相同时（即同一供应商的不同零件），再按 total_qty 降序排列。

GROUP BY 中出现的非聚合列（sno 和 pno）在 SELECT 中出现是合法的，因为它们在组内是唯一的。SELECT 子句中出现的非聚合列必须包含在 GROUP BY 中，否则 SQL 会报错或产生不确定结果（取决于 SQL_MODE 设置）。

常见错误包括：忘记在 ORDER BY 中为 DESC 显式写出，或误将 total_qty 写在 GROUP BY 中。

知识点：3.4（数据查询——GROUP BY、聚合函数 SUM、ORDER BY 多级排序）

参考答案

（答案由解析者提供）

```
1 GRANT DELETE, UPDATE(city) ON TABLE S TO U1, U2
```



```
2 WITH GRANT OPTION;
```

解析：

- DELETE 授予删除权限（对应题目“删除”）。
- UPDATE(city) 授予对 city 列的修改权限（对应题目“修改 city”）。注意此处是列级权限，仅限 city 列，而非整个 S 表的所有列的 UPDATE 权限。
- WITH GRANT OPTION 表示 U1 和 U2 可以将这些权限再转授给其他用户（对应题目“可以将该权限授予其他用户”）。

解析与知识点

本题考察 SQL 数据控制语言（DCL）中的 GRANT 授权语句，属于数据库安全机制的核心内容。涉及权限粒度（表级 vs 列级）、多用户授权和 WITH GRANT OPTION 权限传播机制。

解题需要关注三个细节：(1) 权限粒度：DELETE 是表级权限（删除整行而非删除列中的值），不能指定列范围，因此只写 DELETE 即可；UPDATE 可以细化到列级，通过 UPDATE(列名) 的形式仅授予对特定列的修改权限——本题只需授予修改 city 列，所以写 UPDATE(city) 而非 UPDATE。(2) 多用户授权：可以在一条 GRANT 语句中用逗号分隔多个用户名，同时向 U1 和 U2 授权。(3) 权限传播：WITH GRANT OPTION 子句的作用是允许被授权的用户将这些权限再转授给其他用户，形成权限的传播链。如果不加此子句，则 U1 和 U2 只能自己使用这些权限而不能再授予他人。

GRANT 语句是实现自主存取控制（Discretionary Access Control, DAC）的关键手段——数据所有者可以将自己拥有的权限授予其他用户，并决定是否允许这些用户进一步传播权限。GRANT/REVOKE 机制与角色（ROLE）、审计（AUDIT）共同构成了数据库的安全防护体系。

常见的错误是误认为 DELETE 也可以像 UPDATE 一样限制到列级（实际上删除操作是针对整行的），或遗漏 WITH GRANT OPTION 导致权限传播能力缺失。

知识点：4.2（存取控制——GRANT 语句、WITH GRANT OPTION）

参考答案

（答案由解析者提供）

```
1 CREATE TRIGGER check_status_negative
2 AFTER UPDATE OF status ON S
3 REFERENCING NEW ROW AS NewTuple
```

```
4 FOR EACH ROW
5 WHEN (NewTuple.status < 0)
6 BEGIN
7     UPDATE SPJ
8     SET qty = NULL
9     WHERE sno = NewTuple.sno;
10 END;
```

触发器逻辑说明：

- (1) AFTER UPDATE OF status ON S: 触发器在 S 表的 status 列被更新之后触发执行。由于需要根据修改后的新值进行判断和操作, 所以使用 AFTER 触发。限定 OF status 表示仅在 status 列被修改时才激活触发器。
- (2) REFERENCING NEW ROW AS NewTuple: 引用更新后的新行, 别名 NewTuple。
- (3) FOR EACH ROW: 行级触发器, 即每更新一行就触发一次 (而非语句级)。
- (4) WHEN (NewTuple.status < 0): 触发条件——仅在新 status 值小于 0 时才执行触发体。
- (5) 触发体: UPDATE SPJ SET qty = NULL WHERE sno = NewTuple.sno, 将 SPJ 表中属于该供应商的所有供货记录的 qty 值置为 NULL (空值), 对应题目要求的“改为空值”。

解析与知识点

本题是触发器编写的综合应用题, 考察触发器的设计与实现能力, 涵盖触发时机 (AFTER)、触发粒度 (FOR EACH ROW)、条件过滤 (WHEN) 以及跨表更新操作。

触发器的本质是一段“事件-条件-动作” (ECA, Event-Condition-Action) 规则。本题的 ECA 规则为: 事件是 S 表的 status 列被 UPDATE, 条件是更新后的新 status 值小于 0, 动作是将该供应商的所有 SPJ 供货记录的 qty 改为 NULL。触发时机的选择是第一个关键决策点: 必须使用 AFTER 而非 BEFORE, 因为触发器逻辑需要根据更新后的新 status 值来判断, 并进一步修改 SPJ 表——AFTER 确保 status 已安全写入数据库。第二个关键点是触发粒度: FOR EACH ROW (行级触发器) 意味着 UPDATE 语句每修改一行就触发一次, 每行独立判断 WHEN 条件。第三个关键点是 WHEN 条件过滤: 通过 WHEN (NewTuple.status < 0) 在触发器层面做条件筛选, 避免了不必要的关联更新开销。

第四个关键点是触发动作的关联更新：UPDATE SPJ SET qty = NULL WHERE sno = NewTuple.sno 使用 WHERE 子句将更新范围限定为该供应商的所有 SPJ 记录——一条供应商记录的状态变更可能影响多条 SPJ 记录。从数据库完整性的视角看，触发器是用户定义完整性的重要实现手段，它可以实现 CHECK 约束无法表达的复杂业务规则（如图中需要跨表操作），比 CHECK 约束更灵活但性能开销更大。

常见错误包括混淆 BEFORE 和 AFTER 的适用场景，或误用 FOR EACH STATEMENT（语句级触发器）代替 FOR EACH ROW——前者无法通过 NEW 关键字访问具体行的新值。

知识点：8.3（触发器——CREATE TRIGGER、AFTER/FOR EACH ROW、REFERENCING、WHEN）；5.3（完整性约束——用户定义完整性的触发器实现）；3.4（数据更新——UPDATE 嵌套）

三、关系规范化分析（共 16 分）

设有关系 $R(A, B, C, D, E, P)$ ，函数依赖集 $F = \{ AB \rightarrow C, C \rightarrow D, E \rightarrow A, B \rightarrow C, CA \rightarrow E, BD \rightarrow A \}$

参考答案

（答案由解析者提供）

Step 1：寻找候选键。

首先确定 LHS/RHS 属性分布。写出 F 中所有函数依赖的右部属性：

$$\text{RHS 属性} = \{C\} \cup \{D\} \cup \{A\} \cup \{C\} \cup \{E\} \cup \{A\} = \{A, C, D, E\}$$

属性全集 $U = \{A, B, C, D, E, P\}$ 。

任何候选键都必须包含那些不出现在任何函数依赖右部的属性（LHS 专属属性）： B 和 P 。因此， B 和 P 必须属于所有候选键。

计算 $(BP)^+$ （在 F 下的闭包）：

$$(BP)^+ = \{B, P\} \quad (\text{初始})$$

$$\xrightarrow{B \rightarrow C} \{B, C, P\} \quad (C \text{ 加入})$$

$$\xrightarrow{C \rightarrow D} \{B, C, D, P\} \quad (D \text{ 加入})$$

$$\xrightarrow{BD \rightarrow A} \{A, B, C, D, P\} \quad (B \text{ 和 } D \text{ 确定 } A, A \text{ 加入})$$

$$\xrightarrow{CA \rightarrow E} \{A, B, C, D, E, P\} \quad (C \text{ 和 } A \text{ 确定 } E, E \text{ 加入})$$

因此 $(BP)^+ = \{A, B, C, D, E, P\} = U$ 。 BP 是超键。

验证 BP 是否最小: $(B)^+ = \{B, C, D, A, E\}$ (不含 P), 不足全体属性; $(P)^+ = \{P\}$ (不含其他属性)。所以 BP 的每个真子集均不能推导出全部属性, 故 BP 不可约简。

结论: BP 是 R 的唯一候选键 (Candidate Key)。

Step 2: 判断范式级别。

候选键 BP 中的属性称为主属性 (Prime Attributes): $\{B, P\}$ 。非主属性 (Non-Prime Attributes): $\{A, C, D, E\}$ 。

检查是否存在部分函数依赖 (即非主属性是否函数依赖于候选键的某个真子集):

- $B \rightarrow C$: $B \subset BP$ (B 是候选键的真子集), C 是非主属性。存在非主属性 C 部分依赖于候选键 BP 。

这违反了第二范式 (2NF) 的要求。 R 已经满足第一范式 (1NF) ——所有属性值均为不可分解的原子值。

结论: 关系 R 属于 1NF, 不属于 2NF。

原因: (1) 满足 1NF——所有属性域均含原子值; (2) 不满足 2NF——存在非主属性 C 对候选键 BP 的部分函数依赖 ($B \rightarrow C$, 其中 $B \subset BP$)。

解析与知识点

本题考察关系规范化理论中两个核心技能: 候选键求解和范式级别判定 (1NF/2NF/3NF)。

第一问候选键求解的关键技巧是利用“不在任何函数依赖右部出现的属性必须属于所有候选键”这一引理: 扫描 F 中所有 FD 的右部, 发现 B 和 P 从未出现在右侧, 因此它们必然属于候选键。然后计算闭包 $(BP)^+$ 验证 BP 能否推导出全部属性——通过从 $B \rightarrow C$ 、 $C \rightarrow D$ 、 $BD \rightarrow A$ 、 $CA \rightarrow E$ 的推导链, 确实覆盖了全属性集 U 。最后验证最小性, 确认 BP 是唯一的候选键。第二问范式判定时, 主属性是 $\{B, P\}$, 非主属性是 $\{A, C, D, E\}$ 。发现 $B \rightarrow C$ 中 B 是候选键 BP 的真子集, 而 C 是非主属性, 这就是典型的非主属性对码的部分函数依赖, 直接违反了 2NF。

值得注意的是, $C \rightarrow D$ 虽然构成传递依赖 (违反 3NF), 但在本题中关系连 2NF 都不满足, 因此最终结论为 1NF。由于只需找到一个部分依赖即可判定不满足 2NF, 无需继续检查其他非主属性。

常见错误是将 $B \rightarrow C$ 误判为完全函数依赖 (忘记 B 是候选键 BP 的真子集), 或将“非主属性对码的部分依赖”与“非主属性对码的传递依赖”混淆。

知识点: 6.2 (函数依赖与码——候选码求解、属性闭包计算); 6.3 (范式——1NF、2NF、部分函数依赖)

参考答案

(答案由解析者提供)

Step 1: 将 F 中每个 FD 的右部化为单属性。

原 $F = \{AB \rightarrow C, C \rightarrow D, E \rightarrow A, B \rightarrow C, CA \rightarrow E, BD \rightarrow A\}$ 。

所有 FD 的右部已为单属性, 无需分解。

Step 2: 消除左部冗余属性。

对每个 FD $X \rightarrow Y$, 检查 X 中是否有属性可以去掉。

- $AB \rightarrow C$:
 - 检查 A 是否冗余: 计算 $B_F^+ = \{B, C, D\}$ (由 $B \rightarrow C$ 和 $C \rightarrow D$)。 $C \in B_F^+$, 故 A 冗余。 $AB \rightarrow C$ 可简化为 $B \rightarrow C$, 而 $B \rightarrow C$ 已存在于 F 中, 故删除 $AB \rightarrow C$ 。
- $CA \rightarrow E$:
 - 检查 C 是否冗余: 计算 $A_F^+ = \{A\}$, $E \notin A_F^+$ 。 C 不可去。
 - 检查 A 是否冗余: 计算 $C_F^+ = \{C, D\}$ (由 $C \rightarrow D$), $E \notin C_F^+$ 。 A 不可去。
- $BD \rightarrow A$:
 - 检查 B 是否冗余: 计算 $D_F^+ = \{D\}$, $A \notin D_F^+$ 。 B 不可去。
 - 检查 D 是否冗余: 计算 $B_F^+ = \{B, C, D\}$ (由 $B \rightarrow C$ 和 $C \rightarrow D$), $A \notin B_F^+$ 。 D 不可去。

经消除左部冗余后: $F = \{B \rightarrow C, C \rightarrow D, E \rightarrow A, CA \rightarrow E, BD \rightarrow A\}$ 。

Step 3: 消除冗余的函数依赖。

逐一检验每个 FD 是否可由其余 FD 推导出。

- 检验 $B \rightarrow C$: 从 F 中暂时去掉 $B \rightarrow C$ 。计算 $B_{F-\{B \rightarrow C\}}^+$:

$$F - \{B \rightarrow C\} = \{C \rightarrow D, E \rightarrow A, CA \rightarrow E, BD \rightarrow A\}$$

$B^+ = \{B\}$ (其余 FD 的左部均不匹配单一的 B)。 $C \notin B^+$ 。故 $B \rightarrow C$ 不可去掉。恢复 F 。

- 检验 $C \rightarrow D$: 从 F 中暂时去掉 $C \rightarrow D$ 。计算 $C_{F-\{C \rightarrow D\}}^+$:

$$F - \{C \rightarrow D\} = \{B \rightarrow C, E \rightarrow A, CA \rightarrow E, BD \rightarrow A\}$$

$C^+ = \{C\}$ 。 $D \notin C^+$ 。故 $C \rightarrow D$ 不可去掉。恢复 F 。

- 检验 $E \rightarrow A$: 从 F 中暂时去掉 $E \rightarrow A$ 。计算 E^+ :

$$F - \{E \rightarrow A\} = \{B \rightarrow C, C \rightarrow D, CA \rightarrow E, BD \rightarrow A\}$$

$E^+ = \{E\}$ 。 $A \notin E^+$ 。故 $E \rightarrow A$ 不可去掉。恢复 F 。

- 检验 $CA \rightarrow E$: 从 F 中暂时去掉 $CA \rightarrow E$ 。计算 $\{C, A\}_{F - \{CA \rightarrow E\}}^+$:

$$F - \{CA \rightarrow E\} = \{B \rightarrow C, C \rightarrow D, E \rightarrow A, BD \rightarrow A\}$$

闭包计算: $C \rightarrow D$ 加入 D , 得到 $\{C, A, D\}$ 。无其他 FD 可用 ($BD \rightarrow A$ 需要 B , 无 B ; $E \rightarrow A$ 需要 E , 无 E)。 $E \notin \{C, A\}^+$ 。故 $CA \rightarrow E$ 不可去掉。恢复 F 。

- 检验 $BD \rightarrow A$: 从 F 中暂时去掉 $BD \rightarrow A$ 。计算 $\{B, D\}_{F - \{BD \rightarrow A\}}^+$:

$$F - \{BD \rightarrow A\} = \{B \rightarrow C, C \rightarrow D, E \rightarrow A, CA \rightarrow E\}$$

闭包计算: $B \rightarrow C$ 加入 C , 得到 $\{B, C, D\}$ 。 $CA \rightarrow E$ 需要 A (无 A), $E \rightarrow A$ 需要 E (无 E)。 $A \notin \{B, D\}^+$ 。故 $BD \rightarrow A$ 不可去掉。

最终结果:

$$F_m = \{B \rightarrow C, C \rightarrow D, E \rightarrow A, CA \rightarrow E, BD \rightarrow A\}$$

该最小函数依赖集与原 F 等价, 且满足: (1) 所有右部为单属性; (2) 无左部冗余属性; (3) 无冗余的函数依赖。

解析与知识点

本题考察最小函数依赖集 (最小覆盖/正则覆盖) 的求解, 是规范化理论中步骤严谨的基本功训练。最小函数依赖集必须满足三个条件: 所有 FD 右部为单属性、无左部冗余属性、无冗余的函数依赖。

解题严格按照三步算法执行。第一步——右部单属性化: 检查 F 中所有 FD 的右部, 发现均已为单属性, 无需分解。第二步——消除左部冗余属性: 对每个 FD, 逐一检查其左部的每个属性是否可由剩余属性的闭包推导出右部。在 $AB \rightarrow C$ 的检查中, 去掉 A 后计算 B^+ , 发现在当前 F 下 $B^+ = \{B, C, D\}$ 已包含 C , 因此 A 是冗余的, $AB \rightarrow C$ 可简化为 $B \rightarrow C$ 。由于 $B \rightarrow C$ 已在 F 中, 因此直接删除 $AB \rightarrow C$ 。对于 $CA \rightarrow E$ 和 $BD \rightarrow A$, 分别检查后发现两侧属性都不可去, 保留。第三步——消除冗余函数依赖: 逐一将 F 中的每个 FD 暂时移除, 计算剩余 FD 下左部属性的闭包是否能推导出被移除 FD 的右部。

经过完整验证, $B \rightarrow C$ 、 $C \rightarrow D$ 、 $E \rightarrow A$ 、 $CA \rightarrow E$ 、 $BD \rightarrow A$ 均不可被其他 FD 推导出, 因此每个 FD 都是独立的、不可约简的。最终 F_m 包含 5 个函数依赖。解题中常见的错误包括: 忽略左部冗余的检查顺序 (应从多属性左部开始检查)、遗漏某条 FD 的验证、或在闭包计算时错误地使用了已被移除的 FD。

知识点: 6.2 (函数依赖——最小函数依赖集、Armstrong 公理系统、属性集闭包计算)

参考答案

(答案由解析者提供)

使用 3NF 合成算法 (Synthesis Algorithm), 基于已求得的最小函数依赖集 F_m 进行分解。

已知: $F_m = \{B \rightarrow C, C \rightarrow D, E \rightarrow A, CA \rightarrow E, BD \rightarrow A\}$ 。候选键: BP 。

Step 1: 为 F_m 中每个函数依赖构建一个关系模式。

由 $B \rightarrow C \Rightarrow R_1(B, C)$

由 $C \rightarrow D \Rightarrow R_2(C, D)$

由 $E \rightarrow A \Rightarrow R_3(E, A)$

由 $CA \rightarrow E \Rightarrow R_4(C, A, E)$

由 $BD \rightarrow A \Rightarrow R_5(B, D, A)$

Step 2: 消除被其他模式包含的关系模式。

检查发现 $R_3(E, A) \subset R_4(C, A, E)$, 即 R_3 的属性集是 R_4 的子集。因此删除 R_3 (R_4 保留了 $E \rightarrow A$ 的依赖)。

保留的模式集: $R_1(B, C), R_2(C, D), R_4(C, A, E), R_5(B, D, A)$ 。

Step 3: 检查是否有关系模式包含原关系的候选键。

候选键 $BP = \{B, P\}$ 。逐一检查:

- $R_1(B, C)$: 含 B , 不含 P 。— 未包含。
- $R_2(C, D)$: 不含 B 也不含 P 。— 未包含。
- $R_4(C, A, E)$: 不含 B 也不含 P 。— 未包含。
- $R_5(B, D, A)$: 含 B , 不含 P 。— 未包含。

所有模式均不包含候选键 BP 。因此需要新增一个关系模式来包含候选键。

新增 $R_6(B, P)$, 其候选键为 BP 。

最终分解结果：

$R_1(\underline{B}, C)$	FD: $B \rightarrow C$
$R_2(\underline{C}, D)$	FD: $C \rightarrow D$
$R_4(\underline{CA}, E)$	FD: $CA \rightarrow E, E \rightarrow A$
$R_5(\underline{BD}, A)$	FD: $BD \rightarrow A$
$R_6(\underline{BP})$	候选键关系，无额外 FD

其中下划线标示主键。共计 5 个关系模式。

验证：

- 无损连接性： $R_6(B, P)$ 包含了原关系的候选键 BP ，根据 3NF 合成算法定理，包含候选键的关系模式保证了分解的无损连接性。
- 依赖保持性： F_m 中每个函数依赖在分解后至少有一个关系模式完整保留：
 - $B \rightarrow C$ 保留在 R_1
 - $C \rightarrow D$ 保留在 R_2
 - $E \rightarrow A$ 保留在 R_4 (E 和 A 均在 R_4 中)
 - $CA \rightarrow E$ 保留在 R_4
 - $BD \rightarrow A$ 保留在 R_5
- 3NF 条件：每个关系模式的主键完全函数决定所有非主属性，且不存在传递依赖，均满足 3NF 要求。

解析与知识点

本题考察 3NF 分解的合成算法 (Synthesis Algorithm)，是关系规范化设计的核心内容，也是期末考试中的高频考点和难点。

3NF 合成算法的标准步骤为：第一步，基于已求得的最小函数依赖集 F_m ，为每个函数依赖构造一个关系模式，属性集为该 FD 的左部和右部的并集。第二步，检查并合并被包含的关系模式——本题中 $R_3(E, A)$ 被 $R_4(C, A, E)$ 包含，因此删除 R_3 。第三步（最关键的一步），检查所有保留的关系模式中是否有至少一个包含了原关系的候选键 $BP = \{B, P\}$ 。本题中 P 属性在原函数依赖集中从未出现在任何 FD 的左部或右部，但属于候选键。因此必须新增 $R_6(B, P)$ 来包含候选键，同时确保 P 属性不丢失。

这是保证分解满足无损连接性的必要条件——如果没有任何一个模式包含候选键，则分解后的关系模式自然连接时可能产生虚假元组，导致信息丢失。最终得到 5 个关系模式，经逐一核查：所有 FD 在某个关系模式中被保持（依赖保持性）、存

在一个模式包含候选键（无损连接性）、每个模式的主键完全决定所有非主属性（3NF 条件），三方面全部验证通过。

常见的错误是忘记检查候选键包含性而遗漏新增模式，或因未做模式合并导致分解结果中存在冗余模式。这是 3NF 合成算法中容易被忽略的关键步骤。

知识点：6.3（范式——3NF 分解、合成算法）；6.4（模式分解——无损连接性、依赖保持性）

四、综合应用题（共 20 分）

1. E-R 模型

参考答案

（答案由解析者提供）

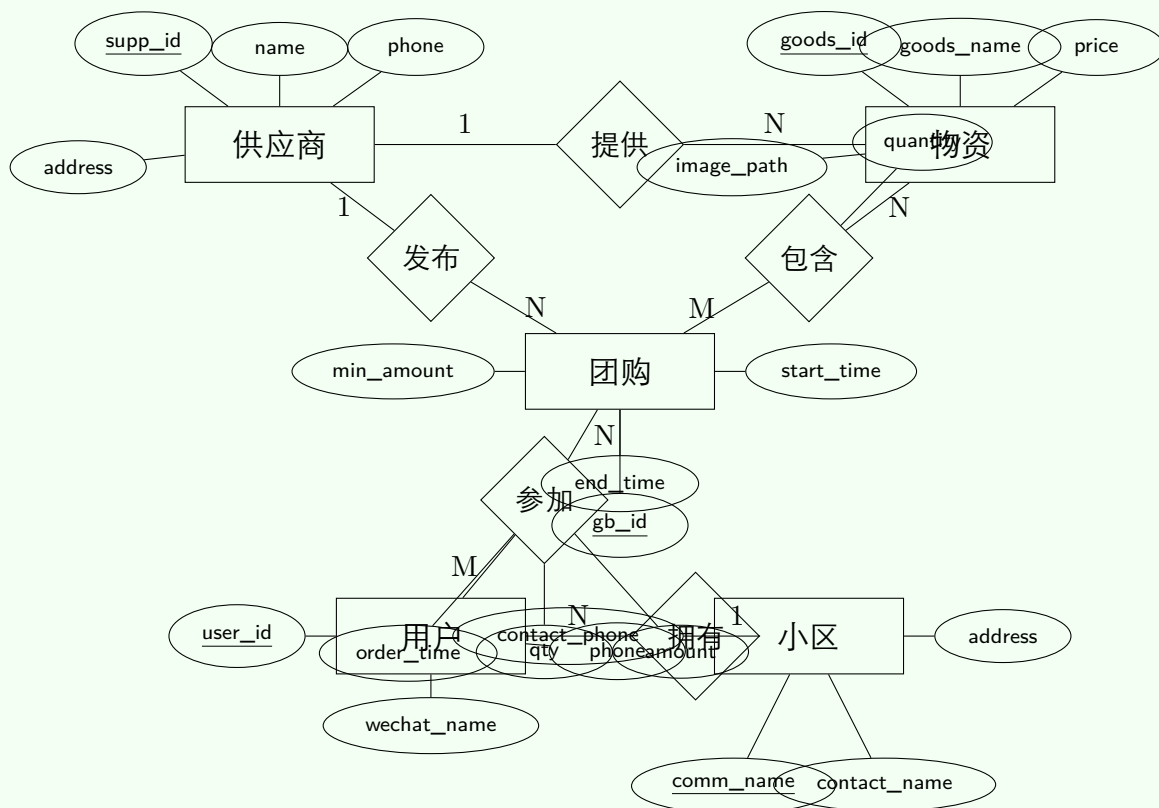
实体及其属性：

- 供应商 (**Supplier**): supp_id、name、phone、address。
- 物资 (**Goods**): goods_id、goods_name、price、image_path。
- 小区 (**Community**): comm_name、contact_name、contact_phone、address。
- 团购 (**GroupBuy**): gb_id、min_amount、start_time、end_time。
- 用户 (**User**): user_id、wechat_name、phone。

联系及其基数：

- 供应商 —[提供 Provide]— 物资: $1:N$ （一名供应商提供多种物资，一种物资由一个供应商提供）。
- 供应商 —[发布 Publish]— 团购: $1:N$ （一名供应商发布多次团购，每次团购由一名供应商发布）。
- 团购 —[包含 Include]— 物资: $M:N$ （一次团购包含多种物资，一种物资可被多次团购包含），联系属性：数量 (quantity)。
- 小区 —[拥有 Own]— 用户: $1:N$ （一个小区拥有多个用户，一个用户属于一个小区）。
- 用户 —[参加 Participate]— 团购: $M:N$ （一个用户可参加多次团购，一次团购可有多个用户参加），联系属性：下单时间 (order_time)、购买数量 (qty)、金额 (amount)。

E-R 图 (TiKZ 绘制):



图例说明：矩形——实体；菱形——联系；椭圆——属性；带下划线的椭圆——主属性（键属性）。联系上的 1/M/N 表示实体参与联系的基数（1 表示至多参与一次，N/M 表示可参与多次）。联系“包含”附有联系属性 quantity；联系“参加”附有联系属性 order_time、qty、amount。

解析与知识点

本题考察 E-R 建模的实际应用能力，要求学生根据社区团购的业务描述，抽象出实体、属性和联系并绘制 E-R 图。

实体识别遵循“独立存在性”原则：供应商、物资、团购、小区和用户都有独立的标识和生命周期，因此被建模为五个实体。属性分配要关注键属性的选择——supp_id、goods_id、gb_id、user_id、comm_name 作为各实体的主标识符。联系基数的确定需要仔细阅读需求描述中的数量词：供应商与物资的“提供”联系为 1:N，因为一种物资由唯一供应商提供；供应商与团购的“发布”联系为 1:N，每次团购由一个供应商发布；团购与物资的“包含”联系为 M:N，这是最容易被误判的地方——“每次团购的物资不一样”暗示一次团购可包含多种物资，必须建模为多对多关系，并在联系上附带 quantity（数量）属性；小区与用户的“拥有”联

系为 1:N；用户与团购的”参加”联系为 M:N，联系上附带下单时间、购买数量和金额三个属性。

绘制 E-R 图时要严格遵守 Chen 氏符号规范：矩形框表示实体、菱形表示联系、椭圆表示属性、主属性以下划线标识、联系线上标注基数比。

供应商虽然描述为”某个供应商”（暗示可能只有一个），但作为 E-R 模型应将其抽象为实体类型（可容纳多个供应商实例），这是建模的基本常识。常见错误是将 M:N 联系误判为 1:N，或将”包含”联系错误地合并到某一方实体。

知识点：7.2（E-R 模型——实体、属性、联系、基数约束）；7.3（E-R 图绘制）

2. 关系模型转换

参考答案

（答案由解析者提供）

遵循题目要求（1:1、1:M 联系不产生新的关系模式），转换规则如下：

- 1:N 联系：将”1”端实体的主键作为外键加入”N”端实体对应的关系模式中，并将联系属性也加入。
- M:N 联系：必须创建独立的关系模式，包含双方实体的主键（作为外键，联合作为主键）及联系自身的属性。

转换后的关系模型（主键以下划线标示，外键以斜体标注）：

(1) 供应商表 **Supplier**：

supp_id, name, phone, address

(2) 物资表 **Goods**：

goods_id, goods_name, price, image_path, *supp_id*

外键 *supp_id* 参照 *Supplier(supp_id)*，来自 1:N 联系”提供”。

(3) 团购表 **GroupBuy**：

gb_id, min_amount, start_time, end_time, *supp_id*

外键 *supp_id* 参照 *Supplier(supp_id)*，来自 1:N 联系”发布”。

(4) 小区表 **Community**：

comm_name, contact_name, contact_phone, address

(5) 用户表 User:

user_id, wechat_name, phone, comm_name

外键 comm_name 参照 Community(comm_name), 来自 1 : N 联系”拥有”。

(6) 团购-物资包含表 GB_Goods ($M : N$ 联系”包含”产生的新关系):

gb_id, goods_id, quantity

主键为 (gb_id, goods_id)。外键 gb_id 参照 GroupBuy, goods_id 参照 Goods。

(7) 用户-团购参加表 Participation ($M : N$ 联系”参加”产生的新关系):

user_id, gb_id, order_time, qty, amount

主键为 (user_id, gb_id)。外键 user_id 参照 User, gb_id 参照 GroupBuy。

共计 7 个关系模式。其中 Goods 通过 supp_id 外键关联 Supplier, GroupBuy 通过 supp_id 外键关联 Supplier, User 通过 comm_name 外键关联 Community; GB_Goods 和 Participation 为独立关系模式, 分别对应 $M : N$ 联系。

解析与知识点

本题考察 E-R 模型向关系模型的转换规则, 是数据库逻辑设计阶段的核心实践技能。转换遵循两大基本规则: 实体转换和联系转换。

1 : N 联系 (包括 1:1) 的处理方式是”外键合并”——将”1”端实体的主键作为外键加入到”N”端实体对应的关系模式中, 联系本身的属性 (如有) 也一并加入。本题中遵循题目约束”1:1、1:M 联系不产生新的关系模式”, 因此 Goods 表中通过 supp_id 合并了”提供”联系, GroupBuy 表中通过 supp_id 合并了”发布”联系, User 表中通过 comm_name 合并了”拥有”联系。 $M : N$ 联系则必须创建独立的关系模式——因为多对多关系无法被单方面吸收, 只能通过独立的交叉引用表来表达。该独立模式的主键为双方实体主键的联合, 同时包含联系自身的属性。本题中的”包含”联系产生了 GB_Goods 表, ”参加”联系产生了 Participation 表。这一规则的根本原因是为了消除冗余: 如果为 1:N 联系单独建表, 将产生不必要的连接开销和重复存储。完整转换后共得到 7 个关系模式。在规范的数据库设计中, 所有外键都应建立参照完整性约束 (FOREIGN KEY ... REFERENCES), 以防止悬空引用。

常见错误是将 M:N 联系错误地通过外键合并到某一方（导致信息丢失），或遗漏联系属性在关系模式中的体现。

知识点：7.4（E-R 模型向关系模型的转换——实体转换、联系转换、外键处理）

3. 约束保证方式

参考答案

（答案由解析者提供）

(1) 保证用户只能在规定的时间内下单：

方式一：应用程序层校验（推荐，最为灵活）。

在用户的微信小程序端（前端或后端服务），当用户提交订单时：

- 先从 GroupBuy 表中查询该团购的 start_time 和 end_time。
- 获取当前系统时间（服务器端时间，不可依赖客户端时间）。
- 若当前时间在 [start_time, end_time] 区间内，允许订单提交并写入 Participation 表；否则返回错误提示“该团购尚未开始”或“该团购已结束”。

实现伪代码：

```
查询：SELECT start_time, end_time FROM GroupBuy WHERE gb_id = :gb_id;
判断：IF NOW() BETWEEN start_time AND end_time THEN
        INSERT INTO Participation VALUES(...);
    ELSE
        返回错误;
    END IF;
```

方式二：数据库级别——使用 CHECK 约束（需函数支持）。

在 Participation 表上添加 CHECK 约束（需 DBMS 支持 CHECK 中使用子查询或自定义函数）：

```
1 CREATE FUNCTION check_gb_time(gb_id_input INT, order_time_input
    DATETIME)
2 RETURNS BOOLEAN
3 BEGIN
4     DECLARE s_time DATETIME;
5     DECLARE e_time DATETIME;
6     SELECT start_time, end_time INTO s_time, e_time
```

```

7      FROM GroupBuy WHERE gb_id = gb_id_input;
8      RETURN (order_time_input >= s_time AND order_time_input <=
          e_time);
9  END;
10
11 ALTER TABLE Participation
12 ADD CONSTRAINT chk_order_time
13 CHECK (check_gb_time(gb_id, order_time) = TRUE);

```

方式三：数据库级别——使用触发器（BEFORE INSERT）。

创建 BEFORE INSERT 触发器，在写入 Participation 之前验证时间：

```

1 CREATE TRIGGER check_order_time
2 BEFORE INSERT ON Participation
3 REFERENCING NEW ROW AS NewOrder
4 FOR EACH ROW
5 BEGIN
6     DECLARE s_time DATETIME;
7     DECLARE e_time DATETIME;
8     SELECT start_time, end_time INTO s_time, e_time
9     FROM GroupBuy WHERE gb_id = NewOrder.gb_id;
10    IF (NewOrder.order_time < s_time OR NewOrder.order_time >
        e_time) THEN
11        SIGNAL SQLSTATE '45000'
12        SET MESSAGE_TEXT = '不在团购时间范围内，无法下单';
13    END IF;
14 END;

```

（SIGNAL 语句抛出异常以阻止非法插入。）

(2) 保证团购金额没达到指定额度本次团购失败：

该约束的核心语义是：在团购截止时间之后，若所有参与订单的总金额未达到 GroupBuy.min_amount，则判定该次团购失败。具体实现方式：

方式一：应用层定时任务（推荐，最实用）。

在服务器端设置定时任务（cron job / 定时调度器），扫描已到达 end_time 的团购记录：

对每个已过期团购 gb_id：

```

total = SELECT SUM(amount) FROM Participation WHERE gb_id = :gb_id;
min    = SELECT min_amount FROM GroupBuy WHERE gb_id = :gb_id;
IF total IS NULL OR total < min THEN

```

```
        UPDATE GroupBuy SET status = 'failed' WHERE gb_id = :gb_id;
        -- 可进一步执行退款等业务逻辑
    ELSE
        UPDATE GroupBuy SET status = 'success' WHERE gb_id = :gb_id;
    END IF;
```

也可向 GroupBuy 表增加一个 status 列（如'pending'/'success'/'failed'）来标记团购状态。

方式二：数据库触发器（UPDATE 触发检查）。

在 Participation 表上创建 AFTER INSERT 和 AFTER DELETE 触发器，维护团购当前累计金额，并在触发器中判断是否超时或在团购结束后检查：

```
-- 每次新增订单后
计算：total = SELECT SUM(amount) FROM Participation
                WHERE gb_id = NewOrder.gb_id;
IF 当前时间 > 该团购的 end_time AND total < min_amount THEN
    UPDATE GroupBuy SET status = 'failed' WHERE gb_id = NewOrder.gb_id;
END IF;
```

方式三：添加状态列 + 定期视图/查询。

在 GroupBuy 表中增加 status 列（VARCHAR(10)，取值'ongoing'、'success'、'failed'）。管理员可通过以下 SQL 在团购结束后进行判定：

```
1 SELECT gb_id, SUM(amount) AS total, min_amount AS required
2 FROM GroupBuy NATURAL JOIN Participation
3 WHERE end_time < NOW() AND status = 'ongoing'
4 GROUP BY gb_id, min_amount
5 HAVING SUM(amount) < min_amount;
```

将上述查询结果中对应的 GroupBuy 记录的 status 更新为'failed'。

综合建议：在实际社区团购系统中，通常采用“应用层校验 + 数据库约束”的混合策略。时间限制（约束 1）适合在应用层做实时校验（用户体验最好，错误信息可直接反馈给用户）；金额门槛（约束 2）适合使用定时任务批量检查（团购失败需要触发后续退款等业务流程，应用层控制更灵活）。

解析与知识点

本题考察数据库完整性的实际工程应用，要求将业务规则（时间窗口约束和金额门槛约束）映射为数据库约束与应用层逻辑相结合的实现方案，是连接数据库理论与系统开发的桥梁题。

第一小问的“时间窗口约束”（用户只能在团购的 `start_time` 至 `end_time` 区间内下单）属于典型的用户定义完整性。数据库层面有三种实现方式：应用层校验（最推荐，在提交订单前查询 `GroupBuy` 表获取时间范围并与系统时间比较）、`CHECK` 约束（需要自定义函数支持）、`BEFORE INSERT` 触发器（在写入前查询并验证，如不满足则通过 `SIGNAL` 抛出异常）。三种方式各有优劣：应用层校验实现最灵活、用户体验最好；触发器提供了数据库级别的最后防线；`CHECK` 约束理论最优雅但受限于 DBMS 的实现。

第二小问的“金额门槛约束”更为复杂——它涉及跨行的聚合计算（`SUM`），且判断时机在团购截止之后，而非实时。数据库的 `CHECK` 约束完全不适用于此场景（`CHECK` 限制为行级表达式且不允许聚合子查询），因此核心实现必然依赖应用逻辑。最实用的方案是在服务器端部署定时任务（`cron job`），周期性扫描已到达 `end_time` 的团购，计算 `Participation` 表中所有记录的 `SUM(amount)`，与 `GroupBuy.min_amount` 比较后更新团购状态。

实际系统中通常采用“应用层校验为主 + 数据库触发器为辅”的双层防御策略。这种“批处理式”完整性检查是处理聚合型业务规则的通用模式，对比“实时触发器式”检查减少了不必要的数据库开销。

并非所有业务规则都适合用数据库约束实现——简单、实时、单行级规则适合 `CHECK`/触发器，复杂、批量、聚合级规则更适合应用层逻辑。考生容易误将所有约束都用数据库实现，忘了考虑聚合计算和跨行操作的限制。

知识点：5.1（实体完整性）；5.2（参照完整性）；5.3（用户定义完整性——`CHECK` 约束、触发器）；8.3（触发器——`BEFORE/AFTER INSERT`）

8 2023–2024 学年《数据库系统》期末试卷

8.1 A 卷

注意

本卷为 2023–2024 学年数据库系统期末试卷 A 卷。原卷无官方答案，所有答案均由解析者提供，标注（答案由解析者提供）。B 卷中大部分大题与 A 卷相同，仅单项选择题部分选项有所调整（主要在第 2、4、6、8 题），多项选择题及第三至第八大题与 A 卷完全一致。

8.2 一、单项选择题（每题 2 分，共 20 分）

参考答案

D（答案由解析者提供）

解析与知识点

数据库系统的三级模式结构（外模式、概念模式、内模式）与外部建模工具（如 UML 类图）的区分。

数据库系统的标准三级模式包括外模式（用户视图层）、概念模式（全局逻辑结构层）和内模式（物理存储层）。选项 A 对应内模式，选项 B 对应模式，选项 C 对应外模式，这三者均属于数据库的三级模式结构。而选项 D 的「类图」是 UML 中面向对象分析与设计的核心概念，属于软件工程领域的建模工具，与关系数据库的三级模式体系结构完全无关。因此正确答案为 D。

三级模式架构通过两级映像（外模式/模式映像和模式/内模式映像）保证了数据独立性。外模式是用户看到的局部数据视图，概念模式是数据库的全局逻辑结构（一个数据库只有一个），内模式描述数据的物理存储方式。区分数据库系统自身的三层抽象架构与外部建模工具的边界是本题的关键。

常见错误是将 UML 类图与 E-R 图混淆，认为类图也是一种数据库模式表示——实际上 E-R 图属于数据库概念设计的工具，而类图是面向对象建模工具，两者属于不同领域。

知识点：1.2 数据库系统的三级模式结构

参考答案

C（答案由解析者提供）

解析与知识点

数据独立性的实现机制——两级映像（模式/内模式映像、外模式/模式映像）各自的作用范围与适用场景。

模式/内模式映像描述了数据的全局逻辑结构如何映射到物理存储结构，当物理存储变化时只需修改该映像，提供物理独立性——选项 A 正确。外模式/模式映像定义了外模式与概念模式的对应关系，当概念模式变更时只需修改该映像，提供逻辑独立性——选项 B 正确。选项 C 声称所有查询都必须经过两级映像变换，这个说法过于绝对，是错误的：当用户直接查询基本表时，只需经过模式/内模式一级变换；只有通过视图查询时才需要两级变换。选项 D 正确描述了直接基于模式查询的场景。

两级映像各自负责不同的独立性类型：模式/内模式映像 → 物理独立性（应对存储变化），外模式/模式映像 → 逻辑独立性（应对逻辑结构变化）。查询经过的映像级数取决于查询是基于基本表还是基于视图。

常见错误是认为所有查询都必须经过两级映像变换——实际上直接基于基本表的查询只需一级变换（模式/内模式），只有基于视图的查询才需两级。

知识点：1.4 数据库系统的数据独立性

参考答案

A（答案由解析者提供）

解析与知识点

关系代数中的对称差（Symmetric Difference）操作——定义、等价表达式及其在数据库查询中的实现。

题目语义是查询「选修了 S_1 但未选修 S_2 ，或选修了 S_2 但未选修 S_1 的学生」，即 $(R_1 \cup R_2) - (R_1 \cap R_2)$ ，这正是对称差的定义。对称差标准定义： $R_1 \oplus R_2 = (R_1 - R_2) \cup (R_2 - R_1)$ 。逐项分析：选项 A 先合并所有学生再排除同时选修两门课的，正是对称差，正确。选项 B 等价于 $R_1 \cup R_2$ （包含了同时选修两门课的学生），不符合题意。选项 C 经等价变换后也等于 $R_1 \cup R_2$ 。选项 D 等价于 R_2 。

对称差对应「互斥或」（XOR）语义——返回恰好属于其中一个集合但不属于两个集合的元素。在实际 SQL 中可以通过 FULL OUTER JOIN ... WHERE ... IS NULL 或组合 UNION + EXCEPT 来实现。本题的陷阱在于选项 B 和 C 形式不同但经过集合运算等价变换后都等于并集。

常见错误是未识别出选项 B 和 C 的等价性——虽然表达式形式不同，但经过等价变换后实际语义相同，需要仔细推导演算而非仅凭直觉判断。知识点：2.4 关系代数——集合运算

参考答案

A（答案由解析者提供）

解析与知识点

关系数据库的基本性质——关系模式的组织方式、关系作为元组集合的数学性质、关系名称的唯一性约束。

以「错误叙述」形式反向考察。选项 A 称「一个应用只能建立一个关系数据库模式」——这是错误的，一个应用完全可以建立多个关系数据库模式来组织不同业务领域的的数据。选项 B 称「模式应该经常变化」虽也不正确，但不如 A 明显错

误。选项 C 称「关系是元组的集合，无固定顺序」——这是关系模型的基本数学性质，正确。选项 D 称「同一数据库内关系名称必须唯一」——这是数据库系统的基本约束，正确。因此 A 为本题答案。

关系是域的笛卡尔积的子集，本质上是元组的集合 (set)，集合中的元素是无序的。B 卷中此题改为问「正确的是」，且选项 A 被修改为「可以建立多个关系数据库模式」，此时 A 变为正确选项。这提醒学生：对于关系的基本性质，要从数学集合论的角度理解。

常见错误是混淆「关系模式的唯一性」与「数据库的唯一性」——一个应用可以有多个数据库模式，同一数据库内关系名称必须唯一，但不同数据库中可以有同名的关系。

知识点：2.1 关系数据结构及形式化定义

参考答案

D（答案由解析者提供）

解析与知识点

候选关键字 (Candidate Key) 与主关键字 (Primary Key) 的概念区别，以及全码 (All-Key) 关系的定义。

逐项分析：选项 A 称必须存在至少一个候选关键字——从数学角度正确（关系是元组集合，需唯一标识），但工程上 DBMS 允许无主键表。选项 B 称最多只能有一个候选关键字——错误，一个关系可有多个候选关键字（如学号和身份证号）。选项 C 称全码关系的候选关键字包含部分属性——错误，全码是全部属性组合才构成候选关键字。选项 D 称可从多个候选关键字中选择一个作为主关键字——完全正确，主键本质上是从小于候选关键字中人工选定，其余为备用码 (Alternate Key)。答案为 D。

实体完整性规则要求主键的任何分量都不能取空值，而主键来自于候选关键字的选择。全码关系是指不存在任何真子集能唯一标识元组的关系。

常见错误有二：一是认为一个关系只能有一个候选关键字（实际可以有多个）；二是混淆全码与普通候选码——全码要求全部属性联合才能唯一标识。

知识点：2.2 关系模型的完整性约束——实体完整性、候选码

参考答案

A（答案由解析者提供）

解析与知识点

候选关键字的求解——属性闭包算法，以及双向依赖关系 ($X \leftrightarrow W$) 对候选码数量的影响。

属性全集 $U = \{X, Y, Z, W\}$, $F = \{X \rightarrow Y, X \rightarrow Z, X \rightarrow W, W \rightarrow X\}$ 。计算 X^+ ：由 $X \rightarrow Y, X \rightarrow Z, X \rightarrow W$ 得 $X^+ = \{X, Y, Z, W\} = U$, X 是候选关键字。计算 W^+ ：由 $W \rightarrow X$ 再经 X 的推导链得 $W^+ = \{W, X, Y, Z\} = U$, W 也是候选关键字。 $Y^+ = \{Y\}$, $Z^+ = \{Z\}$, 均不能推出全部属性。答案为 A (两个候选关键字)。

本题关键点在于 X 和 W 之间存在双向依赖 ($X \rightarrow W$ 且 $W \rightarrow X$)，使它们互相等价，都能各自推导出全部属性。B 卷中移除了 $X \rightarrow W$ ，此时候选关键字只有 W 一个。

常见错误是忘记检查所有属性的闭包，或在计算闭包时遗漏间接传递的依赖—— $W \rightarrow X \rightarrow Y, Z$ 是一条完整的推导链。

知识点：6.2 候选码的求解——属性闭包算法

参考答案

D (答案由解析者提供)

解析与知识点

BCNF (Boyce-Codd 范式) 的判定方法——检查所有函数依赖的决定因素是否均为超码。

从上一题已知候选关键字为 X 和 W 。检查 $F = \{X \rightarrow Y, X \rightarrow Z, X \rightarrow W, W \rightarrow X\}$ 中每个 FD 的决定因素： $X \rightarrow Y$ 中 X 是候选关键字； $X \rightarrow Z$ 中 X 是候选关键字； $X \rightarrow W$ 中 X 是候选关键字； $W \rightarrow X$ 中 W 也是候选关键字。所有非平凡 FD 的决定因素均包含候选码，满足 BCNF 的严格定义。

BCNF 的判定快捷方法：检查所有 FD 的左部是否都是超码。本题中所有 FD 的左部 (X 和 W) 都是候选关键字，自然满足 BCNF。规范化层级： $1NF < 2NF < 3NF < BCNF < 4NF < 5NF$ 。BCNF 比 3NF 更严格，消除了「主属性对候选码的传递依赖」问题。但并非所有关系都能无损地分解为 BCNF 且同时保持函数依赖。

常见错误是在判定 BCNF 时仅检查部分 FD 的左部——必须检查所有非平凡函数依赖的决定因素是否都是超码。

知识点：6.3 范式——BCNF 的定义与判定

参考答案

D（答案由解析者提供）

解析与知识点

事务的 ACID 特性与并发控制策略的分类——悲观并发控制（封锁）与乐观并发控制的哲学差异。

逐项分析：选项 A 称事务有 ACID 四个特性——正确。选项 B 称事务一旦提交便无法用 rollback 撤回——正确，commit 持久化到物理存储，rollback 只能作用于未提交事务。选项 C 称并发执行可能导致数据不一致——正确，可能产生丢失修改、不可重复读、读脏数据等问题。选项 D 称封锁机制是乐观的并发控制方法——错误，封锁是最典型的悲观并发控制（Pessimistic Concurrency Control）方法，假设冲突频繁，事务访问数据前必须先获取锁。正确答案为 D。

悲观策略「预防冲突」（如封锁），乐观策略「检测冲突」（如时间戳排序和 MVCC），事务提交时才验证。乐观策略假设冲突很少发生。

常见错误是混淆悲观与乐观并发控制的分类——封锁（悲观）是预防冲突的手段，而非检测冲突的手段。MVCC 和快照隔离属于乐观范畴。

知识点：10.1 事务的基本概念；10.3 封锁机制

参考答案

B（答案由解析者提供）

解析与知识点

数据库索引的设计原则——复合索引的最左前缀匹配原则、索引选择性、索引冗余的判断标准。

逐项分析：选项 A 正确——主键索引可通过检查主键值唯一性来维护实体完整性。选项 B 错误——根据 B+ 树的最左前缀匹配原则，复合索引 (A, B, C) 已可支持仅针对 A 列的查询，在 A 上再单独建索引是冗余的，浪费存储空间和写入维护开销。选项 C 正确——低基数属性（如性别）索引选择性差，全表扫描反而更高效。选项 D 正确——连接列上建索引可显著加速等值连接中的匹配查找。答案为 B。

索引选择性（Selectivity）= 不同键值数量 / 总行数。选择性越高，索引越有效。复合索引的最左前缀匹配原则：以 (A, B, C) 为例，索引对 A、 (A, B) 、 (A, B, C) 的查询都有效，但对仅涉及 B 或 C 的查询无效。

常见错误是不理解最左前缀匹配原则，在已有复合索引 (A, B, C) 的情况下还应再建 A 的单独索引——这是完全冗余的。另一个易错点是误以为所有列都值得建索

引，忽略了低基数属性的选择性问题。

知识点：7.5 索引技术；9.4 查询优化中索引的作用

参考答案

C（答案由解析者提供）

解析与知识点

数据库封锁中的锁粒度选择策略——锁升级（Lock Escalation）机制及其在大数据量访问场景下的应用。

当事务需要读取关系中 70% 以上元组时，逐元组加锁会产生严重的性能问题——锁管理器需要为每一行分配锁结构，百万行级别的大表逐行加锁的锁开销可能超过数据处理时间。此时最有效的策略是锁升级（Lock Escalation）：将对多个细粒度锁的请求合并为一个粗粒度锁，直接对整个关系加一把 X 锁（排他锁），一次性锁定整个表。

各锁类型分析：SIX 锁（共享意向排他锁）允许其他事务读但不能阻止其他事务修改其他元组；IX 锁（意向排他锁）只表明有意向修改部分元组，还需逐元组加锁；逐行加锁在大量行场景下开销极大；直接加表级 X 锁虽然并发度最低，但在访问大部分数据的极端场景下综合性能最优。实际 DBMS（如 Oracle、MySQL InnoDB）中锁升级是自动进行的——当单事务持有的行锁数量超过阈值时自动升级为表锁。

常见错误是忽视访问比例对锁策略选择的影响——在低比例访问时行级锁最优（高并发），但在高比例访问时表级锁最优（低开销），不存在一种永远最优的锁粒度。

知识点：10.3 封锁——多粒度封锁与锁升级策略

8.3 二、多项选择题（每题均有两个答案，每题 2 分，共 10 分）

参考答案

A、B（答案由解析者提供）

解析与知识点

SQL 中 GROUP BY 子句与 SELECT 子句之间的语法约束——非聚合表达式必须出现在 GROUP BY 中（或其超集）。

当查询包含 GROUP BY 时，SELECT 中的非聚合表达式必须出现在 GROUP BY 中。本题 SELECT 中有 A、B 和 $C \times D$ 三个非聚合表达式。选项 A 的 A、B、 $C \times D$

完全匹配，正确。选项 B 的 A, B, C, D 是更细粒度的分组，由于 $C \times D$ 在 C 和 D 的组内唯一确定，语义合法。选项 C 缺少 B 和 $C \times D$ ，选项 D 缺少 A 和 $C \times D$ ，均非法。答案为 A 和 B。

核心规则：GROUP BY 中的表达式不需要与 SELECT 严格字面一致，但必须保证 SELECT 中的每个非聚合表达式在 GROUP BY 的分组内是唯一确定的。这比死记硬背更有助于写出正确且灵活的 SQL。

常见错误是认为 GROUP BY 必须与 SELECT 中非聚合列完全一致——实际上超集合法，只要保证唯一确定性即可。

知识点：3.5 SQL——GROUP BY 与聚合查询的语法约束

参考答案

A、C（答案由解析者提供）

解析与知识点

数据库物理设计阶段的主要任务——区分物理设计与逻辑设计的边界，掌握物理设计关注的核心内容。

数据库设计分四个阶段：需求分析、概念设计（ER 建模）、逻辑设计（ER 转关系模式）、物理设计。物理设计关注数据在物理存储介质上的组织方式，核心任务包括确定存储结构、设计索引、决定数据聚集存储策略、选择分区策略、设置存储参数等。选项 A「选择存储介质」属于物理设计。选项 C「将相同部门数据聚集存储」是典型的物理设计决策。选项 B「设计 CHECK 约束」属于逻辑设计。选项 D「将 ER 图转换为关系模式」是逻辑设计的核心任务。答案为 A 和 C。

区分技巧：涉及「如何存储」的属物理设计，涉及「存储什么规则」的属逻辑设计。物理设计的去规范化和数据聚集策略可将相关数据存储于相邻物理块中以减少磁盘随机读取。

常见错误是将 CHECK 约束设计误认为是物理设计任务——完整性约束定义的是数据的逻辑规则，不涉及物理存储细节，属于逻辑设计范畴。

知识点：7.4 数据库物理设计

参考答案

A、C（答案由解析者提供）

解析与知识点

SQL 中全称量词与存在量词的逻辑转换—— $\forall x(P(x)) \equiv \neg \exists x(\neg P(x))$ ，以及 NOT IN / NOT EXISTS 在实现全称量词查询中的应用。

题目语义是「查询所有成绩均大于等于 60 分的学生姓名」，即全称量词查询。选项 A 先找出有不及格成绩的学号，再用 NOT IN 排除，等价于「不存在不及格成绩」，正确表达全称量词。选项 C 用 EXISTS 仅要求至少有一门 ≥ 60 ，这是存在量词而非全称量词。选项 B 的 NATURAL JOIN 语法不规范，选项 D 中 SC 表没有 sname 属性。答案为 A 和 C（题目可能将二者作为正确答案对）。

SQL 编程关键技巧：全称量词语义（「所有... 都...」）应通过 NOT EXISTS（或 NOT IN）配合否定条件来实现——先找出不满足条件的，再从全集中排除。直接在存在量词上添加正面条件只能表达存在量词（「至少有一个...」），而非全称量词。常见错误是用 EXISTS + 正面条件来试图表达全称量词语义——如本题选项 C 的错误示范。正确做法是 NOT EXISTS / NOT IN + 否定条件。

知识点：3.4 SQL——嵌套子查询（IN / NOT IN）

参考答案

A、D（答案由解析者提供）

解析与知识点

数据库恢复技术中检查点（Checkpoint）机制下的缓冲区、磁盘与日志三者的时序关系。

事务 A 将 A 改为 25，事务 B 覆盖改为 20，检查点将脏页刷新。检查点时刻缓冲区值为 20（B 的最后修改），刷新后磁盘值 13→20。选项 A「13→25」不可能——检查点时刻缓冲区为 20 而非 25。选项 B「25→20」可能——若 A 先强制刷盘写入了 25，检查点刷新 B 的修改后磁盘变为 20。选项 C「13→20」可能——原始值 13，检查点刷新后变为 20。选项 D「20→25」不可能——REDO 最多恢复到 20，不会回到 25。答案为 A 和 D。

缓冲区中的值是最近一次修改的结果，磁盘的值是最近一次刷新时的结果，刷新时机由缓冲区管理策略决定。检查点确保该点之前的所有已提交事务的修改都被写入磁盘，缩短恢复时的日志扫描范围。

常见错误是忽略 Force Write 的可能性——事务提交后可能立即将修改写盘（在检查点之前），因此 B 选项「25→20」虽反直觉但在技术上可能。

知识点：11.3 具有检查点的恢复技术；11.2 日志文件与故障恢复

参考答案

C、D（答案由解析者提供）

解析与知识点

并发调度的可串行化分析——通过计算两种串行顺序的结果值，判断哪些中间值在可串行化调度下不可能作为最终结果出现。

T1 (转账) : ID101 A -50, ID102 A +50, 总金额守恒 $150 + 200 = 350$ 。T2 (加息) : 所有 A $\times 1.1$ 。串行 T1→T2: ID101 = $(150 - 50) \times 1.1 = 110$, ID102 = $(200 + 50) \times 1.1 = 275$ 。串行 T2→T1: ID101 = $150 \times 1.1 - 50 = 115$, ID102 = $200 \times 1.1 + 50 = 270$ 。在可串行化调度下, ID101 只能是 110 或 115, ID102 只能是 275 或 270。选项 C 中 ID102=220 (T2 的中间值)、选项 D 中 ID102=250 (T1 的中间值) 不可能作为最终结果。答案为 C 和 D。

可串行化调度的本质是最终结果必须等价于某一种串行执行顺序。本题中总金额守恒提供了一种验证方法: 选项 A 总金额 = $110 + 275 = 385$ (350×1.1 , 正确), 选项 B 总金额 = $115 + 270 = 385$ (正确), 选项 C 总金额 = 330 违反守恒。因此”不属于”正确答案后 R 中可能的值是 C 和 D。

常见错误是将中间状态值 (如 T1 执行完但 T2 未执行时的值) 误认为是可串行化调度的可能结果——可串行化要求最终值等价于完整串行执行的最终值。

知识点: 10.4 并发调度的可串行化; 10.5 两段锁协议

8.4 三、规范化设计 (15 分, 每题 5 分)

参考答案

(答案由解析者提供)

(1) 函数依赖:

city $\rightarrow$ population (FD1)

(city, r-name) $\rightarrow$ street, bldg-num (FD2)

(street, bldg-num) $\rightarrow$ r-name, city (FD3)

推导说明:

- FD1: 城市的人口数量仅由城市决定。城市的名字具有全局唯一性 (题目已明示城市名称均不相同), 故 city 唯一决定其 population。
- FD2: 同城内餐馆名不重复, 意味着 (city, r-name) 能唯一标识一条餐馆记录, 故可决定其街道和门牌号。
- FD3: 一个门牌号码中只有一家餐馆, 且 street + bldg-num 共同确定唯一地址, 故能决定该餐馆的名称和所在城市。

解析与知识点

从自然语言的语义约束中提取形式化函数依赖的能力——这是规范化分析的起点和关键步骤。

从三条语义约束提取函数依赖：FD1: $\text{city} \rightarrow \text{population}$ （城市名称全局唯一，city 唯一决定 population）；FD2: $(\text{city}, \text{r-name}) \rightarrow \text{street}, \text{bldg-num}$ （同城内餐馆名不重复，城市名与餐馆名组合唯一标识餐馆记录）；FD3: $(\text{street}, \text{bldg-num}) \rightarrow \text{r-name}, \text{city}$ （一个门牌号码中只有一家餐馆，街道 + 门牌号组合唯一确定餐馆）。FD2 和 FD3 的双向唯一性表明 $(\text{city}, \text{r-name})$ 和 $(\text{street}, \text{bldg-num})$ 互为候选码。推导利用了「命名唯一性」到函数依赖的映射原则：如果属性 A 的每个值唯一对应于属性 B 的一个值，则 $A \rightarrow B$ 成立。复合主键的情形中需组合多个属性才能唯一标识。在现实数据库设计中，从业务规则到函数依赖的映射是最关键的步骤——如果函数依赖分析错误，后续所有规范化结论都将无效。

常见错误是遗漏隐含的函数依赖——如仅看到「同城内餐馆名不重复」推导出 FD2，但忽略了「一个门牌号码中只有一家餐馆」也能推导出反向的唯一决定关系 FD3。

知识点：6.1 函数依赖——从语义约束推导函数依赖

参考答案

（答案由解析者提供）

(2) 范式判断：1NF。

推导步骤：

1. 确定候选关键字。由 FD2 知 $(\text{city}, \text{r-name})^+ = \{\text{city}, \text{r-name}, \text{street}, \text{bldg-num}, \text{population}\} = U$ 。由 FD3 知 $(\text{street}, \text{bldg-num})^+ = \{\text{street}, \text{bldg-num}, \text{r-name}, \text{city}, \text{population}\} = U$ 。两者均不可约简，故候选关键字为 $\{\text{city}, \text{r-name}\}$ 和 $\{\text{street}, \text{bldg-num}\}$ 。
2. 检查部分依赖：FD1 为 $\text{city} \rightarrow \text{population}$ ，其中 city 是候选关键字 $\{\text{city}, \text{r-name}\}$ 的真子集，而 population 为非主属性。存在非主属性对候选码的部分函数依赖。
3. 因此该关系最高为 1NF（不满足 2NF 条件）。

解析与知识点

范式判定——候选关键字求解与 2NF 判定（检查非主属性对候选码的部分函数依赖）。

候选关键字的求解：由 FD2 知 (city, r-name) 的闭包 = U，由 FD3 知 (street, bldg-num) 的闭包 = U，两者各自无法约简，因此有两个候选关键字。主属性：city, r-name, street, bldg-num，非主属性：population。检查 FD1 $\text{city} \rightarrow \text{population}$ ：city 是候选码 (city, r-name) 的真子集，population 是非主属性——这是典型的非主属性对码的部分函数依赖，违反 2NF。因此最高为 1NF。

2NF 要求对所有候选码都不存在部分依赖——只要对任意一个候选码存在部分依赖，关系就不满足 2NF。现实数据库设计中常见两个等价但不同的候选码，如学号和身份证号。本题的关键洞察：虽然对于候选码 (street, bldg-num)，population 不部分依赖于它的任何真子集，但对于候选码 (city, r-name)，city 确实是其真子集。

常见错误有二：一是只检查了主键而忽略了其他候选关键字；二是在判断部分依赖时没有认识到 city 确实是候选码 (city, r-name) 的真子集。

知识点：6.2 范式——1NF/2NF 的定义与判定；候选码的求解

参考答案

(答案由解析者提供)

(3) 3NF 分解：

Step 1 ——求最小函数依赖集 F_m ：

初始 $F = \{\text{city} \rightarrow \text{population}, (\text{city, r-name}) \rightarrow \text{street, bldg-num}, (\text{street, bldg-num}) \rightarrow \text{r-name, city}\}$ 。

右部单属性化：

$$F_1 = \left\{ \begin{array}{l} \text{city} \rightarrow \text{population}, \\ (\text{city, r-name}) \rightarrow \text{street}, \\ (\text{city, r-name}) \rightarrow \text{bldg-num}, \\ (\text{street, bldg-num}) \rightarrow \text{r-name}, \\ (\text{street, bldg-num}) \rightarrow \text{city} \end{array} \right\}$$

去掉左部冗余属性：逐一检查各 FD 的左侧属性是否可以去掉。

- $(\text{city, r-name}) \rightarrow \text{street}$ ：去掉 r-name 后 $\text{city}_{F_1}^+ = \{\text{city, population}\}$ 不含 street；去掉 city 后 $\text{r-name}^+ = \{\text{r-name}\}$ 不含 street。不可去。
- $(\text{city, r-name}) \rightarrow \text{bldg-num}$ ：同理，两侧均不可去。
- $(\text{street, bldg-num}) \rightarrow \text{r-name/city}$ ：同理不可去。

去掉冗余函数依赖：逐一尝试从 F_1 中删除某个 FD，计算剩余 FD 下左部属性闭包是否仍包含被删 FD 的右部。

- 尝试删除 $\text{city} \rightarrow \text{population}$: $\text{city}^+ = \{\text{city}\}$, 不含 population 。不可删。
- $(\text{city}, \text{r-name}) \rightarrow \text{street}$: $(\text{city}, \text{r-name})^+ = \{\text{city}, \text{r-name}, \text{population}, \text{bldg-num}\}$ (由其它 FD), 不含 street 。不可删。
- $(\text{city}, \text{r-name}) \rightarrow \text{bldg-num}$: $(\text{city}, \text{r-name})^+ = \{\text{city}, \text{r-name}, \text{population}, \text{street}\}$, 不含 bldg-num 。不可删。
- $(\text{street}, \text{bldg-num}) \rightarrow \text{r-name}$: $(\text{street}, \text{bldg-num})_{F_1 \setminus \{\dots \rightarrow \text{r-name}\}}^+ = \{\text{street}, \text{bldg-num}, \text{city}, \text{population}\}$, 不含 r-name 。不可删。
- $(\text{street}, \text{bldg-num}) \rightarrow \text{city}$: $(\text{street}, \text{bldg-num})_{F_1 \setminus \{\dots \rightarrow \text{city}\}}^+ = \{\text{street}, \text{bldg-num}, \text{r-name}\}$, 不含 city 。不可删。

故 $F_m = F_1$, 即 F 本身已为最小函数依赖集。

Step 2 ——按 F_m 中每个 FD 构造关系模式并合并:

$$R_1(\underline{\text{city}}, \text{population})$$

$$R_{23}(\underline{\text{city}}, \text{r-name}, \text{street}, \text{bldg-num})$$

R_{23} 中 $\{\text{street}, \text{bldg-num}\}$ 为另一候选关键字。

Step 3 ——验证无损连接性:

$R_1 \cap R_{23} = \{\text{city}\}$, 且 $\text{city} \rightarrow \text{population} = R_1 - R_{23}$, 满足无损连接判定条件。

验证保持函数依赖:

- $\text{city} \rightarrow \text{population}$ 在 R_1 中保持。
- $(\text{city}, \text{r-name}) \rightarrow \text{street}, \text{bldg-num}$ 在 R_{23} 中保持。
- $(\text{street}, \text{bldg-num}) \rightarrow \text{r-name}, \text{city}$ 在 R_{23} 中保持。

所有函数依赖均被保持。该分解满足无损连接与函数依赖保持的 3NF 分解要求。

解析与知识点

3NF 分解算法的完整全过程——最小函数依赖集的求解、按左部相同原则构造关系模式、以及分解质量验证 (无损连接性 + 依赖保持性)。

第一步——求最小函数依赖集 F_m : 将复合右部拆分为单属性, 检查左部冗余和 FD 冗余, 经完整验证后所有五个单属性右部 FD 均不可约简。第二步——按 F_m 中每个 FD 构造关系模式: 将左部相同的 FD 合并, $\text{city} \rightarrow \text{population}$ 产生 $R_1(\text{city}, \text{population})$, $\text{city}, \text{r-name} \rightarrow \text{street}$ 和 $\text{city}, \text{r-name} \rightarrow \text{bldg-num}$ 因左部相同合并产

生 $R_{23}(\text{city}, \text{r-name}, \text{street}, \text{bldg-num})$ 。第三步——验证： $R_1 \cap R_{23} = \{\text{city}\}$ 且 $\text{city} \rightarrow \text{population}$ 满足无损条件，所有原始 FD 均在分解后的子模式中保持。

3NF 合成算法是一个教科书级别的标准流程，关键认知是合并相同决定因素的模式可以显著减少子模式数量。 $\text{street}, \text{bldg-num} \rightarrow \text{r-name}$ 和 $\text{street}, \text{bldg-num} \rightarrow \text{city}$ 的闭包覆盖全部属性，因此 R_{23} 中有两个候选关键字。

常见错误是在第三步只做无损性验证而忘记依赖保持性验证——3NF 分解算法的目标是同时满足两个要求。另一个易错点是合并子模式时未检查左部是否确实相同。

知识点：6.4 模式分解——3NF 分解算法；最小函数依赖集；无损连接判定

8.5 四、关系模式规范化（15 分，每题 5 分）

参考答案

（答案由解析者提供）

(1) 计算 $(IC)^+$ （属性闭包）：

1. 初始： $(IC)^+ = \{I, C\}$
2. 由 $C \rightarrow D$ ：加入 D ， $(IC)^+ = \{I, C, D\}$
3. 由 $I \rightarrow B$ ：加入 B ， $(IC)^+ = \{I, C, D, B\}$
4. 由 $B \rightarrow O$ ：加入 O ， $(IC)^+ = \{I, C, D, B, O\}$
5. 由 $IC \rightarrow A$ ：加入 A ， $(IC)^+ = \{I, C, D, B, O, A\} = U$

结果： $(IC)^+ = U$ ，故 IC 是 R 的超码(superkey)。进一步验证： $I^+ = \{I, B, O\} \neq U$ ， $C^+ = \{C, D\} \neq U$ ，故 IC 为候选码。

解析与知识点

本问考察属性闭包的计算算法，这是规范化理论中最基础也最重要的计算技能。属性闭包 (Attribute Closure) 定义为：给定属性集 X 和函数依赖集 F ， X^+ 是从 X 出发通过反复应用 F 中的函数依赖所能推导出的所有属性的集合。闭包计算的算法是一个迭代过程：从 X 自身开始，反复扫描 F 中的所有函数依赖，如果某个 FD 的左部是当前闭包的子集，则将其右部加入闭包，直到闭包不再增长为止。

本题中从 $\{I, C\}$ 出发：首先应用 $C \rightarrow D$ (C 在闭包中) 加入 D ；然后应用 $I \rightarrow B$ (I 在闭包中) 加入 B ；接着应用 $B \rightarrow O$ (B 刚加入闭包) 加入 O ；最后应用

$IC \rightarrow A$ (I 和 C 都在闭包中) 加入 A 。此时闭包达到 $\{I, C, D, B, O, A\}$, 已等于全属性集 U , 故 IC 是超码。进一步验证最小性: $I^+ = \{I, B, O\}$ (不含 C, D, A), $C^+ = \{C, D\}$ (不含 I, B, O, A), 两者的闭包都不等于 U , 因此 IC 中的任何一个属性都不可删除, IC 是最小的超码, 即候选码。

属性闭包算法的应用场景非常广泛: 除了判断候选键外, 还用于验证函数依赖是否蕴含、判断属性集的等价性以及检验分解的无损连接性。

常见错误包括: 迭代只做一轮就停止, 未检查是否有新的 FD 因新加入的属性而变得可用; 将超码与候选码混淆, 忘记验证最小性 (分别检查每个属性的闭包是否小于全属性集)。

知识点: 6.2 属性闭包的计算算法 (Armstrong 公理系统)

参考答案

(答案由解析者提供)

(2) 最小函数依赖集 F_m :

Step 1: 右部单属性化。 F 右部已均为单属性, 无需分解。

Step 2: 去掉左部冗余属性。

- $IC \rightarrow A$: 去掉 I , $C_F^+ = \{C, D\}$ 不含 A , I 不可去。去掉 C , $I_F^+ = \{I, B, O\}$ 不含 A , C 不可去。

其余 FD 左部均为单属性。

Step 3: 去掉冗余函数依赖。逐一用剩余 FD 计算闭包验证:

- 删 $C \rightarrow D$: $C^+ = \{C\}$, 不含 D 。保留。
- 删 $I \rightarrow B$: $I^+ = \{I\}$, 不含 B 。保留。
- 删 $IC \rightarrow A$: $(IC)^+ = \{I, C, B, O, D\}$, 不含 A 。保留。
- 删 $B \rightarrow O$: $B^+ = \{B\}$, 不含 O 。保留。

结果: $F_m = F = \{C \rightarrow D, I \rightarrow B, IC \rightarrow A, B \rightarrow O\}$ 。

解析与知识点

本问考察最小函数依赖集 (正则覆盖) 的求解算法。最小函数依赖集 F_m 是与原函数依赖集 F 等价的、最小的函数依赖集合, 它必须满足三个条件: 所有 FD 右部为单属性、无左部冗余属性、无冗余的函数依赖。求解过程严格按照三步法执行。第一步“右部单属性化”: 题目中 F 的所有 FD 右部已经都是单属性, 因此此步直接跳过——若遇到如 $A \rightarrow BC$ 的 FD, 需分解为 $A \rightarrow B$ 和 $A \rightarrow C$ 两条。第二

步”消除左部冗余属性”：仅针对左部包含多个属性的 FD，检查每个左部属性是否可以移除。对于 $IC \rightarrow A$ ，分别检查 I 和 C ：去掉 I 后计算 $C^+ = \{C, D\}$ ，不含 A ，故 I 必要；去掉 C 后计算 $I^+ = \{I, B, O\}$ ，不含 A ，故 C 必要。两者均不可去， $IC \rightarrow A$ 保持不变。其余 FD 左部均为单属性，不存在冗余可能性。第三步”消除冗余函数依赖”：逐一将每个 FD 从 F 中暂时移除，在剩余的 FD 集合下计算左部属性的闭包，检查是否仍包含被移除 FD 的右部。若包含，则该 FD 是冗余的（可由其他 FD 推导出）；否则保留。经完整验证，所有四个 FD 均不可由其他 FD 推导出， F 本身即为最小函数依赖集。这个结果说明原函数依赖集已经是”最精简”的形式，不存在任何冗余信息。

最小函数依赖集的求解是规范化理论中标准化 FD 集合的核心步骤，它直接服务于后续的 3NF/BCNF 分解算法。三步法的执行顺序不可颠倒：必须先右部单属性化，再消除左部冗余，最后才消除冗余 FD——若顺序错乱会导致结果错误。实际考试中，很多时候原 FD 集已经接近最小覆盖（如本题），此时只需完整验证三步即可确认。

典型错误包括：混淆步骤顺序（在未做左部冗余消除前就开始消除冗余 FD）；用肉眼判断冗余 FD 而不执行严格的闭包计算验证；忘记检查左部多属性 FD（如 $IC \rightarrow A$ ）中的每个左部属性是否可独立删除。

知识点：6.3 最小函数依赖集（正则覆盖）的求解算法

参考答案

（答案由解析者提供）

(3) 无损连接判定——追踪算法 (Chase)：

初始表（行 = 分解子模式 $\rho = \{CD, IBA, IC, BO\}$ ，列 = U ）：

	C	D	I	B	A	O
CD	a_1	a_2	b_{13}	b_{14}	b_{15}	b_{16}
IBA	b_{21}	b_{22}	a_3	a_4	a_5	b_{26}
IC	a_1	b_{32}	a_3	b_{34}	b_{35}	b_{36}
BO	b_{41}	b_{42}	b_{43}	a_4	b_{45}	a_6

第 1 轮：

- $C \rightarrow D$ ： CD 行 $C = a_1$ ， IC 行 $C = a_1$ 。令 IC 的 $b_{32} \rightarrow a_2$ 。
- $I \rightarrow B$ ： IBA 行 $I = a_3$ ， $B = a_4$ ； IC 行 $I = a_3$ ， $B = b_{34}$ 。令 $b_{34} \rightarrow a_4$ 。
- $IC \rightarrow A$ ：需 $(I, C) = (a_3, a_1)$ 的两行——仅 IC 行满足，无其它行匹配。跳过。

- $B \rightarrow O$: IBA 行 ($B = a_4, O = b_{26}$), IC 行 ($B = a_4, O = b_{36}$), BO 行 ($B = a_4, O = a_6$)。令 $b_{26} \rightarrow a_6, b_{36} \rightarrow a_6$ 。

更新后:

	C	D	I	B	A	O
CD	a_1	a_2	b_{13}	b_{14}	b_{15}	b_{16}
IBA	b_{21}	b_{22}	a_3	a_4	a_5	a_6
IC	a_1	a_2	a_3	a_4	b_{35}	a_6
BO	b_{41}	b_{42}	b_{43}	a_4	b_{45}	a_6

第 2 轮: 重新扫描所有 FD, 无更多变化可应用 ($IC \rightarrow A$ 仍无匹配行, 其余 FD 均已一致)。

判定: 检查是否有一行为全 a :

- CD 行缺失 I, B, A, O ; IBA 行缺失 C, D ; IC 行缺失 A (b_{35}); BO 行缺失 C, D, I, A 。

无全 a 行, 故 ρ 不是无损连接分解。

解析与知识点

本问考察使用追踪算法 (Chase Algorithm) 判定关系模式分解是否满足无损连接性。Chase 算法是处理模式分解无损连接性判定的经典形式化方法, 其核心思想是模拟自然连接的过程: 创建一个初始表格, 每行代表分解后的一个子模式, 每列代表原关系的一个属性, 用 a_i 标记该行包含此属性的已知值, 用 b_{ij} 标记未知值。然后反复应用函数依赖集 F 来“追踪” (chase) ——如果某条 FD $X \rightarrow Y$ 中左部 X 在两行或多行中取值相同, 则将这几行在右部 Y 上的符号统一为 a 值或编号最小 b 值。

追踪过程持续进行直到符号不再变化, 最后检查是否存在一行全为 a 符号——若有, 则分解是无损连接的; 若无, 则不是。本题的追踪过程中, $C \rightarrow D$ 使 IC 行的 D 从 b_{32} 变为 a_2 ; $I \rightarrow B$ 使 IC 行的 B 从 b_{34} 变为 a_4 ; $B \rightarrow O$ 使 IBA 行和 IC 行的 O 都变为 a_6 。但关键的 $IC \rightarrow A$ 始终无法应用——因为 IC 行虽然有 $C = a_1, I = a_3$, 但 IBA 行的 C 为 b_{21} (未知值), 无法与 IC 行在 C 上匹配, 因此 IC 行的 A 始终停留在 b_{35} 。

最终没有任何一行达到全 a , 分解 ρ 不是无损连接的。这意味着原关系投影到这些子模式后再自然连接, 可能产生原关系中没有的虚假元组 (伪元组), 信息会丢失。本题的关键启示是: $IC \rightarrow A$ 这个 FD 的存在对追踪过程构成了障碍——分解方案中没有哪个子模式同时包含 I 、 C 和 A 三个属性, 导致 A 的值无法通过

Chase 在行间传播。

常见错误包括：初始表格符号标注错误（行与子模式属性对应关系搞混）；Chase 过程中只做一轮就下结论，未迭代直到符号稳定；错误地在不同 b 值之间进行统一（应统一为 a 值或编号最小的 b 值，而非任意 b 值）。

知识点：6.4 模式分解——追踪算法（Chase）判定无损连接

8.6 五、SQL 综合查询（15 分，每题 3 分）

参考答案

（答案由解析者提供）

(1) 查询厂商 C 生产的产品的型号和价格：

```
1 SELECT model, price FROM Product NATURAL JOIN PC WHERE maker = '
   C'
2 UNION
3 SELECT model, price FROM Product NATURAL JOIN Laptop WHERE maker
   = 'C'
4 UNION
5 SELECT model, price FROM Product NATURAL JOIN Printer WHERE
   maker = 'C';
```

由于不同产品类型的价格存储在不同表中，需用 UNION 合并三张表的结果。

(2) 查询只生产笔记本的厂商：

```
1 SELECT maker FROM Product
2 WHERE type = 'Laptop'
3     AND maker NOT IN (
4         SELECT maker FROM Product
5         WHERE type IN ('PC', 'Printer')
6     );
```

先找出生产笔记本的厂商，再排除那些同时生产 PC 或 Printer 的厂商。

(3) 查询相同速度和内存的 PC 对，每对仅列出一次：

```
1 SELECT PC1.model AS i, PC2.model AS j
2 FROM PC PC1, PC PC2
3 WHERE PC1.speed = PC2.speed
4     AND PC1.ram = PC2.ram
5     AND PC1.model < PC2.model;
```

通过 $PC1.model < PC2.model$ 确保每对 (i, j) 仅列出一次，不会出现对称的 (j, i) 。

(4) 生产笔记本且有 3 种型号以上的厂商：

```
1 SELECT maker FROM Product
2 WHERE type = 'Laptop'
3 GROUP BY maker
4 HAVING COUNT(model) >= 3;
```

(5) PC、Laptop 和 Printer 均生产的厂商：

```
1 SELECT maker FROM Product WHERE type = 'PC'
2 INTERSECT
3 SELECT maker FROM Product WHERE type = 'Laptop'
4 INTERSECT
5 SELECT maker FROM Product WHERE type = 'Printer';
```

利用集合交运算 (INTERSECT) 找出同时出现在三种产品类型中的 maker。等价写法：

```
1 SELECT DISTINCT P1.maker FROM Product P1
2 WHERE EXISTS (SELECT 1 FROM Product P2 WHERE P2.maker = P1.maker
3               AND P2.type = 'PC')
4   AND EXISTS (SELECT 1 FROM Product P3 WHERE P3.maker = P1.maker
5               AND P3.type = 'Laptop')
6   AND EXISTS (SELECT 1 FROM Product P4 WHERE P4.maker = P1.maker
7               AND P4.type = 'Printer');
```

解析与知识点

本题是一道 SQL 综合应用题，通过五个子问题全方面考察学生的 SQL 编程能力，涉及多表连接、集合运算、分组聚合、自连接和嵌套子查询等核心技术点。

(1) 题是 UNION 操作的典型场景：三种产品类型 (PC、Laptop、Printer) 的价格存储在不同表中，且三张表的属性不完全一样，因此不能直接 UNION ALL 整张表，需要分别从每张表中选取 model 和 price 两列（列数、类型和顺序一致）然后用 UNION 合并去重。(2) 题是“排除型”查询——“只生产笔记本”等价于“生产笔记本且不生产 PC 且不生产 Printer”，用 NOT IN 子查询排除多类型厂商，语义清晰且执行高效。

(3) 题是经典的自连接配对问题——将同一张表做笛卡尔积连接并施加对称条件， $PC1.model < PC2.model$ 是关键技巧，用小于而非不等于或小于等于，确保每对 PC 只出现一次（不会出现 (A,B) 和 (B,A) 两个对称对，也不会出现 (A,A) 自己与自己配对的情况）。(4) 题是典型的 HAVING 分组过滤查询——先用 WHERE

type='Laptop' 筛选产品类型, 再按 maker 分组, 通过 HAVING COUNT(model) ≥ 3 过滤出型号数达标的厂商, 注意 WHERE 和 HAVING 的执行顺序区别。(5) 题考察集合交集操作——“三种产品均生产”即“同时存在于三个集合中”, 最简单的写法是三个子查询取 INTERSECT; 如果数据库不支持 INTERSECT, 可用多个 EXISTS 子查询等价替代, 也是实际工程中的常用写法。

常见错误包括: UNION 各分查询的列数和类型不一致导致语法错误; (3) 题中写成 PC1.model $\leq$ PC2.model 导致输出对称对自己配对; WHERE 和 HAVING 执行顺序混淆导致分组过滤条件写错位置。

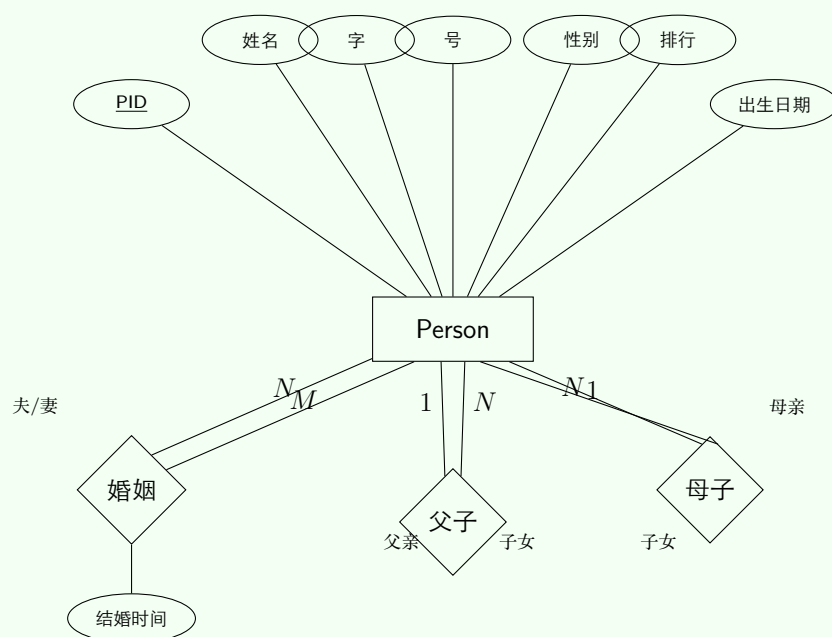
知识点: 3.3 SQL——连接查询与集合运算; 3.5 SQL——分组与聚合; 3.4 SQL——嵌套子查询

8.7 六、ER 图设计与关系模式转化 (10 分)

参考答案

(答案由解析者提供)

ER 图:



约束说明:

- 婚姻: 一个人(男性/女性)可参与多次婚姻(历史上的一夫多妻或改嫁), 故为 $M:N$ 联系, 带属性“结婚时间”。
- 父子: 一个父亲可有多子女 ($1:N$), 一个子女只有一位父亲 ($N:1$)。
- 母子: 一个母亲可有多子女 ($1:N$), 一个子女只有一位母亲 ($N:1$)。

转化为关系模型：

1. Person(PID, name, zi, hao, gender, birth_order, birth_date)
2. Marriage(HusbandID, WifeID, MarriageDate)
外键：HusbandID 参照 Person(PID), WifeID 参照 Person(PID)
3. FatherChild(FatherID, ChildID)
外键：FatherID 参照 Person(PID), ChildID 参照 Person(PID)
4. MotherChild(MotherID, ChildID)
外键：MotherID 参照 Person(PID), ChildID 参照 Person(PID)

想知道 a 是第几代人——修改方案：

在 Person 表中增加属性列 generation (代数)，类型为 INT。代数计算方法：从族谱最早的祖先（无父无母记录者）开始，设定其 generation = 1；其子女 generation = 2，依此类推。对于任意人 a，只需查询 SELECT generation FROM Person WHERE PID = a 即可获知其代数。代数可在数据录入时计算并存储，或通过递归查询（WITH RECURSIVE）动态计算。

解析与知识点

本题考察递归联系的 E-R 建模能力，是 ER 图设计中的进阶题型。族谱数据的特点是所有实体均为同一类型“Person”，而人与人之间存在多种关系，这些关系是实体与其自身之间的递归联系（Recursive Relationship）。本题需要建模三种递归联系：婚姻（对称递归）、父子（非对称递归）和母子（非对称递归）。

建模递归联系的关键挑战在于角色标注——同一个 Person 实体在“父子”联系中同时扮演“父亲”和“子女”两个不同角色，在 ER 图中必须通过连线上的角色标签来区分这两个参与方向。联系基数的确定需要根据现实语义：婚姻联系为 M:N，因为历史上存在一夫多妻和改嫁（一女嫁多次）的情形；父子/母子联系为 1:N，因为一个父亲/母亲可以有多个子女，但每个子女只有一个生物学上的父亲/母亲。转化为关系模式时，M:N 的婚姻联系必须创建独立的 Marriage 关系表，联合双方 PID 作为主键，并附带结婚时间属性。

1:N 的父子/母子联系通常可以选择两种策略：在子女方（N 端）的 Person 记录中添加外键，或创建独立的 FatherChild 和 MotherChild 关系表。本题采用独立建表策略（更清晰地表达关系）。对于“想知道 a 是第几代人”的查询需求，最简单的方案是在 Person 表中增加一个 generation 字段存储代数信息。代数的计算可以通过从根祖先（无父亲的记录）开始，逐代递推：根为第 1 代，其子女为第 2 代，依此类推。这种设计使得代数查询变成一次简单的 SELECT，避免了昂贵的

递归查询。如果数据不支持预计算，也可以使用 WITH RECURSIVE 在查询时动态计算。

典型错误包括：忘记在递归联系的连线上标注角色，导致无法区分参与方向；将婚姻联系的基数误标为 1:1（忽略一夫多妻和改嫁的语义）；在关系模式转化中将 1:N 递归联系也建独立表（可用外键简化）；代数查询方案中未考虑代数的存储与计算方式。

知识点：7.3 E-R 模型——递归联系；7.4 E-R 图向关系模式的转换

8.8 B 卷

注意

B 卷中大部分大题与 A 卷相同。仅单项选择题的第 2、4、6、8 题在选项或问法上有所调整，其余单项选择题及多项选择题、第三至第六大题均与 A 卷完全一致，此处不再重复。注意 B 卷单项选择题的题号从 3 开始编号（即 B 卷第一道选择题编号为第 3 题），以下按 B 卷实际题号给出答案与解析。与 A 卷答案相同且不涉及选项调整的题目（第 3、5、7、9、11、12 题）仅列出答案，详细解析参见 A 卷对应题目。

8.9 一、单项选择题（每题 2 分，共 20 分，题号 3–12）

参考答案

D（答案由解析者提供）

解析与知识点

（B 卷第 3 题与 A 卷第 1 题相同）本题考察数据库系统的三级模式结构。

外模式（用户视图层）、概念模式（全局逻辑结构层）和内模式（物理存储层）构成了 ANSI/SPARC 三级体系结构，而“类图”是 UML 中面向对象建模的概念，不属于关系数据库的三级模式。答案选 D——类图不属于数据库系统的三级模式结构。

三级模式通过两级映像（外模式/模式映像和模式/内模式映像）保证了逻辑独立性和物理独立性。

详细解析见 A 卷第 1 题。

知识点：1.2 数据库系统的三级模式结构

参考答案

C（答案由解析者提供）

解析与知识点

（B 卷第 4 题对应 A 卷第 2 题，选项有调整）本题考察数据库两级映像与数据独立性的关系。

模式/内模式映像定义的是全局逻辑结构与物理存储结构之间的映射关系，当物理存储结构改变时只需修改此映像，因此提供的是物理独立性而非逻辑独立性。外模式/模式映像定义的是用户视图与全局逻辑结构之间的映射关系，当全局逻辑结构改变时只需修改此映像，因此提供的是逻辑独立性。选项 C 声称“所有查询都必须经历两级变换”过于绝对，因此是不正确的叙述，选 C。

两者提供的独立性类型恰好对应各自映像的名称：模式/内模式映像是内层（物理层）的屏障，提供物理独立性；外模式/模式映像是外层（用户层）的屏障，提供逻辑独立性。关于两级变换：当用户直接查询基本表时只需要模式/内模式一级变换，只有通过视图（外模式）查询时才需要两级变换。

常见的混淆是将两者提供的独立性类型记反——记忆技巧是“内映像护物理，外映像护逻辑”。

知识点：1.4 数据库系统的数据独立性

参考答案

A（答案由解析者提供）

解析与知识点

（B 卷第 5 题与 A 卷第 3 题相同）本题考察关系代数中的对称差操作。

题目语义是查询“选修了 S1 但未选修 S2，或选修了 S2 但未选修 S1 的学生”，即对称差 $R_1 \oplus R_2 = (R_1 - R_2) \cup (R_2 - R_1) = (R_1 \cup R_2) - (R_1 \cap R_2)$ 。选项 A 的表达式 $(R_1 \cup R_2) - (R_1 \cap R_2)$ 恰好表达了对称差的语义：先取并集再移除交集。其余选项均不等价于对称差。答案选 A。

对称差在 SQL 中可通过 FULL OUTER JOIN 配合 NULL 检查实现，或通过左/右差集取并集实现。

详细解析见 A 卷第 3 题。

知识点：2.4 关系代数——集合运算

参考答案

A（答案由解析者提供）

解析与知识点

(B 卷第 6 题对应 A 卷第 4 题, B 卷改为问“正确的是”, 且选项 A 被修改为“一个应用可以建立多个关系数据库模式”) 本题考察关系数据库模式的基本性质。一个应用完全可以建立多个关系数据库模式来组织不同业务领域的数据库——例如同一系统中的人事数据、财务数据和库存数据可以分别放在不同的数据库模式下, 以提高模块化和安全性。因此修改后的选项 A 是正确的。其他选项: 关系数据库的模式应当相对稳定, 不应频繁变化 (选项 B 错误, 频繁变更会导致应用层大规模修改); 关系是元组的集合, 集合中的元素无固定顺序 (选项 C 正确但其不是本题问的选项); 在同一数据库内关系 (表) 的名称必须唯一 (选项 D 正确但也非本题目标答案)。本题明确问“正确的是”且 A 在 B 卷中被修改为正确叙述后即成为正确答案。

这一题体现了单选题中“问法反转”的命题技巧——同样选项的知识点不变, 但通过改变问法 (错误 → 正确) 改变了答案选择。

做这类“问法反转”题时, 惯性思维容易导致仍然按“错误叙述”去选, 必须仔细阅读题。

知识点: 2.1 关系数据结构及形式化定义

参考答案

D (答案由解析者提供)

解析与知识点

(B 卷第 7 题与 A 卷第 5 题相同) 本题考察候选关键字与主关键字的关系。关系的候选关键字可以有多个 (如学生关系中学号和身份证号都是候选关键字), 可以从中选择一个作为主关键字, 其余称为备用码。全码关系的候选关键字包含全部属性而非部分属性。答案选 D。

候选码需满足唯一性和最小性两个条件——能唯一标识每个元组, 且任意真子集不能唯一标识。

详细解析见 A 卷第 5 题。

知识点: 2.2 关系模型的完整性约束——实体完整性、候选码

参考答案

C (答案由解析者提供)

解析与知识点

(B 卷第 8 题对应 A 卷第 6 题, B 卷的函数依赖集中缺少 $X \rightarrow W$) 本题考察候选关键字的求解, 核心方法是属性闭包算法。

B 卷的函数依赖集为 $F = \{X \rightarrow Y, X \rightarrow Z, W \rightarrow X\}$ (缺少了 A 卷中的 $X \rightarrow W$)。计算各属性闭包: $X^+ = \{X, Y, Z\}$ ——由 $X \rightarrow Y$ 加入 Y , 由 $X \rightarrow Z$ 加入 Z , 但无法推导出 W , 因此 $X^+ \neq U$, X 不是超码。 W^+ 的计算: 从 W 出发, $W \rightarrow X$ 加入 X , 再由 X 的推导链得到 Y 和 Z , $W^+ = \{W, X, Y, Z\} = U$, 故 W 是超码, 且 W 是单属性无法约简, 所以 W 是唯一的候选关键字。 $Y^+ = \{Y\}$, $Z^+ = \{Z\}$, 均不是超码。因此该关系有且仅有一个候选关键字 W , 答案为 C。

对比 A 卷 (函数依赖集包含 $X \rightarrow W$ 和 $W \rightarrow X$ 双向依赖), A 卷中有两个候选关键字 X 和 W ; B 卷因缺少 $X \rightarrow W$, X 无法推导出 W , 候选关键字缩减为一个 W 。这道题很好地展示了“函数依赖的微小变化可以导致候选关键字的显著不同”这一重要原理。

常见错误: 只计算 X^+ 发现能推导出 Y 和 Z 就认为 X 是候选码, 忽略了 W 属性; 补算 W^+ 时忘了通过 X 的推导链继续扩展。

知识点: 6.2 候选码的求解——属性闭包算法

参考答案

D (答案由解析者提供)

解析与知识点

(B 卷第 9 题与 A 卷第 7 题相同) 本题基于前一问的候选关键字结果判定 BCNF。由 B 卷第 8 题知候选关键字为 W 。现在检查 $F = \{X \rightarrow Y, X \rightarrow Z, W \rightarrow X\}$ 中每个 FD 的决定因素: $X \rightarrow Y$ 的决定因素 X 不是候选关键字 (候选关键字是 W), $X \rightarrow Z$ 的决定因素 X 也不是候选关键字, $W \rightarrow X$ 的决定因素 W 是候选关键字。存在非平凡函数依赖 $X \rightarrow Y$ 和 $X \rightarrow Z$ 的决定因素 X 不是超码, 违反了 BCNF 的定义。因此该关系不满足 BCNF, 最高为 3NF。故选 D——该关系不满足 BCNF。

对比 A 卷, A 卷中所有 FD 的决定因素均为候选关键字 (X 和 W 都是), 因此 A 卷的关系满足 BCNF。B 卷由于缺少 $X \rightarrow W$, 关系不再满足 BCNF。这再次说明: 函数依赖的微小改变不仅影响候选码的确定, 也直接影响范式级别。

常见错误: BCNF 判定时只检查了 $W \rightarrow X$ 这条 FD 的决定因素是超码就认为满足 BCNF, 忽略了其他 FD 的决定因素也需要检查。

知识点: 6.3 范式——BCNF 的定义与判定

参考答案

D（答案由解析者提供）

解析与知识点

（B 卷第 10 题对应 A 卷第 8 题，选项有所调整）本题考察事务处理与并发控制概念中的错误叙述识别。

事务的 ACID 特性（原子性、一致性、隔离性、持久性）是事务区别于普通程序的核心特征，选项 A 正确。事务一旦提交（commit），其修改就被永久性地写入数据库，无法再通过 rollback 回滚——rollback 只能撤销尚未提交的事务修改，因此选项 B 正确。并发事务不加控制地执行确实可能导致丢失修改、不可重复读和脏读等经典的 inconsistency 问题，选项 C 正确。关于选项 D，在并发控制的理论和实践中，封锁（Locking）机制是一种典型的悲观并发控制方法——它假设事务冲突频繁发生，因此在数据访问之前就必须获取锁以预防冲突。因此选项 D 将封锁归类为“乐观”方法是根本性的概念错误，封锁是悲观并发控制的核心手段。本题选 D。乐观并发控制与悲观并发控制是两种互补的策略：乐观方法假设冲突很少，事务不加锁执行，提交时检测冲突；悲观方法（以封锁为代表）假设冲突频繁，先加锁再访问。选择哪种取决于应用场景的冲突频率。

常见错误是将封锁机制误认为是乐观方法——记忆技巧是“封锁 = 先下手为强 = 悲观”。

知识点：10.1 事务的基本概念；10.3 封锁机制

参考答案

B（答案由解析者提供）

解析与知识点

（B 卷第 11 题与 A 卷第 9 题相同）本题考察索引设计策略。

根据 B+ 树索引的最左前缀匹配原则，建立在 (A, B, C) 上的复合索引已经可以支持仅针对 A 列的等值和范围查询，因此在 A 上单独建索引是冗余的。低基数属性（如性别）的索引选择性极差，建索引无益。答案选 B。

最左前缀原则是复合索引设计的核心原则：索引中列的顺序决定了查询可以利用索引的方式。

详细解析见 A 卷第 9 题。

知识点：7.5 索引技术；9.4 查询优化中索引的作用

参考答案

C（答案由解析者提供）

解析与知识点

（B 卷第 12 题与 A 卷第 10 题相同）本题考察多粒度封锁与锁升级策略。

当事务需要读取关系中 70% 以上的元组时，逐元组加锁的锁管理器开销极大，此时最有效的策略是锁升级——对整个关系加一把 X 锁（排他锁），一次性完成全部读写保护。答案选 C。

锁升级是多粒度封锁机制中的优化策略：当事务获取大量低粒度锁时，系统自动将它们升级为更高粒度的锁，减少锁管理开销。

详细解析见 A 卷第 10 题。

知识点：10.3 封锁——多粒度封锁与锁升级策略

注意

B 卷多项选择题与 A 卷完全一致，题号与答案均不变。详见 A 卷第二部分的题目与解析。

注意

B 卷第三至第六大题（规范化设计、关系模式规范化、SQL 综合查询、ER 图设计）与 A 卷完全一致，此处不再重复。请参阅 A 卷对应部分。

9 2021–2022 学年《数据库系统》期末试卷

注意

说明：原试卷为扫描版无法提取文本，以下题目由答案逆向还原，可能与原题表述有差异。

9.1 一、选择题（本大题共 10 小题，每题 2 分，共 20 分）

1. 关于数据库系统的三级模式结构，以下说法正确的是（ ）。
 - A. 外模式/模式映像保证了数据的逻辑独立性
 - B. 一个数据库可以有多个模式，但只有一个内模式
 - C. 模式是数据库中全体数据的逻辑结构和特征的描述
 - D. 模式/内模式映像定义在模式中，保证了数据的物理独立性

参考答案

答案：A

解析与知识点

数据库系统的三级模式结构由外模式、模式和内模式构成，这是数据库系统最核心的体系架构。外模式（也称子模式或用户模式）是数据库用户能看见和使用的局部数据的逻辑结构描述，一个数据库可以有多个外模式。模式（也称概念模式或逻辑模式）是数据库中全体数据的逻辑结构和特征的描述，一个数据库仅有一个模式。

内模式（也称存储模式）是数据物理结构和存储方式的描述，一个数据库也仅有一个内模式。在三层结构之间，存在两级映像：外模式/模式映像定义了外模式与模式之间的对应关系，当模式改变时，只需修改该映像而使外模式保持不变，从而保证了数据的逻辑独立性；模式/内模式映像定义了模式与内模式之间的对应关系，当内模式（物理存储结构）改变时，只需调整该映像而保持模式不变，从而保证了数据的物理独立性。

选项 B 的错误在于「一个数据库可以有多个模式」——模式是唯一的（虽然可以有多个外模式），且一个数据库只有一个内模式也是正确的陈述，使 B 成为表述不严谨的复合选项。选项 C 对模式的定义本身正确，但原题设计意味不在 C。选项 D 的错误在于模式/内模式映像是唯一的（一个模式对应一个内模式），但它并非「定义在模式中」，而是独立存在的映射机制。

常见错误是混淆外模式/模式映像与模式/内模式映像各自保证的独立性类型，或者在记忆模式数量时错误地认为模式可以有多个——实际上，一个数据库只有一个模式和一个内模式，但可以有多个外模式。

知识点：1.3 数据库系统的三级模式结构（外模式、模式、内模式的定义与数量特征）；两层映像机制（外模式/模式映像——逻辑独立性，模式/内模式映像——物理独立性）；数据独立性的形式化定义。

2. 以下属于关系代数基本运算的是（ ）。

A. 连接 B. 投影 C. 交 D. 除

参考答案

答案：B

解析与知识点

关系代数的五种基本运算为：并（ $\cup$ ）、差（ $-$ ）、笛卡尔积（ $\times$ ）、选择（ σ ）、投影（ π ）。这五种运算构成了关系代数的完备操作集——任何可表达的关系查询都可以

由这五种基本运算的组合来实现。

连接（包括 θ -连接、等值连接、自然连接）是导出运算，可以通过笛卡尔积和选择的组合等价表达： $R \bowtie_{X\theta Y} S = \sigma_{X\theta Y}(R \times S)$ 。除法运算也是导出运算，等价于 $\pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$ ，涉及两次差运算、两次投影和一次笛卡尔积。交运算同样可以通过差运算导出： $R \cap S = R - (R - S)$ 。因此只有投影属于基本运算，而连接、交、除均为导出运算。

掌握基本运算与导出运算的划分是理解关系代数体系完备性的基础，也是考试中的经典考点。关系代数分为基本运算和导出运算两个层次，基本运算的选取基于不可再分性和操作集的完备性标准。

常见错误是将选择 σ 与投影 π 混淆——选择是水平方向的元组筛选（WHERE 子句的功能），投影是垂直方向的属性筛选（SELECT 子句中列的选择）。自然连接也常被误认为是基本运算。

知识点：2.4 关系代数——五种基本运算（并 $\cup$ 、差 $-$ 、笛卡尔积 $\times$ 、选择 σ 、投影 π ）；基本运算与导出运算的区分原则；连接、交、除运算由基本运算表达的等价公式。

3. 在 SQL 语言中，删除表中所有元组但保留表结构定义的语句是（ ）。

- A. DROP TABLE 表名 B. TRUNCATE TABLE 表名 C. DELETE FROM 表名 D. ALTER TABLE 表名 DROP ALL

参考答案

答案：C

解析与知识点

本题考查 SQL 中 DELETE、DROP 和 TRUNCATE 三种删除操作的语义对比。DELETE FROM 表名；语句删除指定表的所有元组（数据行），但保留表的结构定义（包括列定义、约束、索引、触发器等数据库对象元数据），且 DELETE 操作是 DML 语句，被记入事务日志，可以在事务中回滚。与之对比，DROP TABLE 是 DDL 语句，会删除整个表——既包括数据也包括表的结构定义，操作不可回滚。TRUNCATE TABLE 也可以删除所有行，且在大多数 DBMS 中比 DELETE 更快（因为它不逐行记录日志），但在标准 SQL 语境与国内多数教材中，DELETE 被认为是更通用、更基础的答案。另外 TRUNCATE 在有些数据库系统中不可回滚，而 DELETE 总可以回滚。选项 D「ALTER TABLE ... DROP ALL」并非标准 SQL 语法，不存在这样的语句。

DELETE 是 DML（数据操纵语言），操作以行为单位，逐行记录日志，支持 WHERE 条件过滤和事务回滚；DROP 是 DDL（数据定义语言），直接从数据字典中移除

表定义；TRUNCATE 在多数数据库中属于 DDL，通过释放数据页清空表，速度最快但不可回滚。本题考查的是 DELETE 与 DROP 的本质区别：一个删除数据保留结构，一个两者皆删。

容易混淆 TRUNCATE 与 DELETE 的事务可回滚性——在 MySQL (InnoDB) 中 TRUNCATE 可以回滚，但在 Oracle 和 PostgreSQL 中不可以。考试中应遵循标准教材定义：DELETE 可回滚，TRUNCATE 和 DROP 不可回滚。

知识点：3.5 数据更新——DELETE、DROP、TRUNCATE 三种删除操作的语义对比与使用场景（DELETE 删数据保结构，DROP 删结构，TRUNCATE 快速清表）；DDL（数据定义语言）与 DML（数据操纵语言）的区分；事务回滚行为差异。

4. 关于事务的 ACID 特性，以下说法错误的是（ ）。
- A. 原子性保证事务的所有操作要么全部执行，要么全部不执行
 - B. 一致性确保事务执行完毕后数据库从一个一致性状态转换到另一个一致性状态
 - C. 隔离性保证并发事务的执行互不干扰，每个事务都感觉不到其他事务的存在
 - D. 持久性保证事务对数据库的所有修改都是永久的

参考答案

答案：D

解析与知识点

事务的 ACID 特性（原子性 Atomicity、一致性 Consistency、隔离性 Isolation、持久性 Durability）的完整定义与辨析，这是数据库并发控制与恢复理论的基础。

逐项分析四个选项：原子性（Atomicity）保证事务中的所有操作要么全部成功执行，要么全部不执行（即当任一操作失败时，事务回滚到执行前的状态），选项 A 正确。一致性（Consistency）确保事务的执行将数据库从一个一致性状态转变为另一个一致性状态，事务单独执行时总能保证一致性，选项 B 正确。隔离性（Isolation）保证并发事务的执行互不干扰——多个事务并发执行的结果应与它们串行执行的结果相同，每个事务都感觉不到系统中其他事务的并发执行，选项 C 正确。选项 D 的表述关键缺陷在于缺少对「提交成功」这一前提条件的强调。

在严谨的教材表述中，持久性（Durability）的完整定义为：一旦事务成功提交，其对数据库的修改就是永久的，即使系统发生故障也不会丢失。D 选项仅说「事务对数据库的所有修改都是永久的」，遗漏了「提交成功」这一关键限定——如果一个事务最终回滚了，其修改显然不是永久的。因此在四个选项中，D 是表述有误的那个。

常见错误是讨论持久性时忽略「提交成功」这一前提条件。许多学习者将持久性

简单理解为「修改永久保存」，但持久性的严格定义要求事务必须先成功提交。未提交的事务（如回滚的事务）的修改不会被持久化，这是持久性概念中容易混淆的关键点。

知识点：10.1 事务的概念——ACID 特性（原子性 Atomicity、一致性 Consistency、隔离性 Isolation、持久性 Durability）的完整定义与辨析；持久性中「提交成功」作为前提条件的关键性；事务回滚与持久性的关系。

5. 关于封锁机制和两段锁协议，以下说法正确的是（ ）。
- A. S 锁之间相互排斥，一个数据对象上最多只能有一个 S 锁
 - B. X 锁之间可以共享，多个事务可以同时持有同一数据对象的 X 锁
 - C. 遵守两段锁协议（2PL）的调度一定是冲突可串行化的
 - D. 一次封锁法可以保证系统永远不会出现死锁

参考答案

答案：C

解析与知识点

两段锁协议（Two-Phase Locking, 2PL）的定义与性质，以及封锁机制中共享锁（S 锁）和排他锁（X 锁）的相容性，一次封锁法与死锁预防的关系。

逐项分析四个选项：选项 A 错误，因为共享锁（S 锁）之间是完全兼容的——多个事务可以同时为同一数据对象加 S 锁进行读操作，互不阻塞。选项 B 严重错误，排他锁（X 锁）不仅与其他 X 锁不兼容，也与 S 锁不兼容——任何时刻一个数据对象最多只能有一个 X 锁，且不能再有其他任何类型的锁。选项 D 错误在于，一次封锁法要求事务在开始时就一次性申请所有需要的锁，虽然可以预防死锁（破坏了「保持并等待」条件），但在实际应用中事务往往无法预知所有需要访问的数据，且一次性锁住大量数据会严重降低系统并发度，并非「一定能避免死锁」的可靠方案。选项 C 正确：遵守 2PL 协议的所有调度一定是冲突可串行化的，因此 2PL 是冲突可串行化的充分条件。

两段锁协议的核心规则是：每个事务必须分两个阶段进行加锁与解锁——在释放任何一个锁之后，不能再申请任何新的锁。2PL 是冲突可串行化的充分条件但并非必要条件——存在符合冲突可串行化但不满足 2PL 的调度。关于一次封锁法：它要求事务在开始时就一次性申请所有需要的锁，虽然可以预防死锁（破坏了「保持并等待」条件），但降低了系统并发度。

常见错误有三：一是混淆 2PL 与冲突可串行化的充分/必要关系——2PL 是充分条件而非必要条件；二是误认为 S 锁之间互斥（实际上 S-S 兼容）；三是将一次封锁法等同于万能的死锁预防方案，忽略了其并发度损失大的实际缺陷。

知识点：10.3 封锁机制——共享锁（S 锁）与排他锁（X 锁）的相容矩阵（S-S 兼容、S-X 不兼容、X-X 不兼容）；两段锁协议（2PL）的定义与性质（2PL 是冲突可串行化的充分条件但非必要条件）；一次封锁法与死锁预防的原理及局限性。

6. 如果关系模式 R 中所有非主属性都完全函数依赖于候选码，且所有非主属性都不传递函数依赖于候选码，则 R 至少属于（ ）。

- A. 1NF B. 2NF C. 3NF D. BCNF

参考答案

答案：C

解析与知识点

关系数据库规范化理论中各范式的递进定义——2NF、3NF 与 BCNF 的条件与区别。

2NF 的定义为：关系模式属于第一范式，且所有非主属性完全函数依赖于候选码（即不存在非主属性对候选码的部分函数依赖）。**3NF** 的定义为：关系模式属于第一范式，且所有非主属性既不部分函数依赖于候选码，也不传递函数依赖于候选码。题干所述的「所有非主属性都完全函数依赖于候选码」满足了 2NF 的条件（排除了部分依赖），「所有非主属性都不传递函数依赖于候选码」进一步满足了 3NF 的条件，因此关系 R 至少属于 3NF。注意这里说的是「至少」——3NF 是满足题干条件的最低范式等级。BCNF 在 3NF 的基础上进一步要求所有非平凡函数依赖的左部都包含候选码（即为超码），而题干的条件并未保证这一点（例如全主属性关系中 BCNF 可能不满足但 3NF 满足），因此不能得出 BCNF 的结论。范式等级之间的递进关系：1NF 仅要求属性不可分；2NF 在 1NF 基础上消除非主属性对候选码的部分函数依赖；3NF 在 2NF 基础上消除非主属性对候选码的传递函数依赖；BCNF 在 3NF 基础上要求所有决定因素均为超码。题干的两个条件正好对应 2NF 和 3NF 的消除目标。

常见错误是看到题干条件较强就误选 BCNF。「至少」一词是关键——题干条件恰好是 3NF 的定义，不能推出 BCNF，因为 BCNF 还要求所有函数依赖的决定因素都是超码，而题干未涉及主属性对候选码的传递依赖等情况。

知识点：6.2 范式——2NF 的定义（无非主属性对候选码的部分函数依赖）；3NF 的定义（无非主属性对候选码的部分依赖和传递依赖）；BCNF 的定义（所有非平凡 FD 左部均为超码）；范式等级之间的递进关系与约束差异。

7. 设有关系 R(A, B) 和 S(B)，则 $\pi_A(\sigma_{R.B > S.B}(R \times S))$ 的等价关系代数表达式为（ ）。

- A. $\pi_A(R \bowtie_{R.B > S.B} S)$ B. $\pi_A(R \times S)$

C. $\pi_A(R \bowtie_{R.B=S.B} S)$ D. $\pi_A(\sigma_{R.B=S.B \wedge R.B>S.B}(R \times S))$

参考答案

答案：A

解析与知识点

关系代数中 θ -连接的定义及其与选择、笛卡尔积之间的等价变换关系。

θ -连接的形式化定义为 $R \bowtie_{A\theta B} S = \sigma_{A\theta B}(R \times S)$ ，即先对 R 和 S 做笛卡尔积，再从结果中选择满足 θ 条件的元组。因此 $\sigma_{R.B>S.B}(R \times S)$ 等价于 $R \bowtie_{R.B>S.B} S$ ，再对其结果取投影 π_A 即为选项 A。选项 B $\pi_A(R \times S)$ 没有经过条件筛选（缺少 $R.B > S.B$ 的限制），与原式不等价。选项 C 使用了等值条件 $R.B = S.B$ ，与原式的不等式条件 $>$ 不符。选项 D 中同时施加了 $R.B = S.B$ 和 $R.B > S.B$ 两个互相矛盾的条件： $R.B = S.B \wedge R.B > S.B$ 恒为假，结果恒为空集，与原式不等价。 θ -连接的这一等价关系是关系代数表达式的化简和等价推演的重要基础。任何 θ -连接都可以表达为笛卡尔积后跟一个选择操作，反之，对笛卡尔积结果施加的选择条件如果连接了两个关系的属性，则可以等价地改写为 θ -连接形式，这在查询优化中非常有用。

常见错误有二：一是未理解 θ -连接的定义，无法识别 $\sigma_{R.B>S.B}(R \times S)$ 与 $R \bowtie_{R.B>S.B} S$ 的等价性；二是在排除错误选项时未能识别选项 D 中两个条件的矛盾性（即 $R.B = S.B$ 与 $R.B > S.B$ 不能同时成立，导致结果恒空）。

知识点：2.4 关系代数—— θ -连接的形式化定义（ $R \bowtie_{X\theta Y} S = \sigma_{X\theta Y}(R \times S)$ ）；连接、选择与笛卡尔积三者之间的等价转换关系；条件矛盾导致结果空集的识别。

8. 关于数据库索引，以下说法正确的是（ ）。

- A. 为表创建越多的索引，查询效率越高
- B. B^+ 树索引中所有数据记录都存储在叶子结点中
- C. 唯一索引主要用于加速范围查询
- D. 一个表上可以根据需要创建多个聚集索引

参考答案

答案：B

解析与知识点

B^+ 树索引的结构特性、聚集索引与非聚集索引的区别、唯一索引的作用、以及索引创建在读写性能之间的权衡。

逐项分析： B^+ 树是一种多级索引结构，所有数据记录（或指向数据记录的指针）

都存储在叶子结点中，内部结点仅存储索引键值用于路由检索，叶子结点之间通过指针链（双向链表）连接，选项 B 正确。选项 A 错误——索引虽然可以加速查询，但每次对表进行插入、删除或更新操作时，所有相关的索引都需要同步维护，过多的索引会严重降低写操作的性能，索引创建需要在查询性能与更新开销之间权衡，并非「越多越好」。选项 C 错误——唯一索引的主要目的是保证列值的唯一性（即约束作用），而不是加速范围查询；范围查询更多依赖 B⁺ 树的叶子层链表或类似结构来高效扫描。选项 D 错误——一个表最多只能有一个聚集索引（Clustered Index），因为聚集索引决定了表中数据的物理存储顺序，数据行只能按照一种顺序物理排列。

B⁺ 树的结构设计兼顾了等值查询（沿树从根到叶子）和范围查询（在叶子层沿链表扫描）的高效支持。聚集索引决定了数据的物理存储顺序，因此每表最多一个；非聚集索引则不改变数据的物理排列，可有多。索引是典型的以空间换时间、以写换读的策略。

常见错误有三：一是认为索引越多越好，忽视更新维护开销；二是混淆聚集索引与非聚集索引，误以为可以创建多个聚集索引；三是将唯一索引与范围查询加速机制混淆——唯一索引解决的是约束问题，B⁺ 树的叶子层链表解决的是范围扫描效率问题。

知识点：9.3 索引——B⁺ 树索引的结构特性（数据记录仅存于叶子结点、内部结点用于路由、叶子结点间指针链支持范围查询）；聚集索引（Clustered Index）与非聚集索引的区别及一个表只能有一个聚集索引的限制；索引创建在读写性能之间的权衡（索引维护开销）。

9. 关于参照完整性（引用完整性），以下说法错误的是（ ）。

- A. 外码的值只要求可以为 NULL
- B. 级联删除（CASCADE DELETE）会破坏参照完整性
- C. 外码的值要么为 NULL，要么等于被参照关系中某个元组的主码值
- D. 参照完整性约束在插入、删除和更新操作时都需要检查

参考答案

答案：B

解析与知识点

参照完整性（引用完整性）的形式化定义与外码约束的策略选项（CASCADE DELETE / SET NULL / NO ACTION）。

参照完整性是关系数据库三条完整性规则之一，其核心要求是：外码（Foreign Key）的值要么取空值 NULL，要么等于被参照关系中某个元组的主码值。选项 C 完整

表述了这一定义，是正确的。选项 D 也是正确的——参照完整性约束在插入从表元组（外码值必须有效或为 NULL）时、删除主表元组（可能触发级联操作或拒绝删除）时、以及更新外码值或主码值时都需要检查。选项 A 表述不完整（仅提及了 NULL 这一种可能性，遗漏了「等于对应主码值」），但整体表述不如 B 的错误明显。选项 B 是错误的——级联删除（CASCADE DELETE）恰恰是维护参照完整性的一种策略，而非破坏它：当事务删除主表中的元组时，DBMS 自动删除从表中所有引用该元组的外码记录，从而保持数据库的参照完整性不被破坏。

外码约束的三种策略：CASCADE DELETE（级联删除从表记录）、SET NULL（从表外码置空）、NO ACTION（拒绝删除主表元组）。题干问「错误的」，B 将维护策略错认为破坏，是明确的错误表述。

常见错误是误认为级联操作会破坏参照完整性。实际上，级联删除/级联更新等操作恰恰是 DBMS 提供的自动维护参照完整性的机制。另一个易错点是忘记外码取值的完整规则（NULL 或对应主码值，两种可能性缺一不可）。

知识点：2.3 关系的完整性——参照完整性（引用完整性）的形式化定义（外码取值为 NULL 或对应主码值）；外码约束的策略选项（CASCADE DELETE / SET NULL / NO ACTION）及其语义；参照完整性在增删改操作中的检查时机。

10. 关于数据库恢复技术，以下说法正确的是（ ）。

- A. 静态转储比动态转储具有更高的灵活性
- B. 日志文件是数据库恢复的核心基础
- C. 检查点机制可以完全取代日志文件的作用
- D. 系统故障后只需对已提交事务进行 REDO 操作，未提交事务无需处理

参考答案

答案：B

解析与知识点

数据库恢复技术——日志文件的作用、检查点技术、静态转储与动态转储的区别、以及系统故障后的恢复策略。

逐项分析：日志文件记录了事务对数据库的所有修改操作（通常包括事务标识、数据项标识、修改前旧值和修改后新值），是数据库恢复的核心基础，无论是事务故障还是系统故障的恢复，都必须依赖日志信息进行 UNDO 和 REDO 操作，选项 B 正确。选项 A 错误——静态转储期间不允许任何数据库修改操作，而动态转储允许并发修改，因此动态转储的灵活性和可用性远高于静态转储。选项 C 错误——检查点（Checkpoint）技术可以缩短恢复时需要扫描的日志范围（从最近一个检查点开始而非从日志头开始扫描），但检查点不能替代日志文件本身，日志文件

仍然是记录修改信息的基础设施。选项 D 错误——系统故障后的恢复通常需要同时进行 UNDO 和 REDO：已提交事务需要 REDO（确保修改写入磁盘），未提交事务需要 UNDO（将数据库恢复到事务开始前的状态），仅做 REDO 会遗漏未提交事务的撤销，导致数据库中残留了未完成事务的部分修改。

恢复技术中的三种关键机制：（1）日志文件——记录所有修改操作，是恢复的核心基础；（2）检查点——缩短恢复时的日志扫描范围，但不能替代日志；（3）转储——静态转储（不可并发）vs 动态转储（可并发），后者更灵活。不同故障类型对应不同恢复策略：事务故障用 UNDO，系统故障用 UNDO + REDO，介质故障需重装副本 + REDO。

常见错误有三：一是误认为检查点可以替代日志文件；二是系统故障恢复时遗漏 UNDO 操作（仅执行 REDO），错误地认为未提交事务无需处理；三是将静态转储与动态转储的灵活性关系记反——实际上是动态转储更灵活。

知识点：11.2 数据库恢复技术——日志文件的作用与内容结构（事务标识、数据项、旧值、新值的记录格式）；检查点技术（缩短恢复扫描范围但不能替代日志）；故障类型与恢复策略（事务故障——UNDO；系统故障——UNDO + REDO；介质故障——重装副本 + REDO）；静态转储与动态转储的对比。

9.2 二、判断题（本大题共 10 小题，每题 2 分，共 20 分。正确的打 ✓，错误的打 ×）

1. 外模式/模式映像保证了数据的逻辑独立性。

参考答案

答案：✓（正确）

解析与知识点

三级模式结构中两层映像与数据独立性（逻辑独立性与物理独立性）的对应关系。外模式/模式映像（也称子模式/模式映像）定义了外模式与概念模式之间的对应关系。当概念模式（模式）发生变化时——例如修改了关系结构的定义、增加了新的属性或关系——数据库管理员只需修改外模式/模式映像中的对应规则，而使外模式对应用程序的接口保持稳定不变。这样，应用程序所依赖的数据逻辑视图（外模式）不需任何修改即可继续运行，从而达到了逻辑数据独立性的目的。与之对应，物理独立性由模式/内模式映像来保障：当物理存储结构（如文件组织方式、索引结构、存储路径等）发生变化时，只需调整模式/内模式映像即可。本题所述「外模式/模式映像保证了数据的逻辑独立性」完全正确，是教材中的标准表述。

两层映像的对应关系：外模式/模式映像 $\rightarrow$ 逻辑独立性（应用程序不变）；模式/内模式映像 $\rightarrow$ 物理独立性（存储结构改变不影响逻辑结构）。

常见错误是将逻辑独立性与物理独立性的对应映像关系记反，认为外模式/模式映像对应物理独立性、模式/内模式映像对应逻辑独立性。

知识点：1.3 数据库系统的三级模式结构——外模式/模式映像的定义与逻辑独立性的保障机制；逻辑独立性 vs. 物理独立性的触发条件与映像对应关系。

2. 在 E-R 图向关系模型的转换中，每个实体型转换为一个关系模式，实体型的属性对应关系的属性，实体型的码对应关系的码。

参考答案

答案：✓（正确）

解析与知识点

E-R 图向关系模型转换中的实体型转换规则，这是概念设计到逻辑设计的核心桥梁。

最基本的转换规则之一是：每个实体型转换为一个关系模式（表），实体型的属性直接映射为该关系的属性（列），实体型的码（或标识符）映射为该关系的主码（Primary Key）。这是关系数据库设计中应用最广泛、最基本的转换规则。例如，在学籍管理系统中，「学生」实体具有属性学号、姓名、年龄、所在系，转换后得到一个学生关系模式：Student(学号, 姓名, 年龄, 所在系)，其中学号为该关系的主码。本题的表述仅针对最基础的实体型转换规则，完全正确。

E-R 图向关系模型的转换还包括联系型的转换规则——1:1 联系可与任一端实体合并或独立成表，1:N 联系可与 N 端实体合并或独立成表，M:N 联系必须转换为独立的关系模式。

常见错误是将联系型的转换规则（尤其是 M:N 联系的处理方式）与实体型的转换规则混淆，或者在判断时过度联想联系型的复杂情况而误判实体型转换规则也有问题。

知识点：7.5 E-R 模型向关系模型的转换——实体型的转换规则（实体型 $\rightarrow$ 关系模式，属性 $\rightarrow$ 属性，码 $\rightarrow$ 主码）；联系型的转换规则体系（1:1、1:N、M:N 各自的转换策略）；数据库设计的逻辑设计阶段定位。

3. GROUP BY 子句可以将查询结果按指定列的值进行分组，使聚集函数可以在每个分组上独立计算。

参考答案

答案：✓（正确）

解析与知识点

SQL 中 GROUP BY 子句的分组机制与聚集函数的协同工作方式。

GROUP BY 子句的作用是将 SELECT 语句的查询结果集按照指定的一个或多个列的值进行分组——所有分组列值相同的行被归入同一组。分组之后，SELECT 子句中的聚集函数（如 COUNT、SUM、AVG、MAX、MIN 等）不再是针对整个结果集进行计算，而是在每个分组内部独立执行。例如：SELECT dept, COUNT(*) FROM student GROUP BY dept; 这条语句按院系分组，COUNT(*) 对每个院系分别计算学生人数。本题的表述准确描述了 GROUP BY 与聚集函数的协同工作机制，完全正确。

同时，SELECT 子句中出现的非聚合列必须出现在 GROUP BY 子句中（这是标准 SQL 的语法约束），因为每个分组中只产生一行输出，非分组列在分组后可能对应多个值，只有出现在 GROUP BY 中的列才能保证每个组中取值唯一。

常见错误是在使用 GROUP BY 时，在 SELECT 子句中包含未出现在 GROUP BY 中的非聚合列，导致语法错误或结果不确定。另一个易错点是误以为聚集函数只能在 GROUP BY 使用——实际上无 GROUP BY 时聚集函数作用于整个结果集。

知识点： 3.4 数据查询——GROUP BY 子句的分组机制与语义；聚集函数（COUNT、SUM、AVG、MAX、MIN）在分组计算中的行为；SELECT 子句中非聚合列必须出现在 GROUP BY 中的语法约束。

4. 自主存取控制（DAC）已经足够满足所有场景的数据库安全需求，不需要强制存取控制（MAC）作为补充。

参考答案

答案：×（错误）

解析与知识点

数据库安全存取控制的两大类型——自主存取控制(DAC)与强制存取控制(MAC)的区别、优缺点及互补关系。

自主存取控制（DAC）通过 SQL 的 GRANT 和 REVOKE 语句实现，允许数据所有者根据自身意愿将对数据对象的操作权限授予其他用户，具有灵活性高的优点。然而 DAC 存在一个根本性的安全漏洞——被授权用户可能将自己获得的权限再授予未经许可的第三方，导致数据泄露，即「权限的传递性」是 DAC 的固有

缺陷。强制存取控制 (MAC) 通过为系统中的每个主体 (用户/进程) 和客体 (数据对象) 分配安全级别 (如密级标记: 绝密、机密、秘密、公开), 并按照 BLP (Bell-LaPadula) 模型中的「不上读、不下写」规则来强制实施访问控制——主体的安全级别必须高于或等于客体的安全级别才能读取 (不上读), 主体的安全级别必须低于或等于客体的安全级别才能写入 (不下写)。MAC 的强约束性使权限的流转完全不受用户意愿控制, 能够防止信息从高安全级向低安全级泄漏。在军事、政府、金融等高安全需求场景中, 必须结合使用 DAC 和 MAC 来构筑多层次的数据库安全保障体系。本题声称「不需要 MAC」显然是错误的。

BLP 模型的「不上读、不下写」规则: 不上读 (No Read Up) 防止低级别主体读取高级别客体; 不下写 (No Write Down) 防止高级别主体将信息写入低级别客体从而间接泄漏。

常见错误是认为 DAC 的灵活性足以满足所有场景的安全需求, 忽视了 DAC 中权限传递性导致的安全漏洞。另一个易错点是混淆 BLP 模型中上下读写的规则方向。

知识点: 4.2 存取控制——自主存取控制 (DAC) 的 GRANT/REVOKE 机制与权限传递性漏洞; 强制存取控制 (MAC) 的安全级标记与 BLP 模型规则 (不上读、不下写); DAC 与 MAC 的互补关系及适用场景。

5. Armstrong 公理系统包含自反律、增广律和传递律三条基本推理规则, 由这三条规则可以推导出函数依赖的所有有效的推理规则。

参考答案

答案: ✓ (正确)

解析与知识点

Armstrong 公理系统的三条基本推理规则 (自反律、增广律、传递律) 及其有效性与完备性, 以及由公理推导出的三条导出规则。

Armstrong 公理系统由 W. W. Armstrong 于 1974 年提出, 是关系数据库函数依赖理论的公理化基础。三条基本公理: 自反律 (Reflexivity) ——若 $Y \subseteq X$, 则 $X \rightarrow Y$; 增广律 (Augmentation) ——若 $X \rightarrow Y$, 则 $XZ \rightarrow YZ$; 传递律 (Transitivity) ——若 $X \rightarrow Y$ 且 $Y \rightarrow Z$, 则 $X \rightarrow Z$ 。这三条公理是完备的 (Complete) 和有效的 (Sound) ——有效意味着由公理推导出的每一个函数依赖都在逻辑上成立, 完备意味着所有逻辑上成立的函数依赖都可以从给定的函数依赖集经由公理推导出来。由这三条基本公理还可以推导出三条常用的导出规则: 合并规则、伪传递规则和分解规则。本题的表述完全正确。

三条导出规则: 合并规则 (若 $X \rightarrow Y$ 且 $X \rightarrow Z$, 则 $X \rightarrow YZ$)、伪传递规则 (若

$X \rightarrow Y$ 且 $WY \rightarrow Z$, 则 $WX \rightarrow Z$)、分解规则 (若 $X \rightarrow YZ$, 则 $X \rightarrow Y$ 且 $X \rightarrow Z$)。有效性和完备性共同保证了 Armstrong 公理系统是函数依赖理论的可靠且完整的推理工具。

常见错误是将导出规则 (合并、分解等) 误认为是基本公理, 或混淆公理与导出规则的证明关系——导出规则是从基本公理推导而来, 非独立公理。另一个易错点是忘记自反律的条件 $Y \subseteq X$, 误以为任意两个属性集之间均可自反。

知识点: 6.1 函数依赖——Armstrong 公理系统 (自反律、增广律、传递律) 的公理化定义; 公理系统的有效性 (Soundness) 与完备性 (Completeness) 证明; 三条导出规则 (合并、伪传递、分解) 的推导方法。

6. 查询优化中, 选择运算应尽可能在连接运算之后进行, 以减少中间结果。

参考答案

答案: $\times$ (错误)

解析与知识点

查询优化中的「选择运算尽早执行」原则 (Selection Pushdown), 即选择运算应在连接运算之前进行。

查询优化的核心原则之一是「选择运算尽早执行」, 即尽可能将选择运算下推到查询执行计划的最底层 (靠近数据源端), 在连接运算之前完成。选择运算具有高度的过滤性——它可以大幅缩减元组数量, 减少参与后续操作的数据规模。如果选择在连接之后才执行, 意味着连接操作需要处理大量会被后续选择丢弃的元组, 不必要地消耗了 I/O 和 CPU 资源。正确的优化策略是: 先对每个参与连接的关系单独执行选择运算 (过滤掉不满足条件的行), 然后再进行连接运算——选择运算越早执行, 中间结果集越小, 整个查询的执行效率越高。本题的表述与这一优化原则完全相反 (主张先连接后选择), 因此错误。

这一策略等价于关系代数表达式优化中的「选择下推」变换: $\sigma_P(R \bowtie S) \equiv \sigma_{P_R}(R) \bowtie \sigma_{P_S}(S)$ (当条件 P 可分解为 P_R 和 P_S 分别作用于 R 和 S 时)。

常见错误是记反选择与连接的先后顺序, 误以为先做连接可以「一步到位」。实际上连接是代价最高的关系代数操作之一, 在执行连接之前尽可能缩小参与连接的数据集, 是查询优化最基本也是最有效的原则。

知识点: 9.2 关系表达式的等价变换与代数优化——选择运算下推策略 (Selection Pushdown); 先选择后连接 vs. 先连接后选择的性能对比; 选择条件的分解与下推的等价变换规则。

7. 封锁粒度越大, 系统并发度越低, 但封锁管理本身的开销也越小。

参考答案

答案：✓（正确）

解析与知识点

封锁粒度（Locking Granularity）的概念及其与并发度、系统管理开销之间的经典权衡关系（Trade-off）。

封锁粒度是指封锁数据对象的大小——可以是属性级、元组级（行级）、页面级、表级乃至整个数据库级。粒度越大（如对整个表加锁），每次加锁覆盖的数据范围越广，意味着多个事务即使访问的是同一表中的不同行也必须互相等待，并发度因此降低；但同时，系统只需维护数量很少的封锁项（一张表只需一条封锁记录），封锁管理的开销较小。粒度越小（如行级封锁），不同事务可以同时操作同一张表的不同行，并发度显著提高；但系统需要维护大量的封锁项（每一加锁的行都需要一条封锁记录），封锁管理的内存和 CPU 开销很大。本题的表述准确总结了封锁粒度与并发度、系统开销之间的权衡关系，完全正确。

实际的数据库管理系统通常支持多粒度封锁（如 MySQL 的 InnoDB 同时支持行级锁和表级锁）以及意向锁机制（IS、IX、SIX）来协调不同粒度锁之间的兼容性，以在并发度和系统开销之间取得动态平衡。

常见错误是片面追求某一方面——要么认为粒度越小越好（忽视管理开销），要么认为粒度越大越好（忽视并发度损失）。理解「权衡」是关键：没有最优的粒度，只有最合适的粒度，取决于应用的并发访问模式和性能需求。

知识点：10.3 封锁机制——封锁粒度（表级、行级、页面级等）的层次与多粒度封锁；封锁粒度与并发度、系统管理开销三者之间的权衡关系（Trade-off）；意向锁（IS、IX、SIX）在多粒度封锁中的作用。

8. 设关系模式 $R(U, F)$ ，其中 $U = A, B, C$ ， $F = A \rightarrow B, B \rightarrow C$ ，则候选码为 A 。

参考答案

答案：✓（正确）

解析与知识点

属性闭包的计算算法与候选码的判定方法（闭包判定法、属性分类法）。

对于给定的函数依赖集 $F = \{A \rightarrow B, B \rightarrow C\}$ ，求属性 A 的闭包 A^+ ：初始 $A^+ = \{A\}$ ；由 $A \rightarrow B$ ，将 B 加入闭包，得 $\{A, B\}$ ；再由 $B \rightarrow C$ ，将 C 加入闭包，得 $\{A, B, C\} = U$ 。 A 的闭包等于全部属性集 U ，说明 A 是候选码。再验证其他属性： $B^+ = \{B, C\} \neq U$ ， $C^+ = \{C\} \neq U$ ，因此 $\{A\}$ 是唯一的候选码。也可

通过属性分类法辅助判定： A 只出现在函数依赖的左侧，属于 L 类属性，根据候选码求解定理， L 类属性必定属于任何候选码。本题的结论正确。

属性闭包计算的步骤是：从初始属性集出发，反复应用函数依赖集将可推导的新属性加入闭包，直到闭包不再增长。若 $X^+ = U$ ，则 X 是候选码。属性分类法将属性分为 L 类（只出现在左侧）、 R 类（只出现在右侧）、 LR 类（两侧均出现）和 N 类（两侧均不出现），其中 L 类属性必定属于候选码。

常见错误是在求出 A 是候选码后停止，未验证是否存在其他候选码（如 AB 等组合），导致遗漏多候选码的情况。另一个易错点是闭包计算不彻底，未反复应用所有可用的函数依赖。

知识点：6.1 函数依赖——属性闭包的计算算法（逐步应用函数依赖扩张闭包集）；候选码的闭包判定法（ $X^+ = U$ 则 X 是候选码）； L 类/ R 类/ LR 类属性分类法在候选码求解中的应用。

9. 视图是一个虚表，其对应的数据并非独立存储于数据库中。

参考答案

答案：✓（正确）

解析与知识点

视图（View）的本质——虚表（Virtual Table）的概念及其与基本表（Base Table）的区别。

视图的本质是一个虚表（Virtual Table），与基本表不同——视图对应的数据并不独立存储在数据库中，而是每次访问视图时，DBMS 根据视图的定义（即视图对应的 SELECT 查询语句）从底层的基本表中实时查询、计算并呈现给用户。视图的定义（即视图名称和对应的查询表达式）存储在数据字典（Data Dictionary）中，但视图本身不占用额外的数据存储空间。对视图的查询操作会通过查询重写（Query Rewriting）或查询修改（Query Modification）技术转化为对底层基本表的查询。本题的表述符合标准教材中对视图的定义，完全正确。

值得注意的是，有一种特殊类型——物化视图（Materialized View），它的查询结果确实被物理存储。但在标准的数据库教材语境中，「视图」通常指普通视图（非物化的），即虚表，这是数据库考试中的默认前提。

常见错误是混淆普通视图与物化视图——普通视图不存储数据（虚表），物化视图存储数据的物理副本。另一个易错点是误以为视图的定义和数据都不存储——实际上视图定义是存储于数据字典中的。

知识点：3.6 视图——虚表的概念与实现机制（定义存于数据字典，数据实时从基本表导出）；视图与基本表的本质区别；查询重写（Query Rewriting）技术；物

化视图 (Materialized View) 与普通视图的区别。

10. 两段锁协议 (2PL) 可以完全避免死锁的发生。

参考答案

答案: × (错误)

解析与知识点

两段锁协议 (2PL) 与死锁的关系——2PL 保证冲突可串行化但不能避免死锁。2PL 协议保证的是调度的冲突可串行化 (Conflict Serializable) ——即遵守 2PL 的任何调度必定等价于某个串行调度。但它不能避免死锁。死锁是两个或多个事务相互等待对方释放锁而形成的循环等待状态。即使在严格执行 2PL 的环境中, 死锁仍然完全可能发生。典型例子: 事务 T1 持有数据对象 A 上的 S 锁, 等待获取 B 上的 X 锁; 事务 T2 持有数据对象 B 上的 S 锁, 等待获取 A 上的 X 锁。此时 T1 等待 T2 释放 B 上的 S 锁, T2 等待 T1 释放 A 上的 S 锁, 形成循环等待——死锁。这两个事务都遵守了 2PL (加锁阶段在前, 解锁阶段在后), 但依然陷入了死锁。因此本题表述错误。

死锁的解决需要专门的机制: 预防方法包括一次封锁法 (破坏保持和等待条件) 和顺序封锁法 (破坏循环等待条件); 检测方法包括等待图 (Wait-for Graph) 检测和超时 (Timeout) 机制, 发现死锁后选择一个事务进行回滚以打破循环等待。

常见错误是将 2PL 与死锁预防混淆, 认为 2PL 自身就能避免死锁。实际上 2PL 解决的是调度的正确性 (可串行化) 问题, 而死锁是资源分配的僵局问题, 两者处于不同的抽象层面。避免死锁需要额外的协议 (如一次封锁法或顺序封锁法), 而非仅靠 2PL。

知识点: 10.3 封锁机制——两段锁协议 (2PL) 保证冲突可串行化但不保证无死锁的严格区分; 死锁的四个必要条件 (互斥、保持并等待、不可抢占、循环等待); 死锁的预防策略 (一次封锁法、顺序封锁法) 和检测恢复策略 (等待图检测、超时回滚)。

9.3 三、综合题

1. 关系模式规范化 (本小题 16 分)

设有关系模式 $R(I, S, B, O, Q, D)$, 属性依次为课程编号、学生学号、教材编号、教师工号、上课地点、所属院系。已知函数依赖集 $F = \{S \rightarrow D, I \rightarrow B, IS \rightarrow Q, B \rightarrow O\}$ 。请完成以下任务。

(1) 候选码求解

参考答案

答案：候选码为 $\{IS\}$ 。

解析与知识点

属性分类法（L 类/R 类/LR 类）与属性闭包计算在候选码求解中的应用，这是规范化理论的起点。

Step 1: 属性分类。遍历 F 中每个函数依赖的左部和右部：

- $S \rightarrow D$: S 出现在左侧, D 出现在右侧
- $I \rightarrow B$: I 出现在左侧, B 出现在右侧
- $IS \rightarrow Q$: I, S 出现在左侧, Q 出现在右侧
- $B \rightarrow O$: B 出现在左侧, O 出现在右侧

分类结果：L 类（只出现在左侧）： I, S ；R 类（只出现在右侧）： Q, D, O ；LR 类（左右均出现）： B 。

由于 L 类属性必属于任何候选码，我们从 $\{I, S\}$ 开始求闭包：

$$\begin{aligned}
 (IS)_F^+ &= \{I, S\} \\
 &\xrightarrow{S \rightarrow D} \{I, S, D\} \\
 &\xrightarrow{I \rightarrow B} \{I, S, D, B\} \\
 &\xrightarrow{B \rightarrow O} \{I, S, D, B, O\} \\
 &\xrightarrow{IS \rightarrow Q} \{I, S, D, B, O, Q\} = U
 \end{aligned}$$

$(IS)^+$ 等于全体属性集 U ，故 $\{IS\}$ 为候选码。再验证是否唯一：

$$I^+ = \{I, B, O\} \neq U, \quad S^+ = \{S, D\} \neq U$$

I 和 S 单独均不能决定全体属性，故候选码唯一，为 $\{IS\}$ 。

属性分类法：L 类（仅出现在 FD 左侧）属性必定属于候选码，R 类（仅出现在 FD 右侧）属性必定不属于候选码，LR 类（左右均出现）属性需要进一步判定。这种分类策略显著缩小了候选码的搜索空间，无需遍历所有可能的属性组合。

常见错误有三：一是属性分类错误，尤其是 LR 类属性的识别（如本题的 B 同时出现在 $I \rightarrow B$ 左侧和 $B \rightarrow O$ 右侧）；二是闭包计算不完整，遗漏某些可应用的函数依赖；三是求出候选码后未验证唯一性。

知识点：6.1 函数依赖与属性闭包（闭包计算算法）；属性分类法（L 类必属候选码、R 类必不属于候选码、LR 类需进一步判定）；候选码的唯一性验证。

(2) 求最小函数依赖集 $F_{\min}$

参考答案

答案: $F_{\min} = \{S \rightarrow D, I \rightarrow B, IS \rightarrow Q, B \rightarrow O\}$ 。

解析与知识点

最小函数依赖集 (Minimal Cover / Canonical Cover) 的求解算法——右部单属性化、消除左部冗余属性、消除冗余函数依赖三步流程。

Step 1: 右部单属性化。 F 中各函数依赖的右部均为单属性, 已满足要求。

Step 2: 消除左部冗余属性。逐一检查每个函数依赖的左部:

- $S \rightarrow D$: 左部单属性, 无需约简。
- $I \rightarrow B$: 左部单属性, 无需约简。
- $B \rightarrow O$: 左部单属性, 无需约简。
- $IS \rightarrow Q$: 左部为 $\{I, S\}$ 。若去掉 I , 求 $S^+ = \{S, D\}$, $Q \notin S^+$, 故 I 不多余; 若去掉 S , 求 $I^+ = \{I, B, O\}$, $Q \notin I^+$, 故 S 也不多余。因此 $IS \rightarrow Q$ 的左部已最小。

Step 3: 消除冗余函数依赖。逐一尝试删去各依赖:

- 删除 $S \rightarrow D$: $S^+ = \{S\}$, $D \notin S^+$, 必须保留。
- 删除 $I \rightarrow B$: $I^+ = \{I\}$, $B \notin I^+$, 必须保留。
- 删除 $IS \rightarrow Q$: $(IS)^+ = \{I, S, B, D, O\}$, $Q \notin (IS)^+$, 必须保留。
- 删除 $B \rightarrow O$: $B^+ = \{B\}$, $O \notin B^+$, 必须保留。

所有依赖均不可删除。综上: $F_{\min} = \{S \rightarrow D, I \rightarrow B, IS \rightarrow Q, B \rightarrow O\}$ 。

最小函数依赖集是 3NF 分解算法的输入。其求解三个步骤依次执行, 不可互换顺序: (1) 右部单属性化——将形如 $X \rightarrow YZ$ 的依赖拆分为 $X \rightarrow Y$ 和 $X \rightarrow Z$; (2) 消除左部冗余属性——对每个多属性左部的 FD, 逐一尝试删除一个属性, 用 $X \setminus \{A_i\}$ 的闭包是否包含右部来判断; (3) 消除冗余 FD——逐一尝试删除整个 FD, 如果剩余 FD 集仍能推导出被删除的 FD, 则该 FD 冗余。本题的 F 本身已经是 最小覆盖, 无需任何约简操作。

常见错误有三: 一是三步操作的顺序不可互换 (如先删 FD 后去左部冗余可能导致不同结果); 二是在消除左部冗余后忘记重新检查冗余 FD (因为左部约简可能改变闭包结果); 三是漏掉多属性左部的逐一检查。

知识点: 6.2 最小函数依赖集 (Minimal Cover / Canonical Cover) 的求解算法——右部单属性化、消除左部冗余属性、消除冗余函数依赖三步流程; 约简操作的不可互换性。

(3) 判断分解 $\rho = \{SD, IB, ISQ, BO\}$ 的无损连接性

参考答案

答案：该分解具有无损连接性。

解析与知识点

Chase 算法（追踪法 / 表格法）判断分解的无损连接性。

使用 Chase 算法判断。将 6 个属性 B, O, I, S, Q, D 作为列标，4 个子模式 SD, IB, ISQ, BO 作为行标，构造初始矩阵。若属性属于子模式则填 a_j ，否则填 b_{ij} 。

初始矩阵：

	B	O	I	S	Q	D
SD	b_{11}	b_{12}	b_{13}	a_4	b_{15}	a_6
IB	a_1	b_{22}	a_3	b_{24}	b_{25}	b_{26}
ISQ	b_{31}	b_{32}	a_3	a_4	a_5	b_{36}
BO	a_1	a_2	b_{43}	b_{44}	b_{45}	b_{46}

根据 F 中的函数依赖依次修改矩阵：(1) 利用 $S \rightarrow D$ ：SD 行 $S = a_4$, $D = a_6$ ；ISQ 行 $S = a_4$ ，将 b_{36} 改为 a_6 。(2) 利用 $I \rightarrow B$ ：ISQ 行 $I = a_3$, $B = b_{31}$ ，将 b_{31} 改为 a_1 。(3) 利用 $B \rightarrow O$ ：IB 行 $B = a_1$ ，将 b_{22} 改为 a_2 ；ISQ 行 B 已变 a_1 ，将 b_{32} 改为 a_2 。(4) 利用 $IS \rightarrow Q$ ：ISQ 行中已满足，无需修改。

修改后的矩阵：

	B	O	I	S	Q	D
SD	b_{11}	b_{12}	b_{13}	a_4	b_{15}	a_6
IB	a_1	a_2	a_3	b_{24}	b_{25}	b_{26}
ISQ	a_1	a_2	a_3	a_4	a_5	a_6
BO	a_1	a_2	b_{43}	b_{44}	b_{45}	b_{46}

检查结果：ISQ 行全部变为 $a_1, a_2, a_3, a_4, a_5, a_6$ （全 a 行）。根据 Chase 算法，若最终矩阵中存在一行为全 a ，则原分解是无损连接的。因此，分解 $\rho = \{SD, IB, ISQ, BO\}$ 具有无损连接性。

Chase 算法的核心规则：若左部属性值在若干行中相同，则这些行的右部属性值也必须相同（符号等同化）。终止性由符号的有限性保证——每次等同化减少不同符号数量，初始符号至多有行数 $\times$ 列数个。ISQ 行率先变为全 a 行，是因为该子模式包含了最多的属性且 IS 恰好是候选码，通过 $I \rightarrow B$ 、 $B \rightarrow O$ 、 $S \rightarrow D$ 的传递依赖链条，将 B 、 O 、 D 三列的 b 符号全部转换为 a 符号。

常见错误有三：一是 Chase 算法中符号等同化规则理解错误——必须左部属性对应的所有列取值都相同，才能将右部符号统一；二是修改矩阵后忘记用后续的 FD

再次检查；三是忽略 Chase 算法的迭代性质——修改可能产生新的等同机会，需要反复应用所有 FD 直到无法再修改。

知识点：6.3 无损连接分解——Chase 算法（追踪法 / 表格法）的完整操作流程；符号等同化规则（左部相同右部统一）；全 a 行判定准则；Chase 算法的终止性与正确性分析。

(4) 保持函数依赖的 3NF 分解

参考答案

答案： $\rho_{3NF} = \{SD, IB, ISQ, BO\}$ 。

解析与知识点

保持函数依赖的 3NF 分解算法——按最小依赖集逐 FD 构造子模式，并确保候选码覆盖以实现无损连接性。

算法步骤：对 $F_{\min}$ 中的每一个函数依赖 $X \rightarrow Y$ ，构造关系模式 XY 。由 $F_{\min} = \{S \rightarrow D, I \rightarrow B, IS \rightarrow Q, B \rightarrow O\}$ ，分别构造：

- $S \rightarrow D \Rightarrow SD(S, D)$ ，码为 S
- $I \rightarrow B \Rightarrow IB(I, B)$ ，码为 I
- $IS \rightarrow Q \Rightarrow ISQ(I, S, Q)$ ，码为 IS
- $B \rightarrow O \Rightarrow BO(B, O)$ ，码为 B

由于候选码 $\{IS\}$ 已包含在 ISQ 模式中，无需额外添加候选码对应的模式。验证：每个子模式都在 3NF 中（各依赖的决定因素即为该子模式的码），且 $F_{\min}$ 中的每个函数依赖都在对应的子模式中得到保持。同时，由（3）已证明该分解具有无损连接性。因此结果为：

$$\rho_{3NF} = \{SD, IB, ISQ, BO\}$$

3NF 分解算法的核心设计目标是在保持函数依赖（确保所有 FD 均可在某个子模式中验证）和保证无损连接性（通过确保候选码被子模式覆盖）之间取得平衡。与 BCNF 分解不同，3NF 分解可以同时满足这两个目标——而 BCNF 分解在消除所有非超码决定因素的过程中，某些函数依赖的属性可能分散到不同子模式中，丧失了依赖保持性。本题的分解结果恰好也满足 BCNF，但这种「巧合」并非常态。常见错误有二：一是忘记在 3NF 分解结果中检查候选码是否已被某子模式覆盖——若候选码未包含在任何子模式中，需额外添加候选码对应的关系模式；二是混淆 3NF 分解与 BCNF 分解的算法和设计目标——3NF 追求依赖保持 + 无损连接，BCNF 追求更高范式但可能丧失依赖保持性。

知识点： 6.4 保持函数依赖的 3NF 分解算法（按最小依赖集逐 FD 构造子模式、确保候选码覆盖）；3NF 分解同时满足无损连接性与依赖保持性的双重目标；3NF 分解与 BCNF 分解的权衡（依赖保持性 vs. 更高范式等级）。

2. 触发器设计（不考）

注意

说明：根据参考答案的标注，本小题考查触发器相关内容，但在 2022–2023 学年及后续考试中已不再考查此题型，故本题略去。

3. 并发控制与封锁（本小题 12 分）

设有两个事务 T_1 和 T_2 ， T_1 ：读取 A 和 B，计算 $B = A - B$ ，将结果写回 B。 T_2 ：读取 A 和 B，计算 $A = A + B$ ，将结果写回 A。A 和 B 的初值均为正数。

(1) 给出一个满足两段锁协议（2PL）的调度，并说明其执行的等价串行顺序。

参考答案

调度序列如下（时间自上而下）：

时间	T_1	T_2
t_1	SLOCK(A)	
t_2	XLOCK(B)	
t_3	R(A)	SLOCK(B) （等待， T_1 持有 X(B)）
t_4	R(B)	
t_5	$B = A - B$	
t_6	W(B)	
t_7	UNLOCK(A)	
t_8	UNLOCK(B)	(T_2 获得 SLOCK(B))
t_9		XLOCK(A)
t_{10}		R(B)
t_{11}		R(A)
t_{12}		$A = A + B$
t_{13}		W(A)

解析与知识点

两段锁协议（2PL）的实际应用验证、S 锁与 X 锁的相容矩阵、以及 2PL 与冲突可串行化的关系。

该调度满足 2PL 协议的验证： T_1 的加锁顺序为 SLOCK(A) $\rightarrow$ XLOCK(B)，解锁阶段为 UNLOCK(A) $\rightarrow$ UNLOCK(B)。 T_1 在 t_7 释放第一个锁 (A) 之后，没有再申请任何新锁，严格遵守了 2PL 的「在释放任何一个锁之后不能再申请新锁」的规则。 T_2 在 t_3 时尝试获取 SLOCK(B)，但因 T_1 持有 X(B) 而阻塞 (S 锁与 X 锁不兼容)，直到 t_8 时刻 T_1 释放 UNLOCK(B) 后才获得 B 上的 S 锁。 T_2 随后申请 XLOCK(A)，获得所需全部锁后执行读写操作，加锁全部在释放任何锁之前完成，也满足 2PL。该调度等价于串行顺序 $T_1 \rightarrow T_2$ ，是冲突可串行化的。

2PL 协议之所以能保证冲突可串行化，本质上是因为它强制所有事务遵循一个全局的加锁-解锁顺序约束，使得并发访问等价于某种串行顺序。在本调度中， T_1 先完成全部操作后 T_2 才开始实质运算，相当于以封锁为手段自然地串行化了。另外注意锁类型的按需选择原则： T_1 对 A 使用了共享锁 S (因为只读 A)，对 B 使用了排他锁 X (因为要写 B) ——只读操作使用 S 锁以允许并发读，写操作使用 X 锁以保护数据一致性。

常见错误是在设计 2PL 调度时忘记「释放任何一个锁之后不能再加新锁」的规则，或在验证 2PL 时将阻塞等待误认为不满足 2PL (阻塞只影响事务进度，不影响是否遵循 2PL 的加锁/解锁阶段分离规则)。

知识点：10.2 并发控制与调度——冲突可串行化的定义与判定；10.3 两段锁协议 (2PL) 的形式化定义 (加锁阶段 + 解锁阶段、释放锁后不可再加锁)；S 锁与 X 锁的相容矩阵 (S-S 兼容、S-X 不兼容、X-X 不兼容) 及其在实际调度中的应用；锁类型的按需选择原则 (读用 S、写用 X)。

(2) 这两个事务的并发执行是否可能引起死锁？如果可能，请给出一个导致死锁的调度序列并分析死锁形成的过程。

参考答案

可能引起死锁。以下给出一种导致死锁的调度序列：

时间	T_1	T_2
t_1	SLOCK(A) (获取 S(A))	
t_2		SLOCK(B) (获取 S(B))
t_3	XLOCK(B) (等待, T_2 持有 S(B))	
t_4		SLOCK(A) (获取 S(A))
t_5		R(B)
t_6		R(A)
t_7		$A = A + B$
t_8		XLOCK(A) (等待, T_1 持有 S(A))

此时 T_1 等待 T_2 释放 S(B)， T_2 等待 T_1 释放 S(A)，形成死锁！

解析与知识点

死锁的形成机制（逐步分析循环等待条件的触发过程）、锁升级操作的死锁风险、以及死锁与 2PL 的关系。

死锁的形成机制（逐步分析）：(1) 在 t_2 时刻， T_2 获得了 SLOCK(B)。 T_1 此时仅持有 A 的 S 锁，并未锁定 B，所以 T_2 可以成功获取 B 的 S 锁。(2) 在 t_3 时刻， T_1 试图申请 XLOCK(B)，但 T_2 已持有 B 上的 S 锁。在标准封锁相容矩阵中，S 锁与 X 锁不兼容， T_1 进入等待状态。(3) 在 t_4 时刻， T_2 申请 SLOCK(A)。 T_1 持有 A 上的 S 锁，但 S 锁与 S 锁完全兼容，所以 T_2 成功获得 SLOCK(A)。 T_2 随后执行读操作和计算。(4) 到 t_8 时刻， T_2 需要将 A 上的 S 锁升级为 X 锁以写入数据 (XLOCK(A))。但 T_1 仍然持有 A 的 S 锁，S 锁与 X 锁不兼容， T_2 也进入等待状态。(5) 此时形成了循环等待： T_1 等待 T_2 释放 B 上的 S 锁，而 T_2 等待 T_1 释放 A 上的 S 锁，双方都在等待对方释放资源，死锁形成。

该例中的锁升级操作 (T_2 试图将 S 锁升级为 X 锁) 是死锁的常见诱因。死锁与 2PL 的关系：即使严格遵循 2PL，死锁仍可能发生——2PL 保证的是可串行化，而非无死锁。死锁需通过专门机制来发现和解除：预防方法包括一次封锁法和顺序封锁法；检测恢复方法包括等待图法（周期性检测等待关系的有向图是否有环）和超时法（事务等待超过设定时间则自动回滚），一旦发现死锁就选择一个事务作为「牺牲者」回滚以打破循环等待。

常见错误有二：一是认为 2PL 能够避免死锁——实际上 2PL 与死锁是两个不同层面的问题，2PL 保证调度正确性（可串行化），而死锁是资源分配僵局；二是混淆锁类型的相容规则，在设计死锁场景时错误地认为 S-S 不兼容或 S-X 兼容。

知识点：10.2 并发控制与调度；10.3 两段锁协议 (2PL) 的定义与性质（保证冲突可串行化，但不保证无死锁）；10.4 死锁的成因分析——循环等待条件的触发过程；S 锁与 X 锁的相容矩阵在死锁场景中的应用；锁升级操作 ($S \rightarrow X$) 的死锁风险；死锁的预防策略（一次封锁法、顺序封锁法）与检测恢复策略（等待图检测、超时回滚、牺牲者选择）。

第二部分 期中试卷解析

10 2008—2009 学年《数据库原理》期中试卷

试卷概述

本试卷共有四道大题，满分 100 分，涵盖关系代数与 SQL 查询 (40 分)、关系模式范式判定与规范化 (36 分)、模式分解 (24 分)。考查内容集中在关系数据理论与 SQL 语言两大核心模块。

10.1 一、关系代数与 SQL 查询 (40 分)

已知某供应数据库包含四个基本表：

- 供应商表 S(SNO, SNAME, CITY) ——SNO: 供应商号, SNAME: 供应商名, CITY: 所在城市
- 工程表 J(JNO, JNAME, CITY) ——JNO: 工程号, JNAME: 工程名, CITY: 所在城市
- 零件表 PART(PNO, PNAME, COLOR, WEIGHT) ——PNO: 零件号, PNAME: 零件名, COLOR: 颜色, WEIGHT: 重量
- 供货表 SPJ(SNO, PNO, JNO, QTY) ——SNO: 供应商号, PNO: 零件号, JNO: 工程号, QTY: 供应数量

1. 用关系代数表示下列查询

(1) 查询不供应 P2 零件的供应商号和供应商名。

参考答案

$$\pi_{SNO, SNAME}(S) - \pi_{SNO, SNAME}(\sigma_{PNO='P2'}(S \bowtie SPJ))$$

思路：先从全量供应商中减去供应过 P2 零件的供应商。先用自然连接获得供应记录，再用选择筛选出 PNO=“P2”的行，投影出供应商号和供应商名构成“供应过 P2 的供应商集合”，最后用差运算从全体供应商中排除这些供应商。

解析与知识点

关系代数的差运算与嵌套表达——「否定」类查询的标准模式（全集减去条件集）。先将「供应了 P2」的供应商找出来，再用全集减去它，得到「没有供应 P2」的供应商。自然连接 $S \bowtie SPJ$ 按同名属性 SNO 做等值连接， $\sigma_{PNO='P2'}$ 筛选出 P2 零件的供应记录，外层的 π 提取 SNO 和 SNAME。注意：差运算要求两个集合的属性列相同，这里两边的投影列一致，满足条件。

此类「否定」查询的标准模式是全集减去满足条件的集合。全集通过投影得到 $(\pi_{SNO, SNAME}(S))$ ，条件集合通过连接 + 选择 + 投影的组合定位。必须注意参与差运算的两边必须「并兼容」(Union-Compatible)，即具有相同的属性个数和对应的域——这正是为何两边都要投影到相同的 $\{SNO, SNAME\}$ 属性组上的原因。若 SPJ 表中完全没有 P2 零件的供应记录，内层选择结果为空，差运算的结果就是全体供应商，语义仍然正确。

常见错误是忘记差运算的并兼容性条件，导致两边属性列不一致（如一边有三列、另一边有两列）。另一个易错点是误用选择直接接在 S 表上——否定条件跨越两个表时不能直接在单表上筛选，必须通过连接引入 SPJ 的信息。

知识点：2.2 关系代数——差运算与嵌套表达（全集减条件集模式）；2.4 关系代数——自然连接（ $\bowtie$ ）与选择运算（ σ ）的组合使用；并兼容性条件；空集参与的差运算语义。

(2) 查询为所有工程供货的供应商名。

参考答案

$$\pi_{\text{SNAME}}\left(S \bowtie \left(\pi_{\text{SNO}, \text{JNO}}(\text{SPJ}) \div \pi_{\text{JNO}}(J)\right)\right)$$

思路：内层用除运算 $\div$ 求出“为所有工程供过货的供应商号”： $\pi_{\text{SNO}, \text{JNO}}(\text{SPJ})$ 给出每个供应商供应过哪些工程，除以全体工程集合 $\pi_{\text{JNO}}(J)$ ，结果就是供应范围覆盖全部工程的供应商号。再与 S 自然连接获得供应商名，最后投影出 SNAME。

解析与知识点

关系代数中除运算 $\div$ 的定义、语义及其与基本运算的等价关系——解决「全部」语义的核心算子。

设 $R(A, B) \div S(B)$ ，结果是那些在 R 中的 A 值，这些 A 值对应的 B 值集合包含 $S(B)$ 的全部元素。本题中 $R = \pi_{\text{SNO}, \text{JNO}}(\text{SPJ})$ 记录了「谁供应了哪些工程」， $S = \pi_{\text{JNO}}(J)$ 是全体工程号，除运算结果即为供应了全部工程的供应商号。最后通过自然连接补充供应商名信息。

除运算的严格定义： $R \div S = \{t[A] \mid t \in R \wedge \forall u \in S, (t[A], u) \in R\}$ ，即结果中的 A 值必须使得对于 S 中的每一个 B 值，组合 (A, B) 都在 R 中出现。除运算不是基本运算，而是由基本运算导出的——等价表达式为 $\pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$ 。在实际 SQL 中没有直接的除运算符，通常使用双重 NOT EXISTS 来实现全称量词语义。

常见错误是将除运算与自然连接混淆，误以为除运算是「包含全部相同值」的连接。另一个易错点是记错除运算等价表达式的结构——减号内外括号层级复杂，需要仔细区分每一步操作的含义。

知识点：2.4 关系代数——除运算 (Division) 的形式化定义 ($R \div S = \{t[A] \mid \forall u \in S, (t[A], u) \in R\}$)；除运算与基本运算的等价关系；除运算在 SQL 中的实现（双重 NOT EXISTS）；自然连接作为属性扩充手段。

(3) 查询至少为工程 J1 和 J2 供货的供应商号。

参考答案

方法一（自连接）：

$$\pi_{\text{SNO}}(\sigma_{[1]=[5] \wedge [3]='J1' \wedge [7]='J2'}(SPJ \times SPJ))$$

其中 $[1] = [5]$ 表示第一条记录的 SNO 等于第二条记录的 SNO（同一供应商）， $[3] = 'J1'$ 与 $[7] = 'J2'$ 分别表示第一条记录对应工程 J1、第二条对应工程 J2。

方法二（除运算）：先构造仅含 $\{J1, J2\}$ 的临时关系 $T = \{\langle J1 \rangle, \langle J2 \rangle\}$ ，则：

$$\pi_{\text{SNO}, \text{JNO}}(SPJ) \div T$$

解析与知识点

关系代数中的自连接（Self-Join）技术与除运算的子集覆盖语义辨析。

自连接方法的核心是笛卡尔积 + 条件筛选。将 SPJ 与自身做笛卡尔积，每条结果记录包含两份供应信息。 $\sigma_{[1]=[5]}$ 确保两份信息来自同一供应商， $\sigma_{[3]='J1' \wedge [7]='J2'}$ 确保该供应商同时有供应 J1 和 J2 的记录。方法二更简洁：构造只包含 J1、J2 的临时关系，用除运算直接求出覆盖 $\{J1, J2\}$ 的所有 SNO。两种方法等价。

自连接的关键技巧是为同一表取不同的别名（或通过笛卡尔积的列位置编号 $[i]$ 来引用），以便在条件表达式中区分两条记录的属性。本题中 $[1] = [5]$ 对应 $\text{SNO}=\text{SNO}$ ，因为 SPJ 表有 4 个属性，第二条记录的 SNO 出现在位置 5。注意「至少为 J1 和 J2 供货」不等同于「为全部工程供货」——前者只需覆盖指定子集 $\{J1, J2\}$ ，后者需覆盖整个 J 表，这也正是方法二中 T 仅包含 J1、J2 而非整个 $\pi_{\text{JNO}}(J)$ 的原因。

常见错误有三：一是将「至少为指定集合供货」与「为全部工程供货」混淆，对应选择的除运算右部不同；二是自连接时忘记别名或列位置编号，导致条件表达式中无法区分两条记录；三是笛卡尔积后忘记关键的 $\text{SNO}=\text{SNO}$ 条件，导致结果包含两个不同供应商分别供应 J1 和 J2 的情况。

知识点：2.2 关系代数——笛卡尔积与自连接（自身笛卡尔积的别名/列号引用）；2.4 关系代数——除运算与子集覆盖语义的区别（至少某几个 vs. 全部）；改名运算（ ρ ）与列位置编号约定。

10.2 2. 用 SQL 语句完成下列查询

(1) 查询为工程 J1 供货且与该工程位于同一城市的供应商名。

参考答案

```
1 SELECT SNAME
2 FROM S, J, SPJ
3 WHERE S.SNO = SPJ.SNO
4       AND J.JNO = SPJ.JNO
5       AND SPJ.JNO = 'J1'
6       AND J.CITY = S.CITY;
```

解析与知识点

SQL 多表连接查询——三表连接的桥接模式，这是 SPJ 查询模式的典型代表。这是典型的三表连接查询。*S* 与 *SPJ* 通过 SNO 连接，*J* 与 *SPJ* 通过 JNO 连接。WHERE 子句中，*SPJ.JNO* = 'J1' 限定为工程 J1 的供应记录，*J.CITY* = *S.CITY* 实现「同一城市」条件。注意区分两个 CITY 属性：*S.CITY* 是供应商所在城市，*J.CITY* 是工程所在城市。

三表连接的连接图结构为 $S \leftrightarrow SPJ \leftrightarrow J$ ，SPJ 在此扮演了「桥接表」的角色——它通过外码 SNO 连接供应商表，通过外码 JNO 连接工程表，从而实现了供应商与工程之间的间接关联。这正是关系数据库中多对多联系的标准实现方式。连接条件 $S.SNO = SPJ.SNO$ 和 $J.JNO = SPJ.JNO$ 必须同时出现，如果遗漏任何一个连接条件，会产生笛卡尔积而导致巨大中间结果集，既不符合语义也会严重降低性能。

常见错误是遗漏某个连接条件（导致笛卡尔积），或在多表查询中混淆不同表的同名属性（如本题的 *S.CITY* 与 *J.CITY*），忘记使用表名前缀限定。

知识点：3.3 SQL——多表连接查询（三表连接的桥接模式）；3.2 SQL——SELECT-FROM-WHERE 基本结构与连接条件的必要性（遗漏连接 = 笛卡尔积）。

(2) 查询重量低于红色螺丝刀的零件号、零件名和颜色。

参考答案

方法一（子查询 + ALL）：

```
1 SELECT PNO, PNAME, COLOR
2 FROM PART
3 WHERE WEIGHT < ALL
4       (SELECT WEIGHT
5        FROM PART
6        WHERE PNAME = '螺丝刀' AND COLOR = '红');
```

方法二（子查询 + MIN）：

```
1 SELECT PNO, PNAME, COLOR
2 FROM PART
3 WHERE WEIGHT <
4     (SELECT MIN(WEIGHT)
5      FROM PART
6      WHERE PNAME = '螺丝刀' AND COLOR = '红');
```

解析与知识点

SQL 子查询中 ALL/ANY 运算符的语义及其与聚合函数（MIN/MAX）的逻辑等价关系。

此题的关键在于理解「低于红色螺丝刀」的含义。由于可能有多把红色螺丝刀（不同型号），需要确保目标零件的重量比所有红色螺丝刀都轻。方法一使用 $< ALL$ （子查询）精确表达这一语义。方法二使用 $< (SELECT MIN(WEIGHT) \dots)$ ，即只要比最轻的那把红色螺丝刀更轻即可，逻辑上等价。方法二的效率通常更高，因为只需计算一个标量值而非逐行比较集合。

ALL/ANY 与聚合函数的等价关系： $x < ALL(S) \iff x < \min(S)$, $x > ALL(S) \iff x > \max(S)$, $x < ANY(S) \iff x < \max(S)$ （以上均假设 S 非空）。DBMS 的查询优化器通常会自动将 $< ALL$ 转换为 $< (SELECT MIN(\dots))$ 。但需注意：当子查询结果为空时，ALL 比较返回 TRUE（空集全称量词为真），而 MIN 返回 NULL， $x < NULL$ 的结果是 UNKNOWN，两者语义出现分歧。

常见错误是混淆 ALL 与 ANY 的语义——ALL 要求比集合中全部元素都满足条件，ANY 只需存在至少一个满足即可。另一个易错点是忽略空子查询结果下 ALL 与 MIN 的语义分歧，导致边界情况的误判。

知识点：3.4 SQL——子查询与 ALL/ANY 运算符（全称量词 $\forall$ 的 SQL 表达）；3.5 SQL——聚合函数 MIN/MAX 与 ALL/ANY 的逻辑等价关系（ $x < ALL(S) \iff x < \min(S)$ ）；空子查询结果下 ALL 与聚合函数的语义分歧。

(3) 查询某工程至少用了 3 家供应商供应的零件的平均数量，并按工程号降序排列。

参考答案

```
1 SELECT JNO, AVG(QTY) AS 平均数量
2 FROM SPJ
3 GROUP BY JNO
4 HAVING COUNT(DISTINCT SNO) >= 3
```



```
5 ORDER BY JNO DESC;
```

解析与知识点

SQL 中 GROUP BY 分组 + HAVING 组级过滤的组合使用，以及 WHERE（行过滤）与 HAVING（组过滤）的本质区别。

GROUP BY JNO 将供应记录按工程号分组。COUNT(DISTINCT SNO) 统计每组中不同供应商的数量（注意 DISTINCT 不能省略，因为同一供应商可能多次为同一工程供应不同零件）。HAVING 子句过滤出至少涉及 3 家的组。AVG(QTY) 对每组计算平均供应数量。最后 ORDER BY JNO DESC 实现降序排列。

WHERE 与 HAVING 的执行时机区别：WHERE 在分组之前对原始行进行过滤，不能使用聚合函数；HAVING 在分组之后对组进行过滤，可以使用聚合函数。如果条件涉及原始行属性（如 QTY > 100），应在 WHERE 中使用；如果条件涉及组聚合值（如 COUNT >= 3），必须在 HAVING 中表达。ORDER BY 是 SQL 执行流程中的最后一步，多列排序时从左到右优先级递减。

常见错误有三：一是在 WHERE 子句中错误使用聚合函数（语法错误）；二是 COUNT(SNO) 忘记加 DISTINCT，导致同一供应商多次供应被重复计数；三是将组级过滤条件错误地放在 WHERE 子句中而非 HAVING 中。

知识点：3.5 SQL——GROUP BY 分组与 HAVING 组级过滤；3.6 SQL——聚合函数（AVG、COUNT）与 DISTINCT 的组合；ORDER BY 排序（ASC/DESC）；WHERE（行过滤）vs. HAVING（组过滤）的执行时机区别。

(4) 查询为所有工程供货的供应商名。

参考答案

使用双重 NOT EXISTS 实现全称量词：

```
1 SELECT SNAME
2 FROM S
3 WHERE NOT EXISTS
4     (SELECT *
5      FROM J
6      WHERE NOT EXISTS
7          (SELECT *
8           FROM SPJ
9           WHERE S.SNO = SPJ.SNO
10              AND J.JNO = SPJ.JNO));
```

解析与知识点

SQL 中双重 NOT EXISTS 实现全称量词（「全部」语义）的标准范式——一阶逻辑变换 $\forall x P(x) \iff \neg \exists x \neg P(x)$ 的 SQL 映射。

SQL 没有直接的除运算，但可通过双重 NOT EXISTS 将全称量词 $\forall$ 转换为双重否定 $\neg \exists \neg$ 。逻辑解读：「不存在这样一个工程，使得该供应商没有向它供过货」，等价于「该供应商向所有工程供过货」。最内层子查询检查供应商 S 与工程 J 之间是否存在供应记录，中间层 NOT EXISTS 检查是否存在缺失的工程，最外层 NOT EXISTS 确保不存在缺失。这是 SQL 中表达「全部」语义的标准范式。

从谓词逻辑的角度分析， $\forall x (P(x)) \iff \neg \exists x (\neg P(x))$ 是经典的一阶逻辑等价变换。SQL 中的三层结构恰好对应了这一逻辑嵌套：最外层 $\neg \exists$ （不存在工程），中间层 $\neg \exists$ （该供应商与工程间不存在供应记录），最内层直接检查供应记录的存在性。理解这种逻辑映射关系后，任何全称量词查询都可以用双重 NOT EXISTS 模板解决。

常见错误有三：一是混淆 EXISTS 与 NOT EXISTS 的层数，少写或多写一层 NOT；二是内层子查询忘记关联条件（如遗漏 $S.SNO = SPJ.SNO$ ），导致子查询变为非相关子查询而结果错误；三是在多重嵌套中混淆各层别名的作用域。

知识点：3.4 SQL——EXISTS/NOT EXISTS 相关子查询与全称量词表达（双重 NOT EXISTS 范式）；2.4 关系代数——除运算在 SQL 中的等价实现；一阶逻辑变换 $\forall x P(x) \iff \neg \exists x \neg P(x)$ 在数据库查询中的应用。

(5) 查询没有使用天津供应商供应的零件的工程号。

参考答案

方法一（NOT IN）：

```
1 SELECT JNO
2 FROM J
3 WHERE JNO NOT IN
4     (SELECT JNO
5      FROM S, SPJ
6      WHERE S.SNO = SPJ.SNO
7           AND S.CITY = '天津');
```

方法二（NOT EXISTS）：

```
1 SELECT JNO
2 FROM J
3 WHERE NOT EXISTS
4     (SELECT *
```

```
5 FROM S, SPJ
6 WHERE S.SNO = SPJ.SNO
7       AND J.JNO = SPJ.JNO
8       AND S.CITY = '天津');
```

解析与知识点

SQL 中 NOT IN 与 NOT EXISTS 的区别及其在否定查询中的应用，特别是 NOT IN 的 NULL 陷阱与三值逻辑。

方法一的思路是先找出所有「使用了天津供应商零件的工程号」，再用 NOT IN 从全体工程中排除。方法二使用相关子查询 NOT EXISTS，检查是否存在该工程使用天津供应商零件的记录。本题子查询结果不含 NULL，故两种方法均正确。

两方法的关键差异在于 NULL 处理。在 SQL 的三值逻辑中， $x \text{ NOT IN } (S) \iff \forall y \in S : x \neq y$ 。当 S 中包含 NULL 时， $x \neq \text{NULL}$ 的值为 UNKNOWN（而非 TRUE），最终 WHERE 过滤掉 UNKNOWN 行，导致 NOT IN 结果为空集。NOT EXISTS 的语义是「检查子查询是否返回至少一行」，子查询返回 NULL 行时仍然算作「存在一行」，因此不受 NULL 影响。生产环境建议优先使用 NOT EXISTS。此外，NOT IN 在语义上等价于 $\neq$ ALL。

常见错误是忽略 NOT IN 的 NULL 陷阱——当子查询结果集中可能包含 NULL 值时，NOT IN 会意外地返回空集而非预期的结果。另一个易错点是混淆 NOT IN 与 NOT EXISTS 的作用域——NOT IN 子查询为非相关子查询（只需执行一次），而 NOT EXISTS 通常为相关子查询（每行都需执行）。

知识点：3.4 SQL——NOT IN 与 NOT EXISTS 的区别及适用场景；3.2 SQL——子查询中的 NULL 处理与三值逻辑（TRUE/FALSE/UNKNOWN）；NOT IN 的 NULL 陷阱（ $x \notin \{\text{NULL}\} = \text{UNKNOWN}$ ）和工程安全建议。

10.3 二、关系模式 BCL 的规范化（18 分）

设有关系模式：

BCL(BNO, CITY, SSETS, CNO, NAME, ADDR, LNO, AMOUNT)

各属性依次为：支行号（BNO）、支行所在城市（CITY）、支行总资产（SSETS）、客户号（CNO）、客户名（NAME）、客户地址（ADDR）、贷款号（LNO）、贷款金额（AMOUNT）。

语义约束：

- 一个客户可贷多笔贷款，一笔贷款可由多个客户共同贷款；
- 贷款由各个支行发出，一笔贷款只能由一个支行发出；

- 贷款号在各支行内唯一（即同一支行不会发出重复的贷款号，但不同支行可能发出相同的贷款号）。

1. 写出关系 BCL 基本的函数依赖。

参考答案

$F = \{ \text{BNO} \rightarrow \text{CITY}, \text{BNO} \rightarrow \text{SSETS}, \text{CNO} \rightarrow \text{NAME}, \text{CNO} \rightarrow \text{ADDR}, (\text{BNO}, \text{CNO}, \text{LNO}) \rightarrow \text{AMOUNT} \}$

依赖分析：

- $\text{BNO} \rightarrow \text{CITY}$ ：支行号唯一决定支行所在城市。
- $\text{BNO} \rightarrow \text{SSETS}$ ：支行号唯一决定支行总资产。
- $\text{CNO} \rightarrow \text{NAME}$ ：客户号唯一决定客户名。
- $\text{CNO} \rightarrow \text{ADDR}$ ：客户号唯一决定客户地址。
- $(\text{BNO}, \text{CNO}, \text{LNO}) \rightarrow \text{AMOUNT}$ ：由支行号、客户号和贷款号共同确定一笔贷款金额。因为 LNO 仅在支行内唯一，不同支行可能有相同 LNO，所以 LNO 单独不能决定 AMOUNT，需要 BNO 和 LNO 联合；又因为一笔贷款可涉及多个客户，CNO 也需加入决定因素。

解析与知识点

函数依赖的语义分析方法——从业务语义和已知约束推导函数依赖集，这是规范化理论的起点与基础。

分析函数依赖需同时考虑：(1) 业务语义——属性之间的自然关系（如支行号决定其所在城市）；(2) 已知约束——贷款号仅在支行内唯一，因此 LNO 不能单独作为任何属性的决定因素，必须与 BNO 组合；(3) 唯一性—— $(\text{BNO}, \text{CNO}, \text{LNO})$ 联合才能唯一确定一笔贷款金额，因为同一贷款可有多个客户参与。注意： $\text{BNO} \rightarrow \text{CITY}$ 和 $\text{BNO} \rightarrow \text{SSETS}$ 是部分函数依赖，它们的存在将直接影响后续的范式判定。

常见的分析步骤：(1) 识别实体标识符——BNO 是支行的标识符、CNO 是客户的标识符，标识符通常函数决定该实体的所有描述属性；(2) 识别复合标识符——LNO 仅在 BNO 范围内唯一，因此 (BNO, LNO) 构成贷款的联合标识；(3) 识别多对多关系——一笔贷款涉及多个客户，金额 AMOUNT 必须由 $\text{BNO} + \text{CNO} + \text{LNO}$ 三者共同决定。这种由业务语义到函数依赖的映射能力是数据库设计者的核心素养。

常见错误有二：一是忽略 LNO 仅在支行内唯一的约束，误以为 LNO 单独即可决定 AMOUNT；二是遗漏 M:N 联系对复合决定因素的影响——由于一个客户可以参与多笔贷款，CNO 也必须参与决定 AMOUNT。

知识点：6.1 函数依赖的概念与语义分析方法（实体标识符规则、复合标识符规

则)；函数依赖的逻辑蕴含与闭包计算；由业务语义约束推导函数依赖集的标准步骤。

2. 判断关系 BCL 最高属于第几范式，并分析该模式存在的问题。

参考答案

Step 1: 求候选码。

从 F 出发求属性集的闭包。首先考察 (BNO, LNO, CNO) ：

- 初始：{BNO, LNO, CNO}
- 由 $BNO \rightarrow CITY$ 得 +CITY: {BNO, LNO, CNO, CITY}
- 由 $BNO \rightarrow SSETS$ 得 +SSETS: {BNO, LNO, CNO, CITY, SSETS}
- 由 $CNO \rightarrow NAME$ 得 +NAME: {BNO, LNO, CNO, CITY, SSETS, NAME}
- 由 $CNO \rightarrow ADDR$ 得 +ADDR: {BNO, LNO, CNO, CITY, SSETS, NAME, ADDR}
- 由 $(BNO, CNO, LNO) \rightarrow AMOUNT$ 得 +AMOUNT: 覆盖全部 8 个属性。

由于 BNO、LNO、CNO 三者均不在任何函数依赖的右部出现（即它们不属于任何非平凡函数依赖的右部），必须在候选码中。又因为三者闭包包含所有属性，候选码为 (BNO, LNO, CNO) 。

Step 2: 判定范式等级。

- 检查 2NF: 存在非主属性对候选码的部分函数依赖吗?
 - CITY 和 SSETS 仅由候选码的部分 (BNO) 决定: $BNO \rightarrow CITY$ 、 $BNO \rightarrow SSETS$ ，而 $BNO \subsetneq (BNO, LNO, CNO)$ 。
 - NAME 和 ADDR 仅由候选码的部分 (CNO) 决定: $CNO \rightarrow NAME$ 、 $CNO \rightarrow ADDR$ 。

存在非主属性对候选码的部分函数依赖，不满足 2NF。

- 由于不满足 2NF，BCL 的最高范式等级为 1NF。

Step 3: 该模式存在的异常问题。

- (1) 数据冗余严重。支行所在城市 (CITY) 和总资产 (SSETS) 对于该支行的每一笔贷款、每一个客户记录都要重复存储；客户名 (NAME) 和地址 (ADDR) 对于该客户的每一笔贷款也要重复存储。数据库膨胀，维护成本高。

- (2) 插入操作异常。候选码为 (BNO, LNO, CNO)，三个属性均为非空。当一家新支行成立但尚未有任何客户和贷款业务时，BNO 可以确定，但 LNO 和 CNO 无值可填（取决于是否允许 NULL），导致支行基本信息无法插入数据库。
- (3) 删除操作异常。当某支行所有客户归还全部贷款且暂无新贷款时，需要删除所有相关供应记录。如果该支行仅在这些记录中出现，删除后该支行的基本信息 (CITY、SSETS) 将一并丢失。
- (4) 更新操作异常。若某支行迁址 (CITY 变更)，需在所有包含该支行的记录中逐一更新 CITY 值，一旦遗漏即造成数据不一致。

解析与知识点

范式等级的逐级判定方法 ($1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$) 以及低范式导致的数据冗余与操作异常（插入、删除、更新、冗余）分析。

范式判定的关键是求候选码闭包，然后逐级检查：2NF 要求无非主属性对候选码的部分函数依赖，3NF 要求无非主属性对候选码的传递函数依赖，BCNF 要求每个非平凡函数依赖的左部都是超码。BCL 不符 2NF 的原因非常典型：BNO 和 CNO 各自独立决定了一批非主属性 (CITY、SSETS 和 NAME、ADDR)，而这些属性与候选码部分相关。该模式存在的异常包括：数据冗余严重（支行城市和资产重复存储）、插入异常（新支行无客户和贷款时无法插入）、删除异常（删除最后一笔贷款记录时支行信息丢失）、更新异常（支行迁址需逐一更新所有记录）。从 1NF 到 BCNF 的逻辑链条：(1) 1NF 约束数据结构的原子性；(2) 2NF 消除部分函数依赖；(3) 3NF 消除传递函数依赖；(4) BCNF 约束所有非平凡 FD 的左部均为超码。这四个等级的递进关系反映了规范化理论的设计哲学——通过逐步施加更强的约束条件消除数据依赖中的异常。BCL 的候选码 (BNO, LNO, CNO) 是三个独立实体标识符的拼接，这种「多实体主码拼接」的模式最容易产生部分函数依赖。

常见错误有二：一是范式判定时跳过 2NF 直接检查 3NF，忽略了部分依赖是影响 2NF 而非 3NF 的问题；二是在分析操作异常时只列举名称而不解释成因机制——插入/删除/更新异常的根源在于非主属性对候选码的部分依赖。

知识点：6.3 范式——1NF / 2NF / 3NF / BCNF 的定义与逐级判定方法（候选码闭包法 + 非主属性依赖类型检测）；6.4 数据冗余与操作异常分析（插入异常、删除异常、更新异常、数据冗余四种异常类型的成因与表现）；多实体主码拼接导致部分依赖的典型模式。

3. 试把 BCL 分解成 3NF 的模式集。

参考答案

按照 3NF 分解算法（先求最小函数依赖集，再按左部相同原则合并，最后确保包含候选码）：

F 本身已是最小函数依赖集（各函数依赖右部均为单属性，且左部无冗余属性），按左部相同原则分组：

- 左部为 $\{BNO\}$ 的函数依赖： $BNO \rightarrow CITY$ 、 $BNO \rightarrow SSETS$
- 左部为 $\{CNO\}$ 的函数依赖： $CNO \rightarrow NAME$ 、 $CNO \rightarrow ADDR$
- 左部为 $\{BNO, CNO, LNO\}$ 的函数依赖： $(BNO, CNO, LNO) \rightarrow AMOUNT$

将左部相同的函数依赖合并为一个关系模式：

$BCL_1(\underline{BNO}, CITY, SSETS)$
$BCL_2(\underline{CNO}, NAME, ADDR)$
$BCL_3(\underline{BNO}, LNO, \underline{CNO}, AMOUNT)$

分解后各模式的范式检查：

- BCL_1 ：候选码为 BNO ，唯一非平凡 FD 为 $BNO \rightarrow CITY, SSETS$ ，左部即码，满足 BCNF。
- BCL_2 ：候选码为 CNO ，唯一非平凡 FD 为 $CNO \rightarrow NAME, ADDR$ ，左部即码，满足 BCNF。
- BCL_3 ：候选码为 (BNO, CNO, LNO) ，唯一非平凡 FD 为 $(BNO, CNO, LNO) \rightarrow AMOUNT$ ，左部即码，满足 BCNF。

三个子模式均达到 BCNF（自然也满足 3NF），且 BCL_3 包含了原关系的候选码，具有无损连接性。所有函数依赖在各子模式中均有对应的关系，保持了依赖保持性。

解析与知识点

保持函数依赖的 3NF 分解算法三步流程——求最小函数依赖集、按左部相同原则分组、确保包含候选码。

3NF 分解的核心步骤：(1) 求最小函数依赖集 F_m ——本题 F 已经最小；(2) 按左部相同原则将 F_m 中的函数依赖分组，每组构成一个关系模式；(3) 检查是否有关系包含原关系的候选码，若没有则需额外添加候选码所在关系（本题 BCL_3 已包含）以确保无损连接性。分解后： BCL_1 管理支行静态信息， BCL_2 管理客户静态信息， BCL_3 管理贷款业务关系。

这种分解体现了「一事一地」的数据库设计原则（One Fact in One Place）：每个关系模式只描述一个主题（实体或联系），属性之间语义高度内聚。更新支行城市只需修改 BCL_1 中的一行，从根本上消除了部分依赖导致的数据冗余与操作异常。

值得指出的是，3NF 分解并不保证每个子模式达到 BCNF——本题分解结果恰好全部达到 BCNF，这归功于原 F 中每个 FD 的左部恰好是各子模式的码。但在一般情形下，3NF 分解算法只保证结果满足 3NF，如需 BCNF 需采用迭代二分法。常见错误有二：一是在 3NF 分解完成后忘记检查候选码是否已被某子模式覆盖——若候选码未覆盖，分解可能不满足无损连接性；二是混淆 3NF 分解与 BCNF 分解的算法与结果——3NF 分解不一定得到 BCNF。

知识点：6.5 模式分解算法——3NF 分解的三步流程（求最小函数依赖集 → 按左部相同原则分组 → 确保包含候选码）；「一事一地」设计原则（One Fact in One Place）；最小函数依赖集的判定条件；无损连接性与依赖保持性的验证方法。

10.4 三、判定关系模式的范式等级（18 分）

指出下列关系模式是第几范式，并说明理由。

1. $R(A, B, C), F = \{B \rightarrow C, AC \rightarrow B\}$

参考答案

Step 1: 求所有候选码。

- 检查 AB : $AB^+ = \{A, B\} \rightarrow B \rightarrow C \rightarrow \{A, B, C\}$, AB 是候选码。
- 检查 AC : $AC^+ = \{A, C\} \rightarrow AC \rightarrow B \rightarrow \{A, B, C\}$, AC 也是候选码。
- 单属性 A 、 B 、 C 的闭包均不覆盖全集，故候选码为 AB 和 AC 。

主属性集合: $\{A, B, C\}$ ——全部属性都是主属性。非主属性集合: $\emptyset$ ——没有非主属性。

Step 2: 范式判定。

- 2NF 要求无非主属性对候选码的部分依赖。无非主属性，自动满足 2NF。
- 3NF 要求无非主属性对候选码的传递依赖。无非主属性，自动满足 3NF。
- BCNF 要求每个非平凡函数依赖的左部都是超码。检查：

– $B \rightarrow C$: $B^+ = \{B, C\} \neq \{A, B, C\}$, B 不是超码，违反 BCNF。

结论: R 的最高范式为 3NF，不是 BCNF。

理由: 全主属性关系自动满足 3NF（因为 2NF 和 3NF 的定义仅约束非主属性对候选码的依赖关系，无非主属性则无违规），但 $B \rightarrow C$ 的左部 B 不是超码，违反 BCNF 条件，故不能达到 BCNF。

解析与知识点

3NF 与 BCNF 的核心区别——3NF 仅约束非主属性，BCNF 约束所有属性的函数依赖。全主属性关系的范式判定是经典高频考点。

这是「全主属性但不满足 BCNF」的经典例子。候选码为 AB 和 AC ，全部属性都是主属性，无非主属性，因此 2NF 和 3NF 自动满足。但 BCNF 要求每个非平凡函数依赖的左部都是超码： $B \rightarrow C$ 中 $B^+ = \{B, C\} \neq U$ ， B 不是超码，违反 BCNF。结论：最高范式为 3NF，不是 BCNF。

3NF 与 BCNF 的关键区别：当所有属性都是主属性时，3NF 的约束自动满足（因为无非主属性可违规），但 BCNF 仍要求每一个决定因素都是超码。本题中 $B \rightarrow C$ 虽然 B 不是超码，但由于 C 也是主属性（存在于候选码 AC 中），不影响 3NF 判定。这类关系模式在实践中通常需要进一步分解以消除由非超码决定因素引起的冗余，BCNF 分解可选择 $\{BC, AB\}$ 。

常见错误是将 3NF 的判定逻辑套用到 BCNF 上，认为「所有属性都是主属性，肯定达到 BCNF」。BCNF 不区分主属性和非主属性，它对每一个非平凡函数依赖平等对待——所有决定因素都必须是超码，即使右部也是主属性。

知识点：6.3 范式——3NF 与 BCNF 的核心区别（3NF 仅约束非主属性，BCNF 约束所有属性）；全主属性关系的范式判定陷阱；非超码决定因素检测的闭包法；BCNF 违规的检测算法。

2. $R(A, B, C)$, $F = \{B \rightarrow C, B \rightarrow A, A \rightarrow BC\}$

参考答案

Step 1: 分析函数依赖，化简 F 。

从 $A \rightarrow BC$ 可分解出 $A \rightarrow B$ 和 $A \rightarrow C$ 。又从 $B \rightarrow A$ 和 $A \rightarrow B$ 可知 A 与 B 相互决定： $A \leftrightarrow B$ 。

等价函数依赖集可写为： $\{A \rightarrow B, B \rightarrow A, A \rightarrow C, B \rightarrow C\}$ 。

Step 2: 求候选码。

- $A^+ = \{A, B, C\}$ ， A 是候选码。
- $B^+ = \{B, C, A\}$ ， B 也是候选码。
- C 不决定其他属性，不包含在候选码中。

候选码： A 和 B 。主属性： $\{A, B\}$ ，非主属性： $\{C\}$ 。

Step 3: 范式判定。

检查每个非平凡函数依赖的左部是否都是超码：

- $B \rightarrow C$: 左部 B 是候选码, 是超码。满足。
- $B \rightarrow A$: 左部 B 是候选码, 是超码。满足。
- $A \rightarrow BC$: 左部 A 是候选码, 是超码。满足。

所有非平凡函数依赖的左部都是超码, 满足 BCNF 定义。

因此 R 自动满足 3NF 和 2NF, 最高范式为 BCNF。

解析与知识点

函数依赖集的等价化简在范式判定中的作用——通过化简可能发现表面复杂的依赖集实际上满足 BCNF。

A 和 B 相互决定 ($A \leftrightarrow B$), 二者都是候选码。从 $A \rightarrow BC$ 分解出 $A \rightarrow B$ 和 $A \rightarrow C$, 又从 $B \rightarrow A$ 和 $A \rightarrow B$ 可得 $A \leftrightarrow B$ 。化简后每个 FD 的左部 (A 或 B) 的闭包都覆盖全部属性, 都是超码。所有非平凡 FD 的左部都是超码, 满足 BCNF。

这个例子展示了函数依赖集等价化简的重要性。原始 $F = \{B \rightarrow C, B \rightarrow A, A \rightarrow BC\}$ 看似有三个 FD, 但经过分解和传递规则化简后, 可以发现本质上只有两个独立的决定因素 A 和 B , 且它们互相等价。 $B \rightarrow C$ 中 B 是候选码 ($B^+ = \{B, A, C\} = U$), $A \rightarrow BC$ 中 A 也是候选码。这提醒我们: 在进行范式判定前, 应先对函数依赖集做等价化简, 可能会发现原本看似不符合 BCNF 的关系实际是充分规范化的。

常见错误是未对原始函数依赖集进行化简就匆匆判定 BCNF——在没有识别出 $A \leftrightarrow B$ 的等价关系前, $B \rightarrow C$ 可能被误判为非超码决定因素 (如果忘记 $B \rightarrow A$ 的存在)。

知识点: 6.3 范式——BCNF 判定 (所有决定因素均为超码) 的闭包验证法; 6.2 函数依赖集的等价化简 (分解规则、传递规则的应用); 属性间相互决定关系 ($A \leftrightarrow B$) 的识别与利用。

3. $R(A, B, C, D), F = \{B \rightarrow D, AB \rightarrow C\}$

参考答案

Step 1: 求候选码。

- $AB^+ = \{A, B\} \rightarrow B \rightarrow D \rightarrow \{A, B, D\} \rightarrow AB \rightarrow C \rightarrow \{A, B, C, D\}$ 。
- A 单独不能推出其他属性, B 单独不能推出 A 和 C 。
- 唯一候选码为 AB 。

主属性：{A, B}。非主属性：{C, D}。

Step 2：范式判定。

- 检查 2NF：是否存在非主属性对候选码的部分函数依赖？
 - $B \rightarrow D$ ：B 是候选码 AB 的真子集，且 D 是非主属性。存在非主属性 D 对候选码 AB 的部分函数依赖。

不满足 2NF。

- 由于不满足 2NF，也必然不满足 3NF 和 BCNF。

结论：R 的最高范式为 1NF。

理由：存在 $B \rightarrow D$ ，其中 $B \subsetneq AB$ （候选码的真子集决定非主属性），构成部分函数依赖，违反 2NF。R 符合 1NF（属性不可分），故最高为 1NF。

解析与知识点

部分函数依赖的形式化定义及其对 2NF 判定的影响——候选码的真子集决定非主属性即违反 2NF。

这是典型的因部分函数依赖导致达不到 2NF 的案例。(1) 求候选码：A 和 B 分别不决定全集，唯一候选码为 AB；(2) 主属性 {A, B}，非主属性 {C, D}；(3) 检查部分依赖： $B \rightarrow D$ 中 $B \subsetneq AB$ 且 D 是非主属性，存在非主属性对候选码的部分函数依赖，不满足 2NF。最高范式为 1NF。要将此模式提升至 2NF，需将 $B \rightarrow D$ 分离为独立关系 (B, D)，剩余部分为 (A, B, C)。

部分函数依赖的形式化定义：设有 $X \rightarrow Y$ ，若存在 $X' \subsetneq X$ 使得 $X' \rightarrow Y$ 也成立，则称 Y 部分函数依赖于 X。在范式判定中，关键是区分候选码和「候选码的真子集」：如果非主属性 Y 可以被候选码 K 的真子集 K' 函数决定，则 Y 对 K 是部分函数依赖。分解为 $R_1(B, D)$ 和 $R_2(A, B, C)$ 后， R_1 中 B 就是候选码（不再是真子集），部分依赖自然消除。

常见错误是混淆部分函数依赖与传递函数依赖——前者是候选码真子集直接决定非主属性，后者是通过中介属性间接决定。本题中 $B \rightarrow D$ 是部分依赖（B 是 AB 的真子集），而非传递依赖。

知识点：6.3 范式——2NF 与部分函数依赖的形式化定义 ($X' \subsetneq X$, $X' \rightarrow Y$)；候选码求解的闭包法；2NF 分解策略（分离部分依赖为独立关系）。

10.5 四、关系模式的分解 (24 分)

设有关系模式 $R(A, B, C, D, E)$, 函数依赖集

$$F = \{ A \rightarrow D, E \rightarrow D, D \rightarrow B, BC \rightarrow D, DC \rightarrow A \}$$

1. 判断 $\rho = \{AB, BE, CE, BD, AC\}$ 是否为无损连接分解。

参考答案

构造初始表格 (行为 5 个分解子模式, 列为 5 个属性), 用 a_i 表示本行对应的子模式包含该属性, b_{ij} 表示不包含。

	A	B	C	D	E
AB	a_1	a_2	b_{13}	b_{14}	b_{15}
BE	b_{21}	a_2	b_{23}	b_{24}	a_5
CE	b_{31}	b_{32}	a_3	b_{34}	a_5
BD	b_{41}	a_2	b_{43}	a_4	b_{45}
AC	a_1	b_{52}	a_3	b_{54}	b_{55}

逐轮应用函数依赖进行符号等同化:

第 1 轮:

- $A \rightarrow D$: 第 1 行 (AB) 和第 5 行 (AC) 的 A 列均为 a_1 , 将二者的 D 列统一。第 1 行 $D = b_{14}$, 第 5 行 $D = b_{54}$, 令 $b_{54} \leftarrow b_{14}$ 。
- $E \rightarrow D$: 第 2 行 (BE) 和第 3 行 (CE) 的 E 列均为 a_5 , 将二者的 D 列统一。第 2 行 $D = b_{24}$, 第 3 行 $D = b_{34}$, 令 $b_{34} \leftarrow b_{24}$ 。
- $D \rightarrow B$: 第 1 行 $D = b_{14}$ 、 $B = a_2$; 第 5 行 $D = b_{14}$ 、 $B = b_{52}$ 。D 相同, 令 $b_{52} \leftarrow a_2$ 。第 2 行 $D = b_{24}$ 、 $B = a_2$; 第 3 行 $D = b_{24}$ 、 $B = b_{32}$ 。令 $b_{32} \leftarrow a_2$ 。

更新后的表格:

	A	B	C	D	E
AB	a_1	a_2	b_{13}	b_{14}	b_{15}
BE	b_{21}	a_2	b_{23}	b_{24}	a_5
CE	b_{31}	a_2	a_3	b_{24}	a_5
BD	b_{41}	a_2	b_{43}	a_4	b_{45}
AC	a_1	a_2	a_3	b_{14}	b_{55}

第 2 轮:

- $BC \rightarrow D$: 第 3 行 ($B = a_2, C = a_3$) 与第 5 行 ($B = a_2, C = a_3$) 的 (B, C) 值相同, 将 D 列统一。第 3 行 $D = b_{24}$, 第 5 行 $D = b_{14}$, 令 $b_{24} \leftarrow b_{14}$ 。因第 2 行也通过 b_{24} 关联, 第 2 行的 D 也变为 b_{14} 。

- $DC \rightarrow A$: 第 3 行 ($D = b_{14}, C = a_3$) 与第 5 行 ($D = b_{14}, C = a_3$) 的 (D, C) 值相同, 将 A 列统一。第 3 行 $A = b_{31}$, 第 5 行 $A = a_1$, 令 $b_{31} \leftarrow a_1$ 。

最终表格:

	A	B	C	D	E
AB	a_1	a_2	b_{13}	b_{14}	b_{15}
BE	b_{21}	a_2	b_{23}	b_{14}	a_5
CE	a_1	a_2	a_3	b_{14}	a_5
BD	b_{41}	a_2	b_{43}	a_4	b_{45}
AC	a_1	a_2	a_3	b_{14}	b_{55}

判断: 在最终表格中, 没有任何一行全部由 a 符号组成 (即不存在 a_1, a_2, a_3, a_4, a_5 的行)。因此,

ρ 为有损连接分解

解析与知识点

Chase 算法 (追踪法/表格法) 判断分解的无损连接性——符号等同化规则与全 a 行判定准则。

Chase 算法的核心思想: 将分解的各子模式按属性展开为表格, 然后逐轮应用函数依赖将符号等同化 (如果 X 值相同, 则 Y 值也必须相同), 直到表格不再变化。若最终某一行全由 a_i 组成, 则分解无损; 否则有损。本题中尽管有多个函数依赖相互作用 (如 $BC \rightarrow D$ 和 $DC \rightarrow A$ 形成循环), 但 D 列始终存在多个不同的符号值 (b_{14} 与 a_4), 无法通过 Chase 过程统一, 导致无一行全 a 。根本原因在于 ρ 缺少单独包含候选码 CE 的子模式, 导致无法从分解片段中无损还原原始关系。最终结论: ρ 为有损连接分解。

Chase 算法的理论基础: 通过反复应用 F 中的 FD 将符号等同化, 等价于在一次关系代数连接中通过函数依赖推导出元组匹配。算法终止性由符号的有限性保证 (行数 $\times$ 列数有限, 每次等同化减少不同符号的数量)。操作技巧: (1) 初始化时准确标记 a_i 和 b_{ij} ; (2) 逐 FD 应用时只关注左部属性值完全相同的行对; (3) 出现符号变化时检查关联行是否也需要跟随变更。

常见错误有三: 一是初始化表格时误标 a_i 和 b_{ij} (属性是否属于子模式决定标记类型); 二是忽略 Chase 算法的迭代性质——一次修改后可能产生新的等同机会, 需要反复扫描所有 FD; 三是符号等同化时忘记同步更新关联行。

知识点: 6.5 无损连接分解——Chase 算法 (表格法) 的完整操作流程与理论基础; 符号等同化规则; 候选码覆盖缺失导致有损分解的根本原因分析。

2. 将 R 分解为 3NF, 并具有无损连接性和依赖保持性。

参考答案

Step 1: 求最小函数依赖集 F_m 。

检查 $F = \{A \rightarrow D, E \rightarrow D, D \rightarrow B, BC \rightarrow D, DC \rightarrow A\}$:

- 各函数依赖右部均为单属性，无需分解。
- 检查左部是否有冗余属性： $BC \rightarrow D$ 中， $B^+ = \{B\}$ 不含 D ， $C^+ = \{C\}$ 不含 D ， B 、 C 均不可删除； $DC \rightarrow A$ 中， $D^+ = \{D, B\}$ 不含 A ， $C^+ = \{C\}$ 不含 A ， D 、 C 均不可删除。
- 检查是否有冗余的函数依赖：
 - 去掉 $A \rightarrow D$ ，求 $A_{F-\{A \rightarrow D\}}^+ = \{A\}$ (不含 D)，不可去。
 - 去掉 $E \rightarrow D$ ， $E_{F-\{E \rightarrow D\}}^+ = \{E\}$ ，不可去。
 - 去掉 $D \rightarrow B$ ， $D_{F-\{D \rightarrow B\}}^+ = \{D\}$ ，不可去。
 - 去掉 $BC \rightarrow D$ ， $BC_{F-\{BC \rightarrow D\}}^+ = \{B, C\}$ ，不可去。
 - 去掉 $DC \rightarrow A$ ， $DC_{F-\{DC \rightarrow A\}}^+ = \{D, C, B\}$ (通过 $D \rightarrow B$ 和 $BC \rightarrow D$ 仍得不到 A)，不可去。

$F_m = F = \{A \rightarrow D, E \rightarrow D, D \rightarrow B, BC \rightarrow D, DC \rightarrow A\}$ 。

Step 2: 求候选码。

- C 和 E 未出现在任何函数依赖的右部，必属于候选码。
- 求 CE^+ :

$$\begin{aligned} \{C, E\} &\rightarrow E \rightarrow D \rightarrow \{C, D, E\} \\ &\rightarrow D \rightarrow B \rightarrow \{B, C, D, E\} \\ &\rightarrow DC \rightarrow A \rightarrow \{A, B, C, D, E\} \end{aligned}$$

CE 的闭包覆盖全部 5 个属性，候选码为 CE 。

Step 3: 按左部相同原则构造 3NF 关系模式。

- 左部 $\{A\}$: $\{A \rightarrow D\} \rightarrow R_1(\underline{A}, D)$
- 左部 $\{E\}$: $\{E \rightarrow D\} \rightarrow R_2(\underline{E}, D)$
- 左部 $\{D\}$: $\{D \rightarrow B\} \rightarrow R_3(\underline{D}, B)$
- 左部 $\{B, C\}$: $\{BC \rightarrow D\} \rightarrow R_4(\underline{B, C}, D)$
- 左部 $\{D, C\}$: $\{DC \rightarrow A\} \rightarrow R_5(\underline{D, C}, A)$

Step 4: 合并可合并的子模式，并确保包含候选码。

观察上面的子模式:

- $R_3(D, B)$ 被 $R_4(B, C, D)$ 包含 (R_4 的属性集是 R_3 的超集且包含 R_3 的码)，可被吸收。

- $R_1(A, D)$ 被 $R_5(D, C, A)$ 包含, 可被吸收。
- $R_2(E, D)$ 保持独立。
- 检查候选码 CE 是否被某个子模式包含: $R_2(E, D)$ 有 E 无 C , R_4 有 C 无 E , R_5 有 C 无 E 。均不包含完整候选码 CE , 需单独添加 $R_6(C, E)$ 。

合并后得到:

$$\rho_{3NF} = \{ \underline{ED}, \underline{BCD}, \underline{DCA}, \underline{CE} \} = \{ DE, BCD, ACD, CE \}$$

验证:

- 无损连接性: 子模式 CE (即 R_6) 包含候选码, 保证了连接的无损性。
- 依赖保持性: 每个函数依赖的左右部属性均保留在相应子模式中 ($A \rightarrow D \in R_5$, $E \rightarrow D \in R_2$, $D \rightarrow B \in R_4$, $BC \rightarrow D \in R_4$, $DC \rightarrow A \in R_5$), 依赖完全保持。
- 3NF 满足性: 各子模式中无非主属性对候选码的传递依赖。

解析与知识点

3NF 分解算法的完整操作流程——求最小函数依赖集、求候选码、按左部相同原则分组、子模式合并与候选码覆盖检查。

步骤要点: (1) 求最小函数依赖集——逐 FD 检查左部属性冗余和 FD 冗余, 确保最小化; (2) 求候选码——利用「未在右部出现的属性必属于候选码」规则, 快速锁定 C 和 E , 再计算闭包确认 CE 为候选码; (3) 按相同左部合并 FD, 生成初始子模式集; (4) 消除被包含的子模式—— $R_3(D, B)$ 被 $R_4(B, C, D)$ 吸收, $R_1(A, D)$ 被 $R_5(D, C, A)$ 吸收; (5) 候选码 CE 不在任何子模式中, 必须额外创建 CE 以保证无损连接。最终结果: $\rho_{3NF} = \{ DE, BCD, ACD, CE \}$ 。

子模式合并策略的优化原理: 当两个子模式 $R_i(X, Y)$ 和 $R_j(X, Z)$ 拥有相同的码, 且一个属性集是另一个的超集时, 可以安全地将较小的子模式吸收掉, 因为较小子模式的数据可以通过较大子模式的投影无损恢复。合并需保证被吸收子模式的函数依赖在吸收子模式中仍然成立。

常见错误是在 3NF 分解后忘记检查候选码是否已被某子模式覆盖——若候选码未覆盖 (如此题的 CE), 分解可能不满足无损连接性, 必须额外添加候选码对应的关系模式。

知识点: 6.5 3NF 分解算法——最小函数依赖集的求解 (左部冗余检测 + FD 冗余检测); 候选码求解技巧 (未在右部出现的属性必属候选码); 子模式合并的优化策略与安全性条件。

3. 将 R 分解为 BCNF, 并具有无损连接性。

参考答案

BCNF 分解采用迭代方式：每次找出违反 BCNF 的函数依赖 $X \rightarrow Y$ （即 X 不是超码），用它将当前关系分解为 $X \cup Y$ 和 $R - (Y - X)$ ，直到所有子关系均满足 BCNF。

初始： $R(A, B, C, D, E)$ ，已求得候选码为 CE 。

第 1 步：检查 BCNF 违规。

遍历 F ，逐一判断左部是否为超码：

- $A \rightarrow D$: $A^+ = \{A, D, B\} \neq ABCDE$, A 非超码，违反 BCNF。
- $E \rightarrow D$: $E^+ = \{E, D, B\} \neq ABCDE$, E 非超码，违反 BCNF。
- $D \rightarrow B$: $D^+ = \{D, B\} \neq ABCDE$, D 非超码，违反 BCNF。
- $BC \rightarrow D$: $BC^+ = \{B, C, D, A\} \neq ABCDE$ (缺少 E)， BC 非超码，违反 BCNF。
- $DC \rightarrow A$: $DC^+ = \{D, C, A, B\} \neq ABCDE$ (缺少 E)， DC 非超码，违反 BCNF。
- 候选码 CE 本身是超码，不构成违规。

选择 $A \rightarrow D$ 进行第一次分解。

$$R_1(\underline{A}, D), \quad R_2(A, B, C, E)$$

第 2 步：对 $R_1(AD)$ 判定 BCNF。

R_1 上投影的函数依赖为 $A \rightarrow D$ 。候选码为 A ，左部即超码。 R_1 满足 BCNF。

第 3 步：对 $R_2(ABCE)$ 求函数依赖投影。

将 F 投影到属性集 $\{A, B, C, E\}$ 上（计算 F^+ 中仅含 $\{A, B, C, E\}$ 属性的非平凡函数依赖）：

$$A^+ \cap ABCE = \{A, D, B\} \cap \{A, B, C, E\} = \{A, B\} \Rightarrow \mathbf{A \rightarrow B}$$

$$E^+ \cap ABCE = \{E, D, B\} \cap \{A, B, C, E\} = \{B, E\} \Rightarrow \mathbf{E \rightarrow B}$$

$$BC^+ \cap ABCE = \{B, C, D, A\} \cap \{A, B, C, E\} = \{A, B, C\} \Rightarrow \mathbf{BC \rightarrow A}$$

R_2 上的函数依赖为 $\{A \rightarrow B, E \rightarrow B, BC \rightarrow A\}$ 。候选码为 CE (CE^+ 在 R_2 上经 $E \rightarrow B$ 和 $BC \rightarrow A$ 可得全集)。

第 4 步：在 R_2 中检查 BCNF 违规。

- $A \rightarrow B$: $A_{R_2}^+ = \{A, B\} \neq ABCE$, A 非超码，违反 BCNF。
- $E \rightarrow B$: $E_{R_2}^+ = \{B, E\} \neq ABCE$, E 非超码，违反 BCNF。
- $BC \rightarrow A$: $BC_{R_2}^+ = \{A, B, C\} \neq ABCE$, BC 非超码，违反 BCNF。

选择 $A \rightarrow B$ 分解 R_2 ：

$$R_{21}(\underline{A}, B), \quad R_{22}(A, C, E)$$

第 5 步：判定 R_{21} 和 R_{22} 。

- $R_{21}(AB)$ ：函数依赖 $A \rightarrow B$ ，候选码 A ，左部即超码。**BCNF**。
- $R_{22}(ACE)$ ：将 R_2 的 FD 投影到 $\{A, C, E\}$ 。经计算仅 $CE \rightarrow A$ 成立（ CE 是 R_{22} 的唯一候选码）。 CE 是超码。**BCNF**。

最终 **BCNF** 分解结果：

$$\rho_{\text{BCNF}} = \{ \underline{AD}, \underline{AB}, \underline{CEA} \} = \{ AD, AB, ACE \}$$

说明：BCNF 分解结果不唯一，取决于每次选择哪个违规 FD 进行分解。若从 $E \rightarrow D$ 开始分解，可能得到 $\{ED, BD, CD, ACE\}$ 等等价结果。但无论次序如何，最终各子模式均满足 BCNF，且分解具有无损连接性（BCNF 分解算法二分性保证）。注意 BCNF 分解不保证依赖保持性——本题中 $E \rightarrow D$ 、 $D \rightarrow B$ 、 $BC \rightarrow D$ 、 $DC \rightarrow A$ 等函数依赖无法在单一子模式中得完整表达（它们的属性分散在不同子模式中），这是 BCNF 分解以牺牲依赖保持性为代价换取更高范式等级的典型体现。

解析与知识点

BCNF 分解算法——迭代式二分法的操作流程、函数依赖投影的计算方法、以及 BCNF 与 3NF 分解的权衡。

BCNF 分解算法的核心是迭代式二分法：在每一步中找到违反 BCNF 的 $X \rightarrow Y$ （ X 不是超码），将关系一分为二—— $X \cup Y$ 包含违规 FD 及其闭包，另一半 $R - (Y - X)$ 保留剩余的属性与依赖。每次分解保证无损（因为共享 X ）。重复此过程直至所有子关系均无 BCNF 违规。本题中选择 $A \rightarrow D$ 首次分解得 $R_1(AD)$ 和 $R_2(ABCE)$ ；在 R_2 中发现 $A \rightarrow B$ 违规，再分解得 $R_{21}(AB)$ 和 $R_{22}(ACE)$ 。最终结果： $\rho_{\text{BCNF}} = \{AD, AB, ACE\}$ 。

函数依赖投影的计算是 BCNF 分解中的技术难点。给定 F 和属性子集 Z ，需计算 $\pi_Z(F) = \{X \rightarrow Y \mid X \rightarrow Y \in F^+, XY \subseteq Z\}$ 。实用方法是计算 Z 中每个子集的闭包（在 F 的作用下），然后与 Z 取交集导出的 FD。BCNF 与 3NF 的权衡：3NF 得到 4 个子模式且保持所有依赖，BCNF 得到 3 个子模式但不保持所有依赖（如 $E \rightarrow D$ 、 $D \rightarrow B$ 等 FD 的属性分散在不同子模式中）。选择哪种分解需权衡数据一致性与查询效率。

常见错误有三：一是分解后忘记重新计算函数依赖投影，仍沿用原 F 来判定子模

式的 BCNF；二是 BCNF 分解结果不唯一（取决于每次选择哪个违规 FD 先分解），但常见错误是认为结果唯一；三是忽略 BCNF 分解不保证依赖保持性的限制——任何 BCNF 分解都以牺牲依赖保持性为代价。

知识点：6.5 BCNF 分解算法——迭代二分法的操作流程、无损分解的二分性定理、函数依赖投影的计算方法（闭包相交法）；6.5 BCNF 与 3NF 分解的权衡（BCNF 以牺牲依赖保持性换取更高范式等级）；分解结果的不唯一性（取决于违规 FD 的选择顺序）。

11 2023–2024 学年《数据库系统》期中测试

注意

试卷说明：本试卷共 15 道大题，覆盖数据库系统概论第一章至第三章的核心内容，包括数据库基本概念、关系模型与关系代数、SQL 语言等。其中第 1–9 题为单选题，第 10–12 题因原始扫描件水印覆盖导致内容无法辨识，仅作参考；第 13–15 题为 SQL 编程应用题，每题包含若干子问题。所有答案均来自官方参考答案，解析与知识点由编者补充。

11.1 一、单项选择题（第 1–9 题）

参考答案

B. 数据库管理系统。

解析与知识点

数据库系统的组成及各组成部分的角色定位，DBMS 的核心地位与其四大功能模块。

数据库系统 (DBS) 由数据库、数据库管理系统 (DBMS)、数据库管理员、应用程序和用户组成。其中 DBMS 是核心组成部分，位于用户与操作系统之间，负责数据的组织、存储、管理、访问和控制。数据库是数据的集合，数据模型是数据特征的抽象，软件工具是辅助性组件——它们都不是系统的核心调度与管理者。DBMS 的引入使得用户无需关心数据的物理存储细节，只需通过 SQL 等接口操作数据，这正是数据库系统区别于文件系统的关键所在。

从功能角度，DBMS 提供了数据定义 (DDL)、数据操纵 (DML)、数据控制 (DCL) 以及事务管理等核心功能。DDL 允许通过 CREATE、ALTER、DROP 等语句定义数据库的结构框架；DML 通过 SELECT、INSERT、UPDATE、DELETE 等语句实现数据的高效查询和修改；DCL 通过 GRANT、REVOKE 等机制保障数据

的安全性和完整性。这四大功能模块共同构成了 DBMS 作为数据库系统核心的完整能力体系。

常见错误是将数据库本身（数据的集合）或数据模型（抽象工具）误认为数据库系统的核心——实际上 DBMS 才是核心调度者与管理者。知识点：1.1 数据库系统概述——数据库系统的组成、DBMS 的核心地位及其数据定义、数据操纵、数据控制、事务管理四大功能模块。

参考答案

C. 数据冗余度大。

解析与知识点

数据库系统的四大基本特点及其与文件系统的核心区别。

数据库系统的四大基本特点为：数据结构化、数据共享性高和冗余度低、数据独立性高（物理与逻辑）、数据由 DBMS 统一管理与控制。选项 C「数据冗余度大」恰好与数据库系统的设计目标相反——数据库正是为了消除文件系统中普遍存在的数据冗余、不一致性和数据孤立等问题而诞生的。通过规范化设计和集中管理，数据库系统将冗余度控制在合理范围内，有时为了查询效率还允许适当保留受控冗余。

文件系统的数据库冗余问题根源在于数据面向具体应用程序组织——每个应用各自维护数据文件，同一份数据（如学生基本信息）可能在教务系统、财务系统和宿舍管理系统中重复存储。这种冗余不仅浪费存储空间，更导致数据不一致。数据库系统通过数据集成和统一管理，从根本上解决了文件系统的这些弊端。

常见错误是混淆数据库系统的特点与文件系统的弊端——数据冗余大、数据孤立、数据不一致等是文件系统的特征，而非数据库系统的特点。知识点：1.1 数据库系统概述——数据库系统的四大特点（数据结构化、共享性高与冗余度低、数据独立性高、统一管理与控制），以及与文件系统的对比。

参考答案

D. 实体-联系模型。

解析与知识点

数据模型的三层抽象（概念模型、逻辑模型、物理模型）及其各自的代表与作用，E-R 模型在概念层的核心地位。

数据模型的抽象层次从高到低依次为：概念模型、逻辑模型和物理模型。概念模型是对现实世界的第一次抽象，直接面向用户，独立于具体 DBMS。实体-联系

模型（E-R 模型）由 P. P. S. Chen 于 1976 年提出，是概念模型中最著名、应用最广泛的一种，用实体、属性和联系三个基本元素来描述现实世界的信息结构。层次模型和网状模型是早期数据库系统的逻辑模型，关系模型是目前主流数据库系统的逻辑模型，三者均属于第二层抽象。

三个抽象层次的区别：概念模型关注「有哪些实体？它们之间有什么联系？」——不涉及计算机如何存储；逻辑模型回答「数据如何组织？」——关系模型用二维表，网状模型用图结构，层次模型用树结构；物理模型解决「数据如何存储？」——涉及文件组织、索引结构、存储路径等底层细节。E-R 模型之所以在概念层占据核心地位，是因为其实体、属性、联系三个基本元素恰好能完整映射现实世界，而后可系统性转换为关系模型中的表和约束。

常见错误是混淆逻辑模型与概念模型的层次——关系模型是逻辑模型而非概念模型，E-R 模型才是概念模型的代表。另一个易错点是将层次模型或网状模型误认为是概念模型。知识点：1.2 数据模型——数据模型的三层抽象（概念模型、逻辑模型、物理模型），实体-联系模型（E-R 模型）的基本概念及其在数据库设计中的定位。

参考答案

C. 用户的应用程序与存储在磁盘上数据库中的数据是相互独立的。

解析与知识点

数据独立性——物理独立性与逻辑独立性的定义、区别及其在三级模式结构中的实现机制。

数据独立性分为逻辑独立性和物理独立性两个层面。物理独立性指应用程序与数据的物理存储结构相互独立——当数据的存储结构（如索引结构、文件组织方式、存储路径等）发生改变时，只需修改模式与内模式之间的映象，而不必修改应用程序。这是通过数据库系统的三级模式结构（外模式、模式、内模式）及其两级映象机制来实现的。

物理独立性和逻辑独立性的保护对象和触发条件不同。物理独立性应对存储层面的变化（如增加 B+ 树索引、迁移数据文件），只需调整模式/内模式映象。逻辑独立性应对概念模式的变化（如新增列、垂直拆分表），只需调整外模式/模式映象，确保用户看到的外模式视图仍包含其需要的属性子集。两级映象机制共同构成了数据库系统实现数据独立性的核心技术保障。

常见错误是将物理独立性与逻辑独立性的对应映像关系记反——物理独立性由模式/内模式映像保障，逻辑独立性由外模式/模式映像保障。知识点：1.3 数据库系统结构——三级模式结构（外模式、模式、内模式）、两层映象（外模式/模式映

象、模式/内模式映像)、物理独立性与逻辑独立性的区别。

参考答案

A. 模式与外模式之间的映像。

解析与知识点

三级模式结构中两级映像与数据独立性的对应关系——外模式/模式映像对应逻辑独立性,模式/内模式映像对应物理独立性。

三级模式结构中,外模式/模式映像定义外模式与概念模式之间的对应关系,当模式发生改变时只需调整该映像即可保持外模式不变,从而保证逻辑数据独立性。模式/内模式映像保证物理数据独立性。本题考察逻辑独立性,需修改外模式与模式之间的映像,选项 A 正确。

逻辑独立性比物理独立性更难实现,因为概念模式的变化(如关系结构调整)往往意味着数据逻辑组织方式的本质改变,外模式/模式映像可能需要在旧外模式与新概念模式之间建立基于视图或推导列的等价关系。而物理独立性的实现通常只需要重定向物理存取路径。这也是为什么数据库设计中强调概念模式的稳定性。常见错误是将两级映像与两种独立性的对应关系记反——逻辑独立性对应外模式/模式映像,物理独立性对应模式/内模式映像。知识点: 1.3 数据库系统结构——外模式/模式映像与逻辑数据独立性,模式/内模式映像与物理数据独立性。

参考答案

D. 关系。

解析与知识点

关系数据模型的基本数据结构(关系/二维表)及其与层次模型、网状模型的本质区别。

关系数据模型由 E. F. Codd 于 1970 年提出,其基本数据结构是关系(Relation),在用户眼中表现为一张二维表。关系模型中所有数据——包括实体和联系——统统用关系来表示,这与层次模型(树形结构)和网状模型(图结构)有本质区别。关系模型建立在严格的数学基础上(集合论和谓词逻辑),具有坚实的理论基础。索引是数据库中的辅助存取结构,不属于数据模型层面的基本数据结构。

关系模型的强大之处在于用同一种结构统一表示实体和联系。在层次模型中,多对多联系需引入冗余节点;在网状模型中,需在程序中显式操纵指针链。而关系模型中,一对一、一对多、多对多联系全部通过外码(Foreign Key)机制表达——只需在从关系中加入一个参照主关系的属性即可。这种统一性极大地简化了数据

操作接口（关系代数、SQL）的设计，也是关系模型能取代早期模型的根本原因。常见错误是将索引误认为是关系模型的基本数据结构——索引属于物理存储层面的辅助结构，而非数据模型的概念层面。另一个易错点是混淆关系模型（逻辑模型）与 E-R 模型（概念模型）的层次。知识点：2.1 关系数据结构——关系模型的基本数据结构（关系/二维表），关系模型与层次模型、网状模型的对比，关系模型的数学基础。

参考答案

C. 车次 + 日期。

解析与知识点

主码的选取原则——唯一性、最小性、稳定性，以及从业务语义分析候选码的思路。

主码应能唯一标识实体集中的每一个实体。分析「列车运营」实体：车次可标识列车线路，但不能区分同一线路的不同运营日；单独用日期无法区分同一天的多个车次。因此需要（车次，日期）的组合作为主码——同一车次在同一天只会运营一趟，该组合可以唯一标识每一次列车运营记录。情况摘要仅是描述性属性，不具有唯一性。

主码选取的三原则：唯一性——候选码的值必须能唯一标识一个元组；最小性——候选码中不能包含冗余属性（去掉任一属性后不能再唯一标识）；稳定性——主码的值不应当频繁变化。实际数据库设计中，有时会选择人工主码（如自增 ID）而不用自然组合键作为主码，因为自然主码可能太长或包含可变属性，虽满足理论唯一性但在实践中不够稳定。

常见错误是将描述性属性（如「情况摘要」）误作为主码组分——只有具有唯一区分能力的属性才能成为主码组分。另一个易错点是忽略最小性，将过多属性纳入候选码。知识点：2.2 关系完整性约束——实体完整性定义、主码的选取原则（唯一性、最小性、稳定性）、候选码与主码的区别。

参考答案

B. $S - (S - R)$ 。

解析与知识点

关系代数中交运算与差运算的等价转换—— $R \cap S = R - (R - S) = S - (S - R)$ ，体现了「通过基本运算表达导出运算」的核心思想。

表达式 $S - (S - R)$ 的含义：先从 S 中去掉所有 S 中有而 R 中没有的元组（即

$S - R$), 剩下的就是 S 和 R 共有的部分, 即 $R \cap S$ 。类似地 $R \cap S = R - (R - S)$ 也成立。选项 D $S - (R - S)$ 中 $R - S$ 是 R 独有的部分, $S - (R - S)$ 的结果可能包含 S 独有的元组, 不等于交集。

关系代数的五种基本运算是并、差、笛卡尔积、选择和投影, 而交、连接、除等均属导出运算。交运算等价于 $R - (R - S)$ 的逻辑是: 从 R 中减去「 R 中有而 S 中没有的元组」, 剩余的就是 R 与 S 共有的元组。这种等价变形在手工推导关系代数表达式和 DBMS 内部查询优化器中都有实际应用——当 DBMS 不直接支持交运算符时, 可自动转换为差运算来执行。

常见错误是混淆 $S - (S - R)$ 与 $S - (R - S)$ 的语义——后者减去的是 R 独有部分, 前者减去的是 S 独有部分, 两者含义完全不同。知识点: 2.4 关系代数——交运算与差运算的等价转换 ($R \cap S = R - (R - S) = S - (S - R)$), 基本运算与导出运算的关系。

参考答案

A. 全外联接。

解析与知识点

外联接的三种变体 (左外连接、右外连接、全外连接) 的语义差异及其适用场景。题意要求同时保留两类「缺失」信息: 没有住宿的学生和空闲的床位。左外联接只能保留左侧关系 (学生) 中的全部元组, 丢失空闲床位信息; 右外联接只能保留右侧关系 (宿舍) 中的全部元组, 丢失未住宿学生信息; 自然联接只返回匹配元组, 两类缺失信息都会丢失。唯有全外联接 (Full Outer Join) 可以同时保留左右两表的所有元组, 匹配时合并, 无匹配时用 NULL 填充, 完整满足题目要求。

外联接三种变体的应用场景: 左外联接适用于「以左表为主体, 补充右表信息」(如查询所有学生及其选修情况, 包括未选课的学生); 右外联接适用于「以右表为主体」; 全外联接适用于「需要完整列出所有可能的匹配与不匹配情况」。全外联接在 MySQL 中不直接支持, 可通过 `LEFT JOIN UNION RIGHT JOIN` 模拟。

常见错误是混淆三种外联接的保留方向——左外保留左表, 右外保留右表, 全外保留两表。另一个易错点是在需要全外联接时仅使用单向外联接, 导致一侧的不匹配元组被遗漏。知识点: 2.4 关系代数——外连接的三种类别 (左外连接、右外连接、全外连接) 及其语义与应用场景; 全外联接在 SQL 中的实现方式。

11.2 二、教材习题（第 10–12 题）

注意

说明：第 10–12 题在原始扫描文件中被水印严重覆盖，题目内容无法辨识。从残留文本可知，这些题目来源于教材《数据库系统概论》第二章（关系数据库）的课后习题，可能涉及关系代数表达式、关系演算或关系模型的完整性约束等内容。由于无法还原具体题目，以下仅列出答案文件中残存的有限信息，供复习时参考。建议读者自行查阅教材第二章课后习题进行针对性练习。

参考答案

（答案文本残缺，仅可辨识答案文件中与第 10 题子题 1 相关的数字「7」，具体含义不明。）

解析与知识点

教材第二章关系数据库的核心内容——关系数据结构、关系操作（五种基本运算）、关系完整性约束。

从题目标题推断，本题为教材第二章的课后习题。第二章的核心内容包括关系的数据结构定义（域、笛卡尔积、关系的性质）、关系操作（基本关系代数运算：并、差、选择、投影、笛卡尔积、连接和除）、关系完整性约束（实体完整性、参照完整性和用户定义完整性）以及关系演算。

五种基本运算（并、差、笛卡尔积、选择、投影）构成了一个完备的操作集——任何可表达的查询都可以由这五种运算的组合来实现。连接运算可以由笛卡尔积加选择来等价表达，交运算可以由差运算两次应用来等价表达，除运算是涉及「全部」语义的最复杂的导出运算。关系代数表达式等价的判定方法：两表达式等价当且仅当它们在任意合法数据库实例上产生相同的结果。

常见错误是将导出运算（连接、交、除）与基本运算混为一谈——牢记五种基本运算是理解关系代数完备性的基础。知识点：2.1–2.4 关系数据结构、关系操作（五种基本运算）、关系完整性（实体完整性、参照完整性、用户定义完整性）、关系代数的完备性。

参考答案

（答案文本被水印完全覆盖，无可用信息。）

解析与知识点

关系代数表达式设计的综合应用，以及关系数据库中的基础概念——关系的三种类型（基本表、视图、查询结果表）与关系模式 vs. 关系实例的区分。

建议复习教材第二章中关于关系代数表达式设计的典型例题，特别是使用选择 σ 、投影 π 、连接 $\bowtie$ 和除运算 $\div$ 完成复杂查询的题目。

关系的三种类型在实际数据库系统中有本质区别：基本表（Base Table）是实际存储在数据库中的表，具有独立的物理存在；视图（View）是定义在基本表之上的虚表，其数据不独立存储，每次访问时从基本表中实时导出；查询结果表是 SELECT 语句执行过程中产生的临时结果集，仅在查询期间存在。关系模式是型（type），关系实例是值（value）——某个关系模式可能不满足 3NF，但其某个特定实例可能碰巧不包含不符合 3NF 的数据。

常见错误是混淆基本表与视图的存储性质——视图定义存储于数据字典，但视图数据不独立存储。另一个易错点是混淆关系模式与关系实例的概念。知识点：2.4 关系代数——综合查询表达式设计；关系的三种类型（基本表、视图、查询结果表）；关系模式与关系实例的概念区分。

参考答案

（答案文本被水印完全覆盖，无可用信息。）

解析与知识点

关系演算（元组关系演算与域关系演算）的基本语法、三类完整性约束的实现方式、以及关系演算与关系代数的等价性（Codd 完备性定理）。

元组关系演算以元组变量为基础，形如 $\{t \mid P(t)\}$ ；域关系演算以域变量为基础，形如 $\{x_1, x_2, \dots, x_n \mid P(x_1, x_2, \dots, x_n)\}$ 。三类完整性约束：实体完整性通过主码保证，参照完整性通过外码保证，用户定义完整性通过 CHECK 等约束保证。

Codd 证明了关系代数、元组关系演算和域关系演算三者具有等价表达能力——任何能用其中一种表达的查询也能用另外两种等价表达，这称为关系模型的完备性。实际 SQL 的设计同时借鉴了两套体系：SELECT-FROM-WHERE 框架更接近关系演算的谓词语义，而 JOIN、UNION、EXCEPT 等关键词直接来源于关系代数。SQL 中的 EXISTS/NOT EXISTS 与关系演算中的存在量词 $\exists$ 直接对应。

常见错误是混淆元组关系演算与域关系演算的语法——前者使用元组变量，后者使用域变量。另一个易错点是忽略三类完整性约束各自的具体实现机制。知识点：

2.2-2.3 关系完整性约束（实体完整性——主码、参照完整性——外码、用户定义完整性——CHECK）；元组关系演算与域关系演算的基本语法与语义；关系演算与关系代数的等价性（Codd 完备性定理）。

11.3 三、SQL 应用题（第 13–15 题）

参考答案

- (1) 找出所有供应商的姓名和所在城市。

```
1 SELECT SNAME, CITY FROM S;
```

- (2) 找出所有零件的名称、颜色、重量。

```
1 SELECT PNAME, COLOR, WEIGHT FROM P;
```

- (3) 找出使用供应商 S1 所供应零件的工程号码。

```
1 SELECT DISTINCT JNO FROM SPJ WHERE SNO = 'S1';
```

使用 DISTINCT 避免重复——一个工程可能从 S1 采购多种零件或多次采购。

- (4) 找出工程项目 J2 使用的各种零件的名称及其数量。

```
1 SELECT PNAME, QTY  
2 FROM SPJ, P  
3 WHERE JNO = 'J2' AND SPJ.PNO = P.PNO;
```

通过 SPJ.PNO = P.PNO 实现 SPJ 表与 P 表的等值连接，以获取零件名称。

- (5) 找出上海厂商供应的所有零件号码。

```
1 SELECT PNO  
2 FROM S, SPJ  
3 WHERE S.CITY = '上海' AND S.SNO = SPJ.SNO;
```

通过 S.SNO = SPJ.SNO 连接供应商表和供应情况表，用 CITY = '上海' 筛选上海供应商。

- (6) 找出使用上海产的零件的工程名称。

```
1 SELECT JNAME  
2 FROM S, SPJ, J  
3 WHERE S.CITY = '上海'  
4 AND S.SNO = SPJ.SNO  
5 AND J.JNO = SPJ.JNO;
```

注意：原答案中 FROM 子句缺少表 J，此处已修正。该查询需要三表连接（S、SPJ、J），通过供应商城市筛选上海的供应商，再通过 SPJ 桥接至工程表获取名称。

(7) 找出没有使用天津产的零件的工程号码。(以下给出三种正确写法)

```
1  -- 方法一：使用 NOT IN 子查询
2  SELECT DISTINCT JNO FROM J WHERE JNO NOT IN
3      (SELECT JNO FROM S, SPJ
4          WHERE CITY = '天津' AND S.SNO = SPJ.SNO);
5
6  -- 方法二：从 SPJ 中排除
7  SELECT DISTINCT JNO FROM SPJ WHERE JNO NOT IN
8      (SELECT JNO FROM S, SPJ
9          WHERE CITY = '天津' AND S.SNO = SPJ.SNO);
10
11 -- 方法三：使用 NOT EXISTS 相关子查询
12 SELECT DISTINCT JNO FROM J WHERE NOT EXISTS
13     (SELECT * FROM S, SPJ
14         WHERE CITY = '天津'
15             AND S.SNO = SPJ.SNO
16             AND J.JNO = SPJ.JNO);
```

注意：原答案中还列出了一种使用 `CITY != '天津'` 的写法，但该方法存在逻辑错误——它返回的是「至少使用了一种非天津供应商零件的工程号」，而非「完全未使用天津供应商零件的工程号」。若某工程同时使用了天津和非天津供应商的零件，该方法会错误地将其纳入结果。上述三种方法通过 `NOT IN` 或 `NOT EXISTS` 正确排除了与天津供应商有供应关系的工程。

(8) 把全部红色零件的颜色改成蓝色。

```
1  UPDATE P SET COLOR = '蓝' WHERE COLOR = '红';
```

(9) 由 S5 供给 J4 的零件 P6 改为由 S3 供应，请作必要的修改。

```
1  UPDATE SPJ SET SNO = 'S3'
2  WHERE SNO = 'S5' AND JNO = 'J4' AND PNO = 'P6';
```

该操作仅修改 SPJ 表中的 SNO 字段，不涉及供应商表 S 本身的修改，因为 S3 供应商在 S 表中是独立于 SPJ 中这一条供应记录而存在的。

(10) 从供应商关系中删除 S2 的记录，并从供应情况关系中删除相应的记录。

```
1  DELETE FROM SPJ WHERE SNO = 'S2';
2  DELETE FROM S WHERE SNO = 'S2';
```

注意：必须先删除 SPJ 中的关联记录，再删除 S 中的记录，否则若存在外键约束（SPJ.SNO 参照 S.SNO）且未设置级联删除，先删 S 表会导致参照完整性违规而失败。若外键设置了 ON DELETE CASCADE，则只需执行 DELETE FROM S 即可自动级联删除 SPJ 中的相关记录。

解析与知识点

SPJ（供应商-零件-工程）综合查询案例——覆盖 SQL 核心操作（SELECT、UPDATE、DELETE）、多表连接、嵌套子查询、外键约束下的删除顺序。

本题系统覆盖了 SQL 的核心操作：单表查询、带 DISTINCT 的投影查询、等值连接查询、多表连接查询、嵌套子查询与 NOT IN/NOT EXISTS、UPDATE 更新以及 DELETE 删除。第 7 子题特别值得关注——「否定」类查询需要使用 NOT IN 或 NOT EXISTS 来实现，简单的!= 比较无法正确表达「不存在此类关系」的语义。第 10 子题涉及外键约束下的删除顺序——先删子表再删主表是安全操作的黄金法则。

SPJ 案例包含三个实体表（S 供应商、P 零件、J 工程）和一个联系表（SPJ 供应情况），构成典型的多对多关系结构。这种多对多关系必须通过联系表来实现，是关系模型中「实体-联系」统一用关系表示的典型实践。本题还覆盖了 CREATE VIEW 和 DROP VIEW 等视图管理语句，使学生理解视图的定义和更新机制与基本表的差异。

常见错误有三：一是在「否定」类查询中错误使用!= 而非 NOT IN/NOT EXISTS，导致语义错误；二是在外键约束下未按正确顺序（先子表后主表）执行 DELETE，导致参照完整性违规；三是多表连接时遗忘某个必要的连接条件，产生笛卡尔积。知识点：3.1–3.6 SQL 语言——SELECT、UPDATE、DELETE 的完整语法；等值连接与多表连接查询；嵌套子查询（NOT IN、NOT EXISTS）的区别与适用场景；DISTINCT 消重；外键约束下的级联删除（ON DELETE CASCADE）与删除顺序。

参考答案

(1) 查询考试成绩不及格的学生学号及成绩。

```
1 SELECT sno, grade
2 FROM s_c
3 WHERE grade < 60;
```

(2) 查询年龄在 19–25 岁（包括 19 和 25 岁）之间的学生的姓名、院系和年龄，并按年龄的降序排序。

```
1 SELECT sname, dept, age
2 FROM student
3 WHERE age BETWEEN 19 AND 25
4 ORDER BY age DESC;
```

- (3) 查询姓名中含「国」字的学生档案情况。

```
1 SELECT *
2 FROM student
3 WHERE sname LIKE '%国%';
```

% 在 LIKE 模式中匹配任意长度的字符串（包括空串），% 国% 表示「国」字可以出现在姓名的任意位置。

- (4) 按院系查询学生总人数。

```
1 SELECT dept, COUNT(*) AS 学生总人数
2 FROM student
3 GROUP BY dept;
```

- (5) 计算选修了 008 号课程的学生平均成绩和最高分及最低分。

```
1 SELECT sno,
2         AVG(grade) AS 平均成绩,
3         MAX(grade) AS 最高分,
4         MIN(grade) AS 最低分
5 FROM s_c
6 WHERE sno IN
7     (SELECT sno
8      FROM s_c
9      WHERE cno = '008')
10 GROUP BY sno;
```

解读：该查询先通过子查询找出所有选修了 008 号课程的学生学号，再对这些学生以学号分组，计算每位学生在所有课程上的平均成绩、最高分和最低分。

- (6) 求平均成绩在 85 分以上（含 85）的各门课程号及平均成绩。

```
1 SELECT cno, AVG(grade) AS 平均成绩
2 FROM s_c
```

```
3 GROUP BY cno
4 HAVING AVG(grade) >= 85;
```

关键点：使用 HAVING 而非 WHERE 来筛选分组后的聚合结果。WHERE 作用于分组前的单行，HAVING 作用于分组后的聚合值。

解析与知识点

SQL 查询的进阶技能——BETWEEN AND、LIKE 模糊匹配、GROUP BY 分组与聚合函数、HAVING 分组过滤、以及 SQL 查询的逻辑执行顺序。

第 1 子题：基本条件查询 (WHERE 比较运算)；第 2 子题：BETWEEN AND 范围查询 + ORDER BY 排序 (DESC 降序, ASC 升序)；第 3 子题：LIKE 模糊匹配 (% 匹配任意字符序列, _ 匹配单字符)；第 4 子题：GROUP BY 分组 + COUNT 聚合函数；第 5 子题：子查询 + IN + 多聚合函数 (AVG、MAX、MIN) 的组合；第 6 子题：HAVING 子句的经典应用——使用 HAVING AVG(grade) >= 85 筛选分组后结果。

SQL 查询的逻辑执行顺序不按语法次序 (SELECT-FROM-WHERE- GROUP BY-HAVING-ORDER BY), 而是: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY。这解释了为什么 WHERE 不能使用聚合函数 (WHERE 在分组前执行), 而 HAVING 可以使用聚合函数 (HAVING 在分组后执行)。

常见错误有二：一是在 WHERE 子句中错误使用聚合函数——聚合条件必须放在 HAVING 中；二是 LIKE 模式中 % 和 _ 的使用混淆——% 匹配任意长度, _ 仅匹配单个字符。知识点：3.3-3.5 数据查询进阶——WHERE 条件筛选与比较运算；BETWEEN AND 范围查询；LIKE 模糊匹配 (% 与 _ 通配符)；GROUP BY 分组；聚合函数 (COUNT、AVG、MAX、MIN)；HAVING 分组过滤 (与 WHERE 的原则性区别)；ORDER BY 排序 (ASC/DESC)；嵌套子查询；IN 操作符；SQL 查询逻辑执行顺序。

参考答案

(1) 查询「张景林」老师所授课程号和课程名。

```
1 SELECT cno, cname
2 FROM C
3 WHERE teacher = '张景林';
```

(2) 查询选修课程名为「C 语言」或者「数据库」的学生号。

```
1 -- 方法一：使用 IN 子查询
```



```
2 SELECT sno
3 FROM SC
4 WHERE cno IN
5     (SELECT cno
6      FROM C
7      WHERE cname = 'C语言' OR cname = '数据库');
8
9 -- 方法二：使用等值连接
10 SELECT SC.sno
11 FROM SC, C
12 WHERE SC.cno = C.cno
13        AND (cname = 'C语言' OR cname = '数据库');
```

注意：方法二中条件 (cname = 'C语言' OR cname = '数据库') 必须加括号，否则 AND 优先级高于 OR 会导致逻辑错误。

(3) 查询「高晓灵」同学所选修课程的课程号及课程名。

```
1 -- 方法一：使用嵌套子查询
2 SELECT cno, cname
3 FROM C
4 WHERE cno IN
5     (SELECT cno
6      FROM SC
7      WHERE sno IN
8          (SELECT sno
9           FROM S
10          WHERE sname = '高晓灵'));
11
12 -- 方法二：使用三表连接查询
13 SELECT C.cno, C.cname
14 FROM SC, C, S
15 WHERE SC.cno = C.cno
16        AND SC.sno = S.sno
17        AND sname = '高晓灵';
```

(4) 查询至少选修课程号为 C3 和 C4 的学生姓名。

```
1 -- 方法一：嵌套子查询
2 SELECT sname
```

```
3 FROM S
4 WHERE sno IN
5     (SELECT sno
6      FROM SC
7      WHERE cno = 'C3'
8      AND sno IN
9          (SELECT sno
10         FROM SC
11         WHERE cno = 'C4'));
12
13 -- 方法二：SC 表自连接
14 SELECT sname
15 FROM S
16 WHERE sno IN
17     (SELECT SC1.sno
18      FROM SC SC1, SC SC2
19      WHERE SC1.sno = SC2.sno
20      AND SC1.cno = 'C3'
21      AND SC2.cno = 'C4');
```

方法二使用了 SC 表的自连接——将 SC 表分别取别名为 SC1 和 SC2，通过 SC1.sno = SC2.sno 确保是同一学生，再分别筛选 SC1 对应 C3 课程、SC2 对应 C4 课程。自连接是解决「至少同时满足多个条件」类查询的常用技巧。

(5) 用 EXISTS 来查询学习课程号为 C5 的学生学号与姓名。

```
1 SELECT sno, sname
2 FROM S
3 WHERE EXISTS
4     (SELECT *
5      FROM SC
6      WHERE S.sno = SC.sno
7      AND SC.cno = 'C5');
```

EXISTS 是相关子查询 (Correlated Subquery) 的关键词。子查询中引用了外层查询的 S.sno，因此 DBMS 对 S 表中每一行都要执行一次内层子查询：检查 SC 表中是否存在该学生选修 C5 的记录。若存在，EXISTS 返回 TRUE，该行被保留。EXISTS 不关心子查询返回的具体值，只关心是否有结果行返回。

(6) 查询不学 C2 课程的学生姓名与性别。

```
1  -- 方法一：使用 NOT IN (不相关子查询)
2  SELECT sname, sex
3  FROM S
4  WHERE sno NOT IN
5      (SELECT sno
6       FROM SC
7       WHERE cno = 'C2');
8
9  -- 方法二：使用 NOT EXISTS (相关子查询)
10 SELECT sname, sex
11 FROM S
12 WHERE NOT EXISTS
13     (SELECT *
14      FROM SC
15      WHERE S.sno = SC.sno
16           AND SC.cno = 'C2');
```

NOT IN 与 NOT EXISTS 的对比：NOT IN 利用子查询构建一个「学了 C2 的学生集合」，然后排除这些学号；NOT EXISTS 则逐行检查每个学生是否存在 C2 的选课记录。两者的结果等价，但 NOT IN 可能因子查询结果包含 NULL 而产生意外行为——如果子查询返回 NULL，sno NOT IN (NULL, ...) 永远为 UNKNOWN，导致查询结果为空。因此对于涉及可能包含 NULL 的场景，NOT EXISTS 更为安全可靠。

解析与知识点

SQL 高级查询技术——EXISTS/NOT EXISTS 相关子查询、IN 与 NOT IN 的区别与 NULL 处理、自连接 (Self-Join)、以及 SQL 中的集合思维。

第 2 子题的 OR 连接与 IN 子查询体现「集合成员资格」的查询思想。第 3 子题的三表连接通过外码逐层桥接跨越多个实体。第 4 子题的自连接解决「同时满足多个条件」的问题——必须使用别名区分同一表的两个实例。第 5 子题的 EXISTS 相关子查询与 IN 的区别在于：IN 先执行子查询构建集合再做判断，EXISTS 对外层每一行循环执行内层子查询。第 6 子题考察 NOT IN vs NOT EXISTS。

EXISTS 与 IN 的执行逻辑差异：当子查询结果集较小、外层表较大时，IN 往往更高效；当外层结果集较小、子查询表较大时，EXISTS 可能更优（找到第一个匹配即可提前终止）。EXISTS 不受 NULL 干扰——它仅判断「是否存在」，而 IN 在遇到 NULL 时因三值逻辑退化为 UNKNOWN。NOT IN 的 NULL 问题更加严重：NOT IN (NULL, ...) = UNKNOWN 会导致整个查询返回空集。

常见错误有四：一是混淆 EXISTS 与 IN 的执行逻辑和 NULL 处理差异；二是在自连接中忘记使用别名区分同一表的两个实例；三是在子查询中使用 OR 条件时忘记加括号（AND 优先级高于 OR）；四是使用 NOT IN 时忽略子查询结果可能包含 NULL 的风险。知识点：3.3-3.4 高级 SQL 查询技术——EXISTS 与 NOT EXISTS 相关子查询；IN 与 NOT IN 的区别及 NULL 值处理差异（三值逻辑）；自连接（Self-Join）；多表连接与别名机制；子查询的两种类型（相关子查询 vs. 不相关子查询）；SQL 中的逻辑运算符优先级（AND 高于 OR）。

第三部分 附录

12 知识点索引

本索引按数据库课程章节组织，列出各知识点在试卷中的出现位置，便于针对性复习。

第一章：绪论

- 数据模型三要素——10-11: 二-1; 08-09: 一-9
- 数据库系统三级模式结构——10-11: 二-8; 08-09: 一-11
- 数据独立性——10-11: 二-8; 08-09: 一-11

第二章：关系数据库

- 关系代数基本运算——10-11: 一-1
- 关系完整性约束——各卷均有涉及
- 关系代数表达式——各卷查询设计题

第三章：SQL 语言

- 数据定义（CREATE TABLE）——10-11: 三-3
- 数据查询（SELECT）——各卷查询设计题
- 数据更新——各卷均有涉及
- 视图——10-11: 一-2; 08-09: 一-14

- 索引——10-11: 一-5; 08-09: 一-8
- GRANT/REVOKE ——08-09: 一-4
- 触发器——08-09: 四-3(5)

第四章：数据库安全性

- 存取控制（GRANT/REVOKE）——08-09: 一-4; 10-11: 四-2(3)
- 数据库安全措施——各卷均有涉及

第五章：数据库完整性

- 实体完整性——各卷设计题
- 参照完整性——各卷设计题
- 用户定义完整性——各卷设计题
- CHECK 约束——10-11: 三-3; 08-09: 四-3(3)
- 断言与触发器——08-09: 四-3(5)

第六章：关系数据理论

- 函数依赖——10-11: 四-1; 08-09: 四-1,2
- 候选码求解——各卷规范化设计题
- 范式（1NF-BCNF）——各卷规范化设计题
- 多值依赖——10-11: 一-7; 08-09: 二-5
- 模式分解——10-11: 四-1; 08-09: 四-2
- 无损连接分解——08-09: 四-2(2)
- Armstrong 公理——10-11: 一-10; 08-09: 二-1

第七章：数据库设计

- E-R 模型——各卷数据库设计题
- E-R 图向关系模式转换——各卷数据库设计题
- 数据库设计阶段——08-09: 一-1,2,3

第八章：数据库编程

- 嵌入式 SQL ——08-09: 二-4
- 存储过程与触发器——08-09: 四-3(5)

第九章：查询处理与优化

- 查询优化方法——10-11: 五-2; 08-09: 四-3(4)
- 代数优化——各卷查询设计题

第十章：事务与并发控制

- 事务 ACID 特性——08-09: 一-15; 10-11: 二-5
- 并发操作问题——08-09: 二-10
- 封锁机制——10-11: 一-6; 08-09: 一-12, 二-9
- 死锁检测——10-11: 二-10
- 可串行化调度——相关试卷

第十一章：数据库恢复

- 故障分类——10-11: 二-7; 08-09: 二-7
- 日志文件——10-11: 一-8; 08-09: 二-8
- 检查点——10-11: 二-3; 08-09: 二-6
- 数据镜像——10-11: 一-9

13 TikZ 样式参考

本解析册中所有 ER 图均使用统一的 TikZ 样式绘制，样式定义如下：

- entity ——实体（矩形框）
- weakentity ——弱实体（双线矩形框）
- relationship ——联系（菱形框）
- weakrelationship ——弱联系（双线菱形框）

- attribute ——属性（椭圆框）
- keyattr ——主属性（下划线标注）
- derivedattr ——派生属性（虚线椭圆框）